

TABLE OF CONTENT

Lecture 1 — Introduction to ReactJS.....	2
Lecture 2 — JSX in ReactJS.....	16
Lecture 3 — Expressions in JSX	34
Lecture 4 — Styling in ReactJS.....	55
Lecture 5 — Basic Event Management in ReactJS.....	79
Lecture 6 — Introduction to Components in ReactJS.....	102
Lecture 7 — Function Components in ReactJS.....	133
Lecture 8 — Class Components in ReactJS.....	165
Lecture 9 — Components in Separate Files in ReactJS	201
Lecture 10 — React Props.....	241
Lecture 11 — React DevTools	281
Lecture 12 — Rendering Arrays in React	309
Lecture 13 — Conditional Rendering in React	350
Lecture 14 — Routing in React	388
Lecture 15 — Parameterized Routes in React	425
Lecture 16 — Introduction to Hooks in React	466
Lecture 17 — useState Hook in React.....	497
Lecture 18 — useEffect Hook in React.....	539
Lecture 19 — Forms in React	581
Lecture 20 — Creating a Mock API.....	627
Lecture 21 — Consuming GET ALL API in React.....	656
Lecture 22 — Consuming GET BY ID API in React	697
Lecture 23 — Consuming DELETE API in React	734
Lecture 24 — Consuming INSERT API in React.....	776
Lecture 25 — Consuming UPDATE API in React.....	822

Note: Content for the material is generated using AI from Video Series "ReactJS for beginner" which is available on Darshan University's Youtube Channel

Youtube Playlist Link: <https://www.youtube.com/playlist?list=PLBmCVCJR032I>

Lecture 1 — Introduction to ReactJS

1. Opening / Hook

Hello everyone, and welcome to this ReactJS course.

If you already know HTML, CSS, and JavaScript, then you already have most of the foundation you need to start learning React.

But before we write our first React component, let's understand one important question:

Why do we actually need React?

You might be thinking:

“I already know JavaScript. I can create HTML elements, modify the DOM, handle events, and build web applications. So why should I learn React?”

That's exactly what we're going to understand in this lecture.

By the end of this video, you'll understand what React is, why it became popular, how it works at a high level, and we'll also create our first React application.

2. What is React?

Let's start with the definition.

React is a JavaScript library for building user interfaces.

React was originally developed at Facebook and was released publicly in 2013.

Today, React is one of the most widely used technologies for building modern web applications.

But don't just memorize the definition.

Let's understand what “building user interfaces” actually means.

Suppose we're creating an e-commerce website.

We may have:

- Header
- Navigation bar

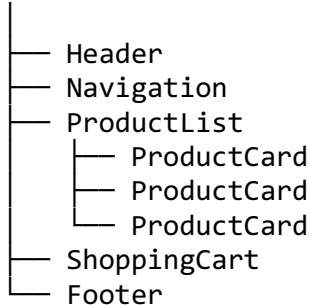
- Product list
- Product card
- Shopping cart
- Login form
- Footer

All of these are parts of the user interface.

With React, we can break these parts into smaller, reusable pieces called **components**.

For example:

Application



We'll spend a lot of time understanding components in the upcoming lectures.

For now, remember one important idea:

React allows us to build the UI using reusable components.

3. Why Do We Need React?

Now let's understand the problem React tries to solve.

Imagine you're building a large application using only JavaScript and the DOM.

You might write code like:

```
const heading = document.createElement("h1");
heading.innerText = "Products";

document.body.appendChild(heading);
```

Then you create buttons, lists, forms, and other elements.

For a small application, this can work perfectly fine.

But as the application becomes larger, managing the UI manually becomes increasingly difficult.

For example, imagine that we have 1,000 products.

When a user adds a product to the shopping cart, we may need to update:

- Product quantity
- Cart count
- Cart total
- Product availability
- Checkout information

We have to carefully manage which parts of the DOM need to change.

This is where React provides a better way of thinking about UI development.

Instead of manually saying:

“Find this element and change this value.”

we describe what the UI should look like based on our application data.

React then helps update the UI when that data changes.

4. Traditional JavaScript vs React

Let's compare the approaches.

Suppose we have:

Count: 0

[Increment]

When the user clicks Increment, the count should become 1.

With traditional JavaScript, we might manually find the element:

```
document.getElementById("count").innerText = count;
```

We are directly manipulating the DOM.

In React, we'll generally think like this:

State = 1

UI should display:

Count: 1

We update the state, and React takes care of updating the appropriate UI.

This is one of the important differences in the programming model.

5. React is Declarative

There is another important word you'll hear frequently when learning React:

Declarative.

Don't worry about the terminology.

Think of it this way.

In an imperative approach, we tell the computer:

Do this. Find this element. Change this text. Hide this button. Show that element.

In a declarative approach, we describe:

"This is what the UI should look like when the application is in this state."

React follows this declarative approach.

For example:

If user is logged in:
Show Dashboard

If user is not logged in:
Show Login

We describe the desired UI based on the state of the application.

React manages the updates.

6. Component-Based Architecture

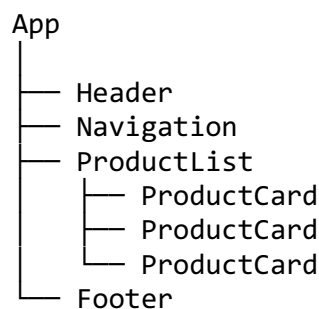
Now let's look at another major concept.

React applications are **component-based**.

Imagine that we're building an Amazon-like product page.

Instead of putting everything into one huge file, we can divide the application into components.

For example:



Each component has its own responsibility.

For example:

Header

Responsible for the website header.

ProductCard

Responsible for displaying one product.

Footer

Responsible for the footer.

The big advantage is **reusability**.

Suppose we need 100 products.

We don't need to write 100 different ProductCard implementations.

We create one ProductCard component and reuse it.

We'll explore this concept in detail when we reach components.

7. React and Single Page Applications

Another term you'll hear frequently is:

SPA — Single Page Application.

Traditionally, when you navigate between pages, the browser may request a new HTML page from the server.

For example:

```
/products  
/about  
/contact
```

In a single-page application, the browser initially loads the application, and JavaScript can dynamically change what is displayed as the user navigates.

React is commonly used to build these kinds of applications.

Later in this course, we'll use React Router to create routes such as:

```
/  
/products  
/products/1  
/about  
/contact
```

We'll cover routing in detail later.

8. Is React a Framework?

This is another common interview question.

React is generally described as a **JavaScript library for building user interfaces**, rather than a complete framework.

The reason is that React primarily focuses on the UI layer.

For example, applications may use additional libraries for:

- Routing
- API communication
- State management
- Forms

- Authentication

The React ecosystem provides tools and libraries for these requirements.

So remember:

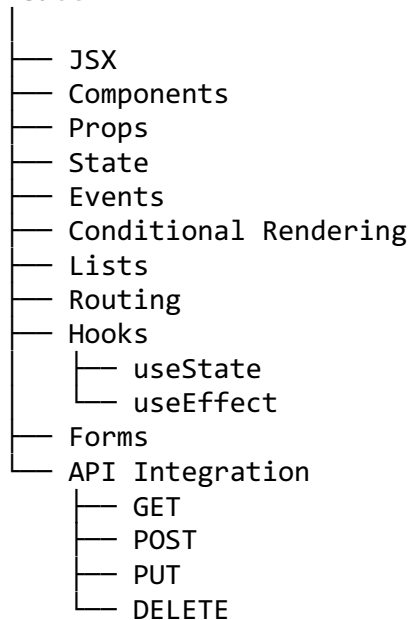
React itself focuses primarily on building the user interface.

9. Important React Concepts

Before we start coding, let's quickly look at the major concepts we'll learn throughout this course.

We'll learn:

React



By the end of the course, we'll build a complete CRUD application that communicates with an API.

10. Let's Create Our First React Application

Now let's move from theory to coding.

We'll create our first React application.

For modern React development, we'll use **Vite** to create the project.

First, make sure Node.js is installed on your computer.

Open the terminal and run:

```
node -v
```

If Node.js is installed, you'll see a version number.

Now create a React project.

Run:

```
npm create vite@latest
```

The command will ask us a few questions.

For example:

Project name:
react-course

Select a framework:
React

Select a variant:
JavaScript

Then move into the project:

```
cd react-course
```

Install the dependencies:

```
npm install
```

And start the development server:

```
npm run dev
```

The terminal will give us a local URL.

Open that URL in the browser.

We now have our first React application.

11. Let's Understand the Project

Now let's look at the project structure.

You'll see something similar to:

```
react-course
├── node_modules
├── public
├── src
│   ├── assets
│   ├── App.jsx
│   ├── App.css
│   ├── main.jsx
│   └── index.css
├── index.html
├── package.json
└── vite.config.js
```

Don't worry about every file right now.

We'll gradually understand the important files throughout the course.

For this lecture, let's focus on three files:

```
index.html
main.jsx
App.jsx
```

12. Understanding App.jsx

Open App.jsx.

Depending on the version of the template, you'll see some example code.

Let's simplify it.

Replace the contents with:

```
function App() {
  return (
    <div>
      <h1>Hello React!</h1>
      <p>Welcome to my first React application.</p>
    </div>
  )
}
```

```
    </div>
  );
}
```

```
export default App;
```

Let's understand this.

We created a function:

```
function App() {
```

This function is our React component.

The component returns:

```
<div>
  <h1>Hello React!</h1>
  <p>Welcome to my first React application.</p>
</div>
```

This looks like HTML.

But technically, this is **JSX**.

We'll learn JSX in the next lecture.

Finally:

```
export default App;
```

allows another file to import and use this component.

13. Understanding main.jsx

Now open:

src/main.jsx

You'll see code that imports React-related functionality and the App component.

Conceptually, it looks like this:

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
```

```
<StrictMode>  
  <App />  
</StrictMode>  
);
```

Let's focus on the important part:

```
<App />
```

We're telling React:

Render the App component.

And:

```
document.getElementById('root')
```

refers to the root element in our HTML.

If we open index.html, we'll find something like:

```
<div id="root"></div>
```

So the basic flow is:

```
index.html  
  ↓  
<div id="root">  
  ↓  
main.jsx  
  ↓  
<App />  
  ↓  
App.jsx  
  ↓  
UI
```

This is the basic architecture of our React application.

14. Modify Our First Application

Let's make the example slightly more interesting.

Change App.jsx to:

```
function App() {  
  return (  
    <div>
```



```
<h1>ReactJS Course</h1>

<h2>Welcome Students!</h2>

<p>
  In this course, we will learn React
  from the basics to API integration.
</p>

  <button>Start Learning</button>
</div>
);
}
```

```
export default App;
```

Save the file.

Look at the browser.

The page updates automatically.

This is one of the nice features of the development experience.

We change our source code, and the browser reflects the change without us manually refreshing the page in the usual workflow.

15. What Have We Learned?

Let's recap today's lecture.

First:

React is a JavaScript library for building user interfaces.

Second:

React uses a **component-based architecture**.

Third:

React encourages a **declarative approach** to building UI.

Fourth:

React is commonly used for **single-page applications**.

And finally, we created our first React application using Vite.

We also saw three important files:

```
index.html  
main.jsx  
App.jsx
```

And the basic flow:

```
index.html  
↓  
main.jsx  
↓  
App.jsx  
↓  
React UI
```

16. Student Exercise

Before moving to the next lecture, I recommend doing a small exercise.

Modify the App component and create your own student profile.

Display:

```
Your Name  
Your Course  
Your College  
Your Skills
```

For example:

```
Arjun
```

```
ReactJS Course
```

```
Skills:  
HTML  
CSS  
JavaScript  
React
```

Don't worry about making it beautiful yet.

We'll learn styling in a later lecture.

17. Closing

So that's our introduction to ReactJS.

In today's lecture, we focused on the big picture: what React is, why it is useful, how components fit into React, and how a basic React application starts.

In the next lecture, we'll start learning one of the most important concepts in React:

JSX.

We'll understand why React code looks like HTML, how JSX works, and the important differences between JSX and normal HTML.

Lecture 2 — JSX in ReactJS

1. Opening / Hook

Hello everyone, and welcome back.

In the previous lecture, we learned what React is, why React is useful, and we created our first React application.

If you remember our `App.jsx` file, we wrote something like this:

```
function App() {  
  return (  
    <div>  
      <h1>ReactJS Course</h1>  
      <p>Welcome to React!</p>  
    </div>  
  );  
}
```

Now, if you are coming from HTML, you might look at this code and think:

“This looks like HTML.”

And you’re right — it looks very similar to HTML.

But technically, this is **not HTML**.

This is called **JSX**.

So in today’s lecture, we’re going to understand:

- What JSX is
- Why React uses JSX
- How JSX works
- The differences between HTML and JSX
- JSX syntax rules
- How to write JavaScript expressions inside JSX
- And we’ll build a small Student Profile application.

Let’s get started.

2. What is JSX?

Let's start with the definition.

JSX stands for JavaScript XML.

JSX is a syntax extension for JavaScript that allows us to write markup-like code inside JavaScript.

For example:

```
const element = <h1>Hello React</h1>;
```

This looks like HTML.

But we're actually writing it inside JavaScript.

Another example:

```
const element = (  
  <div>  
    <h1>ReactJS</h1>  
    <p>Learn React step by step.</p>  
  </div>  
)
```

This is JSX.

The important thing to understand is:

JSX allows us to describe the UI using a syntax that looks very similar to HTML.

3. Why Do We Need JSX?

Now let's ask an important question.

Why doesn't React simply use normal HTML?

To understand that, let's look at how React applications are written.

React components are JavaScript functions.

For example:

```
function App() {  
  return <h1>Hello React</h1>;  
}
```

Our component is JavaScript.

But the component needs to return some UI.

JSX gives us a convenient way to describe that UI.

Without JSX, the same thing would be much more verbose.

Conceptually, we'd have to create React elements using JavaScript function calls.

For example:

```
React.createElement(  
  "h1",  
  null,  
  "Hello React"  
);
```

Compare that with JSX:

```
<h1>Hello React</h1>
```

The JSX version is much easier for developers to read and write.

So JSX gives us a syntax that allows us to combine:

JavaScript
+
UI markup

in a convenient way.

4. JSX Is Not HTML

This is one of the most important things to remember.

JSX looks like HTML, but JSX is not HTML.

For example, in HTML we normally write:

```
<div class="container">  
  <label for="name">Name</label>  
</div>
```

In JSX, we write:

```
<div className="container">
  <label htmlFor="name">Name</label>
</div>
```

There are differences.

One of the biggest differences is:

HTML	→ class
JSX	→ className

And:

HTML	→ for
JSX	→ htmlFor

Why?

Because JSX is ultimately part of JavaScript syntax, and some HTML attribute names conflict with JavaScript terminology or JSX conventions.

We'll see several examples today.

5. Our First JSX Example

Let's open our project from the previous lecture.

Open:

src/App.jsx

Let's replace the existing code with:

```
function App() {
  return (
    <div>
      <h1>Student Profile</h1>
      <p>Name: Arjun</p>
      <p>Course: ReactJS</p>
    </div>
  );
}
```

```
export default App;
```

Save the file and look at the browser.

We have:

Student Profile

Name: Arjun

Course: ReactJS

This is JSX.

The HTML-like structure we're writing inside the JavaScript function is JSX.

6. JSX Must Have One Parent Element

Now let's look at an important JSX rule.

Suppose we write:

```
function App() {  
  return (  
    <h1>Student Profile</h1>  
    <p>Name: Arjun</p>  
  );  
}
```

This will produce an error.

Why?

Because JSX expects the returned elements to have a single parent.

So we can wrap them inside a <div>:

```
function App() {  
  return (  
    <div>  
      <h1>Student Profile</h1>  
      <p>Name: Arjun</p>  
    </div>  
  );  
}
```

Now it works.

Think of it like this:

Incorrect:

h1
p

Correct:

```
div
├── h1
└── p
```

The outer element gives our JSX a single root.

7. Using React Fragment

But what if we don't want to add an unnecessary `<div>`?

React provides a Fragment.

We can write:

```
function App() {
  return (
    <>
      <h1>Student Profile</h1>
      <p>Name: Arjun</p>
    </>
  );
}
```

The empty tags:

```
<>
</>
```

are called a **Fragment**.

A Fragment allows us to group multiple JSX elements without adding an extra element to the HTML DOM.

We'll use this frequently throughout the course.

For now, remember:

```
<div>
  ...
</div>
```

creates an actual HTML element.

Whereas:

```
<>  
...  
</>
```

groups elements without adding a wrapper element.

8. JSX Tags Must Be Properly Closed

Another important rule is that JSX elements must be properly closed.

For example:

```

```

In normal HTML, browsers can handle this.

But in JSX, we write:

```

```

The same applies to other elements.

For example:

```
<input />
```

Instead of:

```
<input>
```

And:

```

```

So remember:

Self-closing elements in JSX must end with />.

9. JSX Uses className

Now let's look at one of the most common differences between HTML and JSX.

Suppose we have this HTML:

```
<div class="student-card">
  <h1>Student Profile</h1>
</div>
```

If we copy this directly into JSX:

```
<div class="student-card">
```

we'll have a problem.

In JSX, we use:

```
<div className="student-card">
```

So let's try it.

Update App.jsx:

```
function App() {
  return (
    <div className="student-card">
      <h1>Student Profile</h1>
      <p>Name: Arjun</p>
      <p>Course: ReactJS</p>
    </div>
  );
}
```

```
export default App;
```

Now let's create some CSS.

Open:

src/App.css

and add:

```
.student-card {
  width: 300px;
  padding: 20px;
  border: 1px solid black;
  border-radius: 10px;
}
```

Make sure the CSS file is imported if your current project doesn't already import it:

```
import "./App.css";
```

Now our student profile has some styling.

We'll cover styling in much more detail in Lecture 4.

10. HTML Attributes Become JSX Props

Let's look at another concept.

In HTML, we have attributes:

```

```

In JSX, we also have attributes:

```

```

Many attributes are exactly the same.

For example:

```
id  
title  
src  
alt  
href
```

But some are different.

For example:

HTML	JSX
class	className
for	htmlFor
tabindex	tabIndex
onclick	onClick
onchange	onChange

We'll learn event handling later, but notice the difference:

HTML:

```
<button onclick="handleClick()">
```

React JSX:

```
<button onClick={handleClick}>
```

The naming and syntax are different.

11. JSX Uses camelCase

Another important rule is that many JSX attributes use camelCase.

For example:

HTML:

`tabindex="1"`

JSX:

`tabIndex="1"`

HTML:

`maxLength="10"`

JSX:

`maxLength="10"`

And events are also written using camelCase.

HTML:

`onclick`

JSX:

`onClick`

HTML:

`onChange`

JSX:

`onChange`

You'll see this pattern repeatedly in React.

12. JavaScript Inside JSX

Now we're getting to one of the most useful features of JSX.

We can use JavaScript expressions inside JSX.

We do this using curly braces:

```
{}
```

For example:

```
const name = "Arjun";

function App() {
  return (
    <h1>Hello {name}</h1>
  );
}
```

The {name} tells JSX:

“Evaluate this JavaScript expression and place the result here.”

The browser displays:

Hello Arjun

This is extremely important.

We’ll spend an entire lecture on expressions in JSX next.

For now, let’s see a few examples.

13. JavaScript Expressions in JSX

We can perform calculations:

```
const age = 20;

<p>Next year: {age + 1}</p>
```

We can call functions:

```
<p>{getName()}</p>
```

We can access object properties:

```
const student = {
  name: "Arjun",
  course: "ReactJS"
};
```

```
<p>{student.name}</p>
```

We can use expressions:

```
<p>{10 + 20}</p>
```

The result will be:

30

We'll explore expressions properly in the next lecture.

14. JSX Can Span Multiple Lines

For small JSX expressions, we can write:

```
const element = <h1>Hello React</h1>;
```

But as our UI becomes larger, we usually format it across multiple lines:

```
const element = (  
  <div>  
    <h1>ReactJS Course</h1>  
    <p>Learn React from basics.</p>  
    <button>Start Learning</button>  
  </div>  
)
```

Notice the parentheses.

The parentheses aren't JSX itself.

They simply allow us to format the JavaScript expression across multiple lines.

15. Comments in JSX

We also need to know how to write comments.

Normal JavaScript:

```
// This is a comment
```

Inside JSX, we use:

```
{/* This is a JSX comment */}
```

For example:

```
function App() {  
  return (  
    <div>  
      {/* Student information */}  
      <h1>Student Profile</h1>  
  
      <p>Name: Arjun</p>  
    </div>  
  );  
}
```

Notice the syntax:

```
{/* comment */}
```

This is a common beginner mistake, so make sure you remember it.

16. Let's Build a Student Profile

Now let's combine what we've learned.

We'll create a small Student Profile.

Our final UI will contain:

Student Profile

Name: Arjun

Course: ReactJS

Experience: Beginner

[View Profile]

Let's write the component.

```
import "../App.css";  
  
function App() {  
  return (  
    <div className="student-card">  
  
      <h1>Student Profile</h1>
```



```
    <p>Name: Arjun</p>

    <p>Course: ReactJS</p>

    <p>Experience: Beginner</p>

    <button>View Profile</button>

  </div>
);
}
```

```
export default App;
```

Now let's add the CSS:

```
.student-card {
  width: 350px;
  padding: 25px;
  border: 1px solid #ccc;
  border-radius: 10px;
}

.student-card h1 {
  margin-bottom: 20px;
}

.student-card button {
  padding: 10px 20px;
  cursor: pointer;
}
```

Save the files.

We now have our first small JSX-based UI.

17. Let's Intentionally Make Some Mistakes

As a beginner, understanding errors is just as important as understanding correct code.

Mistake 1 — Multiple root elements

Don't do:

```
return (  
  <h1>Hello</h1>  
  <p>Welcome</p>  
);
```

Instead:

```
return (  
  <>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </>  
);
```

Mistake 2 — Using class

Don't write:

```
<div class="student-card">
```

Use:

```
<div className="student-card">
```

Mistake 3 — Unclosed tags

Don't write:

```

```

Use:

```

```

Mistake 4 — HTML event naming

Don't write:

```
<button onclick={handleClick}>
```

Use:

```
<button onClick={handleClick}>
```

We'll learn events in detail later.

18. Important JSX Mental Model

Before we finish, I want you to remember one thing.

Don't think of JSX as:

“HTML inside JavaScript.”

A better way to think about it is:

JSX is a syntax that allows us to describe our UI inside JavaScript.

For example:

```
function App() {  
  return (  
    <div>  
      <h1>Hello React</h1>  
    </div>  
  );  
}
```

The component is JavaScript.

The return value describes the UI.

React uses that description to create and update the interface.

This mental model will become much more important when we start using:

- Components
- Props
- State
- Conditional rendering
- Lists
- Hooks

19. Quick Recap

Let's quickly summarize today's lecture.

What is JSX?

JSX is a syntax extension for JavaScript that allows us to write markup-like UI code.

Is JSX HTML?

No.

JSX looks like HTML, but it follows JavaScript/React rules.

Important JSX rules

Remember these:

1. JSX needs a single parent
2. Use Fragment when appropriate
3. Close JSX tags
4. Use className instead of class
5. Use camelCase for many attributes/events
6. JavaScript expressions go inside {}
7. JSX comments use {/* */}

For example:

```
function App() {  
  const name = "Arjun";  
  
  return (  
    <>  
      <h1 className="title">Hello {name}</h1>  
      <p>Welcome to ReactJS.</p>  
    </>  
  );  
}
```

20. Student Exercise

Before the next lecture, create your own **Student Profile** using JSX.

Your profile should contain:

Name
Email
Phone
Course
College
Skills

Use:

- A parent `<div>`
- Headings
- Paragraphs
- An image
- A button
- A CSS class using `className`

Also try using a JavaScript variable:

```
const name = "Your Name";
```

and display it using:

```
{name}
```

Don't worry if you don't completely understand the `{name}` syntax yet.

That's exactly what we'll learn in the next lecture.

21. Closing

So, in today's lecture, we learned one of the most fundamental concepts in React: **JSX**.

We learned what JSX is, why React uses it, how JSX differs from HTML, and several important JSX rules.

We also created a Student Profile using JSX.

Lecture 3 — Expressions in JSX

1. Opening / Hook

Hello everyone, and welcome back.

In the previous lecture, we learned about JSX.

We saw that JSX looks very similar to HTML, but it actually allows us to write UI markup inside JavaScript.

We also saw something very interesting.

We can write JavaScript inside JSX using curly braces.

For example:

```
const name = "Arjun";
```

```
<h1>Hello {name}</h1>
```

And React displays:

Hello Arjun

But this raises several questions.

What exactly can we put inside {}?

Can we perform calculations?

Can we call functions?

Can we access object properties?

Can we use conditions?

Can we put an entire JavaScript statement inside {}?

These are the questions we're going to answer in today's lecture.

By the end of this video, you'll understand how to use JavaScript expressions inside JSX and how to dynamically display data in a React application.

Let's get started.

2. What Is an Expression?

Before we talk about JSX, let's quickly understand what an **expression** is.

Since you already know JavaScript, you have already used expressions many times.

For example:

`10 + 20`

is an expression.

It produces:

`30`

Another example:

`name`

is an expression.

Another:

`student.name`

is an expression.

Another:

`age > 18`

is also an expression.

The important characteristic is:

An expression produces a value.

For example:

`10 + 20`

produces:

`30`

And:

`"Hello " + name`

produces a string.

This becomes very useful in JSX.

3. Expressions Inside JSX

In JSX, we use curly braces:

`{}`

to insert JavaScript expressions.

For example:

```
function App() {  
  const name = "Arjun";  
  return (  
    <h1>Hello {name}</h1>  
  );  
}
```

Here:

`{name}`

is a JavaScript expression inside JSX.

React evaluates the expression and displays its result.

So:

`name = "Arjun"`

becomes:

Hello Arjun

Think of the curly braces as a bridge between:

JavaScript

↓

`{}`

↓

JSX

4. Displaying Variables

Let's start with the simplest example.

Open:

src/App.jsx

and write:

```
function App() {  
  
  const name = "Arjun";  
  const course = "ReactJS";  
  const college = "ABC College";  
  
  return (  
    <div>  
      <h1>Student Profile</h1>  
  
      <p>Name: {name}</p>  
      <p>Course: {course}</p>  
      <p>College: {college}</p>  
    </div>  
  );  
}  
  
export default App;
```

Look at the browser.

We get:

Student Profile

Name: Arjun

Course: ReactJS

College: ABC College

This is much more powerful than hard-coding everything directly into our JSX.

Instead of:

```
<p>Name: Arjun</p>
```

we can write:

```
<p>Name: {name}</p>
```

Now the UI is connected to our JavaScript data.

5. Using Numbers

We can also display numbers.

For example:

```
function App() {  
  
  const age = 20;  
  const marks = 85;  
  
  return (  
    <div>  
      <p>Age: {age}</p>  
      <p>Marks: {marks}</p>  
    </div>  
  );  
}
```

The browser displays:

Age: 20
Marks: 85

There is no need to convert the number into a string manually.

React can render the value.

6. Performing Calculations

Now let's do something more interesting.

Because we're writing JavaScript expressions inside {}, we can perform calculations.

For example:

```
const age = 20;  
  
<p>Current Age: {age}</p>  
<p>Next Year: {age + 1}</p>
```

The browser displays:

Current Age: 20

Next Year: 21

We can also do:

```
<p>Total: {100 + 200}</p>
```

The result is:

Total: 300

Or:

```
const price = 1000;  
const quantity = 3;
```

```
<p>Total Price: {price * quantity}</p>
```

The result is:

Total Price: 3000

This is an important idea:

Anything that is a valid JavaScript expression can generally be used inside JSX braces, subject to what React can render.

7. String Concatenation

We can also combine strings.

For example:

```
const firstName = "Arjun";  
const lastName = "Patel";
```

```
<h1>{firstName + " " + lastName}</h1>
```

The browser displays:

Arjun Patel

We can also use template literals.

```
const firstName = "Arjun";  
const lastName = "Patel";
```

```
<h1>`${firstName} ${lastName}`</h1>
```

Both approaches work.

Since you already know JavaScript, use whichever approach makes your code easiest to understand.

8. Accessing Object Properties

In real applications, data usually doesn't come as individual variables.

Instead, we often have objects.

For example:

```
const student = {  
  name: "Arjun",  
  age: 20,  
  course: "ReactJS",  
  college: "ABC College"  
};
```

We can access the properties inside JSX:

```
function App() {  
  
  const student = {  
    name: "Arjun",  
    age: 20,  
    course: "ReactJS"  
  };  
  
  return (  
    <div>  
      <h1>Student Profile</h1>  
  
      <p>Name: {student.name}</p>  
      <p>Age: {student.age}</p>  
      <p>Course: {student.course}</p>  
    </div>  
  );  
}
```

```
export default App;
```

This is something you'll use constantly in React applications.

Later, when we consume an API, we'll receive objects like:

```
{  
  "id": 1,  
  "name": "iPhone",  
  "price": 79999  
}
```

And we'll display them using exactly this technique:

```
<h2>{product.name}</h2>  
<p>{product.price}</p>
```

So this concept is very important for the API section of our course.

9. Calling Functions Inside JSX

We can also call JavaScript functions.

For example:

```
function getMessage() {  
  return "Welcome to ReactJS";  
}
```

We can call it inside JSX:

```
<h1>{getMessage()}</h1>
```

Complete example:

```
function App() {  
  
  function getMessage() {  
    return "Welcome to ReactJS";  
  }  
  
  return (  
    <div>  
      <h1>{getMessage()}</h1>  
    </div>  
  );  
}
```

```
export default App;
```

The browser displays:

Welcome to ReactJS

The important thing is that we're calling the function.

Notice:

```
{getMessage()}
```

not:

```
{getMessage}
```

The first one executes the function.

The second one refers to the function itself.

We'll see function references again when we learn event handling.

10. Boolean Expressions

We can also evaluate boolean expressions.

For example:

```
const age = 20;
```

```
<p>{age >= 18}</p>
```

The expression:

```
age >= 18
```

evaluates to:

true

However, there's an important React behavior here.

React generally doesn't display true or false directly as visible text.

So this:

```
<p>{true}</p>
```

doesn't display the word true in the normal way you'd expect.

This becomes especially useful when we start learning **conditional rendering**.

For example:

```
{isLoggedIn && <h2>Welcome!</h2>}
```

We'll cover conditional rendering in a later lecture.

For now, just remember that boolean expressions can be used inside JSX, but React handles boolean values differently from strings and numbers.

11. Ternary Operator in JSX

Because the ternary operator is a JavaScript expression, we can use it inside JSX.

For example:

```
const age = 20;
```

```
<p>  
  {age >= 18 ? "Adult" : "Minor"}  
</p>
```

If age is 20:

Adult

If we change:

```
const age = 15;
```

we get:

Minor

This is one of the most common patterns you'll see in React.

For example:

```
{isLoggedIn ? "Logout" : "Login"}
```

We'll study this properly in our Conditional Rendering lecture.

12. What Cannot Be Used Directly?

Now let's look at an important limitation.

Not everything we write in JavaScript can be directly rendered by React.

For example, consider an object:

```
const student = {  
  name: "Arjun",  
  age: 20  
};
```

This will not work as expected:

```
<p>{student}</p>
```

Why?

Because student is an object.

React doesn't render a plain JavaScript object directly as a normal piece of text.

Instead, access a property:

```
<p>{student.name}</p>
```

or:

```
<p>{student.age}</p>
```

Later, we'll learn how to render arrays of objects using `map()`.

13. Expressions vs Statements

This is an important JavaScript concept for JSX.

Inside `{}`, we use **expressions**, not arbitrary JavaScript statements.

For example, this is valid:

```
<p>{age + 1}</p>
```

This is an expression.

This is also valid:

```
<p>{student.name}</p>
```

This is an expression.

And:

```
<p>{isLoggedIn ? "Welcome" : "Please Login"}</p>
```

is an expression.

But you cannot simply put an `if` statement directly inside JSX like this:

```
<p>
  {
    if (age >= 18) {
      return "Adult";
    }
  }
</p>
```

This is invalid.

Why?

Because `if` is a statement, not an expression.

Instead, we can calculate the value before the JSX:

```
let message;

if (age >= 18) {
  message = "Adult";
} else {
  message = "Minor";
}
```

Then:

```
<p>{message}</p>
```

Or use a ternary:

```
<p>
  {age >= 18 ? "Adult" : "Minor"}
</p>
```

We'll explore better approaches when we reach conditional rendering.

14. Practical Example — Student Profile

Let's now create a slightly more realistic example.

Our student object will contain:

```
const student = {
  name: "Arjun",
  age: 20,
  course: "ReactJS",
}
```

```
marks: 85  
};
```

Now let's display it.

```
function App() {  
  
  const student = {  
    name: "Arjun",  
    age: 20,  
    course: "ReactJS",  
    marks: 85  
  };  
  
  return (  
    <div>  
      <h1>Student Profile</h1>  
  
      <p>Name: {student.name}</p>  
  
      <p>Age: {student.age}</p>  
  
      <p>Course: {student.course}</p>  
  
      <p>Marks: {student.marks}</p>  
  
      <p>  
        Next Year Age: {student.age + 1}  
      </p>  
    </div>  
  );  
}
```

```
export default App;
```

Notice what we're doing.

The data lives in JavaScript:

```
const student = {  
  ...  
};
```

And JSX displays that data:

```
{student.name}
```

This separation between **data and UI** is a fundamental React idea.

15. Practical Example — Calculate Result

Let's make the example slightly more useful.

Suppose we have:

```
const marks = 85;
```

We can calculate whether the student passed.

```
<p>  
  Result: {marks >= 40 ? "Pass" : "Fail"}  
</p>
```

If marks are 85:

Result: Pass

Change the marks:

```
const marks = 30;
```

Now:

Result: Fail

This is a good example of using a JavaScript expression to dynamically control our UI.

16. Practical Example — Product

Now let's connect this concept to something we'll use later in the course.

Suppose we have a product:

```
const product = {  
  name: "iPhone",  
  price: 79999,  
  quantity: 2  
};
```

We can display:

```
function App() {  
  
  const product = {
```

```
    name: "iPhone",
    price: 79999,
    quantity: 2
  };

  return (
    <div>
      <h1>{product.name}</h1>

      <p>Price: ₹{product.price}</p>

      <p>Quantity: {product.quantity}</p>

      <p>
        Total: ₹{product.price * product.quantity}
      </p>
    </div>
  );
}

export default App;
```

The output will be:

iPhone

Price: ₹79999
Quantity: 2
Total: ₹159998

This is exactly the kind of code we'll eventually write with API data.

17. Arrays — A Quick Introduction

We can also have arrays in JavaScript.

For example:

```
const skills = [
  "HTML",
  "CSS",
  "JavaScript",
  "React"
];
```

Can we simply write:

```
<p>{skills}</p>
```

React can render an array of renderable values, but this isn't how we normally want to build a list UI.

Instead, we'll usually transform the array into JSX using `map()`:

```
{skills.map(skill => (  
  <p>{skill}</p>  
))}
```

This is a very important React pattern.

But we're not going to explore it fully today.

We'll have a dedicated lecture on **Rendering Arrays** later in the course.

For now, understand that JavaScript expressions allow us to connect our JavaScript data to JSX.

18. Live Coding — Build a Product Card

Let's finish today's lecture with a practical example.

We'll create a Product Card.

Start with:

```
function App() {  
  
  const product = {  
    name: "iPhone 17",  
    price: 79999,  
    brand: "Apple",  
    rating: 4.5,  
    inStock: true  
  };  
  
  return (  
    <div className="product-card">  
  
      <h1>{product.name}</h1>  
  
      <p>Brand: {product.brand}</p>  
  
      <p>Price: ₹{product.price}</p>  
  
    </div>  
  );  
}
```

```
    <p>Rating: {product.rating}</p>

    <p>
      {product.inStock
        ? "Available"
        : "Out of Stock"}
    </p>

  </div>
);
}
```

export default App;

Now let's add some basic CSS:

```
.product-card {
  width: 300px;
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 10px;
}

.product-card h1 {
  margin-bottom: 15px;
}

.product-card p {
  margin: 8px 0;
}
```

Now look at the browser.

We have dynamically generated our Product Card from a JavaScript object.

The important part isn't the CSS.

The important part is this:

```
<h1>{product.name}</h1>
<p>{product.price}</p>
<p>{product.rating}</p>
```

Our JSX is using data from JavaScript.

This is the foundation for dynamic React applications.

19. A Very Important Rule

I want you to remember this rule:

Inside JSX curly braces, think “JavaScript expression”.

For example:

`{name}`

Variable.

`{student.name}`

Object property.

`{price * quantity}`

Calculation.

`{getName()}`

Function call.

`{age >= 18 ? "Adult" : "Minor"}`

Ternary expression.

All of these produce a value that React can use when rendering the UI.

20. Common Beginner Mistakes

Let’s quickly look at some common mistakes.

Mistake 1 — Forgetting the curly braces

Incorrect:

`<h1>name</h1>`

This displays:

name

It doesn’t use the variable.

Correct:

`<h1>{name}</h1>`

Mistake 2 — Putting an object directly into JSX

Incorrect:

`<p>{student}</p>`

Instead:

`<p>{student.name}</p>`

Mistake 3 — Putting an if statement directly inside JSX

Incorrect:

```
<p>
  {if (age > 18) ...}
</p>
```

Use an expression such as:

```
<p>
  {age > 18 ? "Adult" : "Minor"}
</p>
```

or calculate the value before the JSX.

Mistake 4 — Forgetting that {} means JavaScript

For example:

`<p>2 + 2</p>`

displays:

2 + 2

But:

`<p>{2 + 2}</p>`

displays:

4

The curly braces tell JSX:

Evaluate this as JavaScript.

21. What We Learned Today

Let's summarize.

Today we learned that JSX allows us to use JavaScript expressions inside curly braces.

We can use:

Variables

```
{name}
```

Object properties

```
{student.name}
```

Calculations

```
{price * quantity}
```

Function calls

```
{getMessage()}
```

Ternary expressions

```
{isLoggedIn ? "Welcome" : "Login"}
```

And we learned that JSX expects expressions inside `{}` rather than arbitrary JavaScript statements.

22. Student Exercise

Here's your assignment for today.

Create a **Product Card** using a JavaScript object.

Your object should contain:

```
const product = {  
  name: "Laptop",  
  brand: "Dell",  
  price: 65000,  
}
```

```
    quantity: 2,  
    rating: 4.3  
};
```

Display:

Product Name
Brand
Price
Quantity
Rating
Total Price

Calculate the total price using:

```
product.price * product.quantity
```

Also add:

Excellent

if the rating is greater than or equal to 4.

Otherwise display:

Average

Try doing the entire exercise using JSX expressions.

23. Closing

So that's it for today's lecture.

Today we learned how JavaScript expressions can make our JSX dynamic.

This is one of the most important concepts in React because React applications are fundamentally about taking application data and turning that data into UI.

Lecture 4 — Styling in ReactJS

1. Opening / Hook

Hello everyone, and welcome back.

In the previous lecture, we learned how to use JavaScript expressions inside JSX.

We created a Product Card that looked something like this:

```
function App() {  
  
  const product = {  
    name: "iPhone 17",  
    price: 79999,  
    brand: "Apple",  
    rating: 4.5  
  };  
  
  return (  
    <div className="product-card">  
      <h1>{product.name}</h1>  
      <p>Brand: {product.brand}</p>  
      <p>Price: ₹{product.price}</p>  
      <p>Rating: {product.rating}</p>  
    </div>  
  );  
}
```

It works, but our application isn't very attractive yet.

So today's question is:

How do we style a React application?

If you already know CSS, the good news is that you don't have to learn a completely new styling language.

React applications still use CSS.

However, there are some differences in how we connect CSS to JSX.

Today we're going to learn:

- Using normal CSS files
- className
- Inline styles

- JavaScript style objects
- Dynamic styles
- Conditional styling
- Some common styling mistakes

And by the end, we'll build a properly styled Product Card.

Let's get started.

2. Does React Use CSS?

Let's answer the most basic question first.

Yes.

React doesn't replace CSS.

We can continue using:

CSS
SCSS
CSS Modules
Styled Components
Tailwind CSS

and many other styling solutions.

But for this course, we're going to start with standard CSS because it gives us the strongest foundation.

The important difference is that instead of writing HTML like:

```
<div class="product-card">
```

we write JSX:

```
<div className="product-card">
```

So the basic relationship is:

```
React  
↓  
JSX  
↓  
className  
↓  
CSS
```

3. `class` VS `className`

This is probably the most common styling question beginners have.

In HTML, we write:

```
<div class="product-card">
```

But in JSX, we write:

```
<div className="product-card">
```

Why?

Because JSX uses JavaScript syntax, and `class` has a special meaning in JavaScript.

Therefore, React uses:

`className`

instead.

Let's see this practically.

Open:

`src/App.jsx`

and write:

```
function App() {  
  return (  
    <div className="product-card">  
      <h1>iPhone 17</h1>  
      <p>Price: ₹79999</p>  
      <button>Buy Now</button>  
    </div>  
  );  
}
```

```
export default App;
```

Now we need to create the CSS.

4. Using a CSS File

Open:

src/App.css

and add:

```
.product-card {  
  width: 300px;  
  padding: 20px;  
  border: 1px solid #ccc;  
  border-radius: 10px;  
}  
  
.product-card h1 {  
  margin-bottom: 10px;  
}  
  
.product-card p {  
  font-size: 18px;  
}  
  
.product-card button {  
  padding: 10px 20px;  
  cursor: pointer;  
}
```

Now we need to make sure our CSS file is imported into the component.

At the top of App.jsx:

```
import "./App.css";
```

So our complete component becomes:

```
import "./App.css";  
  
function App() {  
  return (  
    <div className="product-card">  
      <h1>iPhone 17</h1>  
      <p>Price: ₹79999</p>  
      <button>Buy Now</button>  
    </div>  
  );  
}
```

```
export default App;
```

Save the file.

Now our Product Card is styled.

This is the most familiar approach for students who already know HTML and CSS.

5. Why Do We Import CSS?

Let's understand this line:

```
import "./App.css";
```

We're telling our application:

“Load the CSS contained in this file.”

The CSS rules then become available to the application.

This is one of the differences between traditional HTML and a modern JavaScript application using a bundler such as Vite.

Instead of writing:

```
<link rel="stylesheet" href="App.css">
```

inside our HTML page, we can import the CSS from JavaScript.

This is a common pattern in React projects.

6. Styling Multiple Elements

Let's make our Product Card a little better.

Update the JSX:

```
import "./App.css";
```

```
function App() {  
  return (  
    <div className="product-card">
```

```
        <h1 className="product-title">
            iPhone 17
        </h1>

        <p className="product-brand">
            Apple
        </p>

        <p className="product-price">
            ₹79999
        </p>

        <button className="buy-button">
            Buy Now
        </button>

    </div>
    );
}
```

```
export default App;
```

Now update the CSS:

```
.product-card {
    width: 300px;
    padding: 20px;
    border: 1px solid #ccc;
    border-radius: 10px;
}

.product-title {
    font-size: 24px;
}

.product-brand {
    font-size: 16px;
}

.product-price {
    font-size: 20px;
    font-weight: bold;
}

.buy-button {
    padding: 10px 20px;
```



```
    cursor: pointer;  
}
```

This should look familiar.

We're simply using CSS classes.

The main JSX difference is:

`className`

instead of:

`class`

7. Inline Styling

Now let's look at another approach.

We can also style an element directly using the `style` attribute.

For example:

```
<h1 style={{ color: "red" }}>  
  iPhone 17  
</h1>
```

Notice something unusual.

We have **two sets of curly braces**:

```
style={{ color: "red" }}
```

Why?

The outer `{}` tells JSX:

“I’m going to provide a JavaScript value.”

The inner `{}` represents the JavaScript object containing the styles.

So:

```
style={{  
  color: "red"  
}}
```

is essentially passing a JavaScript object to the `style` prop.

8. Multiple Inline Styles

We can provide multiple CSS properties:

```
<h1
  style={{
    color: "blue",
    fontSize: "24px",
    backgroundColor: "lightgray"
  }}
>
  iPhone 17
</h1>
```

Notice that CSS property names change slightly.

Traditional CSS:

```
font-size: 24px;
background-color: lightgray;
```

JavaScript style object:

```
{
  fontSize: "24px",
  backgroundColor: "lightgray"
}
```

The property names use **camelCase**.

This is the same camelCase convention we saw with JSX attributes.

9. CSS Properties in JavaScript Objects

Let's compare them side by side.

Traditional CSS:

```
.product-title {
  font-size: 24px;
  background-color: black;
  border-radius: 10px;
}
```

Inline JSX:

```
style={{
  fontSize: "24px",
  backgroundColor: "black",
  borderRadius: "10px"
}}
```

The conversion is generally:

```
font-size      → fontSize
background-color → backgroundColor
border-radius  → borderRadius
```

The hyphen is removed and the next word starts with a capital letter.

10. Using a Style Object

Instead of writing the style directly inside JSX, we can create a JavaScript object.

For example:

```
const titleStyle = {
  color: "blue",
  fontSize: "24px",
  marginBottom: "10px"
};
```

Then:

```
<h1 style={titleStyle}>
  iPhone 17
</h1>
```

Complete example:

```
function App() {

  const titleStyle = {
    color: "blue",
    fontSize: "24px",
    marginBottom: "10px"
  };

  return (
    <div>
      <h1 style={titleStyle}>
```

```
        iPhone 17
      </h1>
    </div>
  );
}
```

```
export default App;
```

This is useful when styles are generated dynamically or shared within a component.

11. Important Difference: CSS Values

When using a JavaScript style object, some values can be numbers.

For example:

```
const titleStyle = {
  fontSize: 24
};
```

React interprets certain numeric CSS properties appropriately.

But for many other properties, we use strings:

```
const boxStyle = {
  width: "300px",
  backgroundColor: "lightblue"
};
```

You can also use:

```
padding: "20px"
```

and:

```
marginTop: "10px"
```

For a beginner, using explicit units such as "20px" is often the clearest approach.

12. Dynamic Styling

Now we're getting to something React makes particularly useful.

Because styles are JavaScript objects, we can make them dynamic.

Suppose we have:

```
const price = 50000;
```

We might want to display a different color depending on the price.

For example:

```
const priceStyle = {  
  color: price > 50000 ? "red" : "green"  
};
```

Then:

```
<p style={priceStyle}>  
  Price: ₹{price}  
</p>
```

Now the style depends on our JavaScript data.

This is a very important React concept:

```
Data  
↓  
JavaScript  
↓  
UI  
↓  
Style
```

The UI and styling can both respond to application data.

13. Practical Example — Product Availability

Let's create a Product object:

```
const product = {  
  name: "iPhone 17",  
  price: 79999,  
  inStock: true  
};
```

Now let's create a dynamic style:

```
const stockStyle = {  
  color: product.inStock ? "green" : "red"  
};
```

Then:

```
<p style={stockStyle}>  
  {product.inStock ? "In Stock" : "Out of Stock"}  
</p>
```

If:

inStock: **true**

we get:

In Stock

with green styling.

If:

inStock: **false**

we get:

Out of Stock

with red styling.

This is where React starts becoming more interesting than simply displaying static HTML.

14. Conditional Classes

We don't always need inline styles.

We can also dynamically choose a CSS class.

Suppose our CSS contains:

```
.in-stock {  
  color: green;  
}  
  
.out-of-stock {  
  color: red;  
}
```

We can choose the class using a ternary expression:

```
<p className={product.inStock ? "in-stock" : "out-of-stock"}>
  {product.inStock ? "In Stock" : "Out of Stock"}
</p>
```

Notice something important:

```
className={...}
```

The value of `className` is a JavaScript expression.

So we can dynamically determine which CSS class should be applied.

This technique is very common in React applications.

15. Live Coding — Build a Dynamic Product Card

Now let's build our final example for today's lecture.

We'll create a Product Card using everything we've learned.

First, our data:

```
const product = {
  name: "iPhone 17",
  brand: "Apple",
  price: 79999,
  rating: 4.5,
  inStock: true
};
```

Now our component:

```
import "./App.css";

function App() {

  const product = {
    name: "iPhone 17",
    brand: "Apple",
    price: 79999,
    rating: 4.5,
    inStock: true
  };

  return (
    <div className="product-card">
```

```

    <h1 className="product-title">
      {product.name}
    </h1>

    <p className="product-brand">
      Brand: {product.brand}
    </p>

    <p className="product-price">
      ₹{product.price}
    </p>

    <p className="product-rating">
      Rating: {product.rating}
    </p>

    <p
      className={
        product.inStock
          ? "in-stock"
          : "out-of-stock"
      }
    >
      {product.inStock
        ? "In Stock"
        : "Out of Stock"}
    </p>

    <button className="buy-button">
      Buy Now
    </button>

  </div>
);
}

```

export default App;

Now let's create our CSS.

```

.product-card {
  width: 320px;
  padding: 25px;
  border: 1px solid #ccc;
  border-radius: 10px;
}

.product-title {

```



```
    margin-bottom: 10px;
}

.product-brand {
    font-size: 16px;
}

.product-price {
    font-size: 22px;
    font-weight: bold;
}

.product-rating {
    font-size: 16px;
}

.in-stock {
    color: green;
    font-weight: bold;
}

.out-of-stock {
    color: red;
    font-weight: bold;
}

.buy-button {
    padding: 10px 20px;
    border: none;
    cursor: pointer;
}
```

Now look at the browser.

Our Product Card is using:

- JavaScript data
- JSX expressions
- CSS classes
- Conditional classes

This is a much more realistic React UI.

16. What Happens If Product Is Out of Stock?

Let's test our dynamic styling.

Change:

inStock: **true**

to:

inStock: **false**

Now look at the browser.

The text changes from:

In Stock

to:

Out of Stock

And the CSS class changes from:

"in-stock"

to:

"out-of-stock"

This is important.

We didn't manually modify the DOM.

We changed our JavaScript data.

The JSX determines what should be displayed.

And the appropriate CSS class is selected based on that data.

This is the kind of declarative UI approach we discussed in Lecture 1.

17. Inline Style vs CSS Classes

Now an important question:

When should we use inline styles, and when should we use CSS classes?

There isn't one universal answer, but a good beginner rule is:

Use CSS classes for:

- General component styling
- Layout
- Spacing
- Typography
- Borders
- Responsive design
- Hover states
- Larger styling systems

For example:

```
<div className="product-card">
```

Use inline styles for:

- Small dynamic styles
- Styles that depend directly on JavaScript values
- One-off situations

For example:

```
<p  
  style={{  
    color: product.inStock ? "green" : "red"  
  }}  
>
```

For larger applications, you'll usually want to keep most reusable styling in CSS or another dedicated styling system.

18. Common Beginner Mistakes

Let's look at some common mistakes.

Mistake 1 — Using `class`

Incorrect:

```
<div class="product-card">
```

Correct:

```
<div className="product-card">
```

Mistake 2 — Using a CSS string for `style`

This is incorrect:

```
<div style="color: red">
```

In JSX, use a JavaScript object:

```
<div style={{ color: "red" }}>
```

Mistake 3 — Using CSS property names directly

Incorrect:

```
style={{  
  font-size: "20px"  
}}
```

Correct:

```
style={{  
  fontSize: "20px"  
}}
```

Mistake 4 — Forgetting to import CSS

If you created:

`App.css`

but didn't import it, your styles may not be applied.

Make sure you have:

```
import "./App.css";
```

when using that stylesheet from `App.jsx`.

Mistake 5 — Putting JavaScript outside {}

For example:

```
<h1>product.name</h1>
```

This displays the literal text:

product.name

Instead:

```
<h1>{product.name}</h1>
```

Now React evaluates the expression.

19. Three Styling Approaches to Remember

At this stage of the course, I want you to remember three approaches.

Approach 1 — CSS class

```
<div className="product-card">
```

with:

```
.product-card {  
  padding: 20px;  
}
```

Approach 2 — Inline style

```
<div  
  style={{  
    padding: "20px"  
  }}  
>
```

Approach 3 — JavaScript style object

```
const cardStyle = {  
  padding: "20px"  
};
```

```
<div style={cardStyle}>
```

We'll use the first approach most often for general styling.

The second and third approaches become useful when styling depends on JavaScript data.

20. A Note About Styling Libraries

Before we finish, you may have heard of libraries such as:

- Tailwind CSS
- Material UI
- Styled Components
- Bootstrap

These are all popular approaches for styling React applications.

We're intentionally not using them yet.

Why?

Because I want you to understand the fundamentals first.

If you understand:

className
CSS
style objects
dynamic classes

then learning a styling library later becomes much easier.

So for this course, we'll continue with standard CSS while learning React fundamentals.

21. Final Example

Let's look at our complete Product Card one more time.

```
import './App.css';
```

```
function App() {
```

```
const product = {
  name: "iPhone 17",
  brand: "Apple",
  price: 79999,
  rating: 4.5,
  inStock: true
};

return (
  <div className="product-card">

    <h1 className="product-title">
      {product.name}
    </h1>

    <p>
      Brand: {product.brand}
    </p>

    <p className="product-price">
      ₹{product.price}
    </p>

    <p>
      Rating: {product.rating}
    </p>

    <p
      className={
        product.inStock
          ? "in-stock"
          : "out-of-stock"
      }
    >
      {product.inStock
        ? "In Stock"
        : "Out of Stock"}
    </p>

    <button className="buy-button">
      Buy Now
    </button>

  </div>
);
}
```

```
export default App;
```

Notice how everything we've learned so far is coming together:

JavaScript data
↓
JSX expressions
↓
UI
↓
className
↓
CSS

This pattern will become the foundation for the rest of the course.

22. Recap

Let's summarize today's lecture.

First:

React applications can use normal CSS.

Second:

In JSX, use:

className

instead of:

class

Third:

We can use inline styles:

```
style={{  
  color: "red"  
}}
```

Fourth:

CSS property names inside JavaScript objects use camelCase:

backgroundColor
fontSize
borderRadius

Fifth:

Styles can be dynamic:

```
style={{  
  color: product.inStock ? "green" : "red"  
}}
```

Sixth:

We can dynamically select CSS classes:

```
className={  
  product.inStock  
    ? "in-stock"  
    : "out-of-stock"  
}
```

23. Student Exercise

Your assignment for today is to create a **Student Profile Card**.

Create an object:

```
const student = {  
  name: "Your Name",  
  course: "ReactJS",  
  college: "Your College",  
  marks: 85,  
  isPassed: true  
};
```

Create a card that displays:

Student Name
Course
College
Marks
Pass/Fail

Add CSS styling for:

- Card

- Heading
- Details
- Button

And make the Pass/Fail text dynamic.

For example:

Pass

should use one CSS class, while:

Fail

should use another.

Try changing:

isPassed: **true**

to:

isPassed: **false**

and observe how your UI changes.

24. Closing

In today's lecture, we learned how to style React applications using CSS and how React's `className` and `style` syntax differs from traditional HTML.

More importantly, we saw how styling can respond to JavaScript data.

Lecture 5 — Basic Event Management in ReactJS

1. Opening / Hook

Hello everyone, and welcome back.

So far in our React course, we've learned how to create UI using JSX, how to display JavaScript data inside JSX, and how to style our React application.

But there's one important thing missing.

Our application isn't really interactive yet.

We have buttons, but clicking them doesn't do anything.

For example, imagine we have:

Product
₹79,999

[Buy Now]

What should happen when the user clicks **Buy Now**?

Maybe we want to:

- Display a message
- Add the product to a cart
- Open a form
- Navigate to another page
- Delete a product
- Submit data

To do these things, our application needs to respond to **events**.

You already know JavaScript events such as:

click
change
submit
mouseover
keydown

React also supports events, but the syntax is slightly different.

So in today's lecture, we'll learn:

- What events are
- How React handles events
- `onClick`
- `onChange`
- `onSubmit`
- Passing functions to events
- Passing parameters
- The event object
- `preventDefault()`
- Common event-handling mistakes

And we'll finish by building an interactive Product Card.

Let's get started.

2. What Is an Event?

Let's start with something you already know.

An **event** is something that happens in the application.

For example:

User clicks a button
User types into an input
User submits a form
User moves the mouse
User presses a keyboard key

JavaScript allows us to respond to these events.

For example, in traditional JavaScript:

```
document
  .getElementById("btn")
  .addEventListener("click", function () {
    console.log("Button clicked");
  });
```

We're saying:

When this button is clicked, execute this function.

React provides a simpler way to connect events to JSX elements.

3. React Event Handling

In React, we can write:

```
jsx id="ys1y0k" <button onClick={handleClick}> Click Me </button>
```

Here:

```
jsx id="7v4hpc" onClick
```

is the React event.

And:

```
jsx id="2v1iz0" handleClick
```

is the function that React should call when the button is clicked.

Let's create our first example.

4. Our First `onClick`

Open:

```
text id="z9xg8v" src/App.jsx
```

and write:

```
`` `jsx id="bryz7m" function App() {  
  function handleClick() {  
    console.log("Button clicked");  
  }  
  
  return (  
    <div>  
      <h1>React Events</h1>  
  
      <button onClick={handleClick}>  
        Click Me  
      </button>  
    </div>  
  );  
};
```

```
}
```

```
export default App;
```

Now open the browser.

Click the button.

Nothing visible may happen on the page, but open the browser Developer Tools and go to the Console.

You should see:

```
```text  
Button clicked
```

every time you click the button.

So the flow is:

```
User clicks button
 ↓
React detects onClick
 ↓
handleClick()
 ↓
Code executes
```

---

## 5. Why Don't We Write `handleClick()`?

Now let's look at something very important.

We wrote:

```
jsx id="jqm2q4" <button onClick={handleClick}>
```

not:

```
jsx id="7fg4sq" <button onClick={handleClick()}>
```

Why?

Because React expects a **function reference**.

We are telling React:

“Here is the function. Call it when the click happens.”

If we write:

```
jsx id="h6p6hm" handleClick()
```

we are actually calling the function immediately while React is rendering the component.

That’s not what we want.

Compare:

```
jsx id="8l4qai" onClick={handleClick}
```

with:

```
jsx id="iq22ht" onClick={handleClick()}
```

The first means:

Call this function when the event happens.

The second means:

Call this function now and use its result.

This is one of the most common beginner mistakes in React.

---

## 6. Inline Event Handler

We can also write the function directly inside JSX.

For example:

```
jsx id="ng5xqk" <button onClick={() => { console.log("Button
clicked"); }} > Click Me </button>
```

This works.

However, when the logic becomes larger, it’s usually cleaner to define the function separately:

```
jsx id="0eqb4y" function handleClick() { console.log("Button clicked"); }
```

and then:

```
jsx id="tzt9ji" <button onClick={handleClick}>
```

Both approaches are valid.

## 7. Updating Text on Click

Let's make our example more interesting.

Suppose we want to display:

Button clicked!

after clicking the button.

We might initially think:

```
jsx id="yl0o4h" let message = "";
and then change it inside handleClick.
```

But there is an important problem.

Changing a normal JavaScript variable doesn't tell React to update the UI.

We'll properly solve this problem when we learn **useState**.

For today's lecture, we'll keep the example focused on event handling and use the browser console for our first demonstrations.

Later, in the useState lecture, we'll make these interactions update the screen.

This distinction is important:

**An event handler performs an action, but changing normal variables doesn't automatically cause React to re-render.**

## 8. The `onChange` Event

Now let's look at input fields.

Suppose we have:

```
jsx id="xv5f9h" <input type="text" />
```

We can listen for changes:

```
jsx id="83s8i7" <input type="text" onChange={handleChange} />
```

And define:



```
jsx id="5rql8y" function handleChange(event) { console.log("Input
changed"); }
```

Now every time the user types something, the function executes.

Let's try it.

```
`` `jsx id="3g3lyb" function App() {
 function handleChange(event) {
 console.log("Input changed");
 }

 return (
 <div>
 <h1>React Events</h1>

 <input
 type="text"
 onChange={handleChange}
 />
 </div>
);
}

export default App;
```

Open the console.

Start typing.

You'll see:

```
`` `text
Input changed
Input changed
Input changed
```

depending on how many changes occur.

---

## 9. The Event Object

Now let's look at the event parameter.

React passes an event object to our event handler.

For example:

```
jsx id="c4v0ai" function handleChange(event) { console.log(event); }
```

Let's type something into the input.

The browser console will show an event object.

We don't need to understand every property right now.

But one property is particularly useful:

```
jsx id="q1d9t9" event.target
```

The target represents the element that triggered the event.

For an input:

```
jsx id="nh8q4q" event.target.value
```

gives us the current value of the input.

Let's demonstrate.

```
jsx id="ypa1in" function handleChange(event)
{ console.log(event.target.value); }
```

Now type:

A  
Ar  
Arj  
Arju  
Arjun

The console will show the corresponding values.

This is extremely important when working with forms.

We'll use this heavily in our Forms lecture later in the course.

---

## 10. onChange vs HTML change

You may notice something interesting.

In React:

`jsx id="gk3i9r" onChange`

uses camelCase.

This is different from how we commonly write HTML attributes.

For example:

```
html id="c11mjb" <input onChange="...">
```

React uses:

```
jsx id="9kt9tj" <input onChange={handleChange} />
```

Similarly:

```
```text id="3bq9ku" HTML React JSX
```

`onClick` `onClicK` `onChange` `onSubmiT` `onmouseover` `onMouseOver`

React event names use camelCase.

11. Passing Parameters to Event Handlers

Now suppose we have multiple products.

For example:

```
```jsx id="wsqb3y"
const product = {
 id: 1,
 name: "iPhone"
};
```

And we want the click handler to know which product was clicked.

We might create:

```
jsx id="oc4kug" function handleBuy(productId) { console.log("Product
ID:", productId); }
```

But we can't simply write:

```
jsx id="xrya1j" <button onClick={handleBuy(product.id)}>
```

because that calls the function immediately.

Instead, we use an arrow function:

```
jsx id="rgr6kb" <button onClick={() => handleBuy(product.id)} > Buy
Now </button>
```

Now the flow is:

```
User clicks
 ↓
Arrow function executes
 ↓
handleBuy(product.id)
 ↓
Product ID is passed
```

Let's try it.

---

## 12. Practical Example — Product Button

Create:

```
` `` `jsx id="5qz1g5" function App() {

 const product = {
 id: 101,
 name: "iPhone 17",
 price: 79999
 };

 function handleBuy(productId) {
 console.log("Buying product:", productId);
 }

 return (
 <div>
 <h1>{product.name}</h1>

 <p>Price: ₹{product.price}</p>

 <button
 onClick={() => handleBuy(product.id)}
 >
 Buy Now
 </button>
 </div>
);
}
```

```
}
```

```
export default App;
```

Click **\*\*Buy Now\*\***.

The console will show:

```
```text  
Buying product: 101
```

This pattern will become very important later when we build:

```
Delete Product  
Edit Product  
View Product  
Add to Cart
```

13. Multiple Buttons

Let's make this example slightly larger.

Suppose we have:

```
jsx id="5ag7n9" function handleAction(action) {      console.log("Action:",  
action); }
```

We can have:

```
```jsx id="b3iw7u" <button onClick={() => handleAction("buy")} > Buy  
<button onClick={() => handleAction("wishlist")} > Add to Wishlist
<button onClick={() => handleAction("share")} > Share
```

Now each button sends a different value.

The console will show:

```
```text  
Action: buy
```

or:

Action: wishlist

or:

Action: share

This is useful when the same function handles multiple related actions.

14. The `onSubmit` Event

Now let's look at forms.

Suppose we have:

```
jsx id="cm9f6m" <form>      <input type="text" />      <button type="submit">
Submit      </button> </form>
```

We can handle form submission using:

```
jsx id="a5i6hf" <form onSubmit={handleSubmit}>
```

And:

```
jsx id="q8x8zn" function handleSubmit(event) {      console.log("Form
submitted"); }
```

Let's create a complete example.

```
`` `jsx id="2t0txk" function App() {
function handleSubmit(event) {
  console.log("Form submitted");
}

return (
  <div>
    <h1>Registration</h1>

    <form onSubmit={handleSubmit}>

      <input
        type="text"
        placeholder="Enter name"
      />
```

```
        <button type="submit">
          Register
        </button>

      </form>
    </div>
  );
}

export default App;
```

Click Register.

You'll see:

```
```text
Form submitted
```

But you may notice something else.

The browser may refresh or navigate as part of the normal HTML form submission behavior.

We usually don't want that in a React single-page application.

That's where `preventDefault()` comes in.

---

## 15. `preventDefault()`

The event object contains a method:

```
jsx id="f0tdfj" event.preventDefault()
```

This prevents the browser's default action.

So our handler becomes:

```
```jsx id="j3c8o7" function handleSubmit(event) {
  event.preventDefault();

  console.log("Form submitted");
}
```

Now the form submission is handled by our React code without the browser performing the normal page reload.

Complete example:

```
``jsx id="2f6nmt"
function App() {

  function handleSubmit(event) {

    event.preventDefault();

    console.log("Form submitted");
  }

  return (
    <form onSubmit={handleSubmit}>

      <input
        type="text"
        placeholder="Enter name"
      />

      <button type="submit">
        Register
      </button>

    </form>
  );
}

export default App;
```

This pattern will be extremely important when we build React forms later.

16. onClick on Different Elements

Event handling isn't limited to buttons.

We can attach events to many JSX elements.

For example:


```
jsx id="a2jvha" <div onClick={handleClick}> Click Here </div>
```

We can use:

```
jsx id="y9t4e1" 
```

Or:

```
jsx id="1s8x7w" <h1 onClick={handleHeadingClick}> ReactJS </h1>
```

However, buttons should generally be used for button-like actions because they're better for accessibility and keyboard interaction.

17. onMouseOver

React supports many DOM events.

For example:

```
jsx id="4y8hqa" function handleMouseOver() { console.log("Mouse over"); }
```

Then:

```
jsx id="x4u9o8" <div onMouseOver={handleMouseOver}> Move your mouse here </div>
```

When the mouse moves over the element, the handler executes.

You may also see:

onMouseEnter
onMouseLeave
onMouseMove

We won't cover every event in this course.

The important thing is to understand the general pattern:

```
jsx id="3ck4bq" eventName={function}
```

18. Live Coding — Interactive Product Card

Now let's build a small practical application.

We'll create a Product Card with three actions:

iPhone 17
₹79,999

[Buy Now]
[Add to Wishlist]

Let's start with the data:

```
jsx id="u7f1ef" const product = {      id: 101,      name: "iPhone 17",  
price: 79999 };
```

Now our handlers:

```
```jsx id="i6r8x1" function handleBuy(productId) { console.log("Buying product:",  
productId); }
```

```
function handleWishlist(productId) { console.log("Adding to wishlist:", productId); }
```

Now our JSX:

```
```jsx id="5rj9wi"  
function App() {  
  
    const product = {  
        id: 101,  
        name: "iPhone 17",  
        price: 79999  
    };  
  
    function handleBuy(productId) {  
        console.log("Buying product:", productId);  
    }  
  
    function handleWishlist(productId) {  
        console.log(  
            "Adding to wishlist:",  
            productId  
        );  
    }  
  
    return (  
        <div className="product-card">  
  
            <h1>{product.name}</h1>  
  
            <p>
```

```
        Price: ₹{product.price}
      </p>

      <button
        onClick={() => handleBuy(product.id)}
      >
        Buy Now
      </button>

      <button
        onClick={() =>
          handleWishlist(product.id)
        }
      >
        Add to Wishlist
      </button>
    </div>
  );
}
```

```
export default App;
```

Now let's test it.

Click **Buy Now**.

Console:

Buying product: 101

Click **Add to Wishlist**.

Console:

Adding to wishlist: 101

Our React application is now responding to user interaction.

19. Adding an Input

Let's add one more example.

Suppose we have:

```
jsx id="ps4l3c" <input      type="text"      placeholder="Enter your name"
onChange={handleChange} />
```

And:

```
jsx id="9f6w0u" function handleChange(event)
{   console.log(event.target.value); }
```

Now when the user types:

Arjun

we can access that value through:

```
jsx id="f2fplq" event.target.value
```

This concept is the foundation of controlled forms.

We'll use it extensively in Lecture 19.

20. A Very Important Distinction

At this point, I want to emphasize something.

An event handler is simply a function.

For example:

```
jsx id="p5b47v" function handleClick() {   console.log("Clicked"); }
```

React's job is to call that function when the specified event occurs.

So this:

```
jsx id="3z9khb" <button onClick={handleClick}>
```

means:

Button clicked



React



handleClick()

And this:

```
jsx id="5hgy7w" <input onChange={handleChange} />
```

means:

Input changes



React



handleChange(event)

Understanding this simple relationship will make event handling much easier.

21. Common Beginner Mistakes

Let's go through the mistakes you should watch for.

Mistake 1 — Calling the function immediately

Incorrect:

```
jsx id="k7p5ms" <button onClick={handleClick()}>
```

Correct:

```
jsx id="7f9w5f" <button onClick={handleClick}>
```

Mistake 2 — Forgetting camelCase

Incorrect:

```
jsx id="m0jv0r" <button onclick={handleClick}>
```

Correct:

```
jsx id="q2vujt" <button onClick={handleClick}>
```

Mistake 3 — Using a string

Don't write:

```
jsx id="w3d4j4" <button onClick="handleClick">
```

Instead:

```
jsx id="6wj9tr" <button onClick={handleClick}>
```

React expects a function, not a string containing the function name.

Mistake 4 — Passing parameters incorrectly

Incorrect:

```
jsx id="pm4x91" <button onClick={handleDelete(id)}>
```

This executes immediately.

Correct:

```
jsx id="oz8tq7" <button onClick={() => handleDelete(id)}>
```

The arrow function waits until the click occurs.

Mistake 5 — Forgetting `preventDefault()`

For forms:

```
jsx id="6ly9k8" function handleSubmit(event) {      event.preventDefault(); }
```

Without this, the browser may perform its normal form submission behavior.

22. Event Handler Naming

You'll often see names such as:

```
jsx id="l4i3bv" handleClick handleChange handleSubmit handleDelete handleSave  
handleLogin handleLogout
```

There's nothing magical about the word handle.

It's simply a naming convention.

For example:

```
jsx id="sg75g5" function handleDelete() {      ... }
```

means:

 This function handles the delete event/action.

You could technically call it:

```
jsx id="w6z52u" function deleteProduct() {      ... }
```

That's valid too.

The important thing is that your function name clearly communicates its purpose.

23. What We Can Do With Events

Now that we understand basic event handling, we can start imagining what our application can do.

For example:

`` ` `text id="6s3z0b" onClick ↓ Delete Product`

`onChange ↓ Read Input`

`onSubmit ↓ Submit Form`

`onMouseEnter ↓ Show Information`

`onClick ↓ Open Modal`

Later, when we introduce state, these events will become much more powerful.

For example:

`` ` `text id="d5s76b"`

User clicks "+"

↓

Event handler

↓

Update state

↓

React re-renders

↓

New UI

That is where React really starts becoming interactive.

24. What We Learned Today

Let's recap today's lecture.

1. React supports events

Examples:

```
onClick  
onChange  
onSubmit  
onMouseOver
```

2. Event names use camelCase

```
jsx id="9fujf0" onClick
```

not:

```
jsx id="p7b2xn" onclick
```

3. Pass a function to an event

```
jsx id="wq3l0w" <button onClick={handleClick}>
```

4. Don't call the function immediately

Avoid:

```
jsx id="zqv4by" onClick={handleClick()}
```

5. Use an arrow function to pass parameters

```
jsx id="q3avx9" onClick={() => handleDelete(id)}
```

6. React passes an event object

```
jsx id="u5ndm7" function handleChange(event)  
{ console.log(event.target.value); }
```

7. Prevent default browser behavior

```
jsx id="axw9mo" event.preventDefault();
```

25. Student Exercise

Today's assignment is to create an **Interactive Product Card**.

Create a product:

```
jsx id="bxw9fu" const product = {      id: 1,      name: "Laptop",      price:  
65000  };
```


Display:

Laptop
₹65000

[Buy Now]
[Add to Wishlist]
[Delete]

Create three event handlers:

handleBuy
handleWishlist
handleDelete

When each button is clicked, print a message in the browser console.

For example:

Buying product: 1

and:

Adding product to wishlist: 1

and:

Deleting product: 1

Bonus exercise

Add an input:

Enter your name

and use onChange to print the current value to the console.

Extra challenge

Create a form with:

Name
Email

[Submit]

Use onSubmit and event.preventDefault().

Lecture 6 — Introduction to Components in ReactJS

1. Opening / Hook

Hello everyone, and welcome back.

In the previous lectures, we learned how to create React UI using JSX, how to use JavaScript expressions, how to style our application, and how to handle events.

But there is a very important question.

Imagine we are building a real e-commerce application.

We might have:

Header
Navigation Bar
Search Box
Product List
Product Card
Shopping Cart
Footer

Now imagine putting all of this UI inside one single component.

Our file could become hundreds or even thousands of lines long.

That would be difficult to:

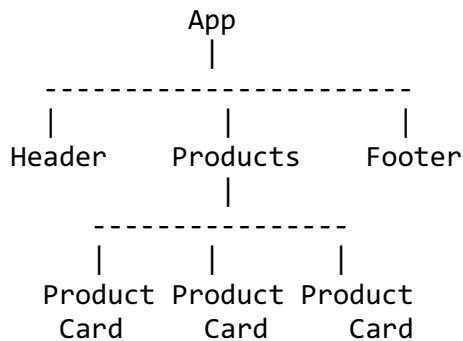
- Understand
- Maintain
- Test
- Reuse
- Modify

So React gives us one of its most important concepts:

Components

Components allow us to break a large user interface into smaller, independent and reusable pieces.

For example:



This is the idea we'll learn today.

2. What Is a Component?

Let's start with a simple definition.

A component is a reusable piece of UI.

For example, suppose we have a button.

Instead of repeatedly writing:

```
<button>Buy Now</button>
```

we can create a component:

```
<BuyButton />
```

And that component can contain the button's JSX, styling and eventually its behavior.

Similarly, we can create:

Header
Footer
Navbar
ProductCard
LoginForm
SearchBox
Sidebar

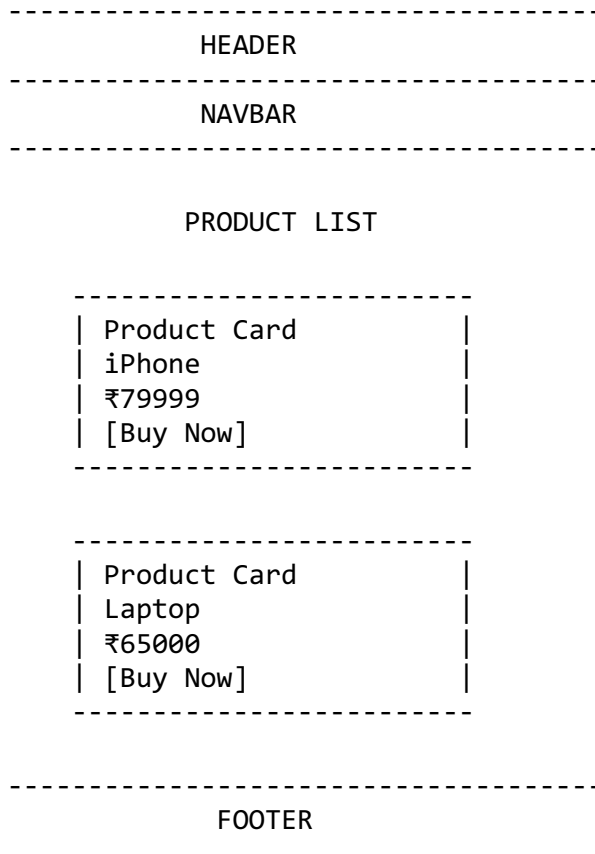
Each one can become a component.

So instead of thinking about an application as one huge page, we think about it as a collection of smaller components.

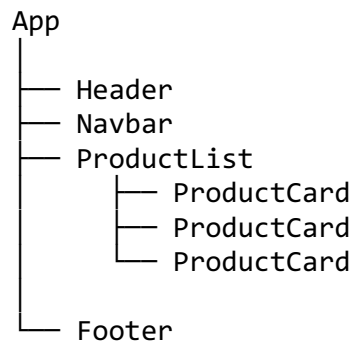
3. Real-World Example

Let's take an e-commerce website.

Suppose the page looks like this:



Instead of creating one huge component, we can divide it:



This is called **component composition**.

We combine smaller components to create a larger application.

4. Why Do We Need Components?

There are several important reasons.

Reason 1 — Reusability

Suppose we need ten product cards.

Without components, we might write the same HTML/JSX ten times.

With components:

```
<ProductCard />  
<ProductCard />  
<ProductCard />
```

We can reuse the same component.

Later, we'll learn how to provide different data to each component using **props**.

Reason 2 — Maintainability

Imagine our Product Card has 100 lines of code.

If we keep it inside `App.jsx`, our App component becomes difficult to manage.

Instead:

```
App.jsx  
ProductCard.jsx  
Header.jsx  
Footer.jsx
```

Each component has a clear responsibility.

Reason 3 — Separation of Responsibility

A component can focus on a specific part of the UI.

For example:

```
Header  
→ Header-related UI
```

ProductCard

→ Product-related UI

Footer

→ Footer-related UI

This makes our application easier to understand.

Reason 4 — Easier Development

When an application is divided into components, multiple developers can work on different parts.

One developer can work on:

Header

while another works on:

ProductCard

and another works on:

Checkout

This becomes very useful in large projects.

5. Our Current Application

Let's look at the type of code we've been writing.

For example:

```
function App() {  
  
  const product = {  
    id: 101,  
    name: "iPhone 17",  
    price: 79999  
  };  
  
  function handleBuy(productId) {  
    console.log("Buying:", productId);  
  }  
}
```

```
    return (  
      <div className="product-card">  
  
        <h1>{product.name}</h1>  
  
        <p>₹{product.price}</p>  
  
        <button  
          onClick={() => handleBuy(product.id)}  
        >  
          Buy Now  
        </button>  
  
      </div>  
    );  
  }  
}
```

export default App;

Everything is inside:

App

Now imagine adding:

Header

Navbar

Sidebar

Product List

Footer

The component would become huge.

So let's break it down.

6. Creating Our First Component

Let's create a simple component called:

ProductCard

In React, a component can be created using a JavaScript function.

For example:

```
function ProductCard() {  
  
  return (  
    <div>  
      <h2>iPhone 17</h2>  
      <p>₹79999</p>  
    </div>  
  );  
}
```

This function returns JSX.

That makes it a React component.

We can then use it inside another component:

```
function App() {  
  
  return (  
    <div>  
      <ProductCard />  
    </div>  
  );  
}
```

Notice the syntax:

```
<ProductCard />
```

It looks like an HTML element.

But it isn't a normal HTML element.

It is our React component.

7. Component Naming Convention

There is an important rule.

React component names should start with a **capital letter**.

For example:

```
function ProductCard() {  
  ...  
}
```


Correct.

But:

```
function productCard() {  
    ...  
}
```

is not the recommended component naming convention.

Why do we use capital letters?

React uses the capitalization to distinguish custom components from built-in HTML elements.

For example:

`<div>`

is a normal HTML element.

But:

`<ProductCard>`

is a React component.

So remember:

HTML element → lowercase

div
button
input
h1

React component → uppercase

ProductCard
Header
Footer
Navbar

This is very important.

8. Creating a Header Component

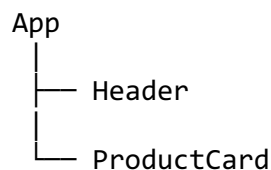
Let's create another component.

```
function Header() {  
  
  return (  
    <header>  
      <h1>My Store</h1>  
    </header>  
  );  
}
```

Now our App can use it:

```
function App() {  
  
  return (  
    <div>  
  
      <Header />  
  
      <h2>Products</h2>  
  
      <ProductCard />  
  
    </div>  
  );  
}
```

So now we have:



This is our first component hierarchy.

9. Multiple Components

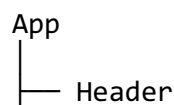
Let's create three components:

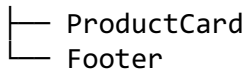
```
function Header() {  
  return (  
    <header>  
      <h1>My Store</h1>  
    </header>  
  );  
}  
  
function ProductCard() {  
  return (  
    <div>  
      <h2>iPhone 17</h2>  
      <p>₹79999</p>  
    </div>  
  );  
}  
  
function Footer() {  
  return (  
    <footer>  
      <p>© 2026 My Store</p>  
    </footer>  
  );  
}
```

Now our App component:

```
function App() {  
  return (  
    <div>  
  
      <Header />  
  
      <ProductCard />  
  
      <Footer />  
  
    </div>  
  );  
}
```

Our application now has a simple structure:





This is much easier to understand.

10. Component Composition

The process of putting components inside other components is called:

Component Composition

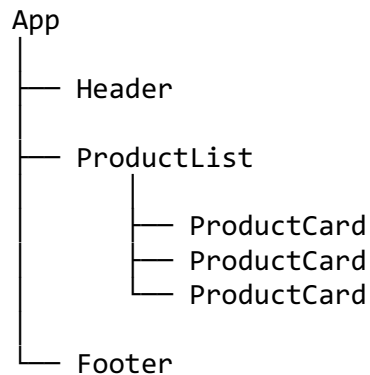
For example:

```
function App() {  
  return (  
    <div>  
  
      <Header />  
  
      <ProductList />  
  
      <Footer />  
  
    </div>  
  );  
}
```

And ProductList can contain ProductCard:

```
function ProductList() {  
  return (  
    <div>  
  
      <ProductCard />  
  
      <ProductCard />  
  
      <ProductCard />  
  
    </div>  
  );  
}
```

Now our hierarchy becomes:



This is how larger React applications are constructed.

11. Components Can Contain Components

This is an important idea.

A component doesn't have to contain only HTML elements.

It can contain other React components.

For example:

```
function ProductList() {  
  return (  
    <div>  
      <ProductCard />  
      <ProductCard />  
    </div>  
  );  
}
```

And:

```
function App() {  
  return (  
    <div>
```

```
        <ProductList />

      </div>
    );
  }
```

So:

```
App
  ↓
ProductList
  ↓
ProductCard
```

This creates a hierarchy.

12. Live Coding — Build an E-Commerce Page

Let's now build a small application.

We'll start with our components.

Header

```
function Header() {

  return (
    <header>
      <h1>My Store</h1>
      <nav>
        Home | Products | Contact
      </nav>
    </header>
  );
}
```

ProductCard

```
function ProductCard() {

  return (
    <div className="product-card">

      <h2>iPhone 17</h2>

      <p>Apple</p>
    </div>
  );
}
```

```
        <p>₹79999</p>

        <button>
            Buy Now
        </button>

    </div>
    );
}
```

Footer

```
function Footer() {

    return (
        <footer>
            <p>© 2026 My Store</p>
        </footer>
    );
}
```

Now our App:

```
function App() {

    return (
        <div>

            <Header />

            <main>

                <h2>Products</h2>

                <ProductCard />

                <ProductCard />

            </main>

            <Footer />

        </div>
    );
}
```

We now have a simple component-based application.

13. A Component Is More Than HTML

It's important not to think of a component as simply “some HTML in a function.”

A component can eventually contain:

- JSX
- +
- JavaScript
- +
- Events
- +
- State
- +
- Props
- +
- Hooks

For example:

```
function ProductCard() {  
  
  function handleBuy() {  
    console.log("Product purchased");  
  }  
  
  return (  
    <div>  
  
      <h2>iPhone 17</h2>  
  
      <button onClick={handleBuy}>  
        Buy Now  
      </button>  
  
    </div>  
  );  
}
```

Our component now contains both:

- UI
- +
- Behavior

This is one of the powerful ideas behind React.

14. Component vs HTML Element

Let's compare.

HTML:

```
<div>
  <h1>Hello</h1>
</div>
```

React:

```
<Header />
<ProductCard />
<Footer />
```

The important difference is that React components are reusable building blocks.

For example:

```
<ProductCard />
<ProductCard />
<ProductCard />
```

can represent three product cards.

Later, each one can receive different data.

For example:

```
<ProductCard name="iPhone" />
<ProductCard name="Laptop" />
<ProductCard name="Tablet" />
```

We'll learn exactly how this works when we study **Props**.

15. Components and Reusability

Let's say our store has:

iPhone
MacBook
iPad
AirPods

We don't want to create:

iPhoneCard
MacBookCard
iPadCard
AirPodsCard

Instead, we create one reusable:

```
<ProductCard />
```

Then we provide different data.

Eventually we'll be able to write something like:

```
<ProductCard  
  name="iPhone 17"  
  price={79999}  
>
```

```
<ProductCard  
  name="MacBook Air"  
  price={99999}  
>
```

```
<ProductCard  
  name="iPad"  
  price={49999}  
>
```

One component.

Multiple products.

This is one of the major benefits of component-based development.

We'll learn how to do this properly in the **Props** lecture.

16. Components Should Have a Responsibility

Here's another important principle.

A component should ideally have a clear responsibility.

For example:

Header

→ Displays application header

ProductCard

→ Displays one product

ProductList

→ Displays a list of products

Footer

→ Displays footer

Avoid creating a component that does everything.

For example, this would eventually become difficult:

App

→ Header

→ Login

→ Product API

→ Product Card

→ Cart

→ Checkout

→ Footer

→ 1000 lines of code

Instead, break it down.

App

├── Header

├── Login

├── ProductList

│ └── ProductCard

├── Cart

├── Checkout

└── Footer

This makes the code easier to manage.

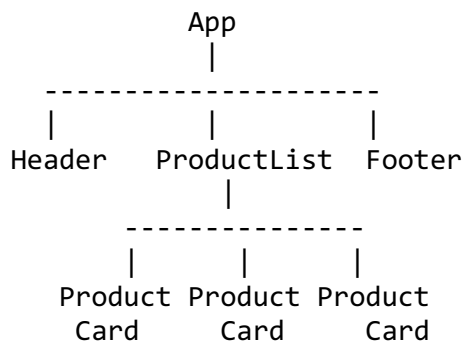
17. Component Hierarchy

Let's introduce an important term:

Component Tree

A React application can be represented as a tree.

For example:



This is our component tree.

Later, data will move through this tree using:

Props

and application data will be managed using:

State

and:

Hooks

So understanding the component tree now will make the later lectures much easier.

18. Where Should We Create Components?

In a small application, you might initially have:

```
function Header() {
  ...
}

function ProductCard() {
  ...
}

function Footer() {
  ...
}

function App() {
```

```
...  
}
```

all inside one file.

That's okay for learning.

But as the application becomes larger, we'll move components into separate files.

For example:

```
src  
├── App.jsx  
├── Header.jsx  
├── ProductCard.jsx  
└── Footer.jsx
```

Then each file can contain a component.

We'll cover this more practically in the next lecture, where we'll focus on **Function Components** and organizing components into files.

19. One Important Rule — Return One Parent

A component must return valid JSX.

For example, this is valid:

```
function ProductCard() {  
  return (  
    <div>  
      <h2>iPhone</h2>  
      <p>₹79999</p>  
    </div>  
  );  
}
```

But if we write:

```
function ProductCard() {
```

```
    return (  
      <h2>iPhone</h2>  
      <p>₹79999</p>  
    );  
  }
```

this is invalid.

There are two sibling elements without a parent.

We can solve this using a wrapper:

```
function ProductCard() {  
  return (  
    <div>  
      <h2>iPhone</h2>  
      <p>₹79999</p>  
    </div>  
  );  
}
```

Or later, we'll learn about React fragments:

```
function ProductCard() {  
  return (  
    <>  
      <h2>iPhone</h2>  
      <p>₹79999</p>  
    </>  
  );  
}
```

For now, remember that the returned JSX needs to form a valid structure.

20. Component Names Should Be Meaningful

Instead of:

```
function Box() {  
  ...  
}
```

prefer:

```
function ProductCard() {  
  ...  
}
```

Instead of:

```
function Part1() {  
  ...  
}
```

prefer:

```
function Header() {  
  ...  
}
```

Good component names make the application easier to understand.

When someone opens your code, they should be able to understand the component's purpose from its name.

21. Live Coding — Improving Our Product Page

Let's now create a slightly better component structure.

```
function Header() {  
  return (  
    <header>  
      <h1>My Store</h1>  
    </header>  
  );  
}
```

Create:

```
function ProductCard() {  
  return (  
    <div className="product-card">  
  
      <h2>iPhone 17</h2>  
  
      <p>Apple</p>  
  
      <p>₹79999</p>  
    </div>  
  );  
}
```

```
        <button>
          Buy Now
        </button>

      </div>
    );
  }
}
```

Create:

```
function ProductList() {

  return (
    <section>

      <h2>Products</h2>

      <ProductCard />
      <ProductCard />
      <ProductCard />

    </section>
  );
}
```

And finally:

```
function Footer() {

  return (
    <footer>
      <p>© 2026 My Store</p>
    </footer>
  );
}
```

Our App:

```
function App() {

  return (
    <div>

      <Header />

      <ProductList />

      <Footer />

    </div>
  );
}
```

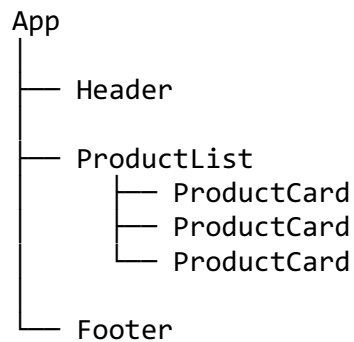


```
    </div>
  );
}

export default App;
```

Now look at our structure.

Instead of having one large component, we've created:



This is much easier to understand.

22. Connecting Today's Lecture With Previous Lectures

Let's connect everything we've learned so far.

In Lecture 2, we learned JSX:

```
<h1>My Store</h1>
```

In Lecture 3, we learned expressions:

```
<h1>{product.name}</h1>
```

In Lecture 4, we learned styling:

```
<div className="product-card">
```

In Lecture 5, we learned events:

```
<button onClick={handleBuy}>
```

And now in Lecture 6, we're putting these ideas inside reusable components.

For example:

```
function ProductCard() {  
  
  const product = {  
    name: "iPhone 17",  
    price: 79999  
  };  
  
  function handleBuy() {  
    console.log("Buying product");  
  }  
  
  return (  
    <div className="product-card">  
  
      <h2>{product.name}</h2>  
  
      <p>₹{product.price}</p>  
  
      <button onClick={handleBuy}>  
        Buy Now  
      </button>  
  
    </div>  
  );  
}
```

This is becoming a real React component.

It has:

Data
+
JSX
+
Styling
+
Events

23. Important Preview — Props

There's still a problem with our ProductCard.

Right now, every time we write:

```
<ProductCard />
```

we get the same product.

But in a real application, we want:

Product 1 → iPhone

Product 2 → Laptop

Product 3 → iPad

How do we send different information to the same component?

That's where **Props** come in.

We'll eventually write:

```
<ProductCard
  name="iPhone 17"
  price={79999}
/>
```

```
<ProductCard
  name="MacBook Air"
  price={99999}
/>
```

And inside the component, we'll receive that data.

But don't worry about that yet.

We'll dedicate an entire lecture to Props.

24. Common Beginner Mistakes

Let's quickly review some mistakes.

Mistake 1 — Lowercase component name

Avoid:

```
function productCard() {
  ...
}
```

Prefer:

```
function ProductCard() {
  ...
}
```

Mistake 2 — Forgetting to return JSX

Incorrect:

```
function ProductCard() {  
  <div>  
    Product  
  </div>  
}
```

Correct:

```
function ProductCard() {  
  return (  
    <div>  
      Product  
    </div>  
  );  
}
```

Mistake 3 — Using the component as lowercase

Avoid:

```
<productCard />
```

Use:

```
<ProductCard />
```

Mistake 4 — Trying to put multiple top-level elements

Avoid:

```
return (  
  <h1>Product</h1>  
  <p>₹79999</p>  
);
```

Use a wrapper:

```
return (  
  <div>
```

```
<h1>Product</h1>
<p>₹79999</p>
</div>
);
```

25. What We Learned Today

Let's summarize.

1. A component is a reusable piece of UI.

For example:

```
<ProductCard />
```

2. Components help us break large applications into smaller pieces.

App

- Header
- ProductList
- Footer

3. Components can contain other components.

```
<ProductList>
  <ProductCard />
</ProductList>
```

Conceptually, the component tree looks like:

App

↓

ProductList

↓

ProductCard

4. Component names should start with an uppercase letter.

ProductCard
Header
Footer

5. Components can contain more than JSX.

They can eventually contain:

JavaScript
Events

Props
State
Hooks

6. Components make our UI reusable and maintainable.

Instead of duplicating UI, we can create one component and reuse it.

26. Student Exercise

For today's assignment, create a small **College Website** using components.

Create these components:

App

- Header
- StudentInfo
- CourseList
- Footer

Your application should look something like:

```
-----  
      ABC College  
-----
```

Student Information

Name: Your Name
Course: BCA
Semester: 4

Courses

JavaScript
ReactJS
Database
HTML/CSS

```
-----  
      © 2026 ABC College  
-----
```

Create separate components:

```
function Header() {  
  ...  
}  
  
function StudentInfo() {  
  ...  
}  
  
function CourseList() {  
  ...  
}  
  
function Footer() {  
  ...  
}
```

Then combine them inside:

```
function App() {  
  ...  
}
```

Bonus Challenge

Create a:

```
<Course />
```

component and use it multiple times:

```
<Course />  
<Course />  
<Course />
```

Don't worry about making the courses different yet.

In the next lecture, we'll learn how to make reusable components display different data using **Props**.

27. Closing

So today we learned one of the most fundamental concepts in React:

Components.

Remember this simple definition:

A React application is built by combining reusable components.

We can think of components like building blocks.

For example:

LEGO Blocks



Small individual pieces



Combine them



Build a large structure

React works in a similar way.

We create small UI components:

Header

Button

ProductCard

Footer

and combine them to create larger applications.

Lecture 7 — Function Components in ReactJS

1. Opening

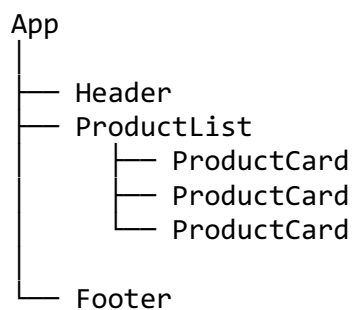
Hello everyone, and welcome back.

In the previous lecture, we learned one of the most important concepts in React:

Components.

We learned that a large application can be divided into smaller reusable pieces.

For example:



Today we're going to look at one specific type of React component:

Function Components

If you've been following along, you've actually already created function components.

For example:

```
function ProductCard() {
    return (
        <div>
            <h2>iPhone 17</h2>
            <p>₹79999</p>
        </div>
    );
}
```

This is a function component.

But today we're going to understand exactly:

- What a function component is
- How to create one
- How it returns JSX
- How to use it
- How components communicate with other components
- How to organize components into separate files
- How to reuse a function component
- Common mistakes
- And finally, we'll build a small component-based application

Let's get started.

2. What Is a Function Component?

Let's start with a simple definition.

A function component is a JavaScript function that returns JSX.

For example:

```
function Welcome() {  
  
  return (  
    <h1>  
      Welcome to React  
    </h1>  
  );  
}
```

Here:

```
function Welcome()
```

is a JavaScript function.

And:

```
<h1>Welcome to React</h1>
```

is JSX.

Because this function is being used as a React component, we call it a **function component**.

We can use it like this:

```
<Welcome />
```

So there are two parts:

Function definition



```
function Welcome() { ... }
```

Component usage



```
<Welcome />
```

3. Simple Function Component

Let's open our React project.

Inside App.jsx, create:

```
function Welcome() {  
    return (  
        <h1>  
            Welcome to ReactJS  
        </h1>  
    );  
}
```

Now use it inside App:

```
function App() {  
    return (  
        <div>  
            <Welcome />  
        </div>  
    );  
}
```

```
export default App;
```

Run the application.

You should see:

Welcome to ReactJS

Our first function component is working.

4. Understanding the Flow

Let's understand what happened.

We created:

```
function Welcome() {  
  
    return (  
        <h1>Welcome to ReactJS</h1>  
    );  
}
```

Then we wrote:

```
<Welcome />
```

React sees this component and renders the JSX returned by the function.

Conceptually:

```
<Welcome />  
  ↓  
Welcome()  
  ↓  
returns JSX  
  ↓  
<h1>Welcome to ReactJS</h1>  
  ↓  
Browser
```

This is the basic idea behind a function component.

5. Component Names Must Start With Capital Letters

This is extremely important.

A React component should normally start with an uppercase letter.

Correct:

```
function Welcome() {  
  ...  
}
```

And:

```
<Welcome />
```

Incorrect:

```
function welcome() {  
  ...  
}
```

and:

```
<welcome />
```

Why?

Because React needs to distinguish between HTML elements and custom components.

For example:

```
<div>
```

is a built-in HTML element.

But:

```
<Welcome />
```

is our custom React component.

The uppercase letter helps React identify it as a component.

Remember:

HTML elements

↓

lowercase

div

h1

button

input

React components

↓

uppercase

App
Welcome
Header
ProductCard

6. Function Components Are JavaScript Functions

One thing students sometimes forget is that a function component is still a JavaScript function.

That means we can use JavaScript inside it.

For example:

```
function Product() {  
  
  const name = "iPhone 17";  
  const price = 79999;  
  
  return (  
    <div>  
  
      <h2>{name}</h2>  
  
      <p>₹{price}</p>  
  
    </div>  
  );  
}
```

Notice that we're using JavaScript variables:

```
const name = "iPhone 17";  
const price = 79999;
```

and then displaying them using JSX expressions:

```
{name}  
{price}
```

This connects directly with what we learned in Lecture 3.

7. A Function Component Can Contain Logic

A function component doesn't have to contain only static JSX.

It can also contain JavaScript logic.

For example:

```
function Product() {  
  
  const price = 79999;  
  const discount = 5000;  
  
  const finalPrice = price - discount;  
  
  return (  
    <div>  
  
      <h2>iPhone 17</h2>  
  
      <p>Original Price: ₹{price}</p>  
  
      <p>Discount: ₹{discount}</p>  
  
      <p>  
        Final Price: ₹{finalPrice}  
      </p>  
  
    </div>  
  );  
}
```

The component:

1. Executes JavaScript
2. Calculates the values
3. Returns JSX
4. React renders that JSX

Conceptually:

```
Function Component  
  ↓  
JavaScript logic  
  ↓  
JSX
```

↓
React UI

8. Function Components and Events

We can also use event handling inside function components.

For example:

```
function ProductCard() {  
  
  function handleBuy() {  
    console.log("Buy button clicked");  
  }  
  
  return (  
    <div>  
  
      <h2>iPhone 17</h2>  
  
      <button onClick={handleBuy}>  
        Buy Now  
      </button>  
  
    </div>  
  );  
}
```

Notice that both:

```
function handleBuy()
```

and:

```
return (...)
```

are inside the component.

This means the component contains:

UI
+
Behavior

This is one of the reasons React components are so useful.

9. Reusing a Function Component

Now let's create a component:

```
function ProductCard() {  
  return (  
    <div className="product-card">  
      <h2>Product</h2>  
      <button>  
        Buy Now  
      </button>  
    </div>  
  );  
}
```

Now inside App:

```
function App() {  
  return (  
    <div>  
      <ProductCard />  
      <ProductCard />  
      <ProductCard />  
    </div>  
  );  
}
```

What happens?

React renders the same component three times.

The result is conceptually:

Product
[Buy Now]

Product
[Buy Now]

Product
[Buy Now]

This demonstrates **reusability**.

But there's a problem.

All three cards display the same product.

How can we display different products?

That's where **Props** come in.

We'll cover Props in Lecture 10.

For now, just understand that function components can be reused multiple times.

10. Live Coding — Product Card

Let's build a slightly more realistic component.

Create:

```
function ProductCard() {  
  
  const productName = "iPhone 17";  
  const productPrice = 79999;  
  
  function handleBuy() {  
    console.log("Buying product");  
  }  
  
  return (  
    <div className="product-card">  
  
      <h2>{productName}</h2>  
  
      <p>₹{productPrice}</p>  
  
      <button onClick={handleBuy}>  
        Buy Now  
      </button>  
  
    </div>  
  );  
}
```

Then:

```
function App() {  
  return (  
    <div>  
      <h1>My Store</h1>  
      <ProductCard />  
    </div>  
  );  
}  
  
export default App;
```

Now we have:

My Store

iPhone 17
₹79999

[Buy Now]

Click the button.

The console should show:

Buying product

So our component contains:

Product data
+
JSX
+
Event handling

11. Separating Components Into Files

Now we're going to make an important improvement.

Until now, we may have written everything in `App.jsx`.

For example:

App.jsx

App
Header
ProductCard
Footer

That's fine while learning.

But imagine a real application with 50 components.

We don't want 50 components in one file.

So we create separate files.

For example:

```
src
├── App.jsx
├── Header.jsx
├── ProductCard.jsx
└── Footer.jsx
```

Each component gets its own file.

12. Creating ProductCard.jsx

Inside the src folder, create:

ProductCard.jsx

Write:

```
function ProductCard() {
  return (
    <div className="product-card">
      <h2>iPhone 17</h2>
      <p>₹79999</p>
      <button>
        Buy Now
      </button>
    </div>
  )
}
```

```
    </div>
  );
}

export default ProductCard;
```

Notice the last line:

```
export default ProductCard;
```

This allows another file to import this component.

13. Importing the Component

Now go to:

App.jsx

At the top:

```
import ProductCard from "../ProductCard";
```

Then use it:

```
function App() {
  return (
    <div>

      <h1>My Store</h1>

      <ProductCard />

    </div>
  );
}

export default App;
```

Now the structure is:

```
App.jsx
  |
  | imports
```



ProductCard.jsx

And the browser displays the component.

14. Understanding Export and Import

Let's slow down here because this is important.

In ProductCard.jsx:

```
export default ProductCard;
```

means:

Make this component available to other files.

Then in App.jsx:

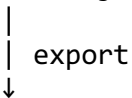
```
import ProductCard from "../ProductCard";
```

means:

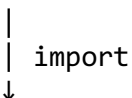
Bring the ProductCard component into this file.

The relationship is:

ProductCard.jsx



ProductCard



App.jsx

This is standard JavaScript module functionality.

React uses JavaScript's module system to organize components.

15. Creating a Header Component

Let's create another file:

Header.jsx

Write:

```
function Header() {  
  
  return (  
    <header>  
  
      <h1>My Store</h1>  
  
      <nav>  
        Home | Products | Contact  
      </nav>  
  
    </header>  
  );  
}  
  
export default Header;
```

Now in App.jsx:

```
import Header from "../Header";  
import ProductCard from "../ProductCard";
```

And:

```
function App() {  
  
  return (  
    <div>  
  
      <Header />  
  
      <ProductCard />  
  
    </div>  
  );  
}  
  
export default App;
```

Now our application has multiple files.

16. Creating a Footer Component

Let's create:

Footer.jsx

Code:

```
function Footer() {  
  
    return (  
        <footer>  
  
            <p>  
                © 2026 My Store  
            </p>  
  
        </footer>  
    );  
}  
  
export default Footer;
```

Then import it:

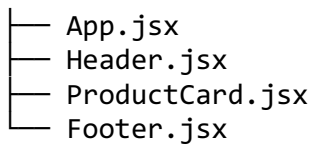
```
import Footer from "../Footer";
```

And use it:

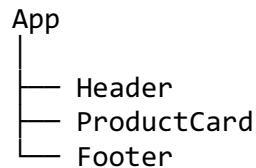
```
function App() {  
  
    return (  
        <div>  
  
            <Header />  
  
            <ProductCard />  
  
            <Footer />  
  
        </div>  
    );  
}
```

Our project now looks like:

```
src  
|
```

And our component tree is:



17. One Component Can Be Used Multiple Times

Let's see the real benefit.

In App.jsx:

```
function App() {  
  return (  
    <div>  
  
      <Header />  
  
      <ProductCard />  
      <ProductCard />  
      <ProductCard />  
  
      <Footer />  
  
    </div>  
  );  
}
```

We only created ProductCard once.

But we're using it three times.

This is component reusability.

Later, Props will allow us to turn these three identical cards into different products.

18. Component Return Value

A function component must return something that React can render.

For example:

```
function Welcome() {  
  
    return (  
        <h1>Hello</h1>  
    );  
}
```

It can return:

```
<div>  
    ...  
</div>
```

or:

```
<>  
    ...  
</>
```

It can also return null:

```
function Message() {  
  
    return null;  
}
```

Returning null means the component renders nothing.

For now, the most important thing is:

A function component normally returns JSX that describes its UI.

19. Function Component With a Parameter

You may notice that a JavaScript function can receive parameters.

For example:

```
function add(a, b) {  
    return a + b;  
}
```

Similarly, React function components can receive information.

For example:

```
function ProductCard(props) {  
  
    return (  
        <h2>  
            {props.name}  
        </h2>  
    );  
}
```

Then:

```
<ProductCard name="iPhone 17" />
```

This is how we start making components reusable with different data.

We're not going to go deeply into this today because **Props** have their own dedicated lecture.

But remember:

```
Parent Component  
    |  
    | data  
    ↓  
Child Component
```

We'll explore this properly in Lecture 10.

20. Function Component vs Normal Function

A common question is:

“What’s the difference between a normal JavaScript function and a function component?”

Technically, a function component is a JavaScript function used by React to describe UI.

For example:

Normal function:

```
function calculateTotal(price, quantity) {
```

```
    return price * quantity;
}
```

Function component:

```
function ProductCard() {

    return (
        <div>
            Product
        </div>
    );
}
```

The major difference is what the function is used for.

A normal function performs some JavaScript operation.

A React function component returns UI.

21. Function Component With Multiple Elements

Let's create a complete example.

```
function Student() {

    const name = "Rahul";
    const course = "ReactJS";
    const semester = 4;

    return (
        <div>

            <h2>{name}</h2>

            <p>
                Course: {course}
            </p>

            <p>
                Semester: {semester}
            </p>

        </div>
    );
}
```

```
    );  
  }  
}
```

Then:

```
function App() {  
  return (  
    <div>  
      <h1>Student Information</h1>  
      <Student />  
    </div>  
  );  
}
```

The component is responsible for displaying student information.

22. Components Can Be Nested

Let's create:

```
function Student() {  
  return (  
    <div>  
      <StudentHeader />  
      <StudentDetails />  
    </div>  
  );  
}
```

Now we have:

```
App  
↓  
Student  
↓  
-----
```

```
|  
StudentHeader  StudentDetails
```

This is perfectly valid.

Components can be nested as deeply as the application requires.

However, we shouldn't create unnecessary complexity.

The goal is to create a component structure that is easy to understand and maintain.

23. Live Coding — College Application

Let's build one complete example.

We'll create:

```
src  
├── App.jsx  
├── Header.jsx  
├── StudentCard.jsx  
└── Footer.jsx
```

Header.jsx

```
function Header() {  
  
  return (  
    <header>  
  
      <h1>ABC College</h1>  
  
      <nav>  
        Home | Students | Courses  
      </nav>  
  
    </header>  
  );  
}
```

```
export default Header;
```

StudentCard.jsx

```
function StudentCard() {
```

```
const name = "Rahul";
const course = "BCA";
const semester = 4;

return (
  <div>

    <h2>{name}</h2>

    <p>Course: {course}</p>

    <p>Semester: {semester}</p>

  </div>
);
}
```

```
export default StudentCard;
```

Footer.jsx

```
function Footer() {

  return (
    <footer>

      <p>
        © 2026 ABC College
      </p>

    </footer>
  );
}
```

```
export default Footer;
```

App.jsx

```
import Header from "./Header";
import StudentCard from "./StudentCard";
import Footer from "./Footer";

function App() {

  return (
    <div>

      <Header />
```

```
    <main>

      <h2>Student Information</h2>

      <StudentCard />

    </main>

    <Footer />

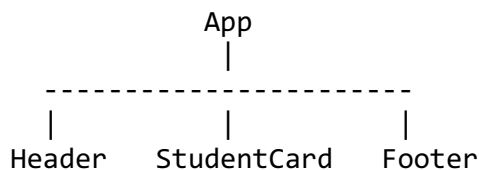
  </div>
);
}

export default App;
```

Now our application has a clear structure.

24. Component Tree of Our Application

Let's visualize it:



This is the component tree.

The App component is the parent.

Header, StudentCard, and Footer are child components.

This parent-child relationship becomes extremely important when we start working with Props and State.

25. Parent and Child Components

Let's define these terms.

If we have:

```
function App() {  
    return (  
        <StudentCard />  
    );  
}
```

Then:

App
↓
StudentCard

App is the **parent component**.

StudentCard is the **child component**.

This relationship will allow us to pass data:

Parent
|
| Props
↓
Child

For example:

```
<StudentCard name="Rahul" />
```

We'll learn this in detail later.

26. Common Mistakes

Let's look at some common beginner mistakes.

Mistake 1 — Lowercase component name

Incorrect:

```
function productCard() {  
    ...  
}
```

Correct:

```
function ProductCard() {  
    ...  
}
```

Mistake 2 — Forgetting the return

Incorrect:

```
function ProductCard() {  
    <div>  
        Product  
    </div>  
}
```

Correct:

```
function ProductCard() {  
    return (  
        <div>  
            Product  
        </div>  
    );  
}
```

Mistake 3 — Forgetting to export

If the component is in another file:

```
function ProductCard() {  
    ...  
}
```

you need to export it:

```
export default ProductCard;
```

Then import it:

```
import ProductCard from "../ProductCard";
```

Mistake 4 — Incorrect import path

If your file is:

`src/components/ProductCard.jsx`

your import may need to be:

```
import ProductCard from "../components/ProductCard";
```

The path depends on where the importing file is located.

Mistake 5 — Treating the component like a normal HTML tag

Remember:

```
<ProductCard />
```

is your React component.

And:

```
<div />
```

is an HTML element.

The capital letter makes the distinction.

27. Function Components and React

Modern React development primarily uses function components.

You'll see code like:

```
function App() {  
  return <h1>Hello React</h1>;  
}
```

or:

```
const App = () => {  
  return <h1>Hello React</h1>;  
};
```

Both are function components.

We'll primarily use the standard function syntax throughout this course because it's easier to understand when you're learning the fundamentals.

28. Arrow Function Components

You may also see components written as arrow functions.

For example:

```
const Welcome = () => {  
  return (  
    <h1>  
      Welcome to React  
    </h1>  
  );  
};
```

This is also a valid function component.

You can use it like:

```
<Welcome />
```

There are two common styles:

Standard function

```
function Welcome() {  
  return <h1>Welcome</h1>;  
}
```

Arrow function

```
const Welcome = () => {  
  return <h1>Welcome</h1>;  
};
```

For this course, we'll primarily use:

```
function Welcome() {  
  ...  
}
```

because it clearly shows that a component is a JavaScript function.

29. What We Learned Today

Let's summarize the important points.

1. A function component is a JavaScript function that returns JSX.

```
function Welcome() {  
  
    return (  
        <h1>Welcome</h1>  
    );  
}
```

2. Components should start with uppercase letters.

ProductCard
Header
Footer

3. Components are used like custom JSX elements.

```
<ProductCard />
```

4. Components can contain JavaScript logic.

```
function Product() {  
  
    const price = 79999;  
  
    return (  
        <p>₹{price}</p>  
    );  
}
```

5. Components can handle events.

```
function ProductCard() {  
  
    function handleBuy() {  
        console.log("Buy");  
    }  
  
    return (  
        <button onClick={handleBuy}>  
            Buy Now  
        </button>  
    );  
}
```

6. Components can be reused.

```
<ProductCard />  
<ProductCard />  
<ProductCard />
```

7. Components can be stored in separate files.

```
Header.jsx  
ProductCard.jsx  
Footer.jsx
```

and imported using:

```
import Header from "./Header";
```

8. Components have parent-child relationships.

```
App  
↓  
ProductCard
```

And later we'll use Props to pass data from parent to child.

30. Student Exercise

For today's exercise, create a small **Employee Management UI**.

Create these components:

```
App  
├── Header  
├── EmployeeCard  
├── EmployeeCard  
├── EmployeeCard  
└── Footer
```

Your EmployeeCard should display:

```
-----  
Name: Rahul  
Department: IT  
Salary: ₹50000  
  
[View Details]  
-----
```

Create a function component:

```
function EmployeeCard() {  
  ...  
}
```

Add a button:

```
<button onClick={handleView}>  
  View Details  
</button>
```

When the button is clicked, print:

Viewing employee details

Then move the components into separate files:

```
src  
├── App.jsx  
├── Header.jsx  
├── EmployeeCard.jsx  
└── Footer.jsx
```

Bonus Challenge

Try rendering:

```
<EmployeeCard />  
<EmployeeCard />  
<EmployeeCard />
```

You will notice that all three display the same employee.

That's okay.

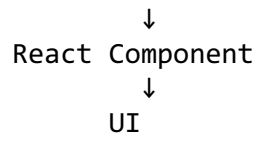
In a future lecture, we'll solve exactly this problem using **Props**.

31. Closing

So today we learned how to create **Function Components**.

The most important idea to remember is:

```
JavaScript Function  
  ↓  
  JSX
```



Function components are the foundation of modern React applications.

Lecture 8 — Class Components in ReactJS

1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned about **Function Components**.

We created components like:

```
function ProductCard() {  
  
  return (  
    <div>  
      <h2>iPhone 17</h2>  
      <p>₹79999</p>  
    </div>  
  );  
}
```

Function components are the style you'll see most often in modern React applications.

But there is another type of React component that you will still encounter when working with older React applications:

Class Components

If you work on an existing React project, you may see code like:

```
class ProductCard extends React.Component {  
  
  render() {  
  
    return (  
      <div>  
        <h2>iPhone 17</h2>  
      </div>  
    );  
  }  
}
```

At first, this may look very different from what we've learned.

So in today's lecture, we'll understand:

- What class components are
- Why they existed
- How to create a class component
- The `render()` method
- `this`
- Class component state at a basic level
- Class component events
- Function component vs class component
- Why modern React generally prefers function components
- How to recognize class components in existing projects

Let's get started.

2. What Is a Class Component?

Let's start with the definition.

A class component is a React component created using a JavaScript class that extends `React.Component`.

For example:

```
class Welcome extends React.Component {  
  
  render() {  
  
    return (  
      <h1>  
        Welcome to React  
      </h1>  
    );  
  }  
}
```

Then we can use it:

```
<Welcome />
```

Conceptually:

```
JavaScript Class  
  ↓  
extends React.Component
```

↓
render()
↓
JSX
↓
React UI

Compare this with our function component:

```
function Welcome() {  
    return (  
        <h1>Welcome to React</h1>  
    );  
}
```

Both create a React component.

The syntax is different.

3. Why Were Class Components Used?

You might ask:

If function components are simpler, why were class components created?

Historically, class components were important because they provided features such as:

- State
- Lifecycle methods
- Event handling
- Access to component lifecycle

Before modern React introduced Hooks, function components were more limited.

For example, developers commonly used class components when they needed state or lifecycle functionality.

A typical older React application might contain many classes like:

```
class App extends React.Component {  
    ...  
}  
  
class Product extends React.Component {  
    ...
```

```
}
```

```
class User extends React.Component {  
    ...  
}
```

Later, React introduced Hooks, which allowed function components to use state and other React features.

That's why modern React development generally favors function components.

But class components haven't disappeared completely.

You may still encounter them in:

- Older projects
- Legacy applications
- Existing company codebases
- Older tutorials
- Older libraries and examples

So learning how to read them is useful.

4. Creating Our First Class Component

Let's create a simple class component.

At the top of the file, import React:

```
import React from "react";
```

Now:

```
class Welcome extends React.Component {  
    render() {  
        return (  
            <h1>  
                Welcome to ReactJS  
            </h1>  
        );  
    }  
}
```

Then our App:

```
function App() {  
  
  return (  
    <div>  
  
      <Welcome />  
  
    </div>  
  );  
}  
  
export default App;
```

Run the application.

You'll see:

Welcome to ReactJS

Our class component is working.

5. Understanding `extends React.Component`

Let's look at this line:

```
class Welcome extends React.Component {
```

This is JavaScript class inheritance.

We are saying:

Create a class called `Welcome` that inherits functionality from `React.Component`.

React provides the base class:

`React.Component`

Our component extends it:

```
class Welcome extends React.Component
```

This gives React the information and behavior it needs to treat our class as a component.

6. The `render()` Method

Now let's look at the most important part of a basic class component:

`render()`

For example:

```
class Welcome extends React.Component {  
  
  render() {  
  
    return (  
      <h1>  
        Welcome  
      </h1>  
    );  
  }  
}
```

The `render()` method returns the JSX that should be displayed.

Think of it as:

```
Class Component  
  ↓  
render()  
  ↓  
JSX  
  ↓  
UI
```

A basic class component needs a `render()` method.

This is one of the biggest differences from function components.

Function Component

```
function Welcome() {  
  
  return (  
    <h1>Welcome</h1>  
  );  
}
```

Class Component

```
class Welcome extends React.Component {  
  
  render() {
```

```
    return (  
      <h1>Welcome</h1>  
    );  
  }  
}
```

In the function component, the function itself returns JSX.

In the class component, the `render()` method returns JSX.

7. Importing React

Depending on your React project setup, you may see class components written as:

```
import React from "react";  
  
class Welcome extends React.Component {  
  ...  
}
```

This allows us to reference:

`React.Component`

Another common syntax is:

```
import { Component } from "react";  
  
class Welcome extends Component {  
  render() {  
    return (  
      <h1>Welcome</h1>  
    );  
  }  
}
```

Both approaches represent the same basic concept.

For beginners, we'll use:

```
import React from "react";
```

and:

```
class Welcome extends React.Component
```

because the relationship is very explicit.

8. Class Component With JSX

Let's create a slightly more realistic example.

```
class ProductCard extends React.Component {  
  
  render() {  
  
    return (  
      <div className="product-card">  
  
        <h2>iPhone 17</h2>  
  
        <p>₹79999</p>  
  
        <button>  
          Buy Now  
        </button>  
  
      </div>  
    );  
  }  
}
```

Then:

```
function App() {  
  
  return (  
    <div>  
  
      <h1>My Store</h1>  
  
      <ProductCard />  
  
    </div>  
  );  
}
```

The browser displays:

My Store

iPhone 17
₹79999

[Buy Now]

So a class component can contain the same type of JSX that we used inside a function component.

9. JavaScript Inside a Class Component

Now suppose we want to define some data.

With a function component, we might write:

```
function ProductCard() {  
  
  const name = "iPhone 17";  
  const price = 79999;  
  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>₹{price}</p>  
    </div>  
  );  
}
```

In a class component, we can use class properties.

For example:

```
class ProductCard extends React.Component {  
  
  name = "iPhone 17";  
  price = 79999;  
  
  render() {  
  
    return (  
      <div>  
  
        <h2>{this.name}</h2>  

```

```
        <p>₹{this.price}</p>
    </div>
  );
}
}
```

Notice something new:

`this.name`

and:

`this.price`

This brings us to an important JavaScript concept:

`this`

10. Understanding `this`

In a class component, `this` refers to the current component instance.

For example:

```
class ProductCard extends React.Component {
  name = "iPhone 17";

  render() {
    return (
      <h2>
        {this.name}
      </h2>
    );
  }
}
```

Here:

`this.name`

means:

The name property belonging to this component instance.

You will see this frequently in older React code.

For example:

`this.props`

and:

`this.state`

and:

`this.setState()`

These are especially important when reading legacy React applications.

11. Class Components and Props

We haven't formally covered Props yet—that's Lecture 10—but we can briefly see how props look in class components.

Suppose we use:

```
<ProductCard  
  name="iPhone 17"  
  price={79999}  
>
```

Inside the class component:

```
class ProductCard extends React.Component {  
  
  render() {  
  
    return (  
      <div>  
  
        <h2>  
          {this.props.name}  
        </h2>  
  
        <p>  
          ₹{this.props.price}  
        </p>  
  
      </div>  
    );  
  }  
}
```

```
}  
}
```

Notice:

`this.props.name`

and:

`this.props.price`

In a function component, you might see:

`props.name`

In a class component, you'll commonly see:

`this.props.name`

We'll study Props properly in Lecture 10.

12. Class Component State

Now let's look at one of the historical reasons class components were so popular.

They could manage component state.

For example:

```
class Counter extends React.Component {  
  
  state = {  
    count: 0  
  };  
  
  render() {  
  
    return (  
      <div>  
  
        <h2>  
          Count: {this.state.count}  
        </h2>  
  
      </div>  
    );  
  }  
}
```

```
}  
}
```

Here we have:

```
state = {  
  count: 0  
};
```

And we access it using:

```
this.state.count
```

So the UI displays:

Count: 0

We'll learn state much more thoroughly later when we reach the `useState` lecture.

For now, the important thing is simply recognizing this syntax.

13. Updating State With `setState`

In a class component, we shouldn't directly modify state like this:

```
this.state.count = this.state.count + 1;
```

Instead, class components use:

```
this.setState()
```

For example:

```
this.setState({  
  count: this.state.count + 1  
});
```

Let's create a complete counter.

```
class Counter extends React.Component {  
  
  state = {  
    count: 0  
  };  
  
  handleIncrement = () => {  
  
    this.setState({
```

```
        count: this.state.count + 1
      });

    };

    render() {

      return (
        <div>

          <h2>
            Count: {this.state.count}
          </h2>

          <button
            onClick={this.handleIncrement}
          >
            Increment
          </button>

        </div>
      );
    }
  }
}
```

Click the button.

The count changes:

Count: 0

↓ click

Count: 1

↓ click

Count: 2

This is a very important example because it shows how older React applications managed interactive data.

14. Why `this.setState()`?

React needs to know that the state has changed.

If we directly modify:

```
this.state.count = 1;
```

React doesn't get the proper state update mechanism.

Instead:

```
this.setState({  
  count: 1  
});
```

tells React:

The component's state has changed. Update the UI accordingly.

This is similar conceptually to what we'll later do with:

```
setCount(...)
```

when using the `useState` Hook.

15. Event Handling in Class Components

Let's look at events.

In a function component, we might write:

```
function ProductCard() {  
  
  function handleBuy() {  
    console.log("Buying product");  
  }  
  
  return (  
    <button onClick={handleBuy}>  
      Buy Now  
    </button>  
  );  
}
```

In a class component:

```
class ProductCard extends React.Component {  
  
  handleBuy = () => {  
    console.log("Buying product");  
  };  
  
  render() {  
  
    return (  
      <button onClick={this.handleBuy}>  
        Buy Now  
      </button>  
    );  
  }  
}
```

Notice:

`this.handleBuy`

because handleBuy belongs to the class.

16. Why Do We See Arrow Functions in Class Components?

You may encounter:

```
handleBuy = () => {  
  console.log("Buying");  
};
```

instead of:

```
handleBuy() {  
  console.log("Buying");  
}
```

One reason is handling the JavaScript this context.

Class methods can lose their expected this context when passed around as callbacks.

Using a class field arrow function:

```
handleBuy = () => {  
  ...  
};
```


keeps the method bound to the component instance.

This is one of the confusing areas of older class-based React code.

You don't need to memorize every detail right now.

The important thing is that when reading older React code, you may see:

```
handleClick = () => {  
  ...  
};
```

and:

```
onClick={this.handleClick}
```

17. Older Binding Syntax

You may also encounter older class components written like this:

```
class ProductCard extends React.Component {  
  
  constructor(props) {  
  
    super(props);  
  
    this.handleClick =  
      this.handleClick.bind(this);  
  }  
  
  handleClick() {  
  
    console.log("Buying product");  
  }  
  
  render() {  
  
    return (  
      <button onClick={this.handleClick}>  
        Buy Now  
      </button>  
    );  
  }  
}
```

You might wonder:

Why is `bind()` being used?

Because older class-based React code often needed explicit binding to make sure this referred to the component instance.

This is one reason class components can feel more complicated than function components.

Modern React code generally avoids this complexity by using function components and Hooks.

18. Constructor

Another thing you may see in class components is a constructor.

For example:

```
class Counter extends React.Component {  
  
  constructor(props) {  
  
    super(props);  
  
    this.state = {  
      count: 0  
    };  
  }  
  
  render() {  
  
    return (  
      <h2>  
        Count: {this.state.count}  
      </h2>  
    );  
  }  
}
```

Let's understand the important pieces.

`constructor`
`constructor(props)`

is a JavaScript class constructor.

```
super(props)
```

```
super(props);
```

calls the constructor of the parent class.

Since our class extends:

```
React.Component
```

we call:

```
super(props);
```

State

Then:

```
this.state = {  
  count: 0  
};
```

initializes the component's state.

You don't need to memorize the constructor syntax for new React development.

But you should recognize it when you see it in an older project.

19. Complete Class Counter

Let's put the concepts together.

```
import React from "react";
```

```
class Counter extends React.Component {
```

```
  constructor(props) {
```

```
    super(props);
```

```
    this.state = {  
      count: 0
```

```
    };
```

```
  }
```

```
  handleIncrement = () => {
```

```
    this.setState({
      count: this.state.count + 1
    });
  };

  render() {

    return (
      <div>

        <h1>
          Counter
        </h1>

        <h2>
          Count: {this.state.count}
        </h2>

        <button
          onClick={this.handleIncrement}
        >
          Increment
        </button>

      </div>
    );
  }
}

export default Counter;
```

This is a complete class component.

It has:

```
Class
↓
State
↓
Event Handler
↓
setState
↓
render()
↓
JSX
```

20. Function Component vs Class Component

Now let's compare them.

Function Component

```
function Counter() {  
  
  return (  
    <h1>  
      Counter  
    </h1>  
  );  
}
```

Class Component

```
class Counter extends React.Component {  
  
  render() {  
  
    return (  
      <h1>  
        Counter  
      </h1>  
    );  
  }  
}
```

The function component is much shorter.

Now compare state.

Function Component

Modern React uses Hooks:

```
const [count, setCount] = useState(0);
```

Class Component

```
this.state = {  
  count: 0  
};
```

And:

```
this.setState({  
  count: this.state.count + 1  
});
```

So class components generally require more syntax.

21. Side-by-Side Example

Let's display the same counter using both approaches.

Function Component

```
import { useState } from "react";

function Counter() {

  const [count, setCount] = useState(0);

  return (
    <div>

      <h2>Count: {count}</h2>

      <button
        onClick={() => setCount(count + 1)}
      >
        Increment
      </button>

    </div>
  );
}
```

Class Component

```
class Counter extends React.Component {

  state = {
    count: 0
  };

  handleIncrement = () => {

    this.setState({
      count: this.state.count + 1
    });
  };

  render() {
```

```
    return (  
      <div>  
  
        <h2>  
          Count: {this.state.count}  
        </h2>  
  
        <button  
          onClick={this.handleIncrement}  
        >  
          Increment  
        </button>  
  
      </div>  
    );  
  }  
}
```

Both can produce the same UI.

But the function component is generally simpler.

22. Why Modern React Uses Function Components

This is an important question.

Why aren't we writing class components in our new React applications?

One major reason is **Hooks**.

Hooks allow function components to use features that historically required class components.

For example:

`useState()`

allows state.

`useEffect()`

allows effects.

And there are many other Hooks.

This means we can build powerful components without the extra class syntax.

So when creating a new React application today, you'll generally use:

```
function ProductCard() {  
  ...  
}
```

rather than:

```
class ProductCard extends React.Component {  
  ...  
}
```

23. When Will You Still See Class Components?

Even though function components are preferred for new development, class components haven't become irrelevant.

You may encounter them when:

Working on an old project

For example:

```
class User extends React.Component {  
  ...  
}
```

Maintaining legacy code

Your company might have an older React application that hasn't been fully migrated.

Reading older tutorials

Older tutorials may demonstrate:

```
class App extends React.Component
```

Reading older Stack Overflow answers

You may find class-based solutions.

So knowing the syntax helps you understand existing code.

Think of this lecture as teaching you:

How to understand and work with legacy React code.

Not necessarily how you should build every new component.

24. Class Component Lifecycle — Introduction

Class components also introduced an important concept called:

Lifecycle methods

A React component goes through different stages.

Conceptually:

```
Component Created
  ↓
Component Rendered
  ↓
Component Updated
  ↓
Component Removed
```

Class components provided methods such as:

```
componentDidMount()
componentDidUpdate()
componentWillUnmount()
```

These allowed developers to perform actions at different stages of the component's lifecycle.

For example:

```
componentDidMount() {
  console.log("Component mounted");
}
```

We won't go deeply into lifecycle methods in this course because modern React commonly handles these kinds of side effects using Hooks such as:

```
useEffect()
```

We'll learn useEffect later.

For today's lecture, just recognize these names when you see them in legacy code.

25. A Legacy React Example

You may encounter code like this:

```
class UserList extends React.Component {  
  
  state = {  
    users: []  
  };  
  
  componentDidMount() {  
  
    console.log("Component mounted");  
  
  }  
  
  render() {  
  
    return (  
      <div>  
  
        <h1>User List</h1>  
  
      </div>  
    );  
  }  
}
```

Don't panic when you see this.

Break it down:

```
class UserList  
  ↓  
React.Component  
  ↓  
state  
  ↓  
componentDidMount()  
  ↓  
render()  
  ↓  
JSX
```

Once you understand the structure, legacy class components become much easier to read.

26. Important Difference — `render()`

One thing you'll immediately notice when comparing the two styles:

Function Component

```
function Product() {  
  
  return (  
    <h1>Product</h1>  
  );  
}
```

There is no `render()`.

Class Component

```
class Product extends React.Component {  
  
  render() {  
  
    return (  
      <h1>Product</h1>  
    );  
  }  
}
```

The JSX comes from:

`render()`

Remember this.

If you open an older React application and see:

```
class Something extends React.Component
```

look for:

`render()`

That's where the component's UI is usually returned.

27. Common Beginner Mistakes

Let's look at some common mistakes.

Mistake 1 — Forgetting `extends React.Component`

Incorrect:

```
class ProductCard {  
  
  render() {  
    ...  
  }  
}
```

Correct:

```
class ProductCard extends React.Component {  
  
  render() {  
    ...  
  }  
}
```

Mistake 2 — Forgetting `render()`

A basic class component needs a render method:

```
class ProductCard extends React.Component {  
  
  render() {  
  
    return (  
      <div>  
        Product  
      </div>  
    );  
  }  
}
```

Mistake 3 — Forgetting `this`

Inside a class component, you may need:

`this.state`

rather than:

state

And:

`this.props`

rather than:

props

Mistake 4 — Directly modifying state

Avoid:

```
this.state.count++;
```

Use:

```
this.setState({  
  count: this.state.count + 1  
});
```

Mistake 5 — Confusing class syntax with function syntax

Don't mix them.

Function component:

```
function Product() {  
  
  return <h1>Product</h1>;  
}
```

Class component:

```
class Product extends React.Component {  
  
  render() {  
  
    return <h1>Product</h1>;  
  }  
}
```

Understand which style you're working with.

28. Live Coding — Legacy Product Component

Let's create one final practical example.

Suppose we're maintaining an older e-commerce application.

We'll create:

```
import React from "react";

class ProductCard extends React.Component {

  state = {
    liked: false
  };

  handleLike = () => {

    this.setState({
      liked: !this.state.liked
    });

  };

  render() {

    return (
      <div className="product-card">

        <h2>iPhone 17</h2>

        <p>₹79999</p>

        <button
          onClick={this.handleLike}
        >
          {this.state.liked
            ? "Liked"
            : "Like"}
        </button>

      </div>
    );
  }
}
```

```
    );  
  }  
}
```

```
export default ProductCard;
```

Now the component has:

```
Product  
  ↓  
State  
  ↓  
Like button  
  ↓  
setState()  
  ↓  
UI update
```

When we click:

```
[Like]
```

it becomes:

```
[Liked]
```

Click again:

```
[Like]
```

This demonstrates how class components used state before Hooks became the standard approach.

29. The Big Picture

At this point, we've now seen two ways to create React components.

Function Component

```
function ProductCard() {  
  return (  
    <div>  
      Product  
    </div>  
  );  
}
```

Class Component

```
class ProductCard extends React.Component {  
  
  render() {  
  
    return (  
      <div>  
        Product  
      </div>  
    );  
  }  
}
```

Both are components.

The biggest difference is their programming model.

Function components are based on:

JavaScript Functions
+
Hooks

Class components are based on:

JavaScript Classes
+
this
+
state
+
Lifecycle Methods

For modern React development, you'll primarily use the first approach.

30. What We Learned Today

Let's summarize today's lecture.

1. Class components use JavaScript classes.

```
class ProductCard extends React.Component {  
  ...  
}
```


2. They extend `React.Component`.

```
extends React.Component
```

3. The UI is returned from `render()`.

```
render() {  
  
    return (  
        <h1>Product</h1>  
    );  
}
```

4. Class components commonly use `this`.

```
this.props  
this.state  
this.setState()
```

5. Class components can handle events.

```
handleClick = () => {  
    ...  
};
```

and:

```
<button onClick={this.handleClick}>
```

6. Class components can have state.

```
state = {  
    count: 0  
};
```

7. State is updated with `setState()`.

```
this.setState({  
    count: this.state.count + 1  
});
```

8. Class components were historically important.

They are still useful to understand when maintaining older React applications.

9. Modern React generally prefers function components.

With Hooks, function components can handle state, effects, and other React features without the class-based syntax.

31. Student Exercise

For today's exercise, create a **Class-Based Counter**.

Create:

```
class Counter extends React.Component {  
  ...  
}
```

Initialize:

```
state = {  
  count: 0  
};
```

Display:

Count: 0

Create three buttons:

[Increment]

[Decrement]

[Reset]

Your component should behave like this:

Count: 0

Click Increment

Count: 1

Click Increment

Count: 2

Click Decrement

Count: 1

Click Reset

Count: 0

Use:

```
this.setState()
```

to update the count.

Bonus Challenge

Create a class component called:

UserCard

with:

Name
Email
City

Display the information using class properties and `this`.

32. Final Takeaway

The most important thing from today's lecture isn't memorizing every piece of class syntax.

The important thing is being able to recognize this:

```
class Something extends React.Component
```

and understand:

“This is a class-based React component.”

Then look for:

```
render()
```

to find the UI.

And if you see:

```
this.state  
this.props  
this.setState()
```

you know you're looking at the class component programming model.

For new React applications, we'll continue using **function components**.

But if you join a company and discover a React project written several years ago, you may encounter class components everywhere.

Now you'll know how to read them.

Lecture 9 — Components in Separate Files in ReactJS

1. Opening

Hello everyone, and welcome back.

In the previous lectures, we learned about React components.

We started with the basic concept of components.

Then we learned about **Function Components**.

After that, we also looked at **Class Components**, mainly so that you can understand older React applications.

Now we have one very important practical question.

Suppose our application has:

Header
Footer
ProductCard
ProductList
Sidebar
Navbar
Login
Register
Profile
Dashboard

Would we put all of these components inside one file?

Obviously, that would become difficult to manage.

So in today's lecture, we're going to learn how to:

- Create a component in a separate file
- Export a component
- Import a component
- Create a components folder
- Build a proper component structure
- Use multiple components together

- Understand relative import paths
- Understand default exports
- Understand named exports at a basic level
- Organize a small React application

By the end of this lecture, we'll have a much cleaner React project.

Let's get started.

2. Why Separate Components Into Files?

Let's first imagine that everything is inside `App.jsx`.

We might have:

```
function Header() {  
  ...  
}  
  
function ProductCard() {  
  ...  
}  
  
function Footer() {  
  ...  
}  
  
function Sidebar() {  
  ...  
}  
  
function ProductList() {  
  ...  
}  
  
function App() {  
  ...  
}
```

Technically, this can work.

But as the application becomes larger, this file becomes very difficult to maintain.

Imagine having:

App.jsx
↓
500 lines
↓
1000 lines
↓
2000 lines

It becomes difficult to:

- Find a component
- Modify a component
- Reuse a component
- Understand the project
- Work with other developers

So instead, we can create separate files.

For example:

Header.jsx
ProductCard.jsx
Footer.jsx

Each file contains one component.

3. Our First Separate Component

Let's start with a simple example.

Suppose we have:

```
function Header() {  
  return (  
    <header>  
      <h1>  
        My Website  
      </h1>  
    </header>  
  );  
}
```

Instead of keeping this inside `App.jsx`, let's create a new file:

`Header.jsx`

And put the component there.

So our project becomes:

```
src
├──
├── App.jsx
└── Header.jsx
```

Now `Header.jsx` contains:

```
function Header() {

  return (
    <header>

      <h1>
        My Website
      </h1>

    </header>
  );
}
```

But there's one problem.

How does `App.jsx` know about this component?

That's where **export** and **import** come in.

4. Exporting a Component

At the bottom of `Header.jsx`, write:

```
export default Header;
```

So the complete file becomes:

```
function Header() {

  return (
    <header>
```



```
    <h1>
      My Website
    </h1>

  </header>
);
}
```

```
export default Header;
```

The line:

```
export default Header;
```

means:

This file is making Header available to other files.

We can now import this component somewhere else.

5. Importing a Component

Now go to:

App.jsx

At the top:

```
import Header from "../Header";
```

Then use:

```
function App() {

  return (
    <div>

      <Header />

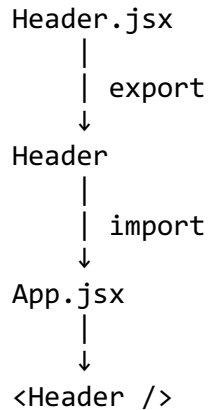
    </div>
  );
}
```

```
export default App;
```

That's it.

We have now moved our component into a separate file.

The flow is:



6. Understanding ./

You may be wondering about this:

```
import Header from "./Header";
```

What does:

```
./
```

mean?

It means:

Look in the current folder.

Suppose our files are:

```
src
├──
├── App.jsx
└── Header.jsx
```

App.jsx and Header.jsx are in the same folder.

So we write:

```
import Header from "./Header";
```

The ./ means the current directory.

7. What About the .jsx Extension?

Our file is:

Header.jsx

But we wrote:

```
import Header from "./Header";
```

We didn't write:

```
import Header from "./Header.jsx";
```

Both can work depending on the project/tooling, but the shorter form is commonly used:

```
import Header from "./Header";
```

The build tool knows how to resolve the component file.

So you'll commonly see:

```
import Header from "./Header";  
import Footer from "./Footer";  
import ProductCard from "./ProductCard";
```

8. Live Coding — Header Component

Let's do this practically.

Open your React project.

Inside src, create:

Header.jsx

Write:

```
function Header() {  
  return (  
    <header>  
      <h1>  
        My Store
```

```
    </h1>

    <nav>
      Home | Products | Contact
    </nav>

  </header>
);
}
```

```
export default Header;
```

Now open:

App.jsx

At the top:

```
import Header from "../Header";
```

Then:

```
function App() {
  return (
    <div>

      <Header />

      <h2>
        Welcome to My Store
      </h2>

    </div>
  );
}
```

```
export default App;
```

Run the application.

You should see:

My Store

Home | Products | Contact

Welcome to My Store

Our component is now stored in a separate file.

9. Creating a Components Folder

For a small project, putting files directly inside `src` is okay.

But as the application grows, we'll have many files.

So a common structure is:

```
src
├── App.jsx
├── components
│   ├── Header.jsx
│   ├── Footer.jsx
│   └── ProductCard.jsx
```

This is cleaner.

Let's create a folder:

`components`

inside:

`src`

So:

`src/components`

Now move `Header.jsx` into:

`src/components/Header.jsx`

Our structure becomes:

```
src
├── App.jsx
└── components
    └── Header.jsx
```

10. Updating the Import Path

Now there is a small change.

Previously:

App.jsx
Header.jsx

were in the same folder.

So we used:

```
import Header from "./Header";
```

But now:

App.jsx
components/
Header.jsx

So we need:

```
import Header from "./components/Header";
```

Let's understand the path.

Starting from:

App.jsx

we go into:

components

and then find:

Header

So:

./components/Header

means:

current folder

↓

components

↓

Header

11. Creating ProductCard

Now let's create another component.

Inside:

src/components

create:

ProductCard.jsx

Write:

```
function ProductCard() {  
  return (  
    <div className="product-card">  
      <h2>  
        iPhone 17  
      </h2>  
      <p>  
        ₹79999  
      </p>  
      <button>  
        Buy Now  
      </button>  
    </div>  
  );  
}
```

```
export default ProductCard;
```

Now import it into App.jsx:

```
import Header from "../components/Header";  
import ProductCard from "../components/ProductCard";
```

Then:

```
function App() {  
  return (  
    <div>
```

```
        <Header />

        <ProductCard />

    </div>
  );
}

export default App;
```

Now our structure is:

```
src
├── App.jsx
└── components
    ├── Header.jsx
    └── ProductCard.jsx
```

12. Creating Footer

Let's create one more component.

File:

src/components/Footer.jsx

Code:

```
function Footer() {

  return (
    <footer>

      <p>
        © 2026 My Store
      </p>

    </footer>
  );
}
```



```
export default Footer;
```

Now import it:

```
import Footer from "../components/Footer";
```

And use it:

```
function App() {  
  return (  
    <div>  
      <Header />  
      <ProductCard />  
      <Footer />  
    </div>  
  );  
}
```

```
export default App;
```

Our project now looks like:

```
src  
├── App.jsx  
└── components  
    ├── Header.jsx  
    ├── ProductCard.jsx  
    └── Footer.jsx
```

13. Understanding the Component Tree

Let's visualize the application.

```
App  
├──  
└──
```



App is the parent component.

Header, ProductCard, and Footer are child components.

Each component lives in its own file:

App



App.jsx

Header



Header.jsx

ProductCard



ProductCard.jsx

Footer



Footer.jsx

This is much easier to manage.

14. Why This Structure Is Better

Suppose tomorrow we want to change the header.

We don't need to search through App.jsx.

We simply open:

Header.jsx

and modify it.

If we want to change the product card:

ProductCard.jsx

If we want to change the footer:

Footer.jsx

Each component has a clear responsibility.

This gives us:

- Better organization
- Easier maintenance
- Reusability
- Easier debugging
- Easier teamwork
- Cleaner code

15. One Component Can Be Reused

Now let's demonstrate an important benefit.

In `App.jsx`, we can write:

```
<ProductCard />  
<ProductCard />  
<ProductCard />
```

Because `ProductCard` is a reusable component.

The result will be:

iPhone 17
₹79999
[Buy Now]

iPhone 17
₹79999
[Buy Now]

iPhone 17
₹79999
[Buy Now]

We're using the same component three times.

But all three products are identical.

This raises an important question:

How can we send different product information to each `ProductCard`?

For example:

iPhone 17
Samsung Galaxy
OnePlus

The answer is:

Props.

And that's exactly what we'll learn in Lecture 10.

16. Components Can Be Imported Into Other Components

The importing doesn't have to happen only in App.jsx.

One component can import another component.

For example:

App
↓
ProductList
↓
ProductCard

We can have:

```
src
├── App.jsx
└── components
    ├── ProductList.jsx
    └── ProductCard.jsx
```

ProductList.jsx can import ProductCard.

For example:

```
import ProductCard from "../ProductCard";

function ProductList() {

  return (
    <div>
```

```
        <ProductCard />

        <ProductCard />

        <ProductCard />

    </div>
  );
}
```

```
export default ProductList;
```

Then App.jsx only needs:

```
import ProductList from "../components/ProductList";
```

and:

```
<ProductList />
```

This allows us to create layers of components.

17. Building a Product List

Let's implement this.

Our structure:

```
src
├── App.jsx
└── components
    ├── Header.jsx
    ├── ProductList.jsx
    ├── ProductCard.jsx
    └── Footer.jsx
```

ProductCard.jsx

```
function ProductCard() {

  return (
    <div>
```

```
        <h2>
            iPhone 17
        </h2>

        <p>
            ₹79999
        </p>

        <button>
            Buy Now
        </button>

    </div>
    );
}

export default ProductCard;

ProductList.jsx
import ProductCard from "../ProductCard";

function ProductList() {

    return (
        <section>

            <h2>
                Products
            </h2>

            <ProductCard />
            <ProductCard />
            <ProductCard />

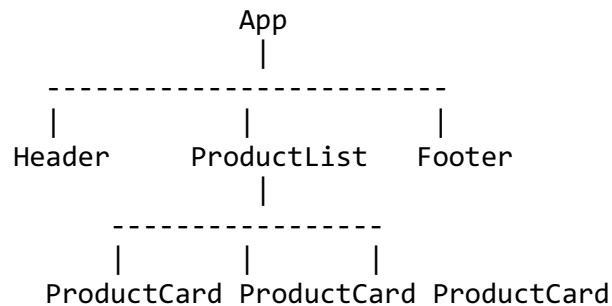
        </section>
    );
}

export default ProductList;

App.jsx
import Header from "../components/Header";
import ProductList from "../components/ProductList";
import Footer from "../components/Footer";
```

```
function App() {  
  return (  
    <div>  
  
      <Header />  
  
      <ProductList />  
  
      <Footer />  
  
    </div>  
  );  
}  
  
export default App;
```

Now look at our component tree:



This is a much more realistic React application structure.

18. Understanding `export default`

Let's focus on this line:

```
export default ProductCard;
```

This is called a **default export**.

A file can have one default export.

For example:

```
function ProductCard() {  
  ...  
}
```

```
export default ProductCard;
```

Then we can import it as:

```
import ProductCard from "../ProductCard";
```

The important thing is that the importing file can choose the local name.

For example:

```
import MyProduct from "../ProductCard";
```

This can work because it is a default export.

But although it is technically possible, it's better practice to keep the name consistent:

```
import ProductCard from "../ProductCard";
```

The component name should tell us what the component actually represents.

19. Named Exports

Now let's briefly introduce another type of export.

We can also use:

```
export function Header() {  
  
    return (  
        <header>  
            Header  
        </header>  
    );  
}
```

This is a **named export**.

Then the import looks different:

```
import { Header } from "../Header";
```

Notice the curly braces:

```
{ Header }
```

Compare:

Default export

```
export default Header;
```

Import:

```
import Header from "./Header";
```

Named export

```
export { Header };
```

or:

```
export function Header() {  
    ...  
}
```

Import:

```
import { Header } from "./Header";
```

For our course, we'll primarily use default exports for components because they are simple and easy to understand.

20. Default vs Named Export

Let's put them side by side.

Default Export

```
function Header() {  
  
    return <h1>Header</h1>;  
}
```

```
export default Header;
```

Import:

```
import Header from "./Header";
```

Named Export

```
export function Header() {
```

```
    return <h1>Header</h1>;  
}
```

Import:

```
import { Header } from "../Header";
```

The curly braces are important.

If you write:

```
import Header from "../Header";
```

for a named export, you'll get an import/export error.

21. Multiple Named Exports

A file can have multiple named exports.

For example:

```
export function Header() {  
    return <h1>Header</h1>;  
}  
  
export function Footer() {  
    return <h1>Footer</h1>;  
}
```

Then:

```
import { Header, Footer } from "../Layout";
```

This is useful in some situations.

But for our React components, we'll generally follow a simple convention:

One component
↓
One file
↓
Default export

For example:

Header.jsx
ProductCard.jsx
Footer.jsx

22. File Naming Convention

Let's discuss naming.

For React component files, you'll commonly see:

Header.jsx
Footer.jsx
ProductCard.jsx
UserProfile.jsx
LoginForm.jsx
ProductList.jsx

Notice that component names use **PascalCase**.

For example:

ProductCard

not:

productcard

and:

UserProfile

not:

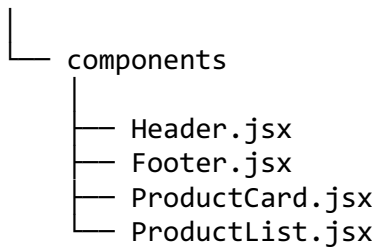
userprofile

This makes it easy to identify component files.

23. Folder Structure for a Small Application

A simple React application might look like:

```
src
├── App.jsx
└── main.jsx
```



The exact structure can vary between projects.

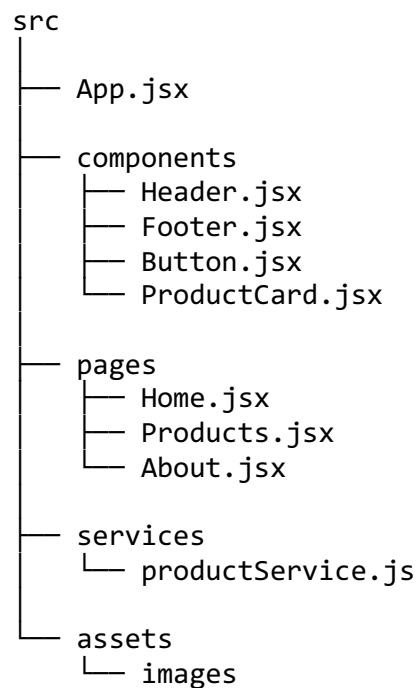
There is no single mandatory folder structure.

The important thing is:

Keep related code organized and easy to find.

24. A Larger Project Structure

As the application grows, you might eventually see something like:



Don't worry about all of these folders yet.

We'll introduce concepts gradually throughout this course.

For now, the important folder is:

components

25. Live Coding — Building a Complete Mini Application

Let's now build a small application from scratch.

Our goal:

My Store

Products

iPhone 17

₹79999

[Buy Now]

Samsung Galaxy

₹59999

[Buy Now]

OnePlus

₹44999

[Buy Now]

© 2026 My Store

For today's lecture, we'll keep the product information hard-coded.

We'll learn how to make the products dynamic using Props in the next lecture.

Step 1 — Create the Folder

Inside src:

components

Step 2 — Create Header

Create:

components/Header.jsx

Code:

```
function Header() {  
  return (  
    <header>  
  
      <h1>  
        My Store  
      </h1>  
  
      <nav>  
  
        <a href="#">  
          Home  
        </a>  
  
        {" | "}  
  
        <a href="#">  
          Products  
        </a>  
  
        {" | "}  
  
        <a href="#">  
          Contact  
        </a>  
  
      </nav>  
  
    </header>  
  );  
}  
  
export default Header;
```

Step 3 — Create ProductCard

Create:

components/ProductCard.jsx

Code:

```
function ProductCard() {  
  
  return (  
    <div>  
  
      <h2>  
        iPhone 17  
      </h2>  
  
      <p>  
        ₹79999  
      </p>  
  
      <button>  
        Buy Now  
      </button>  
  
    </div>  
  );  
}  
  
export default ProductCard;
```

Step 4 — Create ProductList

Create:

components/ProductList.jsx

Code:

```
import ProductCard from "../ProductCard";  
  
function ProductList() {  
  
  return (  
    <main>  
  
      <h2>  
        Products  
      </h2>  
  
      <ProductCard />  
      <ProductCard />  
      <ProductCard />  
    )  
}
```

```
    </main>
  );
}

export default ProductList;
```

Step 5 — Create Footer

Create:

components/Footer.jsx

Code:

```
function Footer() {

  return (
    <footer>

      <p>
        © 2026 My Store
      </p>

    </footer>
  );
}

export default Footer;
```

Step 6 — Update App

Now open:

App.jsx

Write:

```
import Header from "../components/Header";
import ProductList from "../components/ProductList";
import Footer from "../components/Footer";

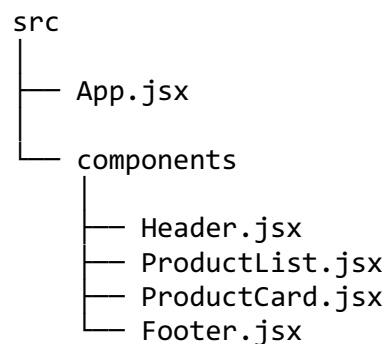
function App() {
```



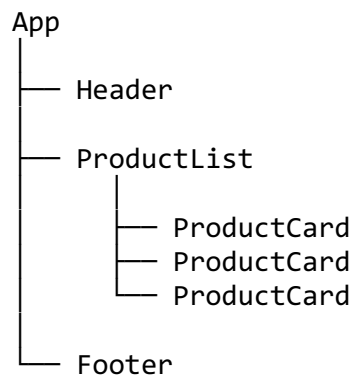
```
    return (  
      <div>  
  
        <Header />  
  
        <ProductList />  
  
        <Footer />  
  
      </div>  
    );  
}  
  
export default App;
```

26. Understanding Our Final Structure

Our project now looks like:



And our component hierarchy is:



This is a clean and understandable component structure.

27. Understanding Import Paths Carefully

Let's look at some common folder structures.

Suppose:

```
src
├── App.jsx
├── components
│   └── Header.jsx
```

From App.jsx:

```
import Header from "../components/Header";
```

Correct.

Now suppose:

```
src
├── App.jsx
├── components
│   ├── Header.jsx
│   └── Product
│       └── ProductCard.jsx
```

From ProductCard.jsx, if we want to import Header, we'd need to go up from Product to components, then to src.

That could look like:

```
import Header from "../../../Header";
```

The .. means:

Go to the parent directory.

So:

.

means:

current directory

and:

..

means:

parent directory

This becomes important as projects get larger.

28. Common Import Path Mistakes

Mistake 1

Writing:

```
import Header from "/components/Header";
```

when you actually need:

```
import Header from "../components/Header";
```

The leading / has a different meaning.

Mistake 2

Using the wrong number of ..

For example, if you're already inside:

src/components

you don't need:

```
../../components/Header
```

unless the directory structure actually requires it.

Always look at where the current file is located.

Mistake 3

Wrong filename capitalization

For example:

ProductCard.jsx

but importing:

```
import ProductCard from "./productcard";
```

Depending on the operating system and environment, this can cause problems.

Keep the filename and import consistent:

```
import ProductCard from "./ProductCard";
```

29. Why One Component Per File?

You might ask:

Is one component per file mandatory?

No.

React doesn't force you to use one component per file.

You technically can have:

```
function Header() {  
  ...  
}  
  
function Footer() {  
  ...  
}
```

in the same file.

But as a project grows, separating components usually makes the code easier to understand.

A useful convention is:

Header.jsx
↓
Header component

Footer.jsx
↓
Footer component

ProductCard.jsx



ProductCard component

This makes the project predictable.

When you want the ProductCard, you know exactly where to look.

30. Components Are Like Building Blocks

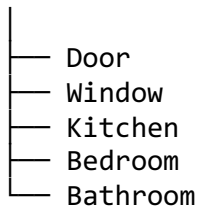
Let's think about components differently.

Imagine we're building a house.

We don't create the entire house as one giant object.

We have:

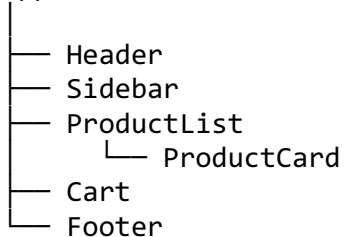
House



React applications work similarly.

We can have:

Application



Each component has a responsibility.

This is one of the most important ideas in React:

Break a large UI into smaller reusable components.

31. Component Reusability

Let's return to our ProductCard.

Currently:

```
<ProductCard />
```

always shows:

iPhone 17
₹79999

We can reuse it:

```
<ProductCard />  
<ProductCard />  
<ProductCard />
```

But we want:

iPhone 17
₹79999

Samsung Galaxy
₹59999

OnePlus
₹44999

We don't want to create:

```
iPhoneCard.jsx  
SamsungCard.jsx  
OnePlusCard.jsx
```

That would defeat the purpose of reusable components.

Instead, we'll create one:

```
ProductCard.jsx
```

and provide different data to it.

We'll do that with:

Props

32. Preview of Props

Let's just preview the concept.

We could eventually write:

```
<ProductCard  
  name="iPhone 17"  
  price={79999}  
>
```

and:

```
<ProductCard  
  name="Samsung Galaxy"  
  price={59999}  
>
```

and:

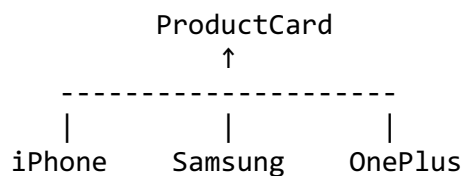
```
<ProductCard  
  name="OnePlus"  
  price={44999}  
>
```

Same component:

ProductCard.jsx

Different data.

Conceptually:



This is one of the major benefits of React components.

And this is exactly what we'll learn in the next lecture.

33. Common Beginner Mistakes

Let's review some common mistakes from today's topic.

Mistake 1 — Forgetting to export

If you create:

```
function Header() {  
  ...  
}
```

in `Header.jsx`, remember:

```
export default Header;
```

Mistake 2 — Forgetting to import

If `App.jsx` uses:

```
<Header />
```

it must have the appropriate import:

```
import Header from "../components/Header";
```

Mistake 3 — Wrong path

If the file is:

`src/components/Header.jsx`

and `App.jsx` is inside `src`, use:

```
import Header from "../components/Header";
```

Mistake 4 — Incorrect component name

If you have:

```
function ProductCard() {  
  ...  
}
```

use:

```
<ProductCard />
```

not:

<Productcard />

Keep naming consistent.

Mistake 5 — Forgetting curly braces for named exports

If you have:

```
export function Header() {  
  ...  
}
```

you import it using:

```
import { Header } from "../Header";
```

not:

```
import Header from "../Header";
```

34. What We Learned Today

Let's summarize.

1. Components can be placed in separate files.

```
Header.jsx  
Footer.jsx  
ProductCard.jsx
```

2. We export components.

```
export default Header;
```

3. We import components.

```
import Header from "../components/Header";
```

4. ./ means the current directory.

```
./components/Header
```

5. .. means the parent directory.

```
../Header
```

6. A components folder helps organize the project.

```
src
├── components
│   ├── Header.jsx
│   ├── Footer.jsx
│   └── ProductCard.jsx
```

7. Components can import other components.

```
import ProductCard from "../ProductCard";
```

8. Components can be reused.

```
<ProductCard />
<ProductCard />
<ProductCard />
```

9. Props allow us to make reusable components dynamic.

We'll learn that next.

35. Student Exercise

For today's exercise, create a small **Student Management UI**.

Your structure should be:

```
src
├── App.jsx
└── components
    ├── Header.jsx
    ├── StudentList.jsx
    ├── StudentCard.jsx
    └── Footer.jsx
```

Your component tree should be:

```
App
├── Header
├── StudentList
│   ├── StudentCard
│   └── StudentCard
```

└─ StudentCard
└─ Footer

Header

Display:

ABC College
Home | Students | Courses

StudentCard

Display:

Name: Rahul
Course: BCA
Semester: 4

[View Details]

Footer

Display:

© 2026 ABC College

StudentList

Render three StudentCard components.

For now, all three can display the same information.

Bonus Challenge

Try to create another component:

Button.jsx

and use it inside StudentCard.

This will give you practice with components importing other components.

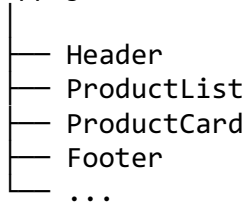
36. Final Takeaway

Let's finish with one simple idea.

A React application is made up of components.

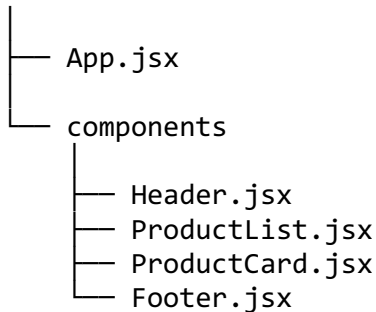
Instead of putting everything in one file:

App.jsx



we can organize it:

src



And connect them using:

```
export default ComponentName;
```

and:

```
import ComponentName from "../path";
```

So remember this flow:

Create Component



Put Component in a File



Export Component



Import Component



Use Component

This simple mechanism is one of the foundations of organizing React applications.

And now we're ready for one of the most important concepts in React:

Props.

Until now, our ProductCard has always displayed the same product.

Lecture 10 — React Props

1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned how to create React components in separate files.

We created components such as:

```
text id="l0w6w3" Header.jsx ProductList.jsx ProductCard.jsx Footer.jsx
```

We also learned how to export and import components.

And we saw that we can reuse a component multiple times:

```
jsx id="j9f4rj" <ProductCard /> <ProductCard /> <ProductCard />
```

But we had a problem.

All three ProductCard components displayed exactly the same information.

For example:

```
text id="w5rv1a" iPhone 17 ₹79999
```

What if we want:

```
` `` text id="o4y9fa" iPhone 17 ₹79999
```

Samsung Galaxy ₹59999

OnePlus ₹44999

Do we need to create three different components?

No.

Instead, we can send data from the parent component to the child component.

And React gives us a mechanism for doing this.

It is called:

```
> **Props**
```

In today's lecture, we'll learn:

- What Props are
- Why Props are needed
- How to pass Props
- How to receive Props
- Accessing Props inside a component
- Passing strings, numbers, booleans, arrays, and objects
- Destructuring Props
- Passing multiple Props
- Passing Props to child components
- Props are read-only
- One-way data flow
- Common Props mistakes

Let's get started.

2. What Are Props?

Let's start with a simple definition.

> ****Props are data passed from a parent component to a child component.****

The word Props is short for:

> ****Properties****

Think about a component as a reusable template.

For example:

```
```jsx id="n1b9js"
function ProductCard() {

 return (
 <div>

 <h2>iPhone 17</h2>

 <p>₹79999</p>

 </div>
)
}
```

```
);
 }
```

The problem is that the component is fixed.

It always displays:

```
text id="d5n3m0" iPhone 17 ₹79999
```

Instead, we can make it dynamic.

We can write:

```
jsx id="z7up7m" <ProductCard name="iPhone 17" price={79999} />
```

and:

```
jsx id="7g5f5c" <ProductCard name="Samsung Galaxy" price={59999} />
```

Now the same component can display different information.

That's the purpose of Props.

---

### 3. Parent and Child Components

Before we write code, we need to understand two terms.

Suppose we have:

```
`` `jsx id="1h7k4d" function App() {
 return (
 <ProductCard />
);
}
```

Here:

```
`` `text id="6m0y6r"
App
↓
ProductCard
```

App is the **parent**.

ProductCard is the **child**.

If App sends information to ProductCard, we call that information:

Props.

So:

text id="m5i0by" Parent      |      | Props      ↓ Child

This is one of the most important relationships in React.

---

## 4. Our First Props Example

Let's create a simple component.

### ProductCard.jsx

```
` ` `jsx id="i6akgk" function ProductCard(props) {
 return (
 <div>
 <h2>
 {props.name}
 </h2>

 <p>
 ₹{props.price}
 </p>

 </div>
);
}

export default ProductCard;
```

Now in `App.jsx`:

```
` ` `jsx id="d8n0p2"
import ProductCard from "../components/ProductCard";

function App() {
```



```
 return (
 <div>

 <ProductCard
 name="iPhone 17"
 price={79999}
 />

 </div>
);
 }
}
```

export default App;

Run the application.

We get:

text id="85xw9u" iPhone 17 ₹79999

We have successfully passed Props.

---

## 5. Understanding the Syntax

Let's focus on this:

```
jsx id="j2f0qt" <ProductCard name="iPhone 17" price={79999} />
```

Here:

text id="u1xxgr" name

is a Prop.

And:

text id="n3j6be" price

is another Prop.

Conceptually:

```
text id="jv3y7o" ProductCard ↑ | | name = "iPhone 17" | price
= 79999 | App
```

The parent is sending:

text id="0c7a11" name price  
to the child.

---

## 6. Receiving Props

Now let's look at the child.

We wrote:

```
jsx id="q4t8od" function ProductCard(props) {
```

The props parameter contains the values sent by the parent.

So:

```
jsx id="q3f8ps" props.name
```

gives us:

```
text id="fzf78w" iPhone 17
```

And:

```
jsx id="a2c6km" props.price
```

gives us:

```
text id="0o2y8n" 79999
```

Therefore:

```
` `` `jsx id="ydh9y0"
```

```
{props.name}
```

```
₹{props.price}
```

becomes:

```
` `` `text id="42o3ed"
iPhone 17
₹79999
```

---

## 7. Passing Multiple Props

A component can receive many Props.

For example:

```
jsx id="z9cl9g" <ProductCard name="iPhone 17" price={79999}
brand="Apple" category="Mobile" inStock={true} />
```

Now the child receives:

```
text id="xqj7us" name price brand category inStock
```

We can use them:

```
`` `jsx id="xky5lc" function ProductCard(props) {
 return (
 <div>

 <h2>
 {props.name}
 </h2>

 <p>
 Brand: {props.brand}
 </p>

 <p>
 Category: {props.category}
 </p>

 <p>
 Price: ₹{props.price}
 </p>

 <p>
 In Stock: {props.inStock ? "Yes" : "No"}
 </p>

 </div>
);
}
```

Now our component is much more flexible.

---

## # 8. Props Can Be Strings

The simplest Prop is a string.

For example:

```
```jsx id="2os7mt"
<ProductCard
  name="iPhone 17"
/>
```

Inside:

```
```jsx id="zuwr7u" function ProductCard(props) {
return (
 <h2>
 {props.name}
 </h2>
);
}
```

Output:

```
```text id="t3n9ce"
iPhone 17
```

String Props are usually written with quotes:

```
jsx id="0x06d5" name="iPhone 17"
```

9. Props Can Be Numbers

Now suppose we want to pass a number.

We can write:

```
jsx id="g2u6fz" price={79999}
```

Notice the curly braces.

Why?

Because:

```
jsx id="t2ahz5" price="79999"
```

is a string.

While:

```
jsx id="i6q3q1" price={79999}
```

is a JavaScript number.

This difference can matter when performing calculations.

For example:

```
jsx id="2agf6d" <p> Price with tax: ₹{props.price * 1.18} </p>
```

If:

```
text id="u0pkhh" price = 79999
```

React evaluates:

```
text id="d7m0sj" 79999 × 1.18
```

and displays the result.

10. Props Can Be Booleans

We can also pass boolean values.

For example:

```
jsx id="7pjr3m" <ProductCard name="iPhone 17" inStock={true} />
```

Or:

```
jsx id="rcx2qs" <ProductCard name="iPhone 17" inStock={false} />
```

Then:

```
` ` `jsx id="qrrzpm" function ProductCard(props) {  
  return (  
    <div>  
  
      <h2>
```

```
        {props.name}
      </h2>

      <p>
        {props.inStock
          ? "Available"
          : "Out of Stock"}
      </p>

    </div>
  );
}
```

If `inStock` is `true`:

```
```text id="h2h2h6"
Available
```

If it is false:

```
text id="yr2l2o" Out of Stock
```

---

## 11. Props Can Be Arrays

Props aren't limited to primitive values.

We can pass an array.

For example:

```
jsx id="k9s7vy" <ProductCard name="iPhone 17" colors={["Black",
"White", "Blue"]} />
```

Inside:

```
```jsx id="f9sp1j" function ProductCard(props) {
  return (
    <div>

      <h2>
        {props.name}
      </h2>
```

```
    <p>
      Available Colors:
    </p>

    <p>
      {props.colors.join(", ")}
    </p>

  </div>
);
}
```

Output:

```
```text id="s2a0cs"
iPhone 17
```

Available Colors:

Black, White, Blue

We'll learn more about rendering arrays later in the series.

For now, understand that Props can contain arrays.

---

## 12. Props Can Be Objects

We can also pass an object.

For example:

```
jsx id="oxfj1a" <ProductCard product={{ name: "iPhone 17",
price: 79999, brand: "Apple" }} />
```

Inside:

```
```jsx id="y9os2g" function ProductCard(props) {
  return (
    <div>

      <h2>
```

```
        {props.product.name}  
      </h2>  
  
      <p>  
        ₹{props.product.price}  
      </p>  
  
      <p>  
        {props.product.brand}  
      </p>  
  
    </div>  
  );  
}
```

The object is available through:

```
```jsx id="z0c0m7"  
props.product
```

and its properties:

```
jsx id="l3r4qk" props.product.name props.product.price props.product.brand
```

We'll use objects frequently when working with APIs later in this course.

---

## 13. Props With JavaScript Expressions

Remember from our JSX lecture that curly braces allow JavaScript expressions.

The same concept applies to Props.

For example:

```
```jsx id="f4w8x5" const productPrice = 79999;
```

We can also write:

```
```jsx id="l2n9fa"
```



```
<ProductCard
 name={"iPhone " + 17}
>
```

Or:

```
jsx id="r1j2u4" <ProductCard price={50000 + 29999} />
```

React evaluates the JavaScript expression and passes the result.

So curly braces mean:

Evaluate this as JavaScript.

## 14. The Most Important Example — Reusable ProductCard

Let's create a realistic example.

### ProductCard.jsx

```
`` `jsx id="xg5h4e" function ProductCard(props) {
 return (
 <div className="product-card">

 <h2>
 {props.name}
 </h2>

 <p>
 Brand: {props.brand}
 </p>

 <p>
 Price: ₹{props.price}
 </p>

 <button>
 Buy Now
 </button>

 </div>
);
};
```

```
}
```

```
export default ProductCard;
```

Now in `App.jsx`:

```
```jsx id="yk2p7a"  
import ProductCard from "../components/ProductCard";
```

```
function App() {  
  return (  
    <div>  
  
      <h1>  
        My Store  
      </h1>  
  
      <ProductCard  
        name="iPhone 17"  
        brand="Apple"  
        price={79999}  
      />  
  
      <ProductCard  
        name="Galaxy S26"  
        brand="Samsung"  
        price={69999}  
      />  
  
      <ProductCard  
        name="OnePlus 14"  
        brand="OnePlus"  
        price={54999}  
      />  
  
    </div>  
  );  
}
```

```
export default App;
```

Now we have one reusable component:

```
text id="c7s9on" ProductCard
```

but three different outputs.

15. Understanding What React Is Doing

Let's visualize it.

We have:

```
jsx id="9ppf2f" <ProductCard      name="iPhone 17"      brand="Apple"
price={79999} />
```

React passes this data to:

```
jsx id="s14tx4" function ProductCard(props)
```

Conceptually:

```
text id="h0uv0v" props | |— name → "iPhone 17" |— brand → "Apple" |— price
→ 79999
```

Then inside the component:

```
jsx id="g2om7u" props.name
```

gives:

```
text id="3nttbr" iPhone 17
```

and:

```
jsx id="x5gk1e" props.price
```

gives:

```
text id="7exf1b" 79999
```

This is the basic Props mechanism.

16. Props Are Read-Only

This is a very important rule.

Props should be treated as read-only.

Suppose we have:

```
``jsx id="ocj2a9" function ProductCard(props) {  
  return (  
    <h2>  
      {props.name}  
    </h2>  
  );  
}
```

We should not try to modify:

```
``jsx id="p4y2h4"  
props.name = "Samsung";
```

The child should not change the Props it receives.

Think of Props as:

text id="6j3z4n" Parent | | sends data ↓ Child

The child receives the data.

It doesn't directly modify the parent's data.

We'll later learn how a child can communicate back to a parent using callback functions.

But for now, remember:

Props flow from parent to child.

17. One-Way Data Flow

React follows a concept called:

One-way data flow

Data normally moves:

text id="m6g9q8" Parent ↓ Child ↓ Grandchild

For example:

text id="7v0j7f" App ↓ ProductList ↓ ProductCard

The App can send data to ProductList.

ProductList can send data to ProductCard.

Conceptually:

text id="zq2x4n" App | | Props ↓ ProductList | | Props ↓ ProductCard

This predictable flow makes React applications easier to understand.

18. Props With a Child Component

Let's create a small example.

App.jsx

```
```jsx id="r4m8kj" function App() {  
 return (
 <UserCard
 name="Rahul"
 city="Rajkot"
 />
);
}
```

### ### UserCard.jsx

```
```jsx id="d4x2o8"  
function UserCard(props) {  
  return (  
    <div>  
      <h2>  
        {props.name}  
      </h2>  
      <p>  
        City: {props.city}  
      </p>  
    </div>  
  );  
}
```

The parent:

text id="0y6bjy" App

sends:

text id="z4bq99" name city

to:

text id="8s2m4y" UserCard

19. Destructuring Props

So far we've been writing:

```
```jsx id="qg1e3s" function ProductCard(props) {  
 return (
 <div>

 <h2>{props.name}</h2>

 <p>{props.price}</p>

 </div>
);
}
```

This is perfectly valid.

But there's a cleaner syntax.

We can destructure the Props:

```
```jsx id="6d8u8n"  
function ProductCard({ name, price }) {  
  
  return (  
    <div>  
  
      <h2>  
        {name}  

```

```
        </h2>

        <p>
            ₹{price}
        </p>

    </div>
    );
}
```

This is JavaScript object destructuring.

Instead of:

```
jsx id="q6v8nt" props.name props.price
```

we directly get:

```
jsx id="1q44d7" name price
```

20. Understanding Destructuring

Suppose the parent sends:

```
jsx id="0b22a9" <ProductCard      name="iPhone 17"      price={79999} />
```

React gives the child something conceptually like:

```
{
  name: "iPhone 17",
  price: 79999
}
```

Then:

```
jsx id="u9r4v5" function ProductCard({ name, price }) {
```

extracts:

```
text id="x9x7j3" name price
```

from that object.

So:

```
jsx id="g6v0f1" {name, price}
```

is shorthand for accessing the relevant properties.

21. Which Syntax Should We Use?

Both are correct.

Using props

```
```jsx id="2c6o5s" function ProductCard(props) {  
 return (
 <h2>
 {props.name}
 </h2>
);
}
```

### ### Destructuring

```
```jsx id="4o3p7c"  
function ProductCard({ name }) {  
  return (  
    <h2>  
      {name}  
    </h2>  
  );  
}
```

For beginners, I recommend understanding the first version first:

```
jsx id="k13wce" props.name
```

because it makes the concept very explicit.

Once you understand Props, destructuring becomes easy.

In modern React code, you'll frequently see:

```
jsx id="9evjkm" function ProductCard({ name, price }) {
```

22. Default Values for Props

Sometimes a Prop may not be provided.

For example:

```
jsx id="7t3n2c" <ProductCard />
```

But our component expects:

```
jsx id="k6g0sa" name price
```

We can provide default values using destructuring:

```
`` `jsx id="qj6e9v" function ProductCard({ name = "Unknown Product", price = 0 }) {  
  return (  
    <div>  
  
      <h2>  
        {name}  
      </h2>  
  
      <p>  
        ₹{price}  
      </p>  
  
    </div>  
  );  
}
```

If no Props are provided, we get:

```
`` `text id="e9jz8n"  
Unknown Product  
₹0
```

This is useful when a component needs sensible fallback values.

23. Boolean Props — Shorthand Syntax

There is a convenient syntax for boolean Props.

Instead of:

```
jsx id="t8u6q2" <ProductCard      inStock={true} />
```

we can write:

```
jsx id="m9u4by" <ProductCard      inStock />
```

This means:

```
text id="x6r5lo" inStock = true
```

For example:

```
```jsx id="1f9v5q" function ProductCard({ inStock }) {  
 return (
 <p>
 {inStock
 ? "Available"
 : "Out of Stock"}
 </p>
);
}
```

Then:

```
```jsx id="r8h4c6"  
<ProductCard inStock />
```

means:

```
text id="9j1b7f" inStock = true
```

24. Passing a Function as a Prop

This is an important concept, although we won't go too deeply into event communication yet.

Props don't only contain data.

We can also pass a function.

For example:

```
```jsx id="2gt5bw" function App() {  
 function handleClick() {
 console.log("Button clicked");
```

```
}

return (
 <ProductCard
 onBuy={handleClick}
 />
);
}
```

The child receives it:

```
```jsx id="t0j5xk"  
function ProductCard(props) {  
  
  return (  
    <button onClick={props.onBuy}>  
      Buy Now  
    </button>  
  );  
}
```

Now the child can call the function provided by the parent.

Conceptually:

```
text id="ppr3wq" Parent    |    | function    ↓ Child    |    | calls  
function    ↓ Parent logic executes
```

This becomes very important when we build interactive applications.

We'll revisit this concept later.

25. Props Are Not the Same as State

Students often confuse these two concepts.

Let's keep it simple.

Props

Props are:

Data received by a component from its parent.

Example:

```
jsx id="qwdl0x" <ProductCard      name="iPhone 17" />
```

The child receives:

```
jsx id="s9i5a8" props.name
```

State

State is:

Data managed by the component itself.

For example:

```
jsx id="n5f0u1" const [count, setCount] = useState(0);
```

We will study State in detail later in the useState lecture.

For now, remember:

```text id="f4b8j7" Props ↓ Passed into component

State ↓ Managed by component

---

## # 26. Props vs Hard-Coded Values

Let's understand why Props are useful.

Without Props:

```
```jsx id="l6p1g2"
function ProductCard() {

    return (
        <div>

            <h2>
                iPhone 17
            </h2>

            <p>
```

```

        ₹79999
      </p>

    </div>
  );
}

```

This component only represents one product.

With Props:

```

```jsx id="uwv7xk" function ProductCard({ name, price }) {
 return (
 <div>

 <h2>
 {name}
 </h2>

 <p>
 ₹{price}
 </p>

 </div>
);
}

```

Now:

```

```jsx id="at0v8j"
<ProductCard
  name="iPhone 17"
  price={79999}
/>

```

or:

```
jsx id="gk0v3w" <ProductCard    name="Samsung Galaxy"    price={59999} />
```

or:

```
jsx id="p2n5jq" <ProductCard    name="OnePlus"    price={44999} />
```

Same component.

Different data.

That's the power of Props.

27. Live Coding — Student Card

Let's build another example.

Create:

text id="byb6ax" StudentCard.jsx

Code:

```
```jsx id="j7y2rx" function StudentCard(props) {
 return (
 <div>

 <h2>
 {props.name}
 </h2>

 <p>
 Course: {props.course}
 </p>

 <p>
 Semester: {props.semester}
 </p>

 </div>
);
}

export default StudentCard;
```

Now in `App.jsx`:

```
```jsx id="5k8vaz"
import StudentCard from "../components/StudentCard";

function App() {

  return (
```

```
<div>

  <h1>
    Students
  </h1>

  <StudentCard
    name="Rahul"
    course="BCA"
    semester={4}
  />

  <StudentCard
    name="Priya"
    course="MCA"
    semester={2}
  />

  <StudentCard
    name="Amit"
    course="B.Tech"
    semester={6}
  />

</div>
);
}
```

```
export default App;
```

The result is:

```
` `` `text id="txj1hh" Students
```

```
Rahul Course: BCA Semester: 4
```

```
Priya Course: MCA Semester: 2
```

```
Amit Course: B.Tech Semester: 6
```

One component:

```
` `` `text id="s1ul4j"
StudentCard
```

Three sets of data.

This is exactly what Props are designed for.

28. Passing an Object as One Prop

There is another useful pattern.

Instead of:

```
jsx id="qf19n6" <StudentCard      name="Rahul"      course="BCA"
semester={4} />
```

we could create an object:

```
jsx id="5mq4fr" const student = {      name: "Rahul",      course: "BCA",
semester: 4 };
```

Then:

```
jsx id="7gb7v2" <StudentCard      student={student} />
```

The child:

```
`` `jsx id="g2n8yb" function StudentCard({ student }) {
  return (
    <div>

      <h2>
        {student.name}
      </h2>

      <p>
        Course: {student.course}
      </p>

      <p>
        Semester: {student.semester}
      </p>

    </div>
  );
}
```

This approach becomes especially useful when working with API responses.

Later in the series, we'll consume API data.

You'll often have objects like:

```
```javascript
{
 id: 101,
 name: "Rahul",
 email: "rahul@example.com"
}
```

and pass them to components.

---

## 29. Props With Children

React also provides a special Prop called:

`text id="6lv0yl" children`

For example:

```
```jsx id="4t1fdd" function Card(props) {
  return (
    <div className="card">

      {props.children}

    </div>
  );
}
```

Then we can write:

```
```jsx id="wqkj8a"
<Card>

 <h2>
 Product
 </h2>
```

```
<p>
 ₹79999
</p>
```

</Card>

Everything placed between:

```
jsx id="32ys8c" <Card>
```

and:

```
jsx id="3qpcg4" </Card>
```

is available through:

```
jsx id="t5r0w6" props.children
```

So:

```
text id="2dy7pf" <Card> Some JSX </Card> ↓ props.children
```

We won't go deeply into this today, but you should recognize it because you'll see it frequently in React applications.

---

## 30. Common Props Mistakes

Let's look at some common mistakes.

### Mistake 1 — Forgetting curly braces for numbers

This:

```
jsx id="2s6b7v" <ProductCard price="79999" />
```

passes a string.

This:

```
jsx id="u3m5o4" <ProductCard price={79999} />
```

passes a number.

---

### Mistake 2 — Trying to modify Props

Avoid:

```
jsx id="h6m4s1" props.name = "New Name";
```

Props should be treated as read-only.

---

## Mistake 3 — Wrong Prop name

Parent:

```
jsx id="i5q1os" <ProductCard productName="iPhone 17" />
```

Child:

```
```jsx id="e7u8k1" function ProductCard(props) {  
  return <h2>{props.name}</h2>;  
}
```

This won't work because:

```
```text id="u7gqgl"  
productName
```

and:

```
text id="m1s5x4" name
```

are different properties.

You need:

```
jsx id="x9h0an" props.productName
```

or change the Prop name.

---

## Mistake 4 — Forgetting to pass the Prop

If your component expects:

```
jsx id="1egv6w" function ProductCard({ name }) {
```

but you render:

```
jsx id="q9c5s1" <ProductCard />
```

then name will be undefined unless you provide a default value.

---

## 31. Best Practices

A few simple practices will help you write better React code.

### Use meaningful Prop names

Good:

```
jsx id="5d6s0g" <ProductCard productName="iPhone 17"
productPrice={79999} />
```

Less clear:

```
jsx id="0u8n9h" <ProductCard x="iPhone 17" y={79999} />
```

---

### Keep components reusable

Instead of:

```
jsx id="h6s4av" function IphoneCard() { ... }
```

prefer:

```
jsx id="2x6b1w" function ProductCard({ name, price }) { ... }
```

Now the component can represent any product.

---

### Keep data and presentation separate

The parent can provide:

```
jsx id="shy0ob" name price brand
```

The child decides how that information should be displayed.

This separation makes components reusable.

---

## 32. A Complete Example

Let's now create a complete mini application.

## ProductCard.jsx

```
`` `jsx id="c2xw1d" function ProductCard({ name, brand, price, inStock }) {
 return (
 <div className="product-card">

 <h2>
 {name}
 </h2>

 <p>
 Brand: {brand}
 </p>

 <p>
 Price: ₹{price}
 </p>

 <p>
 Status:{" "}
 {inStock
 ? "Available"
 : "Out of Stock"}
 </p>

 <button
 disabled={!inStock}
 >
 Buy Now
 </button>

 </div>
);
}
export default ProductCard;
```

Now:

### App.jsx

```
`` `jsx id="x6o8z9"
```

```
import ProductCard from "../components/ProductCard";
```

```
function App() {
 return (
 <div>

 <h1>
 My Store
 </h1>

 <ProductCard
 name="iPhone 17"
 brand="Apple"
 price={79999}
 inStock={true}
 />

 <ProductCard
 name="Galaxy S26"
 brand="Samsung"
 price={69999}
 inStock={true}
 />

 <ProductCard
 name="OnePlus 14"
 brand="OnePlus"
 price={54999}
 inStock={false}
 />

 </div>
);
}
```

```
export default App;
```

Notice what's happening.

The parent controls the data:

```
text id="psqkqa" App | ─ iPhone 17 ─ Galaxy S26 ─ OnePlus 14
```

The child controls the presentation:

```
text id="2xqj4f" ProductCard | ─ Name ─ Brand ─ Price ─ Status ─
Buy Button
```

This separation is extremely useful.

---

## 33. Important Concept — Parent Controls Data

Let's make this concept very clear.

Suppose:

```
jsx id="k7m5pw" <ProductCard name="iPhone 17" price={79999} />
```

The parent decides:

```
text id="f9f1h0" name = iPhone 17 price = 79999
```

The child receives the values.

The child decides how to display them:

```
```jsx id="z1y7m3"
```

```
{name}
```

```
₹{price}
```

So:

```
```text id="z9x9e1"
```

Parent



Data



Child



UI



Browser

This separation is fundamental to React.

---

## 34. What Happens When Props Change?

Suppose the parent initially renders:

```
jsx id="i0q8tw" <ProductCard name="iPhone 17" price={79999} />
```

Later, the parent provides:

```
jsx id="d4byc0" <ProductCard name="iPhone 17" price={74999} />
```

The child receives the updated Prop:

```
text id="2z1i0r" price 74999
```

React updates the component's UI.

So the child doesn't need to know where the data came from.

It simply receives the current Props.

This is another reason Props are powerful.

---

## 35. Props and Component Reusability

Let's look at the bigger picture.

Without Props:

```
text id="e0i4am" IphoneCard SamsungCard OnePlusCard
```

Three components.

With Props:

```
text id="2y0h3w" ProductCard
```

One component.

And:

```
jsx id="bd2k2j" <ProductCard name="iPhone 17" /> <ProductCard name="Samsung Galaxy" /> <ProductCard name="OnePlus" />
```

This is a major React principle:

**Build reusable components and provide them with different data through Props.**

---



## 36. What We Learned Today

Let's summarize everything.

### 1. Props means Properties.

They allow us to pass data to components.

---

### 2. Props flow from parent to child.

text id="txqv71" Parent    ↓ Props    ↓ Child

---

### 3. Props are passed as attributes.

```
jsx id="ip6s5b" <ProductCard name="iPhone 17" price={79999} />
```

---

### 4. Props are received through the component parameter.

```
jsx id="p0rrp4" function ProductCard(props) {
```

---

### 5. We access Props using:

```
jsx id="91j93d" props.name props.price
```

---

### 6. We can destructure Props.

```
jsx id="m40h74" function ProductCard({ name, price }) {
```

---

### 7. Props can contain different data types.

For example:

```
text id="azv7i9" String Number Boolean Array Object Function JSX
```

---

## 8. Props are read-only.

A child should not directly modify its Props.

---

## 9. Props make components reusable.

One:

```
jsx id="c1v3q6" <ProductCard />
```

can represent many different products.

---

## 37. Student Exercise

Now it's time for today's exercise.

Create a reusable:

```
text id="j1n4m5" EmployeeCard
component.
```

The component should accept:

```
text id="r5r4q2" name designation department salary isActive
```

For example:

```
jsx id="r3u4cf" <EmployeeCard name="Rahul" designation="Software
Developer" department="IT" salary={50000} isActive={true} />
```

Display:

```
` `` text id="zq2fmb" Rahul
```

Software Developer IT

Salary: ₹50000

Status: Active

Create three employees with different information.

---

### # 38. Bonus Exercise – ProductCard

Modify your previous `ProductCard`.

It should accept:

```
```text id="u5z9bp"
name
brand
price
category
inStock
```

For example:

```
jsx id="b7f8yy" <ProductCard    name="iPhone 17"    brand="Apple"
price={79999}    category="Smartphone"    inStock={true} />
```

Display all the information.

Then create at least three products.

39. Final Takeaway

Let's finish today's lecture with one simple example.

We have a reusable component:

```
```jsx id="4s5zhc" function ProductCard({ name, price }) {
return (
 <div>

 <h2>
 {name}
 </h2>

 <p>
 ₹{price}
 </p>

 </div>
);
}
```

And the parent provides the data:

```

`jsx id="0gsh0r"
<ProductCard
 name="iPhone 17"
 price={79999}
/>

<ProductCard
 name="Samsung Galaxy"
 price={59999}
/>

<ProductCard
 name="OnePlus"
 price={44999}
/>

```

We have:

```

text id="n7q1h4" ProductCard ↑
|-----| | | |
Samsung OnePlus | | iPhone

```

One component.

Different data.

That's the core idea behind Props.

Remember this sentence:

**Props allow a parent component to pass data to a child component.**

# Lecture 11 — React DevTools

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned about **Props**.

We created reusable components and passed data from a parent component to a child component.

For example:

```
<ProductCard
 name="iPhone 17"
 price={79999}
>
```

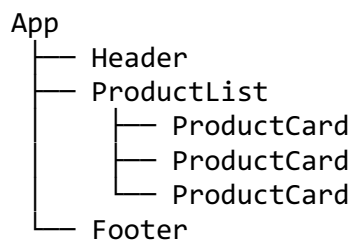
And inside the component:

```
function ProductCard({ name, price }) {

 return (
 <div>
 <h2>{name}</h2>
 <p>₹{price}</p>
 </div>
);
}
```

Now imagine that our application becomes bigger.

We may have:



And something isn't working.

Maybe the wrong product name is displayed.

Maybe the price is incorrect.

Maybe a Prop isn't reaching the child component.

Maybe we want to know which component is currently rendered.

How do we investigate these problems?

We could put lots of:

```
console.log()
```

statements everywhere.

But React provides a much better tool for understanding React applications.

It is called:

**React Developer Tools**, or simply **React DevTools**.

In today's lecture, we'll learn:

- What React DevTools is
- Why we need it
- How to install it
- How to open it
- Components panel
- Component tree
- Inspecting Props
- Inspecting component data
- Selecting components from the page
- Understanding parent-child relationships
- Using DevTools for debugging
- A practical debugging example

Let's get started.

---

## 2. What Is React DevTools?

React DevTools is a browser developer tool designed specifically for React applications.

Normally, when we open browser Developer Tools, we see things such as:

Elements  
Console  
Network

## Sources Application

These are general browser developer tools.

React DevTools adds React-specific information.

It allows us to inspect things such as:

Components  
Props  
State  
Component hierarchy

Instead of looking only at HTML generated by React, we can look at the actual React component structure.

This is extremely useful when developing and debugging React applications.

---

## 3. Browser DevTools vs React DevTools

Let's understand the difference.

Suppose we have:

```
<ProductCard
 name="iPhone 17"
 price={79999}
>
```

The browser eventually renders HTML such as:

```
<div>
 <h2>iPhone 17</h2>
 <p>₹79999</p>
</div>
```

If we inspect this using the browser's **Elements** tab, we see the HTML.

But we don't directly see:

```
ProductCard
name = "iPhone 17"
price = 79999
```

React DevTools lets us inspect the React component itself.

So:

Browser DevTools



HTML / CSS / JavaScript / Network

React DevTools



React Components / Props / State / Hierarchy

Both tools are useful.

They solve different problems.

---

## 4. Installing React DevTools

React DevTools is available as a browser extension for supported browsers.

For this lecture, we'll use a Chromium-based browser such as Chrome.

Open your browser's extension store and search for:

React Developer Tools

Install the official React Developer Tools extension.

After installation, restart or refresh your React application.

You may notice a React icon in the browser toolbar or the DevTools interface.

---

## 5. Opening Browser Developer Tools

Let's open our React application.

For example, our application may be running at:

`http://localhost:5173`

This is the development server created by our React project.

Now open browser Developer Tools.

You can use:

Right Click → Inspect

or the browser's keyboard shortcut.



Once Developer Tools opens, look for tabs related to React.

Depending on the browser and DevTools version, you may see:

Components

Profiler

The exact appearance can change between versions, but the important tabs for us are the React-specific ones.

---

## 6. Our Practice Application

Before using DevTools, let's create a small application.

We'll use the same ProductCard concept from our previous lecture.

Create:

```
components/
 ProductCard.jsx
```

ProductCard.jsx

```
function ProductCard({ name, brand, price }) {

 return (
 <div className="product-card">

 <h2>
 {name}
 </h2>

 <p>
 Brand: {brand}
 </p>

 <p>
 Price: ₹{price}
 </p>

 </div>
);
}

export default ProductCard;
```

Now App.jsx:

```
import ProductCard from "../components/ProductCard";

function App() {

 return (
 <div>

 <h1>
 My Store
 </h1>

 <ProductCard
 name="iPhone 17"
 brand="Apple"
 price={79999}
 />

 <ProductCard
 name="Galaxy S26"
 brand="Samsung"
 price={69999}
 />

 </div>
);
}

export default App;
```

Run the application.

We should see:

My Store

iPhone 17  
Brand: Apple  
Price: ₹79999

Galaxy S26  
Brand: Samsung  
Price: ₹69999

Now let's inspect this application using React DevTools.

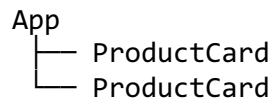
## 7. The Components Panel

Open Developer Tools and select:

Components

You should see a component hierarchy.

Something similar to:

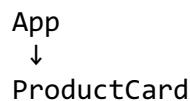


This is one of the most useful features of React DevTools.

It shows the **React component tree**.

Remember from our previous lectures that components have parent-child relationships.

Here:



So App is the parent.

ProductCard is the child.

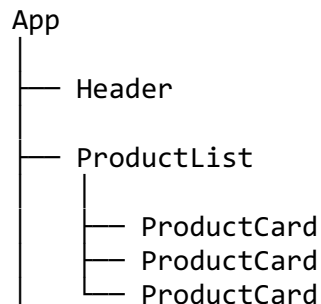
React DevTools displays that relationship visually.

---

## 8. Understanding the Component Tree

Let's look at a slightly larger application.

Suppose we have:



└ Footer

The component tree tells us exactly how our application is structured.

We can think of it as the React application's family tree.

For example:

```
App
├── Header
├── ProductList
│ ├── ProductCard
│ ├── ProductCard
│ └── ProductCard
└── Footer
```

This becomes extremely useful when an application has dozens or hundreds of components.

Instead of searching through all the source code, we can visually inspect the component hierarchy.

---

## 9. Selecting a Component

Now click the first:

ProductCard

component inside React DevTools.

When you select it, the right-hand side should show information about that component.

Depending on the version of React DevTools, you may see information such as:

Props

and other component information.

For our component, we expect to see:

```
name: "iPhone 17"
brand: "Apple"
price: 79999
```

This is very useful.

Remember, our parent wrote:

```
<ProductCard
 name="iPhone 17"
 brand="Apple"
 price={79999}
/>
```

React DevTools lets us verify that these Props actually reached the component.

---

## 10. Inspecting Props

Let's focus on Props.

Select:

ProductCard

You should see its Props.

Conceptually:

Props

name	"iPhone 17"
brand	"Apple"
price	79999

Now select the second ProductCard.

You should see:

Props

name	"Galaxy S26"
brand	"Samsung"
price	69999

Notice something important.

We have only one component definition:

```
function ProductCard({ name, brand, price }) {
```

But we have two instances of that component.

Each instance receives different Props.

React DevTools allows us to inspect those individual instances.

## 11. Component Instance vs Component Definition

This is an important concept.

Our code contains:

```
function ProductCard() {
 ...
}
```

This is the component definition.

But when we write:

```
<ProductCard />
<ProductCard />
```

we create two rendered instances of that component.

React DevTools can show both.

Conceptually:

ProductCard component

↓

Instance 1  
name = iPhone 17

Instance 2  
name = Galaxy S26

This is useful when debugging lists of similar components.

## 12. Practical Debugging Example

Let's intentionally create a bug.

In App.jsx, change:

```
<ProductCard
 name="Galaxy S26"
 brand="Samsung"
```

```
 price={69999}
 />
```

to:

```
<ProductCard
 name="Galaxy S26"
 brand="Samsung"
 price={699999}
/>
```

Notice the extra 9.

Our application now displays:

Price: ₹699999

But suppose we expected:

₹69999

We can inspect the component.

Select the second ProductCard in React DevTools.

We'll see:

price: 699999

Now we immediately know that the problem is with the data being passed from the parent.

This is much faster than searching through the entire application.

---

## 13. Another Debugging Example

Let's make another mistake.

Suppose the parent sends:

```
<ProductCard
 name="iPhone 17"
 brand="Apple"
 price={79999}
/>
```

But inside ProductCard, we accidentally write:

```
<h2>
 {name}
</h2>
```

```
<p>
 {props.brand}
</p>
```

This code is inconsistent because we are using destructuring:

```
function ProductCard({ name, brand, price }) {
```

but then trying to use:

```
props.brand
```

There is no props variable in this function.

The application will produce an error.

React DevTools won't necessarily fix the code for us, but it helps us verify what the component actually received.

This distinction is important:

DevTools helps us investigate problems; it doesn't automatically correct our code.

---

## 14. Selecting Components From the Page

React DevTools also provides ways to identify a component from the rendered page.

This is especially useful when the application is large.

Imagine our page contains:

Header  
Banner  
Product List  
Product Cards  
Footer

We can use the component-selection functionality to identify the corresponding React component.

The exact icon and interaction can vary between DevTools versions.

The idea is:



Rendered UI



Select element/component



React DevTools



Corresponding component

This makes debugging much faster.

---

## 15. Seeing the Relationship Between UI and Component

Let's say we click on:

iPhone 17

on the page.

We want to know:

Which React component generated this?

React DevTools can help us identify:

ProductCard

Then we can inspect its Props.

We may see:

name: iPhone 17

brand: Apple

price: 79999

This gives us a complete debugging path:

Browser UI



React Component



Props



Source Code

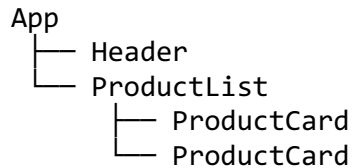
This is one of the main reasons professional React developers use React DevTools.

---

## 16. Inspecting Nested Components

Let's make the application a little bigger.

Create:



### ProductList.jsx

```
import ProductCard from "../ProductCard";
```

```
function ProductList() {
 return (
 <div>

 <ProductCard
 name="iPhone 17"
 brand="Apple"
 price={79999}
 />

 <ProductCard
 name="Galaxy S26"
 brand="Samsung"
 price={69999}
 />

 </div>
);
}
```

```
export default ProductList;
```

And:

### App.jsx

```
import ProductList from "../components/ProductList";
```

```
function App() {
 return (
```

```
<div>

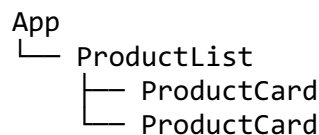
 <h1>
 My Store
 </h1>

 <ProductList />

</div>
);
}

export default App;
```

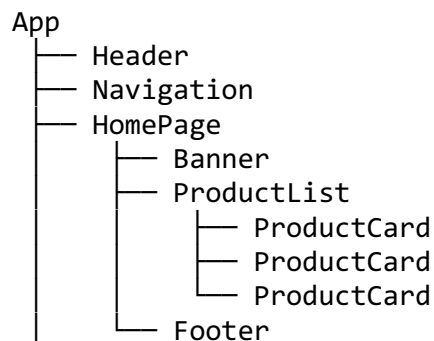
Now React DevTools may show:



This demonstrates how components can be nested.

## 17. Why Component Hierarchy Matters

Suppose the application has:



If something goes wrong with one product card, we can find it through the hierarchy.

We don't need to inspect every HTML element.

We can navigate directly through:

HomePage  
↓

ProductList



ProductCard

This becomes increasingly valuable as our applications grow.

---

## 18. Props Debugging Workflow

Let's establish a simple workflow students can use.

Suppose a component displays the wrong data.

### Step 1

Open React DevTools.

### Step 2

Open:

Components

### Step 3

Find the component.

### Step 4

Select it.

### Step 5

Inspect its Props.

For example:

name  
price  
brand

### Step 6

Compare those values with what you expected.

If the Prop is already wrong:

Problem may be in parent/data source

If the Prop is correct:

Problem may be inside the child component

This gives us a very useful debugging strategy.

---

## 19. Debugging Example — Wrong Prop

Suppose our component displays:

iPhone 17  
Brand: Samsung

We expected:

iPhone 17  
Brand: Apple

Open React DevTools.

Select the component.

Suppose Props show:

name: "iPhone 17"  
brand: "Samsung"

Then the problem isn't the JSX that displays the brand.

The child received the wrong value.

We should inspect the parent:

```
<ProductCard
 name="iPhone 17"
 brand="Samsung"
>
```

We can immediately identify the problem.

It should be:

```
<ProductCard
 name="iPhone 17"
 brand="Apple"
>
```

That's a practical example of using DevTools to reduce debugging time.

---

## 20. React DevTools and Console

React DevTools does not replace the browser Console.

We will use both.

For example, if we write:

```
console.log(props);
```

we can see the object in the Console.

But React DevTools provides a more structured way to inspect React components.

So think of them as complementary tools.

Console



Logs and JavaScript errors

React DevTools



Components, Props, State, React hierarchy

---

## 21. Understanding the Profiler

React DevTools also includes another important feature:

### **Profiler**

For today's lecture, we only need a basic introduction.

The Profiler helps us investigate component rendering and performance.

For example, it can help answer questions such as:

- Which components rendered?
- When did they render?
- How long did rendering take?
- Which component caused a re-render?

We won't use it extensively today because performance optimization is beyond the scope of this lecture.

But students should know that:

Components

is mainly useful for inspecting React structure and data.

And:

Profiler

is mainly useful for investigating rendering performance.

---

## 22. Don't Confuse React DevTools With the Elements Tab

This is another common beginner confusion.

Suppose you inspect:

```
<div class="product-card">
```

in the **Elements** tab.

You are looking at the browser's rendered DOM.

You may see:

```
<h2>iPhone 17</h2>
<p>Brand: Apple</p>
```

But React DevTools lets you think in terms of:

ProductCard

with:

name  
brand  
price

So:

Elements



What the browser rendered

React DevTools



How React components are structured

Both are important.

---

## 23. A Useful Mental Model

Whenever you're debugging a React application, think in three layers.

### Layer 1 — Component

ProductCard

### Layer 2 — Data

name  
brand  
price

### Layer 3 — UI

<h2>  
<p>  
<button>

React DevTools helps us inspect the first two layers.

The browser Elements tab helps us inspect the third layer.

This distinction is extremely useful.

---

## 24. Live Coding — Build a Debugging Scenario

Let's create a small example.

### ProductCard.jsx

```
function ProductCard({
 name,
 brand,
 price
) {

 return (
 <div>

 <h2>{name}</h2>
```



```
 <p>
 Brand: {brand}
 </p>

 <p>
 Price: ₹{price}
 </p>
 </div>
);
 }

export default ProductCard;

App.jsx
import ProductCard from "../components/ProductCard";

function App() {
 return (
 <div>

 <h1>
 Products
 </h1>

 <ProductCard
 name="iPhone 17"
 brand="Apple"
 price={79999}
 />

 <ProductCard
 name="Galaxy S26"
 brand="Samsung"
 price={69999}
 />

 <ProductCard
 name="OnePlus 14"
 brand="OnePlus"
 price={54999}
 />

 </div>
);
}
```

```
}
```

```
export default App;
```

Now open React DevTools.

We should see:

```
App
├── ProductCard
├── ProductCard
└── ProductCard
```

Click the first one.

Inspect:

```
name
brand
price
```

Then click the second.

Notice that the Props change.

Then click the third.

Again, different Props.

This is an excellent demonstration of how one reusable component can receive different data.

---

## 25. Debugging Challenge

Now intentionally introduce a mistake.

Change:

```
<ProductCard
 name="OnePlus 14"
 brand="OnePlus"
 price={54999}
/>
```

to:

```
<ProductCard
 name="OnePlus 14"
```

```
brand="OnePlus"
price={549999}
/>
```

Ask the students:

What is wrong?

Then open React DevTools.

Select the third ProductCard.

Look at:

price

You'll find:

549999

Now ask:

Where did this value come from?

The answer:

The parent passed the wrong Prop value.

This is a simple but realistic debugging exercise.

---

## 26. React DevTools Is Especially Useful With Props

Today's lecture follows directly from our previous lecture.

In Lecture 10, we learned:

Props

Today, we're learning how to inspect them.

For example:

```
<ProductCard
 name="iPhone 17"
 price={79999}
/>
```

React DevTools lets us verify:

ProductCard

Props

name → iPhone 17

price → 79999

So the workflow becomes:

Write Props

↓

Run application

↓

Inspect with React DevTools

↓

Verify data

↓

Debug if necessary

---

## 27. Important Things to Remember

Let's summarize today's important points.

### Point 1

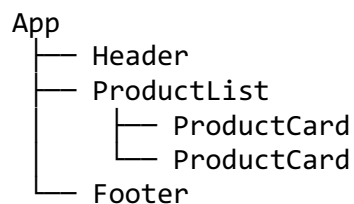
React DevTools is a developer tool specifically designed for React applications.

---

### Point 2

The Components panel lets us inspect the React component hierarchy.

For example:



---

### Point 3

We can select a component and inspect its Props.

For example:

```
name: "iPhone 17"
brand: "Apple"
price: 79999
```

---

#### Point 4

React DevTools helps us understand parent-child relationships.

---

#### Point 5

It is useful for debugging incorrect Props.

---

#### Point 6

React DevTools and browser Developer Tools are complementary.

React DevTools

→ Components / Props / State / React hierarchy

Browser DevTools

→ DOM / CSS / Console / Network

---

#### Point 7

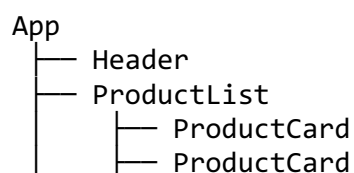
The Profiler helps investigate rendering and performance.

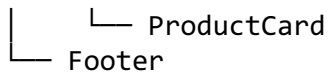
---

## 28. Student Exercise

Now I want you to create a small React application.

Create these components:





Each ProductCard should receive:

name  
brand  
price  
category

For example:

```
<ProductCard
 name="iPhone 17"
 brand="Apple"
 price={79999}
 category="Smartphone"
/>
```

Then open React DevTools and verify:

5. The complete component tree.
6. Each ProductCard.
7. Props for each ProductCard.
8. Different Props for each product.
9. Parent-child relationships.

---

## 29. Bonus Challenge

Intentionally introduce a wrong value.

For example:

```
<ProductCard
 name="iPhone 17"
 brand="Samsung"
 price={79999}
/>
```

Now use React DevTools to find the problem.

The goal isn't simply to fix the code.

The goal is to practice this debugging process:

Problem  
↓

Find component  
↓  
Inspect Props  
↓  
Find incorrect value  
↓  
Check parent  
↓  
Fix the source

This is a very useful habit for React development.

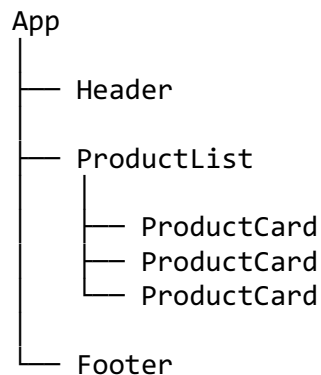
## 30. Final Takeaway

Let's finish with one simple idea.

When you build React applications, your application is not just HTML.

It is a tree of components.

For example:

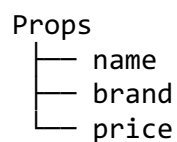


React DevTools allows you to see that tree.

You can select a component and inspect the data it receives.

For example:

ProductCard



So remember:

**React DevTools helps us see and debug what is happening inside our React component tree.**



# Lecture 12 — Rendering Arrays in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned about **React DevTools**.

We saw how we can inspect our component tree and check the Props received by individual components.

We were working with a ProductCard component.

For example:

```
<ProductCard
 name="iPhone 17"
 brand="Apple"
 price={79999}
>
```

```
<ProductCard
 name="Galaxy S26"
 brand="Samsung"
 price={69999}
>
```

```
<ProductCard
 name="OnePlus 14"
 brand="OnePlus"
 price={54999}
>
```

This works.

But imagine we have 100 products.

Are we going to write:

```
<ProductCard />
<ProductCard />
<ProductCard />
```

100 times?

Definitely not.

In a real application, product information will usually come from:

- an array
- a database
- an API
- user input
- some other data source

So today we're going to learn how React can take an array of data and dynamically create UI from that data.

Our main topic today is:

### **Rendering Arrays in React**

By the end of this lecture, you'll understand:

- What an array is
- Why arrays are useful in React
- How `.map()` works
- How to render an array in JSX
- How to render an array of strings
- How to render an array of objects
- How to pass array data to components
- How to render reusable components using `.map()`
- Why React needs a key
- How to choose a good key
- Common mistakes while rendering arrays

Let's get started.

---

## 2. Why Do We Need Arrays?

Let's start with a simple example.

Suppose we have three products:

```
const products = [
 "iPhone 17",
 "Galaxy S26",
 "OnePlus 14"
];
```

This is a JavaScript array.

Instead of storing each product in a separate variable:

```
const product1 = "iPhone 17";
const product2 = "Galaxy S26";
const product3 = "OnePlus 14";
```

we can store related data together:

```
const products = [
 "iPhone 17",
 "Galaxy S26",
 "OnePlus 14"
];
```

This becomes very useful when we have many items.

For example:

```
const products = [
 "iPhone 17",
 "Galaxy S26",
 "OnePlus 14",
 "Pixel 10",
 "Nothing Phone"
];
```

Now we can process all of them using JavaScript.

And React can use the same idea to generate UI.

---

## 3. The Problem With Manually Creating Components

Let's go back to our ProductCard.

Suppose we have:

```
function App() {

 return (
 <div>

 <ProductCard
 name="iPhone 17"
 price={79999}
 />

 </div>
);
}
```

```
 <ProductCard
 name="Galaxy S26"
 price={69999}
 />

 <ProductCard
 name="OnePlus 14"
 price={54999}
 />

 </div>
);
 }
}
```

This is fine for three products.

But what if we have:

10 products  
100 products  
1000 products

We cannot manually create every component.

Instead, we want:

Array of products  
↓  
Loop through array  
↓  
Create ProductCard  
↓  
Display on screen

This is where JavaScript's `.map()` method becomes very important.

---

## 4. Quick Revision — JavaScript Array

Before React, let's quickly revise JavaScript arrays.

We can create:

```
const numbers = [10, 20, 30, 40];
```

We can access an item:

`numbers[0]`

which gives:

10

And:

`numbers[2]`

gives:

30

We can also loop through the array.

One way is:

```
for (let i = 0; i < numbers.length; i++) {
 console.log(numbers[i]);
}
```

But in modern JavaScript, we frequently use:

`numbers.map(...)`

---

## 5. What Does `.map()` Do?

Let's understand `.map()` before using React.

Suppose we have:

```
const numbers = [1, 2, 3, 4];
```

We can write:

```
const result = numbers.map(function(number) {
 return number * 2;
});
```

The result is:

`[2, 4, 6, 8]`

So `.map()` goes through every item in the array.

Conceptually:

1 → 2  
2 → 4  
3 → 6  
4 → 8

Each original item produces a new value.

---

## 6. Arrow Function Version

In React, you'll usually see `.map()` written using an arrow function.

For example:

```
const numbers = [1, 2, 3, 4];

const result = numbers.map(number => {
 return number * 2;
});
```

Or, because there is only one expression:

```
const result = numbers.map(number => number * 2);
```

Both are valid.

We'll use the shorter version when appropriate.

---

## 7. Rendering a Simple Array in React

Now let's bring `.map()` into React.

Suppose:

```
const names = [
 "Rahul",
 "Priya",
 "Amit"
];
```

We want to display:

Rahul  
Priya  
Amit

We can write:

```
function App() {

 const names = [
 "Rahul",
 "Priya",
 "Amit"
];

 return (
 <div>

 {names.map(name => (
 <p>
 {name}
 </p>
))}

 </div>
);
}
```

This is a very important React pattern.

Let's understand it carefully.

---

## 8. Why Are We Using Curly Braces?

Inside JSX:

```
<div>

 {names.map(name => (
 <p>{name}</p>
))}

</div>
```

The curly braces tell React:

Evaluate this as a JavaScript expression.

Inside those curly braces, we're executing:

```
names.map(...)
```

The result is a collection of JSX elements.

React renders those elements.

So conceptually:

```
names array
 ↓
.map()
 ↓
JSX elements
 ↓
React renders them
```

---

## 9. What Actually Happens?

Suppose:

```
const names = [
 "Rahul",
 "Priya",
 "Amit"
];
```

React processes:

```
names.map(name => (
 <p>{name}</p>
))
```

Conceptually, it produces:

```
<p>Rahul</p>
<p>Priya</p>
<p>Amit</p>
```

So `.map()` allows us to dynamically generate JSX.

That's the key idea.

---



## 10. Live Coding — Rendering Names

Let's create our first example.

App.jsx

```
function App() {

 const names = [
 "Rahul",
 "Priya",
 "Amit",
 "Neha"
];

 return (
 <div>

 <h1>
 Students
 </h1>

 {names.map(name => (
 <p>
 {name}
 </p>
))}

 </div>
);
}

export default App;
```

Run the application.

We should get:

Students

Rahul  
Priya  
Amit  
Neha

Notice that we didn't manually write four <p> elements.

We wrote one:

`<p>{name}</p>`

and `.map()` generated it for every array item.

---

## 11. Rendering an Array of Numbers

The same technique works with numbers.

```
function App() {
 const numbers = [10, 20, 30, 40];

 return (
 <div>
 {numbers.map(number => (
 <p>
 {number}
 </p>
))}
 </div>
);
}
```

Output:

10  
20  
30  
40

We can also perform calculations.

For example:

```
{numbers.map(number => (
 <p>
 Double: {number * 2}
 </p>
))}
```

Output:

Double: 20  
Double: 40

Double: 60

Double: 80

Because we're inside JSX, JavaScript expressions can be used.

## 12. Rendering an Array of Objects

This is much more important for real applications.

In real applications, we usually don't have:

```
const products = [
 "iPhone",
 "Samsung",
 "OnePlus"
];
```

We usually have objects containing multiple properties.

For example:

```
const products = [
 {
 id: 1,
 name: "iPhone 17",
 brand: "Apple",
 price: 79999
 },
 {
 id: 2,
 name: "Galaxy S26",
 brand: "Samsung",
 price: 69999
 },
 {
 id: 3,
 name: "OnePlus 14",
 brand: "OnePlus",
 price: 54999
 }
];
```

Now every item has:

id  
name

brand  
price

This is much closer to the kind of data we'll receive from APIs later in this course.

## 13. Rendering Object Data

We can use `.map()`:

```
function App() {
 const products = [
 {
 id: 1,
 name: "iPhone 17",
 brand: "Apple",
 price: 79999
 },
 {
 id: 2,
 name: "Galaxy S26",
 brand: "Samsung",
 price: 69999
 },
 {
 id: 3,
 name: "OnePlus 14",
 brand: "OnePlus",
 price: 54999
 }
];

 return (
 <div>

 {products.map(product => (
 <div>

 <h2>
 {product.name}
 </h2>

 <p>
 Brand: {product.brand}
 </p>
)
)} </div>
)
}
```

```
 <p>
 Price: ₹{product.price}
 </p>
 </div>
)}
 </div>
);
}
```

**export default** App;

Now React generates one block for every product.

---

## 14. Understanding the `product` Variable

This line is important:

```
products.map(product => (
```

The variable:

`product`

represents the current item in the array.

During the first iteration:

```
product = {
 id: 1,
 name: "iPhone 17",
 brand: "Apple",
 price: 79999
}
```

During the second:

```
product = {
 id: 2,
 name: "Galaxy S26",
 brand: "Samsung",
 price: 69999
}
```

During the third:

```
product = {
 id: 3,
 name: "OnePlus 14",
 brand: "OnePlus",
 price: 54999
}
```

So:

```
product.name
```

means:

Give me the name of the current product.

And:

```
product.price
```

means:

Give me the price of the current product.

---

## 15. Using a Separate ProductCard Component

Now let's make our code cleaner.

We already have a reusable component:

ProductCard.jsx

```
function ProductCard({
 name,
 brand,
 price
) {

 return (
 <div className="product-card">

 <h2>
 {name}
 </h2>

 <p>
 Brand: {brand}
 </p>
 </div>
)
}
```

```
 <p>
 Price: ₹{price}
 </p>
 </div>
);
 }
```

```
export default ProductCard;
```

Now App.jsx contains our array.

```
import ProductCard from "../components/ProductCard";
```

```
function App() {
 const products = [
 {
 id: 1,
 name: "iPhone 17",
 brand: "Apple",
 price: 79999
 },
 {
 id: 2,
 name: "Galaxy S26",
 brand: "Samsung",
 price: 69999
 },
 {
 id: 3,
 name: "OnePlus 14",
 brand: "OnePlus",
 price: 54999
 }
];

 return (
 <div>
 <h1>
 Products
 </h1>
 {products.map(product => (
 <ProductCard
 name={product.name}

```

```
 brand={product.brand}
 price={product.price}
 />
)}}

 </div>
);
}
```

**export default** App;

Now we have:

```
Array
↓
map()
↓
ProductCard
↓
Props
↓
UI
```

This is one of the most important patterns in React.

---

## 16. Passing the Entire Object as a Prop

There is another approach.

Instead of:

```
<ProductCard
 name={product.name}
 brand={product.brand}
 price={product.price}
/>
```

we can pass the complete object:

```
<ProductCard
 product={product}
/>
```

Then:



## ProductCard.jsx

```
function ProductCard({ product }) {

 return (
 <div>

 <h2>
 {product.name}
 </h2>

 <p>
 Brand: {product.brand}
 </p>

 <p>
 Price: ₹{product.price}
 </p>

 </div>
);
}

export default ProductCard;
```

This can be useful when an object contains many properties.

For example:

```
id
name
brand
price
category
description
image
rating
stock
```

Instead of passing ten individual Props, we can pass the product object.

---

## 17. Passing the Object — Which Approach?

Both approaches are valid.

## Individual Props

```
<ProductCard
 name={product.name}
 brand={product.brand}
 price={product.price}
/>
```

Advantages:

- Explicit
- Component clearly defines what it needs
- Easy to understand

## Object Prop

```
<ProductCard
 product={product}
/>
```

Advantages:

- Convenient for larger objects
- Less repetitive
- Useful when many fields belong together

For this course, we'll use both approaches depending on the situation.

---

## 18. The `key` Prop

Now we come to one of the most important concepts in today's lecture.

If we render:

```
{products.map(product => (
 <ProductCard
 name={product.name}
 brand={product.brand}
 price={product.price}
 />
))}
```

React will warn us about something.

You may see a warning similar to:

Each child in a list should have a unique "key" prop.

So what is a key?

When React renders a list, it needs a way to identify each item.

We give each rendered item a unique key.

For example:

```
{products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 brand={product.brand}
 price={product.price}
 />
))}
```

Now each ProductCard has a key:

Product 1 → key = 1

Product 2 → key = 2

Product 3 → key = 3

---

## 19. Why Does React Need a Key?

Let's imagine we have:

Product A

Product B

Product C

React needs to keep track of which item is which.

Suppose we later add:

Product X

Product A

Product B

Product C

React needs to understand:

X is new

A already existed

B already existed

C already existed

A stable key helps React identify items across renders.

So a simple way to think about key is:

**A key gives React a stable identity for each item in a list.**

---

## 20. Choosing a Key

The best key is usually a unique ID that comes from the data.

For example:

```
const products = [
 {
 id: 101,
 name: "iPhone 17"
 },
 {
 id: 102,
 name: "Galaxy S26"
 },
 {
 id: 103,
 name: "OnePlus 14"
 }
];
```

Then:

```
key={product.id}
```

This is ideal.

Because the ID uniquely identifies the product.

---

## 21. Why Not Use the Product Name?

We could technically write:

```
key={product.name}
```

if product names are guaranteed to be unique.

But names aren't always unique.

For example:

iPhone 17  
iPhone 17

Now the key isn't unique.

A database ID is generally a much better choice:

key={product.id}

---

## 22. Can We Use the Array Index?

You may see code like:

```
{products.map((product, index) => (
 <ProductCard
 key={index}
 ...
 />
))}
```

Yes, JavaScript's `.map()` gives us the index as the second parameter.

For example:

```
products.map((product, index) => ...)
```

where:

product → current item  
index → current position

So:

iPhone → index 0  
Samsung → index 1  
OnePlus → index 2

Using the index as a key can work for some static lists, but it is generally not the best choice for dynamic lists where items can be inserted, removed, or reordered.

Prefer a stable unique ID when one is available.

---

## 23. Important Rule About Keys

The key is primarily for React's list reconciliation.

For example:

```
<ProductCard
 key={product.id}
 name={product.name}
/>
```

Do not assume that the child receives:

`props.key`

`key` is a special React attribute.

If your component needs the ID as data, pass it separately:

```
<ProductCard
 key={product.id}
 id={product.id}
 name={product.name}
/>
```

Then inside:

```
function ProductCard({ id, name }) {
```

you can access:

`id`

---

## 24. Complete Product Example

Let's now build our final example.

`ProductCard.jsx`

```
function ProductCard({
 name,
 brand,
 price
) {

 return (
 <div className="product-card">

 <h2>
 {name}
 </h2>
```

```
 <p>
 Brand: {brand}
 </p>

 <p>
 Price: ₹{price}
 </p>
 </div>
);
 }

export default ProductCard;

App.jsx

import ProductCard from "../components/ProductCard";

function App() {
 const products = [
 {
 id: 1,
 name: "iPhone 17",
 brand: "Apple",
 price: 79999
 },
 {
 id: 2,
 name: "Galaxy S26",
 brand: "Samsung",
 price: 69999
 },
 {
 id: 3,
 name: "OnePlus 14",
 brand: "OnePlus",
 price: 54999
 },
 {
 id: 4,
 name: "Pixel 10",
 brand: "Google",
 price: 64999
 }
];
};
```

```
 return (
 <div>

 <h1>
 Product List
 </h1>

 {products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 brand={product.brand}
 price={product.price}
 />
))}

 </div>
);
 }
}
```

```
export default App;
```

This is now a dynamic React application.

If we add another product to the array:

```
{
 id: 5,
 name: "Nothing Phone",
 brand: "Nothing",
 price: 39999
}
```

we don't need to create another:

```
<ProductCard />
```

React automatically renders another card because `.map()` processes the new array item.

---

## 25. Understanding the Complete Flow

Let's visualize the entire process.

We start with:



```
const products = [
 {
 id: 1,
 name: "iPhone 17",
 price: 79999
 },
 {
 id: 2,
 name: "Galaxy S26",
 price: 69999
 }
];
```

Then:

```
products.map(product => ...)
```

loops through the array.

For each product:

```
<ProductCard
 key={product.id}
 name={product.name}
 price={product.price}
/>
```

is generated.

Then the ProductCard receives Props.

Finally:

```
Array
↓
.map()
↓
ProductCard
↓
Props
↓
JSX
↓
Browser
```

This is an extremely common React pattern.

## 26. Rendering With `map()` and Index

Let's understand the second parameter of `.map()`.

We can write:

```
{products.map((product, index) => (
 <div>
 <p>
 {index + 1}. {product.name}
 </p>
 </div>
))}
```

Now the output could be:

1. iPhone 17
2. Galaxy S26
3. OnePlus 14
4. Pixel 10

Notice that we used:

`index + 1`

for displaying the serial number.

That's completely fine.

The important distinction is:

Using the index for display is different from using the index as the React key.

For example:

```
<p>
 {index + 1}. {product.name}
</p>
```

is perfectly reasonable.

But for the key, prefer:

`key={product.id}`

when a stable ID exists.

## 27. Rendering an Array of Objects With Destructuring

We can also destructure the object inside `.map()`.

Instead of:

```
{products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 brand={product.brand}
 price={product.price}
 />
))}
```

we can write:

```
{products.map(({ id, name, brand, price }) => (
 <ProductCard
 key={id}
 name={name}
 brand={brand}
 price={price}
 />
))}
```

This is shorter, but for beginners I recommend first becoming comfortable with:

```
product.id
product.name
product.brand
product.price
```

Then introduce destructuring.

---

## 28. Rendering With a Function

There are two common styles.

### Style 1 — Directly inside JSX

```
{products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 price={product.price}
```

```
 />
)})
```

This is the most common and concise approach.

### Style 2 — Create the result separately

```
const productElements = products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 price={product.price}
 />
));
```

Then:

```
return (
 <div>
 {productElements}
 </div>
);
```

Both approaches work.

For most simple cases, putting `.map()` directly inside JSX is convenient.

---

## 29. Common Mistake — Forgetting `return`

Consider:

```
{products.map(product => {
 <ProductCard
 key={product.id}
 name={product.name}
 />
})})
```

This doesn't render the component because the arrow function uses braces:

```
product => {
 ...
}
```

With braces, we need an explicit return:

```
{products.map(product => {
 return (
 <div>
 {products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 />
))}
 </div>
)
})})
```

```
 <ProductCard
 key={product.id}
 name={product.name}
 />
);
 }
 }
}
```

Or use parentheses with an implicit return:

```
{products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 />
))}
```

For simple JSX, the second version is often easier to read.

---

## 30. Common Mistake — Missing `key`

This:

```
{products.map(product => (
 <ProductCard
 name={product.name}
 />
))}
```

may produce a React warning about missing keys.

Instead:

```
{products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 />
))}
```

Always think:

If I'm rendering a list, do I have a stable key?

## 31. Common Mistake — Duplicate Keys

Avoid:

```
key="product"
```

for every item.

That means every item has the same key:

```
product
product
product
product
```

Keys should be unique among the siblings in the rendered list.

Use:

```
key={product.id}
```

when possible.

---

## 32. Common Mistake — Calling `.map()` on Undefined

Suppose we have:

```
let products;
```

and then:

```
products.map(...)
```

This causes an error because `products` is undefined.

For this introductory lecture, we'll keep our array defined directly:

```
const products = [];
```

Later, when we start fetching data from APIs, this becomes an important issue.

We'll learn how to handle API loading states and data safely when we reach the API lectures.

## 33. Rendering an Empty Array

What happens if:

```
const products = [];
```

Then:

```
products.map(...)
```

doesn't produce any components.

That's okay.

The result is simply an empty list.

We can later combine this with conditional rendering to display:

No products found.

We'll study conditional rendering in the next lecture.

---

## 34. Arrays + Props = Reusable UI

Let's connect today's lecture with Lecture 10.

Lecture 10 taught us:

Props allow us to pass data into components.

Lecture 12 now teaches us:

Arrays and `.map()` allow us to create many instances of those components dynamically.

So:

Array



`map()`



Component



Props

For example:

```
const students = [
 {
 id: 1,
 name: "Rahul",
 course: "BCA"
 },
 {
 id: 2,
 name: "Priya",
 course: "MCA"
 }
];
```

Then:

```
{students.map(student => (
 <StudentCard
 key={student.id}
 name={student.name}
 course={student.course}
 />
))}
```

This pattern is used everywhere in React applications.

---

## 35. Real-World Examples

Think about websites you use every day.

A shopping website may have:

Product List



Product Card



100 products

A social media application:

Post List



Post Card



Many posts

A messaging application:



Conversation List



Conversation Item



Many conversations

An admin dashboard:

User List



User Row



Many users

All of these can be represented using:

Array



map()



Reusable Component

---

## 36. Practical Example — Student List

Let's build another example.

StudentCard.jsx

```
function StudentCard({
 name,
 course,
 semester
}) {

 return (
 <div>

 <h2>
 {name}
 </h2>

 <p>
 Course: {course}
 </p>

 <p>
```

```
 Semester: {semester}
 </p>
 </div>
);
}

export default StudentCard;

App.jsx
import StudentCard from "../components/StudentCard";

function App() {
 const students = [
 {
 id: 1,
 name: "Rahul",
 course: "BCA",
 semester: 4
 },
 {
 id: 2,
 name: "Priya",
 course: "MCA",
 semester: 2
 },
 {
 id: 3,
 name: "Amit",
 course: "B.Tech",
 semester: 6
 }
];

 return (
 <div>

 <h1>
 Student List
 </h1>

 {students.map(student => (
 <StudentCard
 key={student.id}
 name={student.name}
 course={student.course}

```

```
 semester={student.semester}
 />
 </div>
);
}

export default App;
```

Now if we add:

```
{
 id: 4,
 name: "Neha",
 course: "BCA",
 semester: 3
}
```

the UI automatically contains another StudentCard.

No additional JSX is required.

---

## 37. A Very Important Connection to Future API Lectures

Remember the array of products we created:

```
const products = [
 {
 id: 1,
 name: "iPhone 17",
 price: 79999
 },
 {
 id: 2,
 name: "Galaxy S26",
 price: 69999
 }
];
```

Later in this series, we'll get this kind of data from an API.

Instead of:

```
const products = [...]
```

we'll eventually have something like:

API  
↓  
JSON data  
↓  
JavaScript array  
↓  
map()  
↓  
ProductCard  
↓  
UI

So today's lecture is actually preparing us for the API lectures later in the course.

---

## 38. React DevTools + Array Rendering

We can also connect this with our previous lecture.

After rendering:

```
{products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 price={product.price}
 />
))}
```

open React DevTools.

You should see:

```
App
├── ProductCard
├── ProductCard
├── ProductCard
└── ProductCard
```

Click each ProductCard.

You'll see different Props:

```
ProductCard 1
name: iPhone 17
price: 79999
```

Then:

ProductCard 2  
name: Galaxy S26  
price: 69999

and so on.

This gives us a very nice way to verify that `.map()` is generating the components correctly.

---

## 39. What We Learned Today

Let's summarize.

### 1. Arrays store multiple values.

```
const products = [
 "iPhone",
 "Samsung",
 "OnePlus"
];
```

---

### 2. `.map()` processes every item in an array.

```
products.map(product => ...)
```

---

### 3. React can render the result of `.map()`.

```
{products.map(product => (
 <p>{product.name}</p>
))}
```

---

### 4. We can render arrays of objects.

```
const products = [
 {
 id: 1,
 name: "iPhone 17"
 }
];
```

---

### 5. We can dynamically create reusable components.

```
{products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 />
))}
```

### 6. Every list item should have a suitable key.

Prefer a stable unique ID:

```
key={product.id}
```

### 7. .map() is one of the most important React patterns.

The basic pattern is:

```
Array
↓
map()
↓
JSX
↓
UI
```

## 40. Student Exercise

Now let's create a small application.

Create an array:

```
const employees = [
 {
 id: 1,
 name: "Rahul",
 designation: "Software Developer",
 salary: 50000
 },
 {
 id: 2,
 name: "Priya",
```

```
 designation: "UI Designer",
 salary: 45000
 },
 {
 id: 3,
 name: "Amit",
 designation: "Project Manager",
 salary: 70000
 }
];
};
```

Create an:

EmployeeCard

component.

Then use `.map()` to render all employees.

The result should look something like:

Employees

Rahul  
Software Developer  
₹50000

Priya  
UI Designer  
₹45000

Amit  
Project Manager  
₹70000

Remember to use:

`key={employee.id}`

---

## 41. Bonus Exercise

Create a ProductCard application with at least five products.

Each product should contain:

id  
name

brand  
price  
category

Render the products dynamically using:

`.map()`

Do not manually write multiple `<ProductCard />` components.

Then open React DevTools and verify that all the ProductCard components are appearing in the component tree.

---

## 42. Final Takeaway

Let's finish with the most important concept from today's lecture.

Suppose we have:

```
const products = [
 {
 id: 1,
 name: "iPhone 17",
 price: 79999
 },
 {
 id: 2,
 name: "Galaxy S26",
 price: 69999
 },
 {
 id: 3,
 name: "OnePlus 14",
 price: 54999
 }
];
```

We don't need to manually create:

```
<ProductCard />
<ProductCard />
<ProductCard />
```

Instead:

```
{products.map(product => (
 <ProductCard
```



```
 key={product.id}
 name={product.name}
 price={product.price}
 />
)})
```

The array drives the UI.

So remember this pattern:

**Data → map() → Components → Props → UI**

This pattern is fundamental to React development.

# Lecture 13 — Conditional Rendering in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned about **Rendering Arrays in React**.

We learned how to take an array like:

```
const products = [
 {
 id: 1,
 name: "iPhone 17",
 price: 79999
 },
 {
 id: 2,
 name: "Galaxy S26",
 price: 69999
 }
];
```

and dynamically generate components using:

```
{products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 price={product.price}
 />
))}
```

But our applications need to do more than simply display data.

For example:

- If the user is logged in, show the dashboard.
- If the user is not logged in, show the login page.
- If a product is available, show the Buy button.
- If a product is out of stock, show “Out of Stock”.
- If there are products, show the product list.
- If there are no products, show “No products found”.

In all these situations, the UI depends on a condition.

This concept is called:

### **Conditional Rendering**

In today's lecture, we'll learn:

- What conditional rendering means
- Using `if` statements
- Conditional rendering outside JSX
- Ternary operator
- Logical AND `&&`
- Conditional rendering with Props
- Conditional rendering with arrays
- Rendering fallback content
- Multiple conditions
- Common mistakes
- A practical product example

Let's get started.

---

## 2. What Is Conditional Rendering?

Let's start with a simple question.

Suppose we have a variable:

```
const isLoggedIn = true;
```

If the user is logged in, we want:

Welcome to Dashboard

But if:

```
const isLoggedIn = false;
```

we want:

Please Login

The UI changes based on the value of `isLoggedIn`.

This is conditional rendering.

We can think of it as:

Condition

↓

Is it true?

↓

Yes → Render UI A

No → Render UI B

React doesn't have a special "conditional rendering" syntax.

Instead, we use normal JavaScript conditions together with JSX.

---

### 3. Why Is Conditional Rendering Important?

Almost every real-world React application needs conditional rendering.

For example, consider an e-commerce website.

A product may have:

```
const inStock = true;
```

If it is in stock:

iPhone 17  
₹79999

[ Buy Now ]

If it is not in stock:

iPhone 17  
₹79999

Out of Stock

The ProductCard is the same component.

Only the displayed UI changes depending on the data.

Another example is authentication.

```
isLoggedIn = true
```

↓

Dashboard

```
isLoggedIn = false
```



Login

So conditional rendering makes our UI dynamic.

---

## 4. Method 1 — Using `if` Statements

Let's begin with the simplest JavaScript approach.

Suppose:

```
const isLoggedIn = true;
```

We can write:

```
if (isLoggedIn) {
 console.log("Welcome");
} else {
 console.log("Please Login");
}
```

This is normal JavaScript.

React can use the same logic.

However, there is one important thing to understand:

We generally cannot directly place an `if` statement inside JSX like this:

```
return (
 <div>

 if (isLoggedIn) {
 <h1>Welcome</h1>
 }

 </div>
);
```

This is invalid JSX.

So we need another approach.

---

## 5. Conditional Rendering Before `return`

One simple solution is to use `if` before the JSX is returned.

For example:

```
function App() {

 const isLoggedIn = true;

 if (isLoggedIn) {
 return (
 <h1>
 Welcome to Dashboard
 </h1>
);
 }

 return (
 <h1>
 Please Login
 </h1>
);
}
```

Let's understand this.

If:

`isLoggedIn = true`

React returns:

```
<h1>
 Welcome to Dashboard
</h1>
```

If:

`isLoggedIn = false`

React reaches the second return:

```
<h1>
 Please Login
</h1>
```

## 6. Live Coding — Login Example

Let's create our first example.

App.jsx

```
function App() {

 const isLoggedIn = true;

 if (isLoggedIn) {
 return (
 <div>
 <h1>
 Welcome to Dashboard
 </h1>

 <p>
 You are successfully logged in.
 </p>
 </div>
);
 }

 return (
 <div>
 <h1>
 Please Login
 </h1>

 <p>
 You need to login first.
 </p>
 </div>
);
}

export default App;
```

Run the application.

We should see:

Welcome to Dashboard

You are successfully logged in.

Now change:

```
const isLoggedIn = true;
```

to:

```
const isLoggedIn = false;
```

The UI changes to:

Please Login

You need to login first.

This is our first conditional rendering example.

---

## 7. When Should We Use `if`?

The `if` approach is useful when the condition controls a large portion of the UI.

For example:

If logged in:  
    Render entire dashboard

Otherwise:  
    Render entire login page

It makes the code very clear.

But sometimes we only want to conditionally display a small part of the UI.

For example:

Show Logout button only if logged in.

For that, React developers commonly use the ternary operator or `&&`.

---

## 8. Method 2 — Ternary Operator

Let's look at the JavaScript ternary operator.

The basic syntax is:

```
condition ? valueIfTrue : valueIfFalse
```

For example:



```
const age = 20;

const message =
 age >= 18
 ? "Adult"
 : "Minor";
```

If the condition is true:

Adult

If false:

Minor

In React, the ternary operator is very useful for JSX.

---

## 9. Ternary Operator in JSX

Let's rewrite our login example.

```
function App() {

 const isLoggedIn = true;

 return (
 <div>

 {
 isLoggedIn
 ? <h1>Welcome to Dashboard</h1>
 : <h1>Please Login</h1>
 }

 </div>
);
}
```

This means:

If isLoggedIn is true



Show Welcome to Dashboard

Otherwise



Show Please Login

This is one of the most common conditional rendering patterns in React.

---

## 10. Cleaner Ternary Formatting

We can format it like this:

```
return (
 <div>

 {isLoggedIn ? (
 <h1>
 Welcome to Dashboard
 </h1>
) : (
 <h1>
 Please Login
 </h1>
)}

 </div>
);
```

This is easier to read when the JSX becomes larger.

---

## 11. Live Coding — Ternary Example

Let's create:

```
function App() {

 const isLoggedIn = true;

 return (
 <div>

 <h1>
 My Application
 </h1>
 </div>
);
}
```

```
 {isLoggedIn ? (
 <p>
 Welcome back!
 </p>
) : (
 <p>
 Please login to continue.
 </p>
)}
 </div>
);
}
```

**export default** App;

Change:

**const** isLoggedIn = **true**;

to:

**const** isLoggedIn = **false**;

and observe the change.

---

## 12. Method 3 — Logical AND &&

Now suppose we don't want an "else" condition.

For example:

Show the Logout button only if the user is logged in.

We can use:

```
{isLoggedIn && (
 <button>
 Logout
 </button>
)}
```

This is another extremely common React pattern.

---

## 13. Understanding &&

Let's understand it using normal JavaScript.

Suppose:

```
const isLoggedIn = true;
```

Then:

```
isLoggedIn && "Welcome"
```

returns:

Welcome

If:

```
const isLoggedIn = false;
```

then:

```
isLoggedIn && "Welcome"
```

doesn't produce "Welcome".

The important idea is:

```
true && something
```

↓

```
something
```

```
false && something
```

↓

```
don't render the right side
```

This makes && useful for conditional rendering.

---

## 14. Live Coding — Conditional Button

Let's create:

```
function App() {
 const isLoggedIn = true;
 return (

```

```
 <div>

 <h1>
 My Application
 </h1>

 {isLoggedIn && (
 <button>
 Logout
 </button>
)}

 </div>
);
}
```

**export default** App;

When:

isLoggedIn = **true**

we see:

Logout

When:

isLoggedIn = **false**

the button doesn't appear.

---

## 15. Ternary vs &&

This is an important distinction.

Use a **ternary** when you have two alternatives.

For example:

```
{isLoggedIn ? (
 <Dashboard />
) : (
 <Login />
)}
```

Meaning:

If true → Dashboard

If false → Login

Use **&&** when you only want to render something when a condition is true.

For example:

```
{isLoggedIn && (
 <LogoutButton />
)}
```

Meaning:

If true → Show LogoutButton

If false → Show nothing

A simple rule:

Two possible UI results

↓

Ternary

Show something only when true

↓

**&&**

---

## 16. Conditional Rendering With Props

Now let's connect today's topic with our previous lectures.

Suppose we have a ProductCard.

We want to show:

Buy Now

if the product is available.

We can pass:

```
<ProductCard
 name="iPhone 17"
 price={79999}
 inStock={true}
/>
```

Now inside the component:

```
function ProductCard({
 name,
 price,
 inStock
}) {

 return (
 <div>

 <h2>
 {name}
 </h2>

 <p>
 ₹{price}
 </p>

 {inStock ? (
 <button>
 Buy Now
 </button>
) : (
 <p>
 Out of Stock
 </p>
)}

 </div>
);
}
```

This is a very realistic React example.

---

## 17. Live Coding — Product Availability

Let's create a complete example.

ProductCard.jsx

```
function ProductCard({
 name,
 price,
 inStock
}) {
```

```
 return (
 <div>

 <h2>
 {name}
 </h2>

 <p>
 Price: ₹{price}
 </p>

 {inStock ? (
 <button>
 Buy Now
 </button>
) : (
 <p>
 Out of Stock
 </p>
)}
 </div>
);
 }

 export default ProductCard;

 App.jsx

 import ProductCard from "../components/ProductCard";

 function App() {

 return (
 <div>

 <h1>
 Products
 </h1>

 <ProductCard
 name="iPhone 17"
 price={79999}
 inStock={true}
 />

 <ProductCard
```



```
 name="Galaxy S26"
 price={69999}
 inStock={false}
 />
 </div>
);
}
```

**export default** App;

The output will be conceptually:

Products

iPhone 17  
Price: ₹79999  
[ Buy Now ]

Galaxy S26  
Price: ₹69999  
Out of Stock

Notice that the same component behaves differently depending on its Props.

---

## 18. Conditional Rendering With Arrays

Now let's combine Lecture 12 and Lecture 13.

Suppose we have:

```
const products = [
 {
 id: 1,
 name: "iPhone 17",
 price: 79999,
 inStock: true
 },
 {
 id: 2,
 name: "Galaxy S26",
 price: 69999,
 inStock: false
 }
];
```

We can render:

```
{products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 price={product.price}
 inStock={product.inStock}
 />
))}
```

Now each ProductCard decides what to display based on:

inStock

This gives us:

```
Array
↓
map()
↓
ProductCard
↓
Props
↓
Conditional Rendering
↓
UI
```

This is becoming a real application architecture.

---

## 19. Rendering “No Products Found”

Here’s another common scenario.

Suppose:

```
const products = [];
```

We don’t want the user to see a blank screen.

We want:

No products found.

We can use a condition.

For example:

```
{products.length > 0 ? (
 <p>
 Products are available.
 </p>
) : (
 <p>
 No products found.
 </p>
)}
```

If the array contains items:

Products are available.

If the array is empty:

No products found.

---

## 20. Combining `.map()` With a Condition

We can also render the products only when the array contains data.

```
{products.length > 0 ? (
 products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 price={product.price}
 />
))
) : (
 <p>
 No products found.
 </p>
)}
```

This is a very useful pattern.

It says:

If products exist

↓

Render ProductCards

Otherwise

↓

Render "No products found"

## 21. Complete Product List Example

Let's build this properly.

### ProductCard.jsx

```
function ProductCard({
 name,
 price,
 inStock
}) {

 return (
 <div>

 <h2>
 {name}
 </h2>

 <p>
 Price: ₹{price}
 </p>

 {inStock ? (
 <button>
 Buy Now
 </button>
) : (
 <p>
 Out of Stock
 </p>
)}

 </div>
);
}
```

```
export default ProductCard;
```

### ProductList.jsx

```
import ProductCard from "../ProductCard";
```

```
function ProductList() {
```

```
const products = [
 {
 id: 1,
 name: "iPhone 17",
 price: 79999,
 inStock: true
 },
 {
 id: 2,
 name: "Galaxy S26",
 price: 69999,
 inStock: false
 },
 {
 id: 3,
 name: "OnePlus 14",
 price: 54999,
 inStock: true
 }
];

return (
 <div>

 <h1>
 Product List
 </h1>

 {products.length > 0 ? (
 products.map(product => (
 <ProductCard
 key={product.id}
 name={product.name}
 price={product.price}
 inStock={product.inStock}
 />
))
) : (
 <p>
 No products found.
 </p>
)}

 </div>
);
}

export default ProductList;
```

Now we have both:

- Array rendering
- Conditional rendering

---

## 22. Multiple Conditions

Sometimes we have more than two possible states.

For example, a product can have:

in stock  
low stock  
out of stock

We might have:

```
const stock = 5;
```

We could write:

```
{stock === 0 ? (
 <p>
 Out of Stock
 </p>
) : stock < 5 ? (
 <p>
 Only a few items left!
 </p>
) : (
 <p>
 In Stock
 </p>
)}
```

Let's understand the conditions:

```
stock === 0
 ↓
Out of Stock
```

```
stock < 5
 ↓
Few items left
```

otherwise



In Stock

This is called a nested ternary.

It works, but be careful.

If the conditions become complicated, it is often better to move the logic outside the JSX.

---

## 23. Avoid Very Complicated JSX

For example, this can become difficult to read:

```
{isLoggedIn ? (
 isAdmin ? (
 <AdminDashboard />
) : (
 <UserDashboard />
)
) : (
 <Login />
)}
```

It works, but the JSX is becoming difficult to understand.

Instead, we could calculate the result before the return.

For example:

```
let page;

if (!isLoggedIn) {
 page = <Login />;
} else if (isAdmin) {
 page = <AdminDashboard />;
} else {
 page = <UserDashboard />;
}
```

Then:

```
return (
 <div>
 {page}
 </div>
);
```

This is easier to maintain.

---

## 24. Conditional Rendering Using Variables

Let's look at a simple example.

```
function App() {

 const isLoggedIn = true;

 let message;

 if (isLoggedIn) {
 message = "Welcome back!";
 } else {
 message = "Please login.";
 }

 return (
 <div>

 <h1>
 {message}
 </h1>

 </div>
);
}
```

Here we're separating:

Decision-making

from:

Rendering

This can make more complex components easier to understand.

---

## 25. Another Example — User Role

Suppose we have:



```
const role = "admin";
```

We want:

admin → Admin Panel  
user → User Dashboard  
guest → Login

We could write:

```
function App() {

 const role = "admin";

 let content;

 if (role === "admin") {
 content = <h1>Admin Panel</h1>;
 } else if (role === "user") {
 content = <h1>User Dashboard</h1>;
 } else {
 content = <h1>Please Login</h1>;
 }

 return (
 <div>
 {content}
 </div>
);
}
```

This is often clearer than deeply nested ternary operators.

---

## 26. Conditional CSS Classes

Conditional rendering isn't limited to showing or hiding components.

We can also conditionally change CSS classes.

Suppose:

```
const isActive = true;
```

We can write:

```
<div className={isActive ? "active" : "inactive"}>
 Account
</div>
```

If isActive is true:

class="active"

If false:

class="inactive"

This is another useful real-world application.

---

## 27. Conditional Button Text

We can also change text.

For example:

```
<button>
 {isLoggedIn ? "Logout" : "Login"}
</button>
```

If:

isLoggedIn = **true**

we get:

Logout

Otherwise:

Login

This is a simple but very common use of the ternary operator.

---

## 28. Conditional Rendering With Events

Let's combine today's topic with our earlier event management lecture.

Suppose we have:

```
const isLoggedIn = true;
```

and:

```
<button onClick={handleLogout}>
 Logout
</button>
```

We could conditionally show it:

```
{isLoggedIn && (
 <button onClick={handleLogout}>
 Logout
 </button>
)}
```

Later, when we learn Hooks and State, this becomes much more interesting.

For example:

```
User clicks Login
 ↓
State changes
 ↓
isLoggedIn becomes true
 ↓
React re-renders
 ↓
Dashboard appears
```

That is the foundation of interactive React applications.

---

## 29. Important Concept — React Re-renders

Suppose we have:

```
const isLoggedIn = true;
```

and React renders:

```
<h1>
 Welcome
</h1>
```

If the value later changes to:

```
false
```

React can render:

```
<h1>
 Please Login
</h1>
```

In this lecture, we're using regular variables, so they don't automatically trigger UI updates when changed.

Later, when we learn useState, we'll learn how state changes cause React to re-render the component.

This distinction will become very important.

---

## 30. Common Mistake — Using `if` Directly Inside JSX

Don't write:

```
return (
 <div>

 if (isLoggedIn) {
 <h1>Welcome</h1>
 }

 </div>
);
```

This is invalid.

Instead, use:

```
{isLoggedIn && (
 <h1>
 Welcome
 </h1>
)}
```

or:

```
{isLoggedIn ? (
 <h1>
 Welcome
 </h1>
) : (
 <h1>
 Please Login
 </h1>
)}
```

```
</h1>
})
```

or perform the if logic before the return.

---

## 31. Common Mistake — Confusing = and ===

Suppose:

```
const role = "admin";
```

We want to check:

```
role === "admin"
```

Remember:

= → assignment

== → loose comparison

=== → strict comparison

In React conditions, prefer:

```
role === "admin"
```

rather than:

```
role = "admin"
```

---

## 32. Common Mistake — Using 0 && ...

There is an important JavaScript behavior beginners should know.

Suppose:

```
const count = 0;
```

and we write:

```
{count && (
 <p>
 Products available
```

```
</p>
)}
```

You might expect nothing to appear.

But JavaScript evaluates:

`0 && something`

and the result is `0`.

React may render that `0`.

A safer condition is:

```
{count > 0 && (
 <p>
 Products available
 </p>
)}
```

This is an important practical detail.

---

## 33. Common Mistake — Forgetting the Boolean Meaning

Suppose:

```
const inStock = false;
```

and we write:

```
{inStock && (
 <button>
 Buy Now
 </button>
)}
```

The button won't appear.

That's correct.

But if we want one UI for true and another for false, use a ternary:

```
{inStock ? (
 <button>
 Buy Now
 </button>
) : (
 <button>
 Out of Stock
 </button>
)}
```

```
<p>
 Out of Stock
</p>
)}
```

Remember:

&&  
→ Show or don't show

? :  
→ Choose between two alternatives

---

## 34. Practical Exercise — User Login

Create a component that has:

```
const isLoggedIn = true;
```

If the user is logged in, display:

Welcome to the application.

[ Logout ]

Otherwise:

Please login to continue.

[ Login ]

Use a ternary operator.

The expected structure is:

```
{isLoggedIn ? (
 // Logged-in UI
) : (
 // Logged-out UI
)}
```

---

## 35. Practical Exercise — Product Stock

Create:

```
const product = {
 name: "iPhone 17",
 price: 79999,
 inStock: true
};
```

Display:

iPhone 17  
₹79999  
[ Buy Now ]

when inStock is true.

When it becomes false:

iPhone 17  
₹79999  
Out of Stock

Use a ternary operator.

---

## 36. Practical Exercise — Product List

Create:

```
const products = [
 {
 id: 1,
 name: "iPhone 17",
 inStock: true
 },
 {
 id: 2,
 name: "Galaxy S26",
 inStock: false
 }
];
```

Render the products using `.map()`.

For each product:

If inStock:  
  Buy Now



Otherwise:

Out of Stock

This exercise combines:

Lecture 12

Array Rendering

+

Lecture 13

Conditional Rendering

---

## 37. Practical Exercise — Empty List

Create:

```
const products = [];
```

If there are products:

Product List

If there are no products:

No products found.

Use:

```
products.length > 0
```

as your condition.

---

## 38. Key Patterns to Remember

There are three patterns I want you to remember from today's lecture.

### Pattern 1 — `if`

Use it when the decision is larger or more complex.

```
if (isLoggedIn) {
 return <Dashboard />;
}
```

```
return <Login />;
```

---

## Pattern 2 — Ternary

Use it when you have two UI alternatives.

```
{isLoggedIn ? (
 <Dashboard />
) : (
 <Login />
)}
```

---

## Pattern 3 — &&

Use it when you want something to appear only if a condition is true.

```
{isLoggedIn && (
 <LogoutButton />
)}
```

---

# 39. Real-World Mental Model

When building a React component, ask:

What should the user see when this condition is true?

Then:

What should the user see when it is false?

For example:

### Authentication

isLoggedIn?



Yes → Dashboard

No → Login

### Product stock

inStock?



Yes → Buy Now  
No → Out of Stock

### Product list

products.length > 0?  
↓  
Yes → Product List  
No → No products found

### Admin access

role === "admin"?  
↓  
Yes → Admin Panel  
No → Access Denied

This way of thinking makes conditional rendering much easier.

---

## 40. Final Live Coding Example

Let's put everything together.

### ProductCard.jsx

```
function ProductCard({
 name,
 price,
 inStock
}) {

 return (
 <div>

 <h2>
 {name}
 </h2>

 <p>
 Price: ₹{price}
 </p>

 {inStock ? (
 <button>
 Buy Now
 </button>
) : (
```

```
 <p>
 Out of Stock
 </p>
)}
 </div>
);
}

export default ProductCard;

App.jsx
import ProductCard from "../components/ProductCard";

function App() {
 const products = [
 {
 id: 1,
 name: "iPhone 17",
 price: 79999,
 inStock: true
 },
 {
 id: 2,
 name: "Galaxy S26",
 price: 69999,
 inStock: false
 },
 {
 id: 3,
 name: "OnePlus 14",
 price: 54999,
 inStock: true
 }
];

 return (
 <div>

 <h1>
 Product Store
 </h1>

 {products.length > 0 ? (
 products.map(product => (
```

```
 <ProductCard
 key={product.id}
 name={product.name}
 price={product.price}
 inStock={product.inStock}
 />
))
) : (
 <p>
 No products found.
 </p>
)}
 </div>
);
}
```

**export default** App;

This small example demonstrates several React concepts we've learned:

- Components
- Props
- Arrays
- map()
- Keys
- Conditional Rendering
- Ternary Operator

This is exactly how our concepts start coming together.

---

## 41. Lecture Summary

Let's quickly summarize today's lecture.

### Conditional rendering

Conditional rendering means:

Displaying different UI depending on a condition.

if

Useful for larger decisions:

```
if (isLoggedIn) {
 return <Dashboard />;
}

return <Login />;
```

## Ternary

Useful for two alternatives:

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

&&

Useful for rendering something only when a condition is true:

```
{isLoggedIn && <LogoutButton />}
```

## With arrays

We can combine:

```
products.map(...)
```

with:

```
product.inStock
```

to create dynamic UI.

## With Props

A component can receive a value such as:

```
inStock={true}
```

and decide what to display.

---

## 42. Final Takeaway

The most important idea from today's lecture is:

**React allows us to make the UI depend on data.**

For example:

Data



Condition



Choose UI



Render

If:

`inStock === true`

show:

Buy Now

Otherwise:

Out of Stock

If:

`products.length > 0`

show the product list.

Otherwise:

No products found.

This is what makes React applications dynamic.

We've now learned how to:

Create Components



Pass Props



Render Arrays



Conditionally Render UI

# Lecture 14 — Routing in React

## 1. Opening

Hello everyone, and welcome back.

So far in our React series, we have learned how to build components, pass data using Props, render arrays, and conditionally render different UI.

But there is one important question.

Suppose we are building an e-commerce application.

We may have different screens:

Home  
Products  
About  
Contact

Normally, on a website, when we click:

Products

the URL changes to something like:

/products

And when we click:

About

the URL becomes:

/about

So how do we handle this in React?

This is where **routing** comes in.

In today's lecture, we'll learn:

- What routing is
- Why React needs a router
- Single Page Applications
- Installing React Router
- Creating routes



- Creating multiple pages
- Navigating between pages
- Link
- NavLink
- BrowserRouter
- Routes and Route
- A complete mini project

And in the next lecture, we'll build on this knowledge and learn **Parameterized Routes**.

Let's get started.

---

## 2. What Is Routing?

Let's start with a simple example.

Imagine we have a website:

My Website

It has four pages:

Home  
About  
Products  
Contact

Each page has a URL.

For example:

/

for Home.

/about

for About.

/products

for Products.

And:

/contact

for Contact.

The process of deciding:

Which UI should be displayed for a particular URL?

is called **routing**.

We can think of it like this:

```
URL
↓
Router
↓
Find matching route
↓
Render component
```

For example:

```
/products
↓
Products component
```

and:

```
/about
↓
About component
```

---

### 3. Traditional Website vs React Application

Before React, websites commonly worked differently.

Suppose we had:

```
index.html
about.html
products.html
contact.html
```

When the user clicked:

About

the browser requested:

```
about.html
```

from the server.

The browser then loaded the new page.

Conceptually:

Click About



Browser requests about.html



Server sends HTML



Browser loads new page

React applications commonly work differently.

A React application is usually a **Single Page Application**, or SPA.

---

## 4. What Is a Single Page Application?

A Single Page Application means that the browser initially loads the application, and then JavaScript changes the displayed UI as the user navigates.

Conceptually:

Browser



React Application



User clicks Products



React changes the displayed component

We don't necessarily need to reload the entire document every time.

For example:

/products

can display:

`<Products />`

while:

/about

can display:

[<About />](#)

The URL changes, but the application can remain loaded.

This provides a smoother user experience.

---

## 5. Does React Have Routing Built In?

This is an important point.

React itself is primarily a library for building user interfaces.

Routing is generally handled using a routing library.

One of the most commonly used choices is:

### **React Router**

For our course, we'll use React Router.

Before coding, let's install it.

---

## 6. Installing React Router

Open the terminal inside your React project.

If you're using a Vite React project, run:

```
npm install react-router-dom
```

After installation, we can use React Router inside our application.

Once the installation completes, restart the development server if necessary.

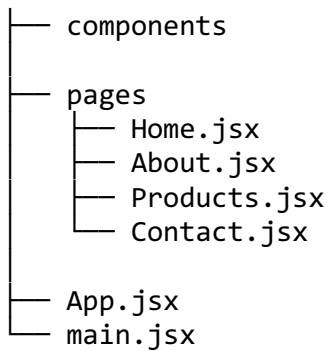
---

## 7. Our Project Structure

Let's create a simple application.

Our project can look like:

```
src
|
```



We'll create a separate component for each page.

This is a good habit because it keeps our project organized.

---

## 8. Creating the Home Page

Let's create:

`src/pages/Home.jsx`

Add:

```
function Home() {
 return (
 <div>
 <h1>
 Home Page
 </h1>

 <p>
 Welcome to our React application.
 </p>
 </div>
);
}

export default Home;
```

---

## 9. Creating the About Page

Create:

src/pages/About.jsx

Add:

```
function About() {
 return (
 <div>
 <h1>
 About Page
 </h1>

 <p>
 This is the about page.
 </p>
 </div>
);
}

export default About;
```

---

## 10. Creating the Products Page

Create:

src/pages/Products.jsx

Add:

```
function Products() {
 return (
 <div>
 <h1>
 Products Page
 </h1>

 <p>
 Here we will display our products.
 </p>
 </div>
);
}

export default Products;
```

---

## 11. Creating the Contact Page

Create:

src/pages/Contact.jsx

Add:

```
function Contact() {
 return (
 <div>
 <h1>
 Contact Page
 </h1>

 <p>
 Contact us for more information.
 </p>
 </div>
);
}

export default Contact;
```

Now we have four components:

Home  
About  
Products  
Contact

But they are not connected to URLs yet.

That's where React Router comes in.

---

## 12. BrowserRouter

Let's open:

main.jsx

Depending on the React project setup, you may have something like:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "../App.jsx";
```

```
import "./index.css";

createRoot(document.getElementById("root")).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

We need to provide routing functionality to our application.

Import:

```
import { BrowserRouter } from "react-router-dom";
```

Then wrap our application:

```
createRoot(document.getElementById("root")).render(
 <StrictMode>
 <BrowserRouter>
 <App />
 </BrowserRouter>
 </StrictMode>
);
```

The important part is:

```
<BrowserRouter>
 <App />
</BrowserRouter>
```

This gives React Router access to the browser's URL and navigation system.

---

## 13. What Does BrowserRouter Do?

You don't need to understand all the internal implementation at this stage.

For now, think of:

```
<BrowserRouter>
```

as the component that enables routing for our application.

Conceptually:

```
Browser URL
 ↓
BrowserRouter
 ↓
```



React Router  
↓  
Matching Route  
↓  
Component

---

## 14. Creating Routes

Now let's open:

App.jsx

Import:

```
import {
 Routes,
 Route
} from "react-router-dom";
```

Then import our pages:

```
import Home from "../pages/Home";
import About from "../pages/About";
import Products from "../pages/Products";
import Contact from "../pages/Contact";
```

Now we can create our routes.

```
function App() {
 return (
 <Routes>

 <Route
 path="/"
 element={<Home />}
 />

 <Route
 path="/about"
 element={<About />}
 />

 <Route
 path="/products"
 element={<Products />}
 />
)
);
}
```

```
 />

 <Route
 path="/contact"
 element={<Contact />}
 />

 </Routes>
);
}

export default App;
```

This is the heart of React Router.

---

## 15. Understanding `Routes`

Let's look at:

```
<Routes>
```

`Routes` is the container that holds our route definitions.

Inside it we define individual:

```
<Route />
```

components.

Think of it like:

`Routes`

```
├── Route → /
├── Route → /about
├── Route → /products
└── Route → /contact
```

React Router looks at the current URL and finds the appropriate route.

---

## 16. Understanding `Route`

A route looks like:

```
<Route
 path="/about"
 element={<About />}
/>
```

There are two important properties here.

```
path
path="/about"
```

This tells React Router:

Match the /about URL.

```
element
element={<About />}
```

This tells React Router:

When this URL matches, render the About component.

So:

```
/about
↓
<About />
```

---

## 17. Our Complete App.jsx

Let's look at the complete file.

```
import {
 Routes,
 Route
} from "react-router-dom";

import Home from "../pages/Home";
import About from "../pages/About";
import Products from "../pages/Products";
import Contact from "../pages/Contact";

function App() {

 return (
 <Routes>
```

```
 <Route
 path="/"
 element={<Home />}
 />

 <Route
 path="/about"
 element={<About />}
 />

 <Route
 path="/products"
 element={<Products />}
 />

 <Route
 path="/contact"
 element={<Contact />}
 />
 </Routes>
);
}

export default App;
```

Now start the application.

---

## 18. Testing the Routes

Open the application.

If the URL is:

/

we should see:

Home Page

Now manually change the URL to:

/about

We should see:

About Page

Now:

/products

should show:

Products Page

And:

/contact

should show:

Contact Page

Our routing is working.

---

## 19. But There Is a Problem

At the moment, we're manually typing URLs into the browser.

That's obviously not how a real application works.

We need navigation links.

For example:

Home | About | Products | Contact

When the user clicks:

Products

we want to navigate to:

/products

For this, React Router provides:

`<Link>`

---

## 20. The `Link` Component

Import:

```
import { Link } from "react-router-dom";
```

We can create:

```
<Link to="/">
 Home
</Link>
```

For About:

```
<Link to="/about">
 About
</Link>
```

Products:

```
<Link to="/products">
 Products
</Link>
```

And Contact:

```
<Link to="/contact">
 Contact
</Link>
```

---

## 21. Creating a Navigation Bar

Let's create:

src/components/Navbar.jsx

Add:

```
import { Link } from "react-router-dom";

function Navbar() {

 return (
 <nav>

 <Link to="/">
 Home
```

```
 </Link>

 {" | "}

 <Link to="/about">
 About
 </Link>

 {" | "}

 <Link to="/products">
 Products
 </Link>

 {" | "}

 <Link to="/contact">
 Contact
 </Link>

 </nav>
);
}

export default Navbar;
```

Now we need to display this navigation bar in our application.

---

## 22. Using Navbar in App

Update App.jsx:

```
import {
 Routes,
 Route
} from "react-router-dom";

import Navbar from "../components/Navbar";

import Home from "../pages/Home";
import About from "../pages/About";
import Products from "../pages/Products";
import Contact from "../pages/Contact";
```

```
function App() {
 return (
 <div>

 <Navbar />

 <Routes>

 <Route
 path="/"
 element={<Home />}
 />

 <Route
 path="/about"
 element={<About />}
 />

 <Route
 path="/products"
 element={<Products />}
 />

 <Route
 path="/contact"
 element={<Contact />}
 />

 </Routes>

 </div>
);
}
```

```
export default App;
```

Now we have:

Home | About | Products | Contact

Clicking the links changes the URL and displays the appropriate component.



## 23. Why Use `Link` Instead of `<a>`?

This is a very important question.

You may wonder:

Why don't we simply use an HTML anchor tag?

For example:

```

 Products

```

We can use regular HTML links, but in a React Router application, we generally use:

```
<Link to="/products">
 Products
</Link>
```

The `Link` component is designed for client-side navigation within the React application.

It allows React Router to handle the navigation instead of treating the destination like a traditional full-page navigation.

So remember:

React Router navigation  
↓  
`<Link>`

---

## 24. `href` VS `to`

An HTML anchor uses:

```

```

React Router's `Link` uses:

```
<Link to="/about">
```

So:

HTML  
`href`

React Router  
to

Don't write:

```
<Link href="/about">
```

That's incorrect.

Use:

```
<Link to="/about">
```

---

## 25. Adding Some CSS

Our navigation currently looks very basic.

Let's add some simple styling.

Navbar.jsx

```
import { Link } from "react-router-dom";

function Navbar() {

 return (
 <nav className="navbar">

 <Link to="/">
 Home
 </Link>

 <Link to="/about">
 About
 </Link>

 <Link to="/products">
 Products
 </Link>

 <Link to="/contact">
 Contact
 </Link>

 </nav>
);
}
```

```
}
```

```
export default Navbar;
```

And in our CSS:

```
.navbar {
 display: flex;
 gap: 20px;
 padding: 20px;
 border-bottom: 1px solid #ddd;
}
```

Now our navigation is easier to use.

---

## 26. Understanding the Complete Flow

Let's stop and understand what is happening.

Suppose the user clicks:

Products

The Link contains:

```
<Link to="/products">
```

React Router sees:

```
/products
```

It checks our route definitions.

We have:

```
<Route
 path="/products"
 element={<Products />}
/>
```

So React Router renders:

```
<Products />
```

The complete flow is:

User clicks Products  
↓

`<Link to="/products">`  
↓  
URL becomes `/products`  
↓  
Routes checks available routes  
↓  
Matches `/products`  
↓  
`<Products />` is rendered

This is the fundamental idea behind routing.

---

## 27. Layout Components

Our navigation should usually appear on every page.

Currently we have:

`<Navbar />`

`<Routes>`

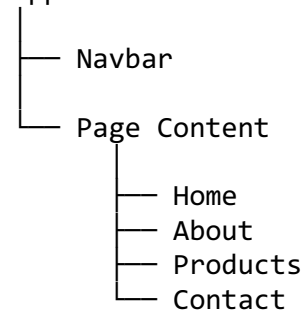
...

`</Routes>`

This is a simple way of creating a common layout.

Conceptually:

Application



Later, we can learn more advanced nested routing and layouts.

For now, this structure is enough.

## 28. NavLink

React Router also provides another useful component:

NavLink

NavLink is similar to Link, but it can tell us which link is currently active.

Import:

```
import { NavLink } from "react-router-dom";
```

Then:

```
<NavLink to="/">
 Home
</NavLink>
```

and:

```
<NavLink to="/products">
 Products
</NavLink>
```

This is particularly useful for navigation bars.

---

## 29. Why Use NavLink?

Suppose we're currently on:

/products

It would be useful if:

Products

were highlighted.

For example:

```
Home About [Products] Contact
 ↑
 Active
```

NavLink provides information about whether the link is active.

For example:

```
<NavLink
 to="/products"
 className={({ isActive }) =>
 isActive ? "active" : ""
 }
>
 Products
</NavLink>
```

When we're on /products, isActive is true.

Then the class becomes:

active

---

## 30. Simple `NavLink` Example

Our `Navbar.jsx` can become:

```
import { NavLink } from "react-router-dom";

function Navbar() {

 return (
 <nav className="navbar">

 <NavLink to="/">
 Home
 </NavLink>

 <NavLink to="/about">
 About
 </NavLink>

 <NavLink to="/products">
 Products
 </NavLink>

 <NavLink to="/contact">
 Contact
 </NavLink>

 </nav>
);
}
```

```
export default Navbar;
```

For today's lecture, the main focus is Link.

NavLink is an additional useful tool that you should know exists.

---

## 31. What Happens If the Route Doesn't Exist?

Suppose the user enters:

```
/random
```

But we haven't defined:

```
<Route path="/random" ... />
```

There is no matching route.

So we can create a fallback route.

For example:

```
<Route
 path="*"
 element={<h1>Page Not Found</h1>}
/>
```

The \* means:

Match anything that hasn't matched another route.

---

## 32. Creating a 404 Page

Let's create:

```
src/pages/NotFound.jsx
```

Add:

```
function NotFound() {
 return (
 <div>
```

```
 <h1>
 404
 </h1>

 <p>
 Page not found.
 </p>
 </div>
);
}

export default NotFound;
```

Then import it:

```
import NotFound from "../pages/NotFound";
```

And add:

```
<Route
 path="*"
 element={<NotFound />}
/>
```

Now if the user enters:

/random

React displays:

404

Page not found.

---

## 33. Complete Application

Let's put everything together.

Our project structure:

```
src
├── components
│ └── Navbar.jsx
└── pages
```



```
|— Home.jsx
|— About.jsx
|— Products.jsx
|— Contact.jsx
|— NotFound.jsx
|
|— App.jsx
|— main.jsx
```

```
main.jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";

import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
 <StrictMode>
 <BrowserRouter>
 <App />
 </BrowserRouter>
 </StrictMode>
);
```

```
App.jsx
import {
 Routes,
 Route
} from "react-router-dom";

import Navbar from "../components/Navbar";

import Home from "../pages/Home";
import About from "../pages/About";
import Products from "../pages/Products";
import Contact from "../pages/Contact";
import NotFound from "../pages/NotFound";

function App() {

 return (
 <div>
```

```
 <Navbar />

 <Routes>

 <Route
 path="/"
 element={<Home />}
 />

 <Route
 path="/about"
 element={<About />}
 />

 <Route
 path="/products"
 element={<Products />}
 />

 <Route
 path="/contact"
 element={<Contact />}
 />

 <Route
 path="*"
 element={<NotFound />}
 />

 </Routes>

 </div>
);
}

export default App;
```

---

```
Navbar.jsx
import { Link } from "react-router-dom";

function Navbar() {

 return (
 <nav className="navbar">
```

```
 <Link to="/">
 Home
 </Link>

 <Link to="/about">
 About
 </Link>

 <Link to="/products">
 Products
 </Link>

 <Link to="/contact">
 Contact
 </Link>

 </nav>
);
}

export default Navbar;
```

---

## 34. Testing the Application

Now let's test each route.

Home

/

Expected:

Home Page

About

/about

Expected:

About Page

Products

/products

Expected:

Products Page

Contact

/contact

Expected:

Contact Page

Invalid URL

/hello

Expected:

404

Page not found.

---

## 35. Important Concept — Route vs Component

Students often confuse these two.

A **component** is a React UI building block.

For example:

```
function Products() {
 return <h1>Products</h1>;
}
```

A **route** connects a URL to a component.

For example:

```
<Route
 path="/products"
 element={<Products />}
>
```

So:

Component



Products

Route



/products → Products

A route is essentially a mapping between a URL and UI.

---

## 36. Important Concept — Routing Doesn't Mean Multiple HTML Files

When we have:

```
/
/about
/products
/contact
```

we don't necessarily need:

```
home.html
about.html
products.html
contact.html
```

Instead, our React application can contain:

```
Home component
About component
Products component
Contact component
```

and React Router determines which component should be displayed.

That's one of the key ideas behind Single Page Applications.

---

## 37. Common Mistake — Forgetting BrowserRouter

If you use:

```
<Routes>
```

without having:

```
<BrowserRouter>
```

above it, React Router will complain because the routing components need router context.

Make sure your application is wrapped:

```
<BrowserRouter>
 <App />
</BrowserRouter>
```

---

## 38. Common Mistake — Wrong Import

Make sure your React Router components are imported from:

react-router-dom

For example:

```
import {
 BrowserRouter,
 Routes,
 Route,
 Link,
 NavLink
} from "react-router-dom";
```

---

## 39. Common Mistake — Using `href` With `Link`

Don't write:

```
<Link href="/products">
 Products
</Link>
```

Use:

```
<Link to="/products">
 Products
</Link>
```

Remember:

```
<a> → href
<Link> → to
```

---

## 40. Common Mistake — Incorrect Route Path

If you define:

```
<Route
 path="/product"
 element={<Products />}
/>
```

and visit:

/products

the route doesn't match.

Make sure the paths match exactly.

For example:

```
<Route
 path="/products"
 element={<Products />}
/>
```

---

## 41. Practical Exercise

Now I want you to create your own mini React website.

Create these pages:

Home  
Students  
Courses  
About  
Contact

Create these routes:

```
/
/students
/courses
/about
/contact
```

Create a navigation bar:

Home | Students | Courses | About | Contact

Each link should navigate to its corresponding page.

---

## 42. Bonus Exercise

Create a NotFound component.

If the user enters:

/anything

display:

404

Page Not Found

Add a link:

Go Home

that takes the user back to:

/

You can use:

```
<Link to="/">
 Go Home
</Link>
```

---

## 43. Interview Questions

Here are some questions you should be able to answer after this lecture.

### Question 1

What is routing?

**Answer:**

Routing determines which UI/component should be displayed for a particular URL.

---



## Question 2

What is a Single Page Application?

**Answer:**

An application where the browser can update the displayed UI without requiring a full page reload for every navigation.

---

## Question 3

Why do we use React Router?

**Answer:**

To implement client-side routing and navigation in a React application.

---

## Question 4

What does BrowserRouter do?

**Answer:**

It provides the routing context that allows React Router to track and handle browser URL navigation.

---

## Question 5

What does Route do?

For example:

```
<Route
 path="/about"
 element={<About />}
>
```

It maps the /about URL to the About component.

---

## Question 6

What is the difference between Link and NavLink?

Link provides navigation.

NavLink provides navigation and can also determine whether the current link is active.

---

## Question 7

Why use Link instead of a normal <a> tag?

Link is designed for navigation within a React Router application and avoids treating internal navigation like a traditional document navigation.

---

## 44. Lecture Summary

Let's summarize everything we learned today.

Routing allows us to connect URLs with React components.

We installed:

```
npm install react-router-dom
```

We used:

BrowserRouter

to enable routing.

We used:

Routes

to contain our routes.

We used:

Route

to map URLs to components.

For example:

```
<Route
 path="/products"
 element={<Products />}
>
```

We used:

Link

for navigation:

```
<Link to="/products">
 Products
</Link>
```

We also learned about:

NavLink

for navigation where we may want to identify the active route.

And finally, we created a fallback route:

```
<Route
 path="*"
 element={<NotFound />}
</Route>
```

---

## 45. Final Takeaway

The most important thing to remember is this:

```
URL
↓
Route
↓
React Component
↓
UI
```

For example:

```
/products
↓
<Route path="/products">
↓
<Products />
↓
Products Page
```

And navigation works like:

User clicks Products  
↓  
<Link to="/products">  
↓  
URL changes  
↓  
React Router finds matching Route  
↓  
Products component renders

At this point, we can build an application with multiple pages.

But there's still one important problem.

Our Products page currently looks like:

/products

What if we want to open a **specific product**?

For example:

/products/101

/products/102

/products/103

We don't want to create a separate route manually for every product.

We need a dynamic value in the URL.

That's exactly what we'll learn in the next lecture:

# Lecture 15 — Parameterized Routes in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned about **Routing in React**.

We created routes such as:

```
/
/about
/products
/contact
```

And we learned how to use:

```
<Route />
```

and:

```
<Link />
```

to navigate between different pages.

But let's think about a real-world application.

Suppose our application has 1,000 products.

We might have:

```
iPhone 17
Galaxy S26
OnePlus 14
MacBook Pro
Dell Laptop
...
```

When the user clicks a product, we want to open a product details page.

For example:

```
/products/101
```

Maybe product 101 is an iPhone.

Another product could be:

/products/102

Maybe product 102 is a Samsung phone.

Another:

/products/103

We cannot create a separate route manually for every product.

We don't want to write:

```
<Route path="/products/101" ... />
<Route path="/products/102" ... />
<Route path="/products/103" ... />
```

Instead, we want one dynamic route:

/products/:id

where :id can be:

101  
102  
103  
104  
...

This is called a **Parameterized Route** or **Dynamic Route**.

In today's lecture, we'll learn:

- What parameterized routes are
- Why we need them
- Route parameters
- :id
- useParams
- Creating product detail pages
- Passing dynamic URLs using Link
- Combining .map() with parameterized routes
- Multiple route parameters
- Common mistakes
- A complete product-details example

Let's get started.

---

## 2. The Problem We Are Trying to Solve

Let's imagine our product list:

Products

iPhone 17  
[View Details]

Galaxy S26  
[View Details]

OnePlus 14  
[View Details]

When the user clicks the first product, we want:

/products/101

Second:

/products/102

Third:

/products/103

The last part of the URL identifies the product.

So we can think of the URL as:

/products/101  
          ↑  
      Product ID

This ID is dynamic.

That's why we need a parameterized route.

---

## 3. Static Route vs Dynamic Route

Let's compare the two.

In the previous lecture, we created:

```
<Route
 path="/products"
 element={<Products />}
/>
```

This is a **static route**.

It always matches:

/products

Now imagine:

/products/101

/products/102

/products/103

We can define:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

Here:

:id

is a route parameter.

It can contain different values.

For example:

/products/101

↓

id = 101

or:

/products/102

↓

id = 102

---

## 4. Understanding :id

This syntax:



:id

is important.

The colon tells React Router:

This part of the URL is a dynamic parameter.

For example:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

The following URLs can match this route:

```
/products/101
/products/102
/products/999
```

The value changes, but the route structure remains the same.

Conceptually:

```
/products/:id
 ↓
 101
```

```
/products/:id
 ↓
 102
```

```
/products/:id
 ↓
 999
```

---

## 5. Creating the Product Details Component

Let's create:

src/pages/ProductDetails.jsx

Initially, we'll create a simple component:

```
function ProductDetails() {
```

```
 return (
 <div>

 <h1>
 Product Details
 </h1>

 <p>
 Product details will appear here.
 </p>

 </div>
);
 }

 export default ProductDetails;
```

Now we need to create a route for it.

---

## 6. Creating the Parameterized Route

Open:

App.jsx

Import:

```
import ProductDetails from "../pages/ProductDetails";
```

Then add:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

So our routes might now look like:

```
<Routes>

 <Route
 path="/"
 element={<Home />}
 />

 <Route
```

```
 path="/products"
 element={<Products />}
 />

 <Route
 path="/products/:id"
 element={<ProductDetails />}
 />

 <Route
 path="/about"
 element={<About />}
 />

</Routes>
```

Now let's test it.

Open:

/products/101

The ProductDetails component should render.

Try:

/products/102

The same component renders.

Try:

/products/500

Again, the same component renders.

So far, we know that the route matches.

But we haven't actually received the 101, 102, or 500.

How do we get that value?

---

## 7. Reading the Parameter With `useParams`

React Router provides a Hook called:

`useParams`

Import it:

```
import { useParams } from "react-router-dom";
```

Then inside our component:

```
function ProductDetails() {

 const params = useParams();

 console.log(params);

 return (
 <div>

 <h1>
 Product Details
 </h1>

 </div>
);
}
```

Now visit:

/products/101

Open the browser console.

You should see something conceptually like:

```
{
 id: "101"
}
```

Notice something important.

The value is:

"101"

not:

101

It is a string.

## 8. Getting `id` Directly

Instead of:

```
const params = useParams();
```

we can use destructuring:

```
const { id } = useParams();
```

Now we can directly use:

`id`

For example:

```
function ProductDetails() {
 const { id } = useParams();

 return (
 <div>
 <h1>
 Product Details
 </h1>

 <p>
 Product ID: {id}
 </p>
 </div>
);
}
```

Now visit:

`/products/101`

We should see:

Product Details

Product ID: 101

Change the URL to:

`/products/102`

and we get:

Product ID: 102

---

## 9. Understanding the Complete Flow

Let's understand exactly what happened.

We defined:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

Then the user visits:

/products/101

React Router matches:

/products/:id

and extracts:

id = "101"

Then inside:

ProductDetails

we use:

```
const { id } = useParams();
```

and get:

101

The complete flow is:

```
URL
↓
/products/101
↓
Route
↓
/products/:id
↓
```

useParams()

↓

id = "101"

↓

ProductDetails component

This is the key concept of today's lecture.

---

## 10. Live Coding — Displaying the Product ID

Let's make our component slightly better.

```
import { useParams } from "react-router-dom";
```

```
function ProductDetails() {
 const { id } = useParams();

 return (
 <div>
 <h1>
 Product Details
 </h1>

 <p>
 Product ID: {id}
 </p>
 </div>
);
}
```

```
export default ProductDetails;
```

Now test:

/products/101

Then:

/products/102

Then:

/products/103

The component remains the same.

Only the parameter changes.

---

## 11. Why Is This Useful?

This becomes extremely useful when working with data.

Imagine we have:

```
const products = [
 {
 id: 101,
 name: "iPhone 17",
 price: 79999
 },
 {
 id: 102,
 name: "Galaxy S26",
 price: 69999
 },
 {
 id: 103,
 name: "OnePlus 14",
 price: 54999
 }
];
```

The product list can generate links dynamically.

For product 101:

/products/101

For product 102:

/products/102

For product 103:

/products/103

And when the user visits the details page, we can read that ID.

This is exactly how many real-world applications work.

---



## 12. Combining `.map()` and Parameterized Routes

Let's return to our Products page.

Suppose:

```
function Products() {

 const products = [
 {
 id: 101,
 name: "iPhone 17",
 price: 79999
 },
 {
 id: 102,
 name: "Galaxy S26",
 price: 69999
 },
 {
 id: 103,
 name: "OnePlus 14",
 price: 54999
 }
];

 return (
 <div>

 <h1>
 Products
 </h1>

 {products.map(product => (
 <div key={product.id}>

 <h2>
 {product.name}
 </h2>

 <p>
 ₹{product.price}
 </p>

 <Link
 to={` /products/${product.id}`}
 >

```

View Details  
</Link>

```
 </div>
)})
</div>
);
}
```

Notice this:

```
to={` /products/${product.id}`} }
```

This is very important.

---

## 13. Template Literals

The following:

```
to={` /products/${product.id}`} }
```

uses a JavaScript template literal.

For example:

```
const id = 101;
```

```
const url = `/products/${id}`;
```

The result is:

```
/products/101
```

If:

```
const id = 102;
```

the result becomes:

```
/products/102
```

So our code dynamically creates the correct URL.

---

## 14. Complete Products Page

Let's create the complete example.

### Products.jsx

```
import { Link } from "react-router-dom";

function Products() {

 const products = [
 {
 id: 101,
 name: "iPhone 17",
 price: 79999
 },
 {
 id: 102,
 name: "Galaxy S26",
 price: 69999
 },
 {
 id: 103,
 name: "OnePlus 14",
 price: 54999
 }
];

 return (
 <div>

 <h1>
 Products
 </h1>

 {products.map(product => (
 <div key={product.id}>

 <h2>
 {product.name}
 </h2>

 <p>
 Price: ₹{product.price}
 </p>

```

```
 <Link
 to={` /products/${product.id}`}
 >
 View Details
 </Link>

 </div>

)))}

</div>
);
}
```

**export default** Products;

Now the screen might look like:

Products

iPhone 17  
Price: ₹79999  
View Details

Galaxy S26  
Price: ₹69999  
View Details

OnePlus 14  
Price: ₹54999  
View Details

---

## 15. What Happens When We Click “View Details”?

Let’s click the first product.

The code generates:

/products/101

React Router sees:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

It extracts:

```
id = "101"
```

Then:

```
const { id } = useParams();
```

gives us:

```
101
```

So the application can now identify which product the user selected.

---

## 16. Building a Real Product Details Page

Now let's make the details page more useful.

Suppose we have:

```
const products = [
 {
 id: 101,
 name: "iPhone 17",
 price: 79999,
 category: "Mobile"
 },
 {
 id: 102,
 name: "Galaxy S26",
 price: 69999,
 category: "Mobile"
 },
 {
 id: 103,
 name: "OnePlus 14",
 price: 54999,
 category: "Mobile"
 }
];
```

Our details page receives:

```
id = 101
```

We can find the matching product:

```
const product = products.find(
 product => product.id === Number(id)
);
```

Notice the:

`Number(id)`

Why?

Because `useParams()` gives us:

`"101"`

as a string.

But our product IDs are:

`101`

numbers.

So we convert:

`"101"`

to:

`101`

---

## 17. Complete Product Details Example

Let's write it.

`ProductDetails.jsx`

```
import { useParams } from "react-router-dom";
```

```
function ProductDetails() {

 const { id } = useParams();

 const products = [
 {
 id: 101,
 name: "iPhone 17",
 price: 79999,
 category: "Mobile"
 }
]
}
```

```
 },
 {
 id: 102,
 name: "Galaxy S26",
 price: 69999,
 category: "Mobile"
 },
 {
 id: 103,
 name: "OnePlus 14",
 price: 54999,
 category: "Mobile"
 }
];

 const product = products.find(
 product => product.id === Number(id)
);

 return (
 <div>

 <h1>
 Product Details
 </h1>

 <h2>
 {product.name}
 </h2>

 <p>
 Price: ₹{product.price}
 </p>

 <p>
 Category: {product.category}
 </p>

 </div>
);
}

export default ProductDetails;

Now:

/products/101
```

shows:

Product Details

iPhone 17

Price: ₹79999

Category: Mobile

And:

/products/102

shows:

Product Details

Galaxy S26

Price: ₹69999

Category: Mobile

---

## 18. An Important Problem — Invalid IDs

What happens if the user enters:

/products/999

but product 999 doesn't exist?

Our:

```
find()
```

will return:

**undefined**

Then this:

```
product.name
```

will cause an error.

So we need conditional rendering.

This connects directly to our previous lecture.

---



## 19. Handling a Missing Product

We can write:

```
if (!product) {
 return (
 <h1>
 Product Not Found
 </h1>
);
}
```

Now our component becomes:

```
import { useParams } from "react-router-dom";

function ProductDetails() {
 const { id } = useParams();

 const products = [
 {
 id: 101,
 name: "iPhone 17",
 price: 79999
 },
 {
 id: 102,
 name: "Galaxy S26",
 price: 69999
 },
 {
 id: 103,
 name: "OnePlus 14",
 price: 54999
 }
];

 const product = products.find(
 product => product.id === Number(id)
);

 if (!product) {
 return (
 <h1>
 Product Not Found
 </h1>
);
 }
}
```

```
}

return (
 <div>

 <h1>
 Product Details
 </h1>

 <h2>
 {product.name}
 </h2>

 <p>
 Price: ₹{product.price}
 </p>

 </div>
);
}

export default ProductDetails;
```

Now:

/products/999

displays:

Product Not Found

This is a practical example of combining:

Parameterized Routes

+

useParams

+

Conditional Rendering

---

## 20. The Complete Flow

Let's pause and visualize the entire process.

We have our product:

```
{
 id: 101,
 name: "iPhone 17"
}
```

The Products page generates:

/products/101

The route is:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
</>
```

React Router extracts:

id = "101"

useParams() gives us:

```
{
 id: "101"
}
```

We convert it:

Number(id)

Then find the product:

```
products.find(
 product => product.id === Number(id)
)
```

And finally render the details.

So:

```
Product
 ↓
Link
 ↓
/products/101
 ↓
Route
 ↓
useParams()
 ↓
id = "101"
```

↓  
find product  
↓  
Product Details

This is the most important flow in today's lecture.

---

## 21. Multiple Route Parameters

Route parameters aren't limited to one parameter.

For example, imagine an e-commerce website with categories.

We might have:

/products/mobile/101

Here we have two dynamic values:

```
category = mobile
id = 101
```

The route can be:

```
<Route
 path="/products/:category/:id"
 element={<ProductDetails />}
>
```

Now:

/products/mobile/101

gives:

```
{
 category: "mobile",
 id: "101"
}
```

And we can retrieve both:

```
const {
 category,
 id
} = useParams();
```

## 22. Example With Category and ID

Suppose:

/products/mobile/101

Our component can do:

```
import { useParams } from "react-router-dom";

function ProductDetails() {

 const {
 category,
 id
 } = useParams();

 return (
 <div>

 <h1>
 Product Details
 </h1>

 <p>
 Category: {category}
 </p>

 <p>
 Product ID: {id}
 </p>

 </div>
);
}

export default ProductDetails;
```

Output:

Product Details

Category: mobile

Product ID: 101

## 23. Route Parameter Names Are Up to You

We don't have to call the parameter:

```
:id
```

We could call it:

```
:productId
```

For example:

```
<Route
 path="/products/:productId"
 element={<ProductDetails />}
/>
```

Then:

```
const { productId } = useParams();
```

If the URL is:

```
/products/101
```

we get:

```
productId = "101"
```

The parameter name should describe what it represents.

So:

```
:id
```

is simple.

But:

```
:productId
```

can sometimes be clearer.

---

## 24. Dynamic Routes Are Not the Same as Query Parameters

This is a useful distinction.

A URL like:

/products/101

uses a **path parameter**.

The 101 is part of the URL path.

A URL like:

/products?category=mobile

uses a **query parameter**.

We will not go deeply into query parameters today.

For this lecture, remember:

/products/101

→ Route parameter.

---

## 25. Common Mistake — Forgetting the Colon

This is incorrect if you want a dynamic parameter:

```
<Route
 path="/products/id"
 element={<ProductDetails />}
/>
```

This means the literal URL:

/products/id

It does **not** mean that id is dynamic.

Correct:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

The colon is important:

:id

## 26. Common Mistake — Wrong Parameter Name

Suppose your route is:

```
<Route
 path="/products/:productId"
 element={<ProductDetails />}
/>
```

Then you should use:

```
const { productId } = useParams();
```

Not:

```
const { id } = useParams();
```

The names must match.

For example:

Route:  
:productId

useParams:  
productId

---

## 27. Common Mistake — Forgetting That Params Are Strings

Suppose:

```
const { id } = useParams();
```

and the URL is:

/products/101

Then:

```
typeof id
```

will be:

"string"

So:



```
id === 101
```

is false because:

```
"101" !== 101
```

That's why we may need:

```
Number(id)
```

when comparing against numeric IDs.

For example:

```
product.id === Number(id)
```

---

## 28. Common Mistake — Assuming the Product Exists

Don't assume that:

```
/products/999
```

must exist.

Users can manually type URLs.

So always consider:

```
Valid ID
Invalid ID
```

and provide a useful fallback:

```
Product Not Found
```

This is good application design.

---

## 29. Improving the Product Details Page

Let's add a Back button.

Import:

```
import {
 Link,
```

```
 useParams
 } from "react-router-dom";
```

Then:

```
<Link to="/products">
 Back to Products
</Link>
```

Our page can now look like:

Product Details

iPhone 17

Price: ₹79999

Category: Mobile

Back to Products

---

## 30. Complete Final Example

Let's now create the complete mini application.

App.jsx

```
import {
 Routes,
 Route
} from "react-router-dom";

import Home from "../pages/Home";
import Products from "../pages/Products";
import ProductDetails from "../pages/ProductDetails";

function App() {

 return (
 <Routes>

 <Route
 path="/"
 element={<Home />}
 />

 <Route
```

```
 path="/products"
 element={<Products />}
 />

 <Route
 path="/products/:id"
 element={<ProductDetails />}
 />

</Routes>
);
}

export default App;
```

---

Products.jsx

```
import { Link } from "react-router-dom";

function Products() {

 const products = [
 {
 id: 101,
 name: "iPhone 17",
 price: 79999
 },
 {
 id: 102,
 name: "Galaxy S26",
 price: 69999
 },
 {
 id: 103,
 name: "OnePlus 14",
 price: 54999
 }
];

 return (
 <div>

 <h1>
 Products
 </h1>


```

```
 {products.map(product => (
 <div key={product.id}>
 <h2>
 {product.name}
 </h2>
 <p>
 Price: ₹{product.price}
 </p>
 <Link
 to={` /products/${product.id}`}
 >
 View Details
 </Link>
 </div>
))}
 </div>
);
}

export default Products;
```

---

```
ProductDetails.jsx
import {
 Link,
 useParams
} from "react-router-dom";

function ProductDetails() {
 const { id } = useParams();
 const products = [
 {
 id: 101,
 name: "iPhone 17",
 price: 79999,
 category: "Mobile"
 },
],
```

```
{
 id: 102,
 name: "Galaxy S26",
 price: 69999,
 category: "Mobile"
},
{
 id: 103,
 name: "OnePlus 14",
 price: 54999,
 category: "Mobile"
}
];

const product = products.find(
 product => product.id === Number(id)
);

if (!product) {
 return (
 <div>

 <h1>
 Product Not Found
 </h1>

 <Link to="/products">
 Back to Products
 </Link>

 </div>
);
}

return (
 <div>

 <h1>
 Product Details
 </h1>

 <h2>
 {product.name}
 </h2>

 <p>
 Price: ₹{product.price}
 </p>
 </div>
);
```

```
 </p>

 <p>
 Category: {product.category}
 </p>

 <Link to="/products">
 Back to Products
 </Link>

 </div>
);
}

export default ProductDetails;
```

## 31. Demonstrating the Application

Start the application.

Navigate to:

/products

You should see:

Products

iPhone 17  
₹79999  
View Details

Galaxy S26  
₹69999  
View Details

OnePlus 14  
₹54999  
View Details

Click:

View Details

for iPhone.

The URL becomes:

`/products/101`

The application reads:

`id = "101"`

and finds:

iPhone 17

Then displays:

Product Details

iPhone 17

Price: ₹79999

Category: Mobile

Back to Products

Click Galaxy:

`/products/102`

Now the same component displays:

Galaxy S26

This is the power of parameterized routes.

---

## 32. Why This Is Better Than Creating Individual Routes

Imagine 10,000 products.

Without parameters, we'd potentially need:

```
<Route path="/products/1" ... />
<Route path="/products/2" ... />
<Route path="/products/3" ... />
...
<Route path="/products/10000" ... />
```

That's obviously not practical.

With a parameterized route:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

one route handles all of them.

```
/products/1
/products/2
/products/3
...
/products/10000
```

This is one of the major benefits of dynamic routing.

---

## 33. Practical Exercise 1 — Student Details

Create a route:

```
/students/:id
```

For example:

```
/students/101
```

should display:

Student ID: 101

Use:

```
const { id } = useParams();
```

---

## 34. Practical Exercise 2 — Course Details

Create:

```
/courses/:courseId
```

For example:

```
/courses/react
```

should display:



Course: react

Then try:

/courses/javascript

and display:

Course: javascript

---

## 35. Practical Exercise 3 — Product List

Create an array:

```
const products = [
 {
 id: 1,
 name: "Laptop"
 },
 {
 id: 2,
 name: "Mobile"
 },
 {
 id: 3,
 name: "Tablet"
 }
];
```

Render them using `.map()`.

Each product should have:

View Details

The links should dynamically become:

```
/products/1
/products/2
/products/3
```

---

## 36. Practical Exercise 4 — Invalid Product

Try:

/products/999

If the product doesn't exist, display:

Product Not Found

Also provide:

Back to Products

This exercise combines everything we've learned.

---

## 37. Interview Questions

### Question 1

What is a parameterized route?

**Answer:**

A route containing dynamic values that can change depending on the URL.

Example:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

---

### Question 2

What does :id mean?

It represents a dynamic route parameter.

---

### Question 3

How do you retrieve a route parameter?

Using:

`useParams()`

For example:

```
const { id } = useParams();
```

---

### Question 4

What value does `useParams()` return?

It returns an object containing the route parameters.

For:

`/products/101`

and:

```
path="/products/:id"
```

we get:

```
{
 id: "101"
}
```

---

### Question 5

Why might we use `Number(id)`?

Because route parameters are strings, while our product ID may be stored as a number.

---

### Question 6

Can we have multiple parameters?

Yes.

For example:

```
<Route
 path="/products/:category/:id"
 element={<ProductDetails />}
</>
```

Then:

```
const {
 category,
```

```
 id
 } = useParams();
```

---

## 38. Lecture Summary

Let's summarize today's lecture.

A static route looks like:

```
<Route
 path="/products"
 element={<Products />}
>
```

A parameterized route looks like:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
>
```

The `:id` is dynamic.

We retrieve it using:

```
const { id } = useParams();
```

We can dynamically create URLs:

```
<Link
 to={` /products/${product.id}`}
>
 View Details
</Link>
```

We can then use that ID to find the correct product:

```
const product = products.find(
 product => product.id === Number(id)
);
```

And we should handle the case where the product doesn't exist:

```
if (!product) {
 return <h1>Product Not Found</h1>;
}
```

---

## 39. Final Takeaway

The most important pattern from today's lecture is:

```
Product List
 ↓
Click View Details
 ↓
/products/101
 ↓
Route: /products/:id
 ↓
useParams()
 ↓
id = "101"
 ↓
Find Product
 ↓
Product Details
```

This pattern is used everywhere in modern applications.

For example:

/users/25

can display a user.

/orders/5001

can display an order.

/students/101

can display a student.

/courses/react

can display a course.

The URL contains the identifier, and the component uses that identifier to determine what data to display.

# Lecture 16 — Introduction to Hooks in React

## 1. Opening

Hello everyone, and welcome back.

So far in this React series, we've covered many important concepts.

We started with:

- ReactJS introduction
- JSX
- Expressions
- Styling
- Events
- Components
- Function Components
- Class Components
- Props
- React DevTools
- Arrays
- Conditional Rendering
- Routing
- Parameterized Routes

Up to this point, most of our examples have been relatively simple.

But now we are going to start one of the **most important parts of modern React**.

And that is:

## React Hooks

In today's lecture, we're going to understand:

- What are Hooks?
- Why were Hooks introduced?
- What problems did Hooks solve?

- Why are Hooks mainly used with function components?
- Common Hooks
- Rules of Hooks
- How to identify a Hook
- A first look at `useState`
- A first look at `useEffect`

We won't go deeply into `useState` and `useEffect` today.

Those will be covered in the next lectures.

Today's goal is to understand the **concept of Hooks**.

---

## 2. What Is a Hook?

Let's start with a simple definition.

**A Hook is a special function provided by React that allows function components to use React features.**

For example, React provides a Hook called:

`useState`

which allows a function component to maintain state.

React also provides:

`useEffect`

which allows us to perform side effects.

There are many other Hooks, such as:

`useState`  
`useEffect`  
`useContext`  
`useReducer`  
`useRef`  
`useMemo`  
`useCallback`

We'll discuss some of these later.

For now, remember:

Hooks allow function components to use important React features.

---

### 3. Why Were Hooks Introduced?

This is one of the most important questions.

If you remember from our earlier lecture, React has two common types of components:

Function Component  
Class Component

Before Hooks were introduced, if we wanted to use certain React features such as state and lifecycle methods, we commonly used **class components**.

For example:

```
class Counter extends React.Component {

 state = {
 count: 0
 };

 render() {

 return (
 <div>

 <h1>
 {this.state.count}
 </h1>

 </div>
);
 }
}
```

This works.

But the syntax is more complicated.

We have:

```
class
extends
this
```



state  
render()

and other concepts associated with class components.

React Hooks gave developers a way to use these capabilities inside function components.

---

## 4. Before Hooks

Let's look at an old-style example.

Suppose we want a counter.

We could write:

```
class Counter extends React.Component {
 constructor(props) {
 super(props);
 this.state = {
 count: 0
 };
 }

 increment = () => {
 this.setState({
 count: this.state.count + 1
 });
 };

 render() {
 return (
 <div>

 <h1>
 {this.state.count}
 </h1>

 <button onClick={this.increment}>
 Increment
 </button>
 </div>
);
 }
}
```

```
 </div>
);
}
}
```

There's nothing wrong with this code.

But for beginners, there are several things to understand:

constructor  
super  
this.state  
this.setState  
this  
class  
render

Now let's see how modern React approaches the same problem.

---

## 5. Function Component + Hook

With Hooks, we can write a function component:

```
function Counter() {
 return (
 <div>
 <h1>
 0
 </h1>
 <button>
 Increment
 </button>
 </div>
);
}
```

This is much simpler.

But there is a problem.

Where do we store the value?

We need some way to remember:

```
count = 0
```

And when the button is clicked:

```
count = 1
```

Then:

```
count = 2
```

This is where:

`useState`

comes in.

---

## 6. First Look at `useState`

Let's write:

```
import { useState } from "react";
```

Then:

```
function Counter() {
 const [count, setCount] = useState(0);

 return (
 <div>
 <h1>
 {count}
 </h1>

 <button
 onClick={() => setCount(count + 1)}
 >
 Increment
 </button>
 </div>
);
}
```

Don't worry if this code isn't completely clear yet.

That's exactly what Lecture 17 will cover.

For today's lecture, just notice that:

```
useState(0)
```

allows our function component to have a value that React can remember.

---

## 7. What Does “State” Mean?

Before understanding Hooks, we should understand the word:

### State

State is basically **data that can change over time and that affects what the component displays or does.**

For example:

Counter

has:

```
count = 0
```

After clicking the button:

```
count = 1
```

After another click:

```
count = 2
```

The value changes.

That changing data is called **state**.

Other examples of state include:

```
isLoggedIn
username
selectedProduct
shoppingCart
searchText
```

isLoading  
isModalOpen

For example:

isLoggedIn = false

Then after login:

isLoggedIn = true

That's state.

---

## 8. State vs Normal Variable

This is a very important concept.

A beginner might write:

```
function Counter() {

 let count = 0;

 return (
 <button>
 {count}
 </button>
);
}
```

Then perhaps try:

count++;

But changing a normal JavaScript variable does not tell React:

“Please render the component again.”

React needs to know when data that affects the UI changes.

That's why we use:

useState

Instead of just:

```
let count = 0;
```

With state, React can keep track of the value and update the UI when the state changes.

---

## 9. Live Coding — Normal Variable Problem

Let's demonstrate this.

Create:

```
function Counter() {
 let count = 0;

 const increment = () => {
 count++;
 console.log(count);
 };

 return (
 <div>
 <h1>
 {count}
 </h1>

 <button onClick={increment}>
 Increment
 </button>

 </div>
);
}

export default Counter;
```

Now click the button.

The console may show:

```
1
2
3
4
```

But the screen doesn't update as expected.

Why?

Because count is just a normal JavaScript variable.

React doesn't know that the value changed.

This is one of the reasons we need state.

---

## 10. Using `useState`

Now let's change the example.

First:

```
import { useState } from "react";
```

Then:

```
function Counter() {
 const [count, setCount] = useState(0);

 const increment = () => {
 setCount(count + 1);
 };

 return (
 <div>
 <h1>
 {count}
 </h1>

 <button onClick={increment}>
 Increment
 </button>

 </div>
);
}
```

Now React knows that:

`setCount()`

is changing state.

React can update the component's UI accordingly.

We'll explain exactly how this works in Lecture 17.

---

## 11. Why Is It Called a Hook?

You may be wondering:

Why do React call these functions “Hooks”?

The idea is that Hooks allow a function component to **hook into React's features**.

For example:

`useState`  
↓  
Hook into React state

`useEffect`  
↓  
Hook into React's effect system

`useContext`  
↓  
Hook into React Context

So the word “Hook” represents the idea of connecting a function component to React functionality.

---

## 12. Common React Hooks

React provides several Hooks.

Let's look at some of the important ones.

`useState`

Used for state.

```
const [count, setCount] = useState(0);
```



We'll study this in detail in the next lecture.

---

### useEffect

Used for side effects.

For example:

```
useEffect(() => {
 console.log("Component rendered");
}, []);
```

We'll study useEffect in Lecture 18.

---

### useContext

Used to access data from React Context.

For example, applications may use Context for:

- Theme
- Authentication
- Language
- User information

---

### useRef

Used to keep a mutable reference or access a DOM element.

For example:

```
const inputRef = useRef();
```

---

### useReducer

Useful for managing more complex state logic.

---

### useMemo

Used to memoize a calculated value.

---

## `useCallback`

Used to memoize a function reference.

We don't need to master all of these right now.

Our immediate focus is:

`useState`  
`useEffect`

---

## 13. Hooks Start With `use`

One easy way to recognize React Hooks is that their names generally start with:

`use`

Examples:

`useState`  
`useEffect`  
`useContext`  
`useRef`  
`useReducer`  
`useMemo`  
`useCallback`

This naming convention is important.

If you create your own Hook, it should also generally start with:

`use`

For example:

`useProducts()`

or:

`useAuthentication()`

These are called **custom Hooks**.

We won't create custom Hooks in this lecture.

---

## 14. Rules of Hooks

Hooks have some important rules.

React expects us to follow these rules so that Hooks work correctly.

There are two fundamental rules we should remember.

### Rule 1 — Only Call Hooks at the Top Level

Don't call Hooks inside:

- if
- for
- while
- nested functions

For example, this is incorrect:

```
function Component() {
 if (isLoggedIn) {
 const [name, setName] = useState("");
 }

 return <div />;
}
```

Don't do this.

Instead:

```
function Component() {
 const [name, setName] = useState("");

 if (!isLoggedIn) {
 return <div>Please Login</div>;
 }

 return <div>{name}</div>;
}
```

The Hook is declared at the top level.

---

## 15. Why Can't We Put Hooks Inside Conditions?

Consider:

```
if (isLoggedIn) {
 useState();
}
```

React needs to know the order in which Hooks are called.

For example:

```
First Hook
Second Hook
Third Hook
```

If we put Hooks inside conditions, that order could change between renders.

That can confuse React about which state belongs to which Hook.

So React requires Hooks to be called consistently.

For beginners, remember:

Keep Hooks at the top level of your function component.

---

## 16. Rule 2 — Only Call Hooks From React Functions

Hooks should be called from:

- React function components
- Custom Hooks

For example:

```
function Counter() {
 const [count, setCount] = useState(0);
 return <h1>{count}</h1>;
}
```

This is correct.

But don't call a Hook from an ordinary JavaScript function:

```
function calculateSomething() {
 const [value, setValue] = useState(0);
}
```

That's not a React component or custom Hook.

Don't do this.

---

## 17. Component Names vs Hook Names

React uses naming conventions.

A React component usually starts with an uppercase letter:

```
function Counter() {}
```

A Hook starts with:

use

For example:

useProducts

So if you create a custom Hook:

```
function useProducts() {
 // Hook Logic
}
```

The name begins with:

use

This helps React tooling and developers understand that it is a Hook.

---

## 18. Hooks and Function Components

An important point for modern React development is:

**Hooks are designed to be used in function components.**

For example:

```
function User() {

 const [name, setName] = useState("Arjun");

 return (
 <h1>
 {name}
 </h1>
);
}
```

This is the style you'll see in most modern React applications.

We can write React applications primarily using function components and Hooks.

---

## 19. Do We Still Need Class Components?

This is a common question.

Class components are still part of React's history and are supported in React applications.

However, modern React development generally favors:

Function Components

+

Hooks

rather than writing new components as classes.

You should still understand class components because:

- Many existing projects use them
- Older tutorials may use them
- You may encounter them in a job
- Understanding them helps you understand React's evolution

But for our series, from this point forward, we'll primarily use:

Function Components

+

Hooks

---

## 20. Hook Example — `useState`

Let's look at another simple example.

Suppose we have a login button.

```
function Login() {
 const [isLoggedIn, setIsLoggedIn] = useState(false);

 return (
 <div>
 <h1>
 {isLoggedIn
 ? "Welcome User"
 : "Please Login"
 }
 </h1>

 <button
 onClick={() => setIsLoggedIn(true)}
 >
 Login
 </button>
 </div>
);
}
```

Initially:

`isLoggedIn = false`

The screen shows:

Please Login

When the button is clicked:

`setIsLoggedIn(true);`

the state changes to:

`true`

and the UI changes to:

Welcome User

This demonstrates an important React concept:

State changes



React re-renders



UI updates

We'll explore this in much more detail in the next lecture.

---

## 21. Hook Example — `useEffect`

Now let's look at the second important Hook in our series.

`useEffect`

Suppose we want to execute some code after the component renders.

For example:

```
import {
 useEffect
} from "react";

function App() {

 useEffect(() => {

 console.log("Component rendered");

 }, []);

 return (
 <h1>
 Hello React
 </h1>
);
}
```

When the component is rendered, the effect runs.

This is useful for things such as:

API calls

Timers

Subscriptions



Browser APIs  
Updating document title  
Event listeners

We'll study this in Lecture 18.

---

## 22. What Is a Side Effect?

The term **side effect** is important when discussing `useEffect`.

A side effect is an operation that interacts with something outside the normal rendering process.

Examples:

- Calling an API
- Starting a timer
- Changing `document.title`
- Adding an event listener
- Reading browser storage
- Connecting to a subscription

For example:

```
document.title = "Products";
```

changes something outside the React component's rendered output.

Or:

```
fetch("/api/products");
```

communicates with an external resource.

These are examples of operations that we may manage using `useEffect`.

---

## 23. `useState` VS `useEffect`

At this point, students often confuse these two Hooks.

Let's make the distinction very simple.

`useState`

Used to store changing data.

```
const [count, setCount] = useState(0);
```

Think:

“My component needs to remember this value.”

---

`useEffect`

Used to perform side effects.

```
useEffect(() => {
 // side effect
```

```
}, []);
```

Think:

“After rendering, I need to perform some operation.”

So:

`useState`  
→ State

`useEffect`  
→ Side Effects

We'll explore both deeply in the next two lectures.

---

## 24. A Small Live-Coding Example

Let's create a component combining the two.

```
import {
 useState,
 useEffect
} from "react";

function Counter() {
 const [count, setCount] = useState(0);
 useEffect(() => {
```

```
 console.log(
 "Current count:",
 count
);
 }, [count]);

 return (
 <div>

 <h1>
 {count}
 </h1>

 <button
 onClick={() => setCount(count + 1)}
 >
 Increment
 </button>

 </div>
);
}

export default Counter;
```

Don't worry about this line yet:

```
}, [count]);
```

We'll explain the dependency array carefully in Lecture 18.

For now, just notice the relationship:

```
useState
 ↓
Stores count

setCount()
 ↓
Changes count

React renders again
 ↓
useEffect runs
 ↓
Console displays new count
```

This is a very important pattern in React.

---

## 25. Hooks Don't Replace JavaScript

Another important point.

Hooks are not a replacement for JavaScript.

We still use normal JavaScript:

variables  
functions  
arrays  
objects  
map()  
filter()  
find()  
if  
ternary operator  
destructuring  
template literals

Hooks simply provide React-specific functionality.

For example:

```
const products = productsArray.filter(
 product => product.price > 50000
);
```

That's normal JavaScript.

But:

```
const [products, setProducts] = useState([]);
```

uses a React Hook.

A good React developer needs both:

Strong JavaScript  
+  
Understanding React  
+  
Understanding Hooks

---

## 26. Hooks Are Not Magic

When you first see:

```
const [count, setCount] = useState(0);
```

it can look strange.

Don't think of Hooks as magic.

Think of them as APIs provided by React.

For example:

```
useState()
```

is a React function that gives your component access to React's state mechanism.

Similarly:

```
useEffect()
```

provides a way to work with side effects.

As we use them more, the syntax becomes natural.

---

## 27. A Simple Mental Model

Here's a simple mental model I recommend.

Think of a function component as:

```
Component
 ↓
Receives data
 ↓
Calculates UI
 ↓
Returns JSX
```

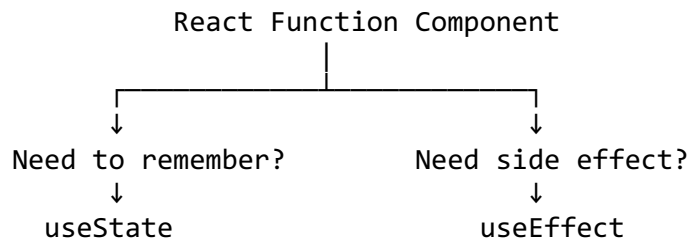
When we need the component to **remember something**:

`useState`

When we need the component to **perform a side effect**:

`useEffect`

So:



This mental model will help you understand the next two lectures.

## 28. Common Beginner Mistakes

Let's discuss some common mistakes.

### Mistake 1 — Calling Hooks inside `if`

Incorrect:

```
if (condition) {
 const [value, setValue] = useState(0);
}
```

Correct:

```
const [value, setValue] = useState(0);

if (condition) {
 // use value
}
```

### Mistake 2 — Calling Hooks inside loops

Incorrect:

```
for (let i = 0; i < 5; i++) {
 useState();
}
```

Don't do this.

---

## Mistake 3 — Calling Hooks inside event handlers

Incorrect:

```
function handleClick() {
 const [value, setValue] = useState(0);
}
```

Hooks belong at the top level of the component or inside a custom Hook.

---

## Mistake 4 — Calling Hooks from ordinary functions

Incorrect:

```
function calculate() {
 useState();
}
```

Unless that function is itself a properly designed custom Hook, don't call Hooks there.

---

## 29. React Developer Tools

Since we covered React DevTools in Lecture 11, this is a good time to revisit it.

When you inspect a component that uses Hooks, React DevTools can show information about the component's state and other React data.

For example, if we have:

```
const [count, setCount] = useState(0);
```

React DevTools can help us inspect the component and understand its state.

This becomes particularly useful when debugging larger applications.

---

## 30. A Small Exercise

Before moving to the next lecture, try this exercise.

Create a component called:

Counter

Use:

`useState`

to create:

`count = 0`

Add three buttons:

Increment

Decrement

Reset

Expected behavior:

Initial:

0

Click Increment:

1

Click Increment:

2

Click Decrement:

1

Click Reset:

0

Don't worry if you struggle with it.

We'll build a similar example in the next lecture.

---

## 31. Another Exercise — Show/Hide

Create a component:



Message

Initially display:

Hello React

Add a button:

Hide

When clicked, hide the message.

Then the button should say:

Show

Clicking it should display the message again.

You'll need a state such as:

```
const [isVisible, setIsVisible] =
 useState(true);
```

This is another very common use of useState.

---

## 32. Interview Questions

Let's finish with some interview questions.

### Question 1

What are Hooks in React?

**Answer:**

Hooks are special React functions that allow function components to use React features such as state and effects.

---

### Question 2

Why were Hooks introduced?

**Answer:**

Hooks were introduced to allow function components to use state and other React features without needing class components.

---

### Question 3

Name some React Hooks.

**Answer:**

Examples include:

useState  
useEffect  
useContext  
useRef  
useReducer  
useMemo  
useCallback

---

### Question 4

What is useState used for?

**Answer:**

useState is used to add state to a function component.

---

### Question 5

What is useEffect used for?

**Answer:**

useEffect is used to perform side effects in a component, such as API calls, timers, subscriptions, or interacting with browser APIs.

---

### Question 6

Can Hooks be called inside an if statement?

**Answer:**

No. Hooks should be called at the top level of a function component or custom Hook.

---

## Question 7

Can Hooks be called inside normal JavaScript functions?

**Answer:**

No, Hooks should be called from React function components or custom Hooks.

---

## 33. Lecture Summary

Let's summarize today's lecture.

We learned that:

### 1. Hooks are special React functions

They allow function components to use React features.

### 2. Hooks became important with modern function components

They allow us to use things like state without relying on class components.

### 3. `useState` manages state

Example:

```
const [count, setCount] = useState(0);
```

### 4. `useEffect` handles side effects

Example:

```
useEffect(() => {
 // side effect
}, []);
```

### 5. Hooks have rules

They should generally be called:

At the top level

and from:

React function components  
or  
custom Hooks

## 6. Hook names start with use

Examples:

useState  
useEffect  
useContext  
useRef

---

## 34. Final Takeaway

If you remember only one thing from today's lecture, remember this:

**Hooks allow function components to use React features such as state and side effects.**

The two Hooks we are going to focus on first are:

useState  
↓  
Manage state

useEffect  
↓  
Manage side effects

And now we are ready for the next lecture.

# Lecture 17 — useState Hook in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we introduced one of the most important concepts in modern React:

## Hooks

We learned that Hooks allow function components to use React features.

We also introduced two very important Hooks:

`useState`  
`useEffect`

Today we're going to focus completely on the first one:

### `useState`

`useState` is one of the most commonly used Hooks in React.

If you're going to build React applications, you will use `useState` again and again.

In today's lecture, we'll understand:

- What is state?
- Why do we need state?
- Normal variable vs state
- `useState()` syntax
- State variable
- State updater function
- Initial state
- Updating state
- Re-rendering
- Multiple state variables
- String state
- Number state
- Boolean state

- Array state
- Object state
- Common mistakes
- A practical Counter application

Let's start.

---

## 2. What Is State?

Before learning useState, we need to understand the word:

### State

In simple terms:

State is data that a React component needs to remember and that can change over time.

For example, imagine a counter.

Initially:

count = 0

After clicking a button:

count = 1

Click again:

count = 2

The value is changing.

That changing value is called **state**.

---

## 3. Real-World Examples of State

State isn't only used for counters.

Think about a login page.

Initially:

```
isLoggedIn = false
```

After successful login:

```
isLoggedIn = true
```

That's state.

A search box:

```
searchText = ""
```

User types:

```
searchText = "react"
```

That's state.

A shopping cart:

```
cart = []
```

User adds a product:

```
cart = [product]
```

That's state.

A modal:

```
isModalOpen = false
```

User clicks "Open":

```
isModalOpen = true
```

Again, state.

So common examples include:

Counter

Login status

Search text

Shopping cart

Selected product

Dropdown state

Modal visibility

Form values

Loading state

Error messages

## 4. Why Can't We Just Use a Normal Variable?

Let's start with a normal JavaScript variable.

Create:

```
function Counter() {
 let count = 0;

 const increment = () => {
 count++;
 console.log(count);
 };

 return (
 <div>
 <h1>
 {count}
 </h1>
 <button onClick={increment}>
 Increment
 </button>
 </div>
);
}
```

At first glance, this looks reasonable.

We have:

```
let count = 0;
```

and:

```
count++;
```

So you might think the screen should change.

Let's test it.



Click:

Increment

The console may show:

1

Click again:

2

Click again:

3

But the UI doesn't update correctly.

Why?

Because count is only a normal JavaScript variable.

React doesn't know that it changed.

---

## 5. React Needs to Know When State Changes

This is the important idea.

Suppose we have:

```
count = 0
```

and then:

```
count = 1
```

React needs to know:

“The data used by this component has changed. Please render the component again.”

A normal variable doesn't provide that mechanism.

That's why React gives us:

`useState()`

With `useState`, we tell React:

“This value is state. Please remember it and update the component when it changes.”

---

## 6. Importing `useState`

To use `useState`, we first import it from React.

```
import { useState } from "react";
```

Then inside our component:

```
function Counter() {
 const [count, setCount] = useState(0);

 return (
 <h1>
 {count}
 </h1>
);
}
```

This line is extremely important:

```
const [count, setCount] = useState(0);
```

Let's understand every part of it.

---

## 7. Understanding `useState(0)`

We write:

```
useState(0)
```

The `0` represents the **initial value**.

So we're telling React:

Create a state value and initially set it to 0.

For example:

```
useState(0)
```

means:

Initial state = 0

We can also have:

```
useState("Hello")
```

which means:

Initial state = "Hello"

Or:

```
useState(false)
```

which means:

Initial state = false

Or:

```
useState([])
```

which means:

Initial state = empty array

Or:

```
useState({})
```

which means:

Initial state = empty object

So the value passed to `useState()` is the initial state.

---

## 8. Understanding the Array Syntax

Now let's look at:

```
const [count, setCount] = useState(0);
```

This uses JavaScript **array destructuring**.

`useState()` returns two values:

1. Current state
2. Function used to update state

So:

```
const [count, setCount]
```

means:

count



Current state value

setCount



Function used to update the state

We could conceptually think of it as:

```
[count, setCount]
```

---

## 9. State Variable

The first value:

count

is the current state.

Initially:

```
count = 0
```

If the state changes to 5:

```
count = 5
```

We use it inside JSX:

```
<h1>
 {count}
</h1>
```

So whatever the current state is, React displays it.

---

## 10. State Updater Function

The second value:

setCount

is a function.

Its job is to update:

count

For example:

```
setCount(10);
```

changes the state to:

```
count = 10
```

Or:

```
setCount(20);
```

changes it to:

```
count = 20
```

The naming convention is usually:

```
state variable
+
set
+
state variable
```

For example:

```
const [count, setCount] = useState(0);
const [name, setName] = useState("");
const [isLoggedIn, setIsLoggedIn] = useState(false);
const [products, setProducts] = useState([]);
```

---

## 11. Our First Complete Counter

Let's build our first real example.

```
import { useState } from "react";

function Counter() {
```

```
const [count, setCount] = useState(0);

return (
 <div>

 <h1>
 {count}
 </h1>

 <button
 onClick={() => setCount(count + 1)}
 >
 Increment
 </button>

 </div>
);
}
```

**export default** Counter;

Let's understand what happens.

Initially:

count = 0

The UI displays:

0

When we click the button:

setCount(count + 1);

Since:

count = 0

we get:

setCount(1)

React updates the state.

The component renders again.

Now:

`count = 1`

The screen displays:

1

Click again:

`setCount(2)`

The screen becomes:

2

---

## 12. The React State Flow

This is an extremely important concept.

Let's visualize it.

Initial State



`count = 0`



Render UI



User clicks button



`setCount(count + 1)`



State changes



React re-renders component



`count = 1`



Updated UI

So the important relationship is:

State changes



Re-render



UI updates

## 13. What Is a Re-render?

You will hear the word **re-render** frequently in React.

A re-render means React runs the component again to determine what the UI should look like based on the latest data.

For example:

Initially:

```
count = 0
```

React renders:

```
<h1>0</h1>
```

Then:

```
setCount(1);
```

The state changes.

React renders the component again.

Now:

```
<h1>1</h1>
```

The browser displays the updated UI.

So:

Updating state causes React to render the component again.

---

## 14. State Is Not the Same as a Variable

This distinction is very important.

Normal variable:

```
let count = 0;
```

React state:

```
const [count, setCount] = useState(0);
```

The normal variable is managed by JavaScript.



The state value is managed by React.

A normal variable changing doesn't automatically tell React to update the UI.

Using the state updater:

```
setCount(...)
```

does.

---

## 15. Updating State

There are different ways we can update state.

For example:

```
setCount(10);
```

This directly sets the state to:

10

We can also calculate a new value:

```
setCount(count + 1);
```

For incrementing:

```
setCount(count + 1);
```

For decrementing:

```
setCount(count - 1);
```

For resetting:

```
setCount(0);
```

---

## 16. Building Increment, Decrement and Reset

Let's create a better counter.

```
import { useState } from "react";
```

```
function Counter() {
```

```
const [count, setCount] = useState(0);

const increment = () => {
 setCount(count + 1);
};

const decrement = () => {
 setCount(count - 1);
};

const reset = () => {
 setCount(0);
};

return (
 <div>

 <h1>
 Count: {count}
 </h1>

 <button onClick={increment}>
 Increment
 </button>

 <button onClick={decrement}>
 Decrement
 </button>

 <button onClick={reset}>
 Reset
 </button>

 </div>
);
}

export default Counter;
```

Now we have three operations:

Increment → count + 1  
Decrement → count - 1  
Reset → count = 0

---

## 17. State Can Store Strings

State doesn't have to be a number.

Let's create a name.

```
const [name, setName] = useState("");
```

Initially:

```
name = ""
```

We can update it:

```
setName("Arjun");
```

Now:

```
name = "Arjun"
```

Let's display it:

```
import { useState } from "react";

function User() {

 const [name, setName] = useState("");

 return (
 <div>

 <h1>
 Hello {name}
 </h1>

 <button
 onClick={() => setName("Arjun")}
 >
 Set Name
 </button>

 </div>
```

```
);
 }
```

Initially:

Hello

After clicking:

Hello Arjun

---

## 18. State With an Input

This is a very common pattern.

```
import { useState } from "react";

function User() {
 const [name, setName] = useState("");

 return (
 <div>
 <input
 value={name}
 onChange={(e) =>
 setName(e.target.value)
 }
 />

 <h2>
 Hello {name}
 </h2>

 </div>
);
}
```

If the user types:

React

then:

name = "React"

If the user changes it to:

ReactJS

then:

```
name = "ReactJS"
```

This is an example of **controlled input**, which we'll use more extensively in Lecture 19 when we study forms.

---

## 19. State Can Store Boolean Values

Boolean state is extremely useful.

For example:

```
const [isVisible, setIsVisible] = useState(true);
```

The initial value is:

true

We can change it:

```
setIsVisible(false);
```

or:

```
setIsVisible(true);
```

Let's build a show/hide example.

```
import { useState } from "react";

function Message() {

 const [isVisible, setIsVisible] =
 useState(true);

 return (
 <div>

 {isVisible && (
 <h2>
 Welcome to React
 </h2>
)}
 </div>
);
}
```

```
)}

 <button
 onClick={() =>
 setIsVisible(!isVisible)
 }
 >
 Show / Hide
 </button>
 </div>
);
}

export default Message;
```

When:

`isVisible = true`

the message is displayed.

When:

`isVisible = false`

the message is hidden.

---

## 20. Toggle State

A very common pattern is:

```
setIsVisible(!isVisible);
```

Let's understand it.

If:

`isVisible = true`

then:

`!isVisible`

becomes:

`false`

If:

```
isVisible = false
```

then:

```
!isVisible
```

becomes:

```
true
```

So this is called a **toggle**.

It is commonly used for:

- Menu open/close
- Modal open/close
- Show/hide password
- Dark/light mode
- Expand/collapse

---

## 21. State Can Store Arrays

We can also store an array in state.

For example:

```
const [products, setProducts] = useState([]);
```

Initially:

```
products = []
```

We can later store:

```
setProducts([
 "Laptop",
 "Mobile",
 "Tablet"
]);
```

Now:

```
products = [
 "Laptop",
 "Mobile",
]
```

```
"Tablet"
]
```

And display them:

```

```

```
 {products.map(product => (
 <li key={product}>
 {product}

))}
```

```

```

We'll work much more with arrays and objects when we start consuming APIs.

---

## 22. State Can Store Objects

We can also store an object.

For example:

```
const [user, setUser] = useState({
 name: "",
 age: 0
});
```

Initially:

```
{
 name: "",
 age: 0
}
```

We might want to update the name.

We can write:

```
setUser({
 ...user,
 name: "Arjun"
});
```

Now:



```
{
 name: "Arjun",
 age: 0
}
```

Notice the:

```
...user
```

This is the JavaScript spread operator.

It copies the existing properties and then changes name.

This pattern becomes very important when working with forms and API data.

---

## 23. Why Should We Not Directly Modify State?

Suppose we have:

```
const [count, setCount] = useState(0);
```

Don't do:

```
count = count + 1;
```

This is incorrect.

We should use:

```
setCount(count + 1);
```

Why?

Because React needs the state updater to know that state has changed.

So remember:

Never directly modify a state variable. Use its setter function.

---

## 24. Example of Incorrect State Update

Incorrect:

```
const [count, setCount] = useState(0);
```

```
const increment = () => {

 count++;
};
```

Correct:

```
const increment = () => {

 setCount(count + 1);
};
```

The setter is the mechanism React provides for updating state.

---

## 25. Multiple State Variables

A component can have multiple state values.

For example:

```
function User() {

 const [name, setName] = useState("");

 const [age, setAge] = useState(0);

 const [isActive, setIsActive] =
 useState(false);

 return (
 <div>

 <h1>
 {name}
 </h1>

 <p>
 Age: {age}
 </p>

 <p>
 Active: {isActive ? "Yes" : "No"}
 </p>

 </div>
);
}
```

```
);
 }
```

Here we have three separate pieces of state:

```
name
age
isActive
```

Each has its own updater:

```
setName
setAge
setIsActive
```

This is perfectly valid.

---

## 26. One State Object vs Multiple State Values

You might ask:

Should I write:

```
const [name, setName] = useState("");
```

```
const [age, setAge] = useState(0);
```

or:

```
const [user, setUser] = useState({
 name: "",
 age: 0
});
```

Both approaches are possible.

For simple independent values, separate state variables can be easier to understand.

For logically related data, an object may sometimes make sense.

For example:

```
const [user, setUser] = useState({
 name: "",
 age: 0,
 email: ""
});
```

The right choice depends on how the data is used.

---

## 27. Important Concept — State Updates Are Asynchronous

This is a concept students often encounter.

Suppose:

```
const [count, setCount] = useState(0);

const increment = () => {
 setCount(count + 1);
 console.log(count);
};
```

You might expect the console to immediately print:

1

But it can still print:

0

Why?

Because calling:

```
setCount(...)
```

does not immediately change the count variable within the currently executing function.

React schedules the state update and then re-renders the component.

For now, remember:

Don't assume the state variable changes immediately after calling its setter.

We'll explore this further with functional state updates.

---

## 28. Functional State Updates

Suppose we want to update a state value based on its previous value.

Instead of:

```
setCount(count + 1);
```

we can write:

```
setCount(
 previousCount => previousCount + 1
);
```

This is called a **functional state update**.

For example:

```
const increment = () => {

 setCount(
 previousCount => previousCount + 1
);
};
```

Here React gives us the previous state:

previousCount

and we return the new state:

previousCount + 1

This pattern is especially useful when the new state depends on the previous state.

---

## 29. Why Functional Updates Matter

Imagine we want to increment the counter multiple times.

We might write:

```
setCount(count + 1);
setCount(count + 1);
setCount(count + 1);
```

This may not produce the result you expect because all three updates can be based on the same current value.

Instead:

```
setCount(
 previousCount => previousCount + 1
);

setCount(
 previousCount => previousCount + 1
);

setCount(
 previousCount => previousCount + 1
);
```

Each update can use the most recent previous state.

This is a pattern worth remembering.

---

## 30. Practical Counter Using Functional Update

Let's modify our counter.

```
import { useState } from "react";

function Counter() {
 const [count, setCount] = useState(0);

 const increment = () => {
 setCount(
 previousCount =>
 previousCount + 1
);
 };

 const decrement = () => {
 setCount(
 previousCount =>
 previousCount - 1
);
 };

 return (

```

```
 <div>

 <h1>
 Count: {count}
 </h1>

 <button onClick={increment}>
 Increment
 </button>

 <button onClick={decrement}>
 Decrement
 </button>

 </div>
);
}

export default Counter;
```

This is a good pattern when the new value depends on the old value.

---

## 31. State and Conditional Rendering

useState works very well with conditional rendering.

For example:

```
const [isLoggedIn, setIsLoggedIn] =
 useState(false);
```

Then:

```
{
 isLoggedIn
 ? <h1>Welcome</h1>
 : <h1>Please Login</h1>
}
```

Button:

```
<button
 onClick={() =>
 setIsLoggedIn(true)
 }
>
```

Login  
</button>

State changes:

false → true

and the UI changes automatically.

This demonstrates one of React's most important ideas:

The UI is a representation of the current state.

---

## 32. State and the UI

Let's think about React this way:

State



UI

If:

isLoggedIn = false

we show:

Please Login

If:

isLoggedIn = true

we show:

Welcome

Similarly:

count = 0

shows:

0

and:

count = 10

shows:



10

The UI is determined by the current state.

---

## 33. A Practical Example — Like Button

Let's build something more realistic.

```
import { useState } from "react";

function LikeButton() {

 const [likes, setLikes] = useState(0);

 return (
 <div>

 <h2>
 Likes: {likes}
 </h2>

 <button
 onClick={() =>
 setLikes(
 previousLikes =>
 previousLikes + 1
)
 }
 >
 Like
 </button>

 </div>
);
}
```

```
export default LikeButton;
```

Every click increases:

likes

So:

0  
↓

1  
↓  
2  
↓  
3

This is a very simple example of state.

---

## 34. Another Practical Example — Character Counter

Let's build a character counter.

```
import { useState } from "react";

function CharacterCounter() {

 const [text, setText] = useState("");

 return (
 <div>

 <textarea
 value={text}
 onChange={(e) =>
 setText(e.target.value)
 }
 />

 <p>
 Characters: {text.length}
 </p>

 </div>
);
}

export default CharacterCounter;
```

Here:

text

is state.

Every time the user types:

```
setText(e.target.value)
```

updates the state.

React renders again.

Then:

```
text.length
```

is recalculated.

This is a very good example of how state drives UI.

---

## 35. A Practical Example — Show Password

Let's create a password field.

```
import { useState } from "react";

function PasswordInput() {
 const [showPassword, setShowPassword] =
 useState(false);

 return (
 <div>

 <input
 type={
 showPassword
 ? "text"
 : "password"
 }
 />

 <button
 onClick={() =>
 setShowPassword(
 previous =>
 !previous
)
 }
 >
 Show / Hide
 </button>
 </div>
);
}
```

```
 </div>
);
}
```

Here the state:

showPassword

controls the UI.

If:

true

we use:

text

If:

false

we use:

password

Again:

State

↓

UI

---

## 36. Important Rule — Don't Mutate Arrays Directly

Suppose:

```
const [products, setProducts] = useState([]);
```

Don't do:

```
products.push(newProduct);
```

and expect React to update correctly.

Instead, create a new array:

```
setProducts([
 ...products,
```

```
 newProduct
]);
```

The spread operator creates a new array.

For example:

```
setProducts([
 ...products,
 {
 id: 1,
 name: "Laptop"
 }
]);
```

We'll use this pattern frequently when working with lists.

---

## 37. Important Rule — Don't Mutate Objects Directly

Suppose:

```
const [user, setUser] = useState({
 name: "Arjun",
 age: 25
});
```

Don't do:

```
user.name = "Rahul";
```

Instead:

```
setUser({
 ...user,
 name: "Rahul"
});
```

We're creating a new object.

This is an important React pattern:

Treat state as immutable and create a new value when updating it.

---

## 38. `useState` Syntax Cheat Sheet

Let's create a quick reference.

### Number

```
const [count, setCount] =
 useState(0);
```

### String

```
const [name, setName] =
 useState("");
```

### Boolean

```
const [isOpen, setIsOpen] =
 useState(false);
```

### Array

```
const [items, setItems] =
 useState([]);
```

### Object

```
const [user, setUser] =
 useState({});
```

The pattern is always:

```
const [state, setState] =
 useState(initialValue);
```

---

## 39. Common Mistakes

Let's look at some common beginner mistakes.

### Mistake 1 — Directly changing state

Incorrect:

```
count++;
```

Correct:

```
setCount(count + 1);
```

## Mistake 2 — Forgetting the setter

Incorrect:

```
const [count] = useState(0);
```

This is valid JavaScript, but if you need to update the state, you won't have the setter.

Usually:

```
const [count, setCount] =
 useState(0);
```

---

## Mistake 3 — Calling the setter during rendering

Be careful with code like:

```
function Counter() {

 const [count, setCount] =
 useState(0);

 setCount(10);

 return <h1>{count}</h1>;
}
```

This can cause repeated rendering and should not be done.

State updates should normally happen in response to events, effects, or other appropriate logic—not unconditionally during rendering.

---

## Mistake 4 — Forgetting that state is component-specific

If two components have:

```
const [count, setCount] =
 useState(0);
```

they don't automatically share the same state.

Each component instance has its own state.

## 40. State Belongs to a Component Instance

Suppose we have:

```
<Counter />
<Counter />
```

We have two Counter components.

Each one can have its own:

count

For example:

Counter 1 → count = 5  
Counter 2 → count = 2

Updating Counter 1 doesn't automatically update Counter 2.

This is because each component instance has its own state.

This concept becomes important when we start discussing sharing state between components.

---

## 41. Live Coding Challenge

Now let's build a small application.

Create:

Counter

Requirements:

Initial value

0

Buttons

Increment

Decrement

Reset

Additional requirement

Don't allow the counter to go below zero.



So:

0

If the user clicks Decrement:

0

should remain:

0

One solution is:

```
setCount(
 previousCount =>
 previousCount > 0
 ? previousCount - 1
 : 0
);
```

This combines:

- useState
- Functional updates
- Conditional operator
- Event handling

---

## 42. Final Counter Application

Here's the completed version.

```
import { useState } from "react";

function Counter() {
 const [count, setCount] = useState(0);

 const increment = () => {
 setCount(
 previousCount =>
 previousCount + 1
);
 };
};
```

```
const decrement = () => {

 setCount(
 previousCount =>
 previousCount > 0
 ? previousCount - 1
 : 0
);
};

const reset = () => {

 setCount(0);
};

return (
 <div>

 <h1>
 Counter
 </h1>

 <h2>
 {count}
 </h2>

 <button onClick={increment}>
 Increment
 </button>

 <button onClick={decrement}>
 Decrement
 </button>

 <button onClick={reset}>
 Reset
 </button>

 </div>
);
}

export default Counter;
```

This is a good first useState application.

## 43. Interview Questions

Let's finish with some common interview questions.

### Question 1

What is useState?

**Answer:**

useState is a React Hook that allows a function component to create and manage state.

---

### Question 2

What does useState() return?

It returns an array containing two values:

Current state

State updater function

For example:

```
const [count, setCount] =
 useState(0);
```

---

### Question 3

What is the purpose of setCount?

It updates the count state and tells React that the component needs to render again.

---

### Question 4

Can useState store objects?

Yes.

```
const [user, setUser] =
 useState({
 name: "",
 age: 0
 });
```

---

### Question 5

Can useState store arrays?

Yes.

```
const [items, setItems] =
 useState([]);
```

---

### Question 6

Can we directly modify state?

No.

Use the setter function.

Incorrect:

```
count++;
```

Correct:

```
setCount(count + 1);
```

---

### Question 7

What is a functional state update?

It is a state update where we receive the previous state and calculate the new state from it.

```
setCount(
 previousCount =>
 previousCount + 1
);
```

---

### Question 8

What happens when state changes?

React schedules the component to render again so the UI can reflect the latest state.

---

## 44. Lecture Summary

Let's summarize everything.

`useState`

The basic syntax is:

```
const [state, setState] =
 useState(initialValue);
```

For example:

```
const [count, setCount] =
 useState(0);
```

Here:

`count`

↓

Current state

`setCount`

↓

State updater

`0`

↓

Initial value

We learned that state can contain:

Number

String

Boolean

Array

Object

We also learned:

State changes

↓

React re-renders

↓

UI updates

And when the new state depends on the previous state, we can use:

```
setCount(
 previousCount =>
```

```
 previousCount + 1
);
```

Finally, remember:

**Don't directly modify state. Use the state updater function.**

---

## 45. Final Takeaway

If you remember only one pattern from today's lecture, remember:

```
const [state, setState] =
 useState(initialValue);
```

For example:

```
const [count, setCount] =
 useState(0);
```

Then:

```
setCount(10);
```

updates the state.

Or when the new value depends on the old value:

```
setCount(
 previousCount =>
 previousCount + 1
);
```

This simple Hook is the foundation for many React applications.

# Lecture 18 — useEffect Hook in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned about:

`useState`

We learned how state allows our React components to remember data and update the UI when that data changes.

Today we're going to learn another extremely important React Hook:

`useEffect`

`useEffect` is used when our component needs to perform a **side effect**.

You will use `useEffect` frequently when building real-world React applications.

For example, you might need to:

- Call an API
- Start a timer
- Stop a timer
- Change the browser title
- Add an event listener
- Remove an event listener
- Read browser APIs
- Subscribe to some external service

These are examples of side effects.

In today's lecture, we'll learn:

- What is a side effect?
- Why do we need `useEffect`?
- Basic `useEffect` syntax
- When `useEffect` runs
- Dependency array
- Empty dependency array

- Effects with dependencies
- Effects without dependencies
- Cleanup functions
- `useEffect` with `useState`
- API call example
- Common mistakes
- Practical examples

Let's get started.

---

## 2. Quick Revision of `useState`

Before learning `useEffect`, let's quickly revise the previous lecture.

We can create state using:

```
const [count, setCount] = useState(0);
```

Here:

`count`



Current state

`setCount`



Function to update state

`0`



Initial value

For example:

```
function Counter() {
 const [count, setCount] = useState(0);
 return (
 <div>
 <h1>
 {count}
 </h1>
 </div>
);
}
```



```
 <button
 onClick={() =>
 setCount(count + 1)
 }
 >
 Increment
 </button>
 </div>
);
 }
}
```

When the state changes:

State changes



React renders component again



UI updates

Now let's ask an important question.

What if we want to execute some additional code because the component rendered or because some state changed?

That's where `useEffect` comes in.

---

## 3. What Is a Side Effect?

Let's first understand the term:

### Side Effect

A side effect is an operation that happens outside the normal process of calculating and returning the UI.

For example, your component might return:

```
<h1>Hello React</h1>
```

That's normal rendering.

But suppose we want to:

```
document.title = "React Application";
```

Now we're changing something outside the component's returned JSX.

That's a side effect.

Another example:

```
fetch("/api/products");
```

We're communicating with an external API.

That's also a side effect.

Other examples include:

- Starting a timer
- Stopping a timer
- Adding an event listener
- Removing an event listener
- Calling an API
- Updating document.title
- Reading browser storage
- Subscribing to a service

---

## 4. Why Can't We Just Put Side Effects Inside the Component?

A beginner might try:

```
function App() {

 document.title = "My React App";

 return (
 <h1>
 Hello React
 </h1>
);
}
```

Although this might appear to work, it's not a good pattern for managing side effects.

Remember that React components can render multiple times.

For example:

State changes



Component renders again



Side-effect code runs again

This can create unwanted behavior.

For example, imagine making an API call directly during rendering:

```
function Products() {

 fetch("/api/products");

 return (
 <h1>
 Products
 </h1>
);
}
```

Every time the component renders, the API call can run again.

That's not what we usually want.

Instead, React provides:

`useEffect`

to manage side effects.

---

## 5. What Is `useEffect`?

The basic definition is:

`useEffect` is a React Hook used to perform side effects in a function component.

The basic syntax is:

```
useEffect(() => {

 // side effect

});
```

Before using it, import it:

```
import { useEffect } from "react";
```

For example:

```
import { useEffect } from "react";

function App() {

 useEffect(() => {

 console.log("Effect executed");

 });

 return (
 <h1>
 Hello React
 </h1>
);
}
```

Now let's understand when this effect runs.

---

## 6. Basic `useEffect`

Consider:

```
useEffect(() => {

 console.log("Effect executed");

});
```

The function passed to `useEffect` is called the **effect function**.

```
() => {

 console.log("Effect executed");

}
```

React executes this effect after the component has rendered.

So conceptually:

Component renders



React updates the UI



Effect runs

This is an important distinction.

---

## 7. Live Coding — First `useEffect`

Let's create a simple component.

```
import {
 useEffect
} from "react";

function App() {

 useEffect(() => {

 console.log(
 "App component rendered"
);

 });

 return (
 <div>

 <h1>
 Hello React
 </h1>

 </div>
);
}

export default App;
```

Open the browser console.

You should see:

App component rendered

This demonstrates the simplest use of useEffect.

---

## 8. useEffect With useState

Now let's combine what we learned in the previous lecture.

```
import {
 useState,
 useEffect
} from "react";

function Counter() {

 const [count, setCount] =
 useState(0);

 useEffect(() => {

 console.log(
 "Count:",
 count
);

 });

 return (
 <div>

 <h1>
 {count}
 </h1>

 <button
 onClick={() =>
 setCount(count + 1)
 }
 >
 Increment
 </button>

 </div>
);
}
```

Initially:

count = 0

The effect runs:

Count: 0

Click the button.

State changes:

0 → 1

React renders again.

Then the effect runs:

Count: 1

Click again:

1 → 2

Effect:

Count: 2

So the flow is:

State changes



Component renders



Effect executes

---

## 9. The Dependency Array

Now we reach one of the most important parts of `useEffect`.

The dependency array.

We can write:

```
useEffect(() => {
 console.log("Effect");
})
```

```
}, []);
```

Notice this:

```
[]
```

This is called the **dependency array**.

The dependency array tells React when the effect should be executed again.

---

## 10. `useEffect` With an Empty Dependency Array

Consider:

```
useEffect(() => {
 console.log("Effect executed");
}, []);
```

The empty array:

```
[]
```

means there are no dependencies.

In common React usage, this means the effect runs after the initial mount and does not re-run because of later state/prop changes.

For example:

```
function App() {
 const [count, setCount] =
 useState(0);
 useEffect(() => {
 console.log(
 "Effect executed"
);
 }, []);
 return (

```



```
 <div>

 <h1>
 {count}
 </h1>

 <button
 onClick={() =>
 setCount(count + 1)
 }
 >
 Increment
 </button>

 </div>
);
}
```

Initially:

Effect executed

Click Increment.

The component renders again.

But the effect does not re-run because:

[]

hasn't changed.

---

## 11. Three Common Forms of `useEffect`

There are three patterns you should understand.

### Pattern 1 — No dependency array

```
useEffect(() => {
```

```
 // effect
```

```
});
```

The effect runs after every render.

### Pattern 2 — Empty dependency array

```
useEffect(() => {
```

```
 // effect
```

```
}, []);
```

The effect runs after the initial mount and does not re-run due to subsequent dependency changes.

---

### Pattern 3 — Dependency array with values

```
useEffect(() => {
```

```
 // effect
```

```
}, [count]);
```

The effect runs after the initial mount and again when count changes.

This third pattern is extremely important.

---

## 12. Effect With a Dependency

Let's create:

```
useEffect(() => {
```

```
 console.log(
 "Count changed:",
 count
);
```

```
}, [count]);
```

The dependency is:

```
[count]
```

This tells React:

Run this effect again when count changes.

Let's see what happens.

Initially:

count = 0

Effect:

Count changed: 0

Click:

count = 1

Effect:

Count changed: 1

Click:

count = 2

Effect:

Count changed: 2

---

## 13. Why Do We Need Dependencies?

Imagine we have:

```
const [name, setName] =
 useState("");
```

```
const [age, setAge] =
 useState(0);
```

Suppose our effect is related only to:

name

We can write:

```
useEffect(() => {

 console.log(
 "Name changed:",
 name
);

}, [name]);
```

Now the effect is associated with the name dependency.

If age changes:

age: 20 → 21

the effect does not need to re-run merely because age changed.

But if:

name: "Arjun" → "Rahul"

then the effect runs again.

This allows us to control when an effect needs to synchronize with something outside React.

---

## 14. Example — Updating Browser Title

One simple and useful example is changing the browser tab title.

Suppose our counter is:

```
const [count, setCount] =
 useState(0);
```

We want the browser title to display:

Count: 0

Then:

Count: 1

Then:

Count: 2

We can write:

```
useEffect(() => {
 document.title = `Count: ${count}`;
}, [count]);
```

Complete example:

```
import {
 useState,
 useEffect
} from "react";

function Counter() {

 const [count, setCount] =
 useState(0);

 useEffect(() => {

 document.title =
 `Count: ${count}`;

 }, [count]);

 return (
 <div>

 <h1>
 {count}
 </h1>

 <button
 onClick={() =>
 setCount(count + 1)
 }
 >
 Increment
 </button>

 </div>
);
}
```

```
export default Counter;
```

Now look at your browser tab.

Initially:

Count: 0

Click:

Count: 1

Click again:

Count: 2

This is a very simple demonstration of a side effect.

---

## 15. Important Concept — Effect Runs After Rendering

Remember this sequence:

State / Props  
↓  
Component renders  
↓  
React updates the UI  
↓  
Effect runs

For example:

```
<h1>{count}</h1>
```

is part of rendering.

While:

```
document.title = `Count: ${count}`;
```

is a side effect.

So React separates the two concepts.

---

## 16. Example — API Call

Now let's look at one of the most important uses of `useEffect`:

### API Calls

Suppose our React application needs to load products from an API.

We might write:

```
useEffect(() => {

 fetch(
 "https://example.com/api/products"
)
 .then(response =>
 response.json()
)
 .then(data => {

 console.log(data);

 });

}, []);
```

Why do we put the API call inside useEffect?

Because calling an API is a side effect.

We don't want to make the API request every time React renders the component.

With:

```
[]
```

we're saying that this effect does not depend on changing component values.

This pattern is commonly used when loading initial data.

We'll build proper API consumption examples later in the series.

---

## 17. Practical API Example

Let's create a simple example using a public test API.

```
import {
 useEffect,
 useState
} from "react";

function Products() {

 const [products, setProducts] =
 useState([]);
```

```
useEffect(() => {

 fetch(
 "https://dummyjson.com/products"
)
 .then(response =>
 response.json()
)
 .then(data => {

 setProducts(
 data.products
);

 });

 }, []);

 return (
 <div>

 <h1>
 Products
 </h1>

 {products.map(product => (

 <div key={product.id}>

 {product.title}

 </div>

))}

 </div>
);
}
```

**export default** Products;

Let's understand what happens.

Initially:

products = []



The component renders.

Then:

`useEffect`

runs.

The API request is sent.

The server returns data.

We call:

`setProducts(data.products);`

The state changes.

React renders again.

Now the products are displayed.

The complete flow is:

```
Component renders
 ↓
useEffect runs
 ↓
API request
 ↓
Response received
 ↓
setProducts()
 ↓
State changes
 ↓
React renders again
 ↓
Products displayed
```

This flow will become extremely important in Lectures 20–25.

---

## 18. Why API Calls Are Usually Not Made Directly in Rendering

Avoid doing this:

```
function Products() {
 fetch(
 "https://dummyjson.com/products"
);
 return (
 <h1>
 Products
 </h1>
);
}
```

Why?

Because rendering can happen multiple times.

That could cause:

Render



API call

Render again



API call again

Render again



API call again

We don't want uncontrolled API requests.

Instead:

```
useEffect(() => {
 fetch(...);
}, []);
```

The effect gives us a proper place to synchronize with the external API.

---

## 19. `useEffect` Cleanup

Now let's discuss another important feature.

Sometimes an effect creates something that needs to be cleaned up.

For example:

Timer  
Event listener  
Subscription  
WebSocket connection

Suppose we create an event listener:

```
window.addEventListener(
 "resize",
 handleResize
);
```

When the component is removed, we should remove the listener.

Otherwise, the listener can remain active unnecessarily.

That's where the **cleanup function** comes in.

---

## 20. Cleanup Function Syntax

An effect can return a function:

```
useEffect(() => {

 // setup

 return () => {

 // cleanup

 };
}, []);
```

For example:

```
useEffect(() => {

 console.log("Effect started");

 return () => {
```

```
 console.log(
 "Effect cleaned up"
);

 };

}, []);
```

The function returned by the effect is called the **cleanup function**.

---

## 21. Timer Example

Let's build a timer.

```
import {
 useEffect,
 useState
} from "react";

function Timer() {

 const [seconds, setSeconds] =
 useState(0);

 useEffect(() => {

 const timer =
 setInterval(() => {

 setSeconds(
 previous =>
 previous + 1
);

 }, 1000);

 return () => {

 clearInterval(timer);

 };

 }, []);
```

```
 return (
 <h1>
 Seconds: {seconds}
 </h1>
);
 }
}
```

**export default** Timer;

Let's understand it.

When the component mounts:

`useEffect` runs

Then:

`setInterval(...)`

starts a timer.

Every second:

`seconds + 1`

The component updates.

But what happens when the component is removed?

We don't want the timer to continue running.

So we return:

```
return () => {
 clearInterval(timer);
};
```

This cleans up the timer.

---

## 22. Why Is Cleanup Important?

Imagine a component that creates an event listener.

Every time the component mounts, we add:

`window.addEventListener(...)`

If we don't remove it when necessary, we can end up with multiple listeners.

Similarly, if we create timers and never clear them, they can continue running after they're no longer needed.

So the general pattern is:

```
Effect starts something
 ↓
Component uses it
 ↓
Component no longer needs it
 ↓
Cleanup removes it
```

---

## 23. Event Listener Example

Let's create a resize listener.

```
import { useEffect } from "react";

function WindowSize() {

 useEffect(() => {

 const handleResize = () => {

 console.log(
 window.innerWidth
);

 };

 window.addEventListener(
 "resize",
 handleResize
);

 return () => {

 window.removeEventListener(
 "resize",
 handleResize
);

 };

 });

}
```

```
 };

 }, []);

 return (
 <h1>
 Resize the browser
 </h1>
);
 }
}
```

When the component mounts:

Event listener added

When it is removed:

Event listener removed

This is a classic use of cleanup.

---

## 24. Dependency Array — Important Examples

Let's summarize the three common patterns.

### No dependency array

```
useEffect(() => {

 console.log("Effect");

});
```

Runs after every render.

---

### Empty dependency array

```
useEffect(() => {

 console.log("Effect");

}, []);
```

Runs after initial mount and does not re-run due to later dependency changes.

---

## Dependency array

```
useEffect(() => {

 console.log("Count changed");

}, [count]);
```

Runs after initial mount and when count changes.

---

## 25. Multiple Dependencies

An effect can have multiple dependencies.

For example:

```
const [name, setName] =
 useState("");

const [age, setAge] =
 useState(0);
```

We can write:

```
useEffect(() => {

 console.log(
 "Name or age changed"
);

}, [name, age]);
```

Now the effect can re-run when either:

name

or:

age

changes.

---



## 26. Example — User Information

Suppose:

```
const [name, setName] =
 useState("Arjun");

const [age, setAge] =
 useState(25);
```

We might have:

```
useEffect(() => {

 console.log(
 "User information changed"
);

}, [name, age]);
```

If we change:

name

the effect runs.

If we change:

age

the effect runs.

If some unrelated state changes, the effect doesn't re-run merely because of that unrelated state.

---

## 27. A Common Mistake — Missing Dependencies

Consider:

```
const [count, setCount] =
 useState(0);

useEffect(() => {

 console.log(count);
```

```
}, []);
```

The effect uses:

count

but the dependency array is:

```
[]
```

This means the effect won't re-run when count changes.

If the intention is to react to count changes, we should normally write:

```
useEffect(() => {
 console.log(count);
}, [count]);
```

Modern React tooling can warn about missing dependencies.

---

## 28. A Common Mistake — Infinite Loops

This is very important.

Suppose:

```
const [count, setCount] =
 useState(0);

useEffect(() => {
 setCount(count + 1);
}, [count]);
```

What happens?

```
count changes
 ↓
effect runs
 ↓
setCount()
 ↓
count changes
```

↓  
effect runs  
↓  
setCount()  
↓  
...

This can create an infinite update loop.

So be careful when an effect updates a state that is also one of its dependencies.

Always ask:

“Why does this effect need to update this state?”

---

## 29. `useEffect` Is Not for Everything

Another important lesson:

Don't use `useEffect` for every piece of logic.

For example:

```
const total =
 price * quantity;
```

This doesn't need `useEffect`.

It's simply a calculation.

We can write:

```
const total =
 price * quantity;
```

Similarly:

```
const fullName =
 firstName + " " + lastName;
```

doesn't require an effect.

Use `useEffect` when you need to synchronize with something outside the component's normal rendering calculation.

---

## 30. Derived Values vs Side Effects

This distinction is very useful.

Suppose:

```
const [price, setPrice] =
 useState(100);

const [quantity, setQuantity] =
 useState(2);
```

We can calculate:

```
const total =
 price * quantity;
```

That's a derived value.

We don't need:

```
useEffect(() => {
 setTotal(price * quantity);
}, [price, quantity]);
```

unless there is a specific reason to store that derived value as state.

Usually, just calculate it:

```
const total =
 price * quantity;
```

So remember:

Calculation  
→ Normal JavaScript

External side effect  
→ useEffect

---

## 31. Another Example — Document Title

Here's another practical example.

```
function UserProfile() {

 const [name, setName] =
 useState("Guest");

 useEffect(() => {

 document.title =
 `Profile - ${name}`;

 }, [name]);

 return (
 <div>

 <h1>
 {name}
 </h1>

 <button
 onClick={() =>
 setName("Arjun")
 }
 >
 Change Name
 </button>

 </div>
);
}
```

Initial title:

Profile - Guest

After clicking:

Profile - Arjun

The effect is synchronizing React state with something outside React:

React state

↓

document.title

That's a perfect use case.

## 32. Understanding Mount, Update and Unmount

You may hear these three terms frequently.

### Mount

When a component is first added to the UI.

Component appears

---

### Update

When its state or props change and React renders it again.

State changes



Component updates

---

### Unmount

When the component is removed from the UI.

Component disappears

Effects can participate in these stages.

For example:

```
useEffect(() => {

 // setup

 return () => {

 // cleanup

 };
}, []);
```

The setup runs after the component mounts, while the cleanup runs when the effect needs to be cleaned up, including when the component unmounts.

---

## 33. Important Note About Development Mode

When testing `useEffect`, you may sometimes notice an effect appearing to run more than once during development.

If your application uses React Strict Mode, React may intentionally perform extra development-only checks to help detect unsafe side effects.

For example, you might see:

```
Effect executed
Effect cleanup
Effect executed
```

during development.

Don't immediately assume that React is broken.

This behavior is related to development checks.

We'll focus on understanding the actual effect lifecycle rather than trying to work around it.

---

## 34. Complete Example — Fetching Products

Let's build a slightly more realistic example.

```
import {
 useEffect,
 useState
} from "react";

function Products() {

 const [products, setProducts] =
 useState([]);

 const [loading, setLoading] =
 useState(true);

 useEffect(() => {

 fetch(
 "https://dummyjson.com/products"
)
 .then(response =>
```

```
 response.json()
)
 .then(data => {

 setProducts(
 data.products
);

 setLoading(false);

 });

}, []);

if (loading) {

 return (
 <h2>
 Loading...
 </h2>
);
}

return (
 <div>

 <h1>
 Products
 </h1>

 {products.map(product => (

 <div key={product.id}>

 <h3>
 {product.title}
 </h3>

 <p>
 ${product.price}
 </p>

 </div>

))}

)
}
```



```
 </div>
);
}
```

**export default** Products;

This example combines several concepts:

- useState
- useEffect
- API call
- Conditional rendering
- Array.map()
- Props/key concepts

We'll build much more advanced versions of this later.

---

## 35. The Complete API Flow

Let's understand that example conceptually.

Initially:

```
products = []
loading = true
```

The component renders:

Loading...

Then:

```
useEffect
 ↓
API request
```

The server returns:

Product data

Then:

```
setProducts(data.products);
```

and:

```
setLoading(false);
```

State changes.

React renders again.

Now:

```
loading = false
```

and:

```
products = [...]
```

The product list appears.

So:

Initial Render



Effect



API Request



Response



Update State



Re-render



Display Data

This is one of the most important React patterns you'll use in real applications.

---

## 36. Exercise for Students

Let's create a simple component called:

Clock

Requirements:

10. Display the current time.
11. Update the time every second.
12. Use `useState`.
13. Use `useEffect`.
14. Use `setInterval`.
15. Clean up the interval using `clearInterval`.

The basic structure is:

```
const [time, setTime] =
 useState(new Date());

useEffect(() => {

 const timer =
 setInterval(() => {

 setTime(new Date());

 }, 1000);

 return () => {

 clearInterval(timer);

 };

}, []);
```

Then display:

```
<h1>
 {time.toLocaleTimeString()}
</h1>
```

This exercise is excellent for understanding:

useState  
+  
useEffect  
+  
cleanup

---

## 37. Interview Questions

Let's finish with some common interview questions.

### Question 1

What is useEffect?

**Answer:**

useEffect is a React Hook used to synchronize a component with external systems and perform side effects such as API calls, timers, subscriptions, and browser API interactions.

---

## Question 2

When does useEffect run?

It runs after React commits the render. Its re-execution depends on the dependency array.

---

## Question 3

What does an empty dependency array mean?

```
useEffect(() => {

}, []);
```

It means the effect doesn't depend on changing reactive values in the dependency array, so it doesn't re-run due to subsequent dependency changes.

---

## Question 4

What happens if there is no dependency array?

```
useEffect(() => {

});
```

The effect runs after every render.

---

## Question 5

What does [count] mean?

```
useEffect(() => {

}, [count]);
```

The effect runs after the initial mount and again when count changes.

---

### Question 6

What is a cleanup function?

A cleanup function is a function returned from an effect to clean up resources created by that effect.

Example:

```
useEffect(() => {

 const timer =
 setInterval(...);

 return () => {

 clearInterval(timer);

 };
}, []);
```

---

### Question 7

Why are API calls commonly made inside useEffect?

Because an API request is a side effect, and useEffect provides a controlled place to synchronize with the external API.

---

### Question 8

Can every calculation be placed inside useEffect?

No.

Simple calculations and derived values should generally be calculated directly during rendering rather than being unnecessarily stored or synchronized through an effect.

---

## 38. Lecture Summary

Let's summarize today's lecture.

We learned:

### 1. useEffect

Used to synchronize with external systems and perform side effects.

```
useEffect(() => {

 // effect

});
```

### 2. Dependency array

```
useEffect(() => {

}, [count]);
```

The effect re-runs when count changes.

### 3. Empty dependency array

```
useEffect(() => {

}, []);
```

The effect runs after the initial mount and doesn't re-run because of later dependency changes.

### 4. Cleanup

```
useEffect(() => {

 // setup

 return () => {

 // cleanup

 };

}, []);
```

Useful for:

Timers  
Event listeners  
Subscriptions  
Connections

## 5. API calls

A common pattern is:

```
Render
↓
useEffect
↓
API call
↓
setState
↓
Re-render
↓
Display data
```

## 6. Don't use `useEffect` unnecessarily

Use normal JavaScript for normal calculations.

Use effects when you need to synchronize with something outside React.

---

## 39. Final Takeaway

If you remember only one thing from this lecture, remember:

**`useEffect` is used to synchronize your React component with something outside its normal rendering process.**

The basic pattern is:

```
useEffect(() => {

 // side effect

}, [dependencies]);
```

And the most important examples are:

- API calls
- Timers
- Event listeners
- Subscriptions
- Browser APIs

Now you understand the two most important Hooks we'll use throughout this series:

useState



Manage component state

useEffect



Synchronize with external systems / perform side effects



# Lecture 19 — Forms in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned about the `useEffect` Hook.

We learned how React can perform side effects such as:

- Calling an API
- Starting a timer
- Adding an event listener
- Updating the browser title
- Cleaning up resources

Today we're going to learn another very important topic:

## Forms in React

Forms are everywhere in web applications.

For example:

Login form  
Registration form  
Contact form  
Search form  
Product form  
Employee form  
Student form  
Checkout form

If you're building a real-world React application, you will work with forms constantly.

In this lecture, we'll learn:

- How HTML forms work in React
- Form events
- `onChange`
- Controlled components
- Managing input using `useState`
- Text inputs

- Number inputs
- Checkboxes
- Radio buttons
- Select dropdowns
- Multiple form fields
- Form submission
- `preventDefault()`
- Basic validation
- Building a complete React form

Let's get started.

---

## 2. Quick Revision — React Events

Before creating forms, let's quickly revise React events.

In normal HTML, we might write:

```
<button onclick="saveData()">
 Save
</button>
```

In React, we use camelCase:

```
<button onClick={saveData}>
 Save
</button>
```

Similarly, for an input:

```
<input onChange={handleChange} />
```

So remember:

HTML  
onclick

React  
onClick

And:

HTML  
onchange

React  
onChange

---

### 3. Creating a Basic HTML Form

Let's start with a normal form.

```
function App() {
 return (
 <form>

 <label>
 Name
 </label>

 <input
 type="text"
 />

 <label>
 Email
 </label>

 <input
 type="email"
 />

 <button>
 Submit
 </button>

 </form>
);
}
```

This looks very similar to HTML.

We have:

form  
↓  
label  
↓  
input  
↓  
button

But there's an important difference.

In React, we generally want to control and manage the form data using React state.

This brings us to an important concept.

## Controlled Components

---

### 4. What Is a Controlled Component?

A controlled component is a form element whose value is controlled by React state.

For example:

```
const [name, setName] =
 useState("");
```

Then:

```
<input
 value={name}
>
```

Now the input's value comes from:

React State  
↓  
value  
↓  
Input

When the user types something, we update the state.

```
onChange={(e) =>
 setName(e.target.value)
}
```

So the complete flow is:

User types  
↓  
onChange  
↓  
setName()  
↓  
State changes  
↓  
React renders  
↓  
value updates

This is one of the most important concepts when working with React forms.

---

## 5. First Controlled Input

Let's create a simple example.

```
import { useState } from "react";

function App() {

 const [name, setName] =
 useState("");

 return (
 <div>

 <input
 type="text"
 value={name}
 onChange={(e) =>
 setName(e.target.value)
 }
 />

 <h2>
 {name}
 </h2>

 </div>
);
}

export default App;
```

Let's understand what's happening.

Initially:

```
name = ""
```

The input receives:

```
value={name}
```

So it's empty.

Now the user types:

A

The onChange event executes.

```
setName(e.target.value)
```

Now:

```
name = "A"
```

React renders again.

Then the user types:

Arjun

The state becomes:

```
name = "Arjun"
```

And the <h2> displays:

Arjun

---

## 6. Understanding event

You will frequently see:

```
onChange={(e) =>
 setName(e.target.value)
}
```

Here:

e

is the event object.

We can inspect it:

```
onChange={(e) => {
 console.log(e);
}}
```

The event object contains information about the event.

The most important property for forms is:

`e.target`

`e.target` refers to the input element that generated the event.

And:

`e.target.value`

gives us the current value of that input.

So:

`e`  
↓  
Event object

`e.target`  
↓  
Input element

`e.target.value`  
↓  
Current input value

This is extremely important.

---

## 7. Using a Separate Event Handler

Instead of writing:

```
onChange={(e) =>
 setName(e.target.value)
}
```

we can create a function.

```
const handleNameChange = (e) => {

 setName(e.target.value);

};
```

Then:

```
<input
 value={name}
 onChange={handleNameChange}
/>
```

Complete example:

```
import { useState } from "react";

function App() {

 const [name, setName] =
 useState("");

 const handleNameChange = (e) => {

 setName(e.target.value);

 };

 return (
 <div>

 <input
 type="text"
 value={name}
 onChange={handleNameChange}
 />

 <h2>
 Hello {name}
 </h2>

 </div>
);
}

export default App;
```



This approach becomes useful when the form logic becomes more complex.

---

## 8. Building a Login Form

Now let's build something more realistic.

We'll create a login form containing:

Email  
Password  
Login button

First, create state:

```
const [email, setEmail] =
 useState("");

const [password, setPassword] =
 useState("");
```

Then:

```
<input
 type="email"
 value={email}
 onChange={(e) =>
 setEmail(e.target.value)
 }
</>
```

And:

```
<input
 type="password"
 value={password}
 onChange={(e) =>
 setPassword(e.target.value)
 }
</>
```

Our form now has two controlled inputs.

---

## 9. Handling Form Submission

Now let's talk about the Submit button.

We can write:

```
<form>
```

and:

```
<button type="submit">
 Login
</button>
```

But when we click Submit, the browser normally performs its default form submission behavior.

In a traditional web application, this could cause the page to reload or navigate.

In React applications, we commonly handle the submission ourselves.

We use:

`onSubmit`

For example:

```
<form onSubmit={handleSubmit}>
```

Then:

```
const handleSubmit = (e) => {

 e.preventDefault();

 console.log(email);
 console.log(password);

};
```

---

## 10. What Is `preventDefault()`?

This line is extremely important:

```
e.preventDefault();
```

The browser has default behavior for form submission.

preventDefault() tells the browser:

Don't perform the default form submission behavior.

So our React application can handle the form submission.

The flow becomes:

```
User clicks Submit
 ↓
onSubmit executes
 ↓
preventDefault()
 ↓
React handles the form
```

---

## 11. Complete Login Form

Let's put everything together.

```
import { useState } from "react";

function Login() {

 const [email, setEmail] =
 useState("");

 const [password, setPassword] =
 useState("");

 const handleSubmit = (e) => {

 e.preventDefault();

 console.log("Email:", email);
 console.log("Password:", password);

 };

 return (
 <form onSubmit={handleSubmit}>

 <div>

 <label>
```

```
 Email
 </label>

 <input
 type="email"
 value={email}
 onChange={(e) =>
 setEmail(e.target.value)
 }
 />

 </div>

 <div>

 <label>
 Password
 </label>

 <input
 type="password"
 value={password}
 onChange={(e) =>
 setPassword(e.target.value)
 }
 />

 </div>

 <button type="submit">
 Login
 </button>

 </form>
);
}
```

**export default** Login;

Now enter:

Email:  
student@example.com

Password:  
123456

Click Login.

The console should display:

Email: student@example.com

Password: 123456

Notice that the page doesn't reload because we called:

```
e.preventDefault();
```

---

## 12. Handling Multiple Inputs

Now imagine we have a registration form.

We need:

First Name

Last Name

Email

Password

Phone

One approach is to create separate state variables:

```
const [firstName, setFirstName] =
 useState("");
```

```
const [lastName, setLastName] =
 useState("");
```

```
const [email, setEmail] =
 useState("");
```

```
const [password, setPassword] =
 useState("");
```

```
const [phone, setPhone] =
 useState("");
```

This works.

But when the number of fields becomes large, it can become repetitive.

A common approach is to store the entire form in one object.

---

## 13. Form State as an Object

We can write:

```
const [formData, setFormData] = useState({

 firstName: "",
 lastName: "",
 email: "",
 password: "",
 phone: ""

});
```

Now our complete form data exists in one object.

Initially:

```
{
 firstName: "",
 lastName: "",
 email: "",
 password: "",
 phone: ""
}
```

---

## 14. Updating Object State

Suppose the user enters:

Arjun

into the first name field.

We need to update:

firstName

while keeping the other properties unchanged.

We can use the spread operator:

```
setFormData({

 ...formData,
```

```
 firstName: e.target.value
```

```
 });
```

The ...formData copies the existing properties.

Then:

firstName

is updated.

This is an important JavaScript concept when working with React state.

---

## 15. Complete Multiple-Input Example

```
import { useState } from "react";

function Registration() {

 const [formData, setFormData] =
 useState({

 firstName: "",
 lastName: "",
 email: "",
 password: ""

 });

 const handleSubmit = (e) => {

 e.preventDefault();

 console.log(formData);

 };

 return (
 <form onSubmit={handleSubmit}>

 <div>

 <label>
 First Name
```

```
 </label>

 <input
 type="text"
 value={formData.firstName}
 onChange={(e) =>
 setFormData({

 ...formData,

 firstName:
 e.target.value

 })
 }
 />

 </div>

 <div>

 <label>
 Last Name
 </label>

 <input
 type="text"
 value={formData.lastName}
 onChange={(e) =>
 setFormData({

 ...formData,

 lastName:
 e.target.value

 })
 }
 />

 </div>

 <div>

 <label>
 Email
```



```
 </label>

 <input
 type="email"
 value={formData.email}
 onChange={(e) =>
 setFormData({

 ...formData,

 email:
 e.target.value

 })
 }
 />

 </div>

 <div>

 <label>
 Password
 </label>

 <input
 type="password"
 value={formData.password}
 onChange={(e) =>
 setFormData({

 ...formData,

 password:
 e.target.value

 })
 }
 />

 </div>

 <button type="submit">
 Register
 </button>
```

```
 </form>
);
}
```

```
export default Registration;
```

This approach is useful, but we can make it much cleaner.

---

## 16. Using the `name` Attribute

We can give each input a name.

For example:

```
<input
 name="firstName"
/>
```

Then:

```
<input
 name="lastName"
/>
```

and:

```
<input
 name="email"
/>
```

Now we can create one common change handler.

```
const handleChange = (e) => {
```

```
 const {
 name,
 value
 } = e.target;

 setFormData({
 ...formData,
 [name]: value
 });
}
```

```
});
};
```

This is a very useful pattern.

---

## 17. Understanding Computed Property Names

This code:

```
[name]: value
```

may initially look confusing.

Suppose:

```
name = "email"
```

Then:

```
[name]: value
```

becomes:

```
email: value
```

If:

```
name = "firstName"
```

then it becomes:

```
firstName: value
```

So one function can handle multiple inputs.

---

## 18. Cleaner Registration Form

Now our form becomes much cleaner.

```
import { useState } from "react";

function Registration() {
 const [formData, setFormData] =
 useState({
```

```
 firstName: "",
 lastName: "",
 email: "",
 password: ""

 });

 const handleChange = (e) => {

 const {
 name,
 value
 } = e.target;

 setFormData({

 ...formData,

 [name]: value

 });
 };

 const handleSubmit = (e) => {

 e.preventDefault();

 console.log(formData);

 };

 return (
 <form onSubmit={handleSubmit}>

 <input
 type="text"
 name="firstName"
 placeholder="First Name"
 value={formData.firstName}
 onChange={handleChange}
 />

 <input
 type="text"
 name="lastName"
```

```
 placeholder="Last Name"
 value={formData.lastName}
 onChange={handleChange}
 />

 <input
 type="email"
 name="email"
 placeholder="Email"
 value={formData.email}
 onChange={handleChange}
 />

 <input
 type="password"
 name="password"
 placeholder="Password"
 value={formData.password}
 onChange={handleChange}
 />

 <button type="submit">
 Register
 </button>

 </form>
);
}
```

**export default** Registration;

Now all four inputs use:

`onChange={handleChange}`

This is a pattern students should remember.

---

## 19. Checkbox

Now let's look at checkboxes.

Suppose our registration form contains:

☐ I agree to the terms and conditions

We can create state:

```
const [agree, setAgree] =
 useState(false);
```

Then:

```
<input
 type="checkbox"
 checked={agree}
 onChange={(e) =>
 setAgree(e.target.checked)
 }
>
```

Notice something important.

For normal text input, we use:

`e.target.value`

For checkbox, we use:

`e.target.checked`

Because a checkbox has a checked state rather than a text value.

---

## 20. Checkbox Example

```
function Terms() {

 const [agree, setAgree] =
 useState(false);

 return (
 <div>

 <label>

 <input
 type="checkbox"
 checked={agree}
 onChange={(e) =>
 setAgree(
 e.target.checked
)
 }
 >
 </label>
 </div>
)
}
```

I agree to the terms

```
</label>
```

```
<p>
```

Agreement:

```
{agree ? " Yes" : " No"}
```

```
</p>
```

```
</div>
```

```
);
```

```
}
```

Initially:

Agreement: No

Click the checkbox:

Agreement: Yes

Click again:

Agreement: No

---

## 21. Radio Buttons

Now let's look at radio buttons.

Suppose we want the user to select gender:

- Male
- Female
- Other

We can use state:

```
const [gender, setGender] =
 useState("");
```

Then:

```
<input
 type="radio"
 name="gender"
 value="male"
 checked={gender === "male"}/>
```

```
 onChange={(e) =>
 setGender(e.target.value)
 }
 />
```

For Female:

```
<input
 type="radio"
 name="gender"
 value="female"
 checked={gender === "female"}
 onChange={(e) =>
 setGender(e.target.value)
 }
/>
```

Notice that all radio buttons have the same:

name="gender"

That makes them part of the same radio group.

---

## 22. Select Dropdown

Now let's create a dropdown.

Suppose the user needs to select a city.

```
const [city, setCity] =
 useState("");
```

Then:

```
<select
 value={city}
 onChange={(e) =>
 setCity(e.target.value)
 }
>

 <option value="">
 Select City
 </option>

 <option value="rajkot">
 Rajkot
```



```
</option>

<option value="ahmedabad">
 Ahmedabad
</option>

<option value="surat">
 Surat
</option>
```

```
</select>
```

Again, we use:

```
e.target.value
```

to get the selected value.

---

## 23. Number Input

Suppose we have:

```
<input
 type="number"
 value={age}
 onChange={(e) =>
 setAge(e.target.value)
 }
/>
```

One thing students should know:

Even though the HTML input type is:

number

the value received from the browser is commonly a string.

For example:

"25"

rather than:

25

If we need an actual number, we can convert it:

```
setAge(
 Number(e.target.value)
);
```

or:

```
const age =
 Number(e.target.value);
```

We'll use this concept later when sending numeric values to APIs.

---

## 24. Basic Form Validation

Now let's add simple validation.

Suppose email and password are required.

Inside:

handleSubmit

we can write:

```
if (!email) {
 alert("Email is required");
 return;
}
```

Then:

```
if (!password) {
 alert("Password is required");
 return;
}
```

Complete example:

```
const handleSubmit = (e) => {
 e.preventDefault();
 if (!email) {
```

```
 alert(
 "Email is required"
);

 return;
 }

 if (!password) {

 alert(
 "Password is required"
);

 return;
 }

 console.log(
 "Form submitted"
);
};
```

This is basic validation.

Later, in larger applications, you can use more advanced validation techniques and libraries.

---

## 25. HTML Validation

HTML itself also provides basic validation.

For example:

```
<input
 type="email"
 required
/>
```

The required attribute means the browser expects a value.

Similarly:

```
<input
 type="password"
 required
 minLength={6}
/>
```

This requires at least six characters.

React works together with standard HTML form capabilities.

---

## 26. Complete Student Registration Form

Now let's build one complete example combining today's concepts.

We'll create:

Student Registration

-----

Name

Email

Age

Gender

City

Agree to terms

Register

Let's start with state.

```
const [formData, setFormData] =
 useState({

 name: "",
 email: "",
 age: "",
 gender: "",
 city: "",
 agree: false

 });
```

Then:

```
const handleChange = (e) => {

 const {
 name,
 value,
 type,
 checked
 } = e.target;
```

```
setFormData({
 ...formData,

 [name]:
 type === "checkbox"
 ? checked
 : value
});
};
```

This is a very useful generic handler.

---

## 27. Complete Form Code

```
import { useState } from "react";

function StudentRegistration() {

 const [formData, setFormData] =
 useState({

 name: "",
 email: "",
 age: "",
 gender: "",
 city: "",
 agree: false
 });

 const handleChange = (e) => {

 const {
 name,
 value,
 type,
 checked
 } = e.target;

 setFormData({
```

```
 ...formData,

 [name]:
 type === "checkbox"
 ? checked
 : value

 });
 };

 const handleSubmit = (e) => {

 e.preventDefault();

 if (!formData.name) {

 alert(
 "Name is required"
);

 return;
 }

 if (!formData.email) {

 alert(
 "Email is required"
);

 return;
 }

 if (!formData.agree) {

 alert(
 "Please accept the terms"
);

 return;
 }

 console.log(
 "Form Data:",
 formData
);
 };
 }
}
```

```
};

return (
 <form onSubmit={handleSubmit}>

 <h1>
 Student Registration
 </h1>

 <div>

 <label>
 Name
 </label>

 <input
 type="text"
 name="name"
 value={formData.name}
 onChange={handleChange}
 />

 </div>

 <div>

 <label>
 Email
 </label>

 <input
 type="email"
 name="email"
 value={formData.email}
 onChange={handleChange}
 />

 </div>

 <div>

 <label>
 Age
 </label>

 <input
 type="number"
```

```
 name="age"
 value={formData.age}
 onChange={handleChange}
 />
 </div>

 <div>

 <label>
 Gender
 </label>

 <label>
 <input
 type="radio"
 name="gender"
 value="male"
 checked={
 formData.gender === "male"
 }
 onChange={handleChange}
 />
 Male
 </label>

 <label>
 <input
 type="radio"
 name="gender"
 value="female"
 checked={
 formData.gender === "female"
 }
 onChange={handleChange}
 />
 Female
 </label>

 </div>

 <div>

 <label>
 City
 </label>
```



```
<select
 name="city"
 value={formData.city}
 onChange={handleChange}
>

 <option value="">
 Select City
 </option>

 <option value="rajkot">
 Rajkot
 </option>

 <option value="ahmedabad">
 Ahmedabad
 </option>

 <option value="surat">
 Surat
 </option>

</select>

</div>

<div>

 <label>

 <input
 type="checkbox"
 name="agree"
 checked={
 formData.agree
 }
 onChange={handleChange}
 />

 I agree to the terms

 </label>

</div>

<button type="submit">
 Register
```

```
 </button>

 </form>
);
}

export default StudentRegistration;
```

## 28. Understanding the Complete Form Flow

Let's step through this application.

Initially:

```
formData = {
 name: "",
 email: "",
 age: "",
 gender: "",
 city: "",
 agree: false
}
```

The user types:

Arjun

into the name field.

The event occurs:

```
onChange
 ↓
handleChange
 ↓
name = "name"
value = "Arjun"
 ↓
setFormData()
 ↓
React renders
```

Now:

formData.name

contains:

Arjun

The user selects a city.

Again:

```
onChange
 ↓
handleChange
 ↓
city = "Rajkot"
 ↓
setFormData()
```

The user checks the checkbox.

This time:

type = checkbox

So our code uses:

checked

instead of:

value

That's why this line is useful:

```
type === "checkbox"
 ? checked
 : value
```

---

## 29. Form Submission Flow

When the user clicks:

Register

the following happens:

```
Submit button
 ↓
<form onSubmit={handleSubmit}>
 ↓
handleSubmit()
 ↓
```

```
preventDefault()
```



```
Validation
```



```
Form data
```

If validation succeeds:

```
console.log(formData);
```

Later, instead of:

```
console.log(formData);
```

we'll send the data to an API.

That's exactly what we'll do in the upcoming API lectures.

---

## 30. Connection With Future Lectures

Today's lecture is especially important because our upcoming lectures will involve APIs.

For example, in Lecture 24, we'll eventually do something like:

```
React Form
```



```
User enters product data
```



```
Submit
```



```
POST API
```



```
Server
```



```
Product created
```

And in Lecture 25:

```
Edit Form
```



```
User changes product
```



```
Submit
```



```
PUT/PATCH API
```



```
Server updates product
```

So forms are the bridge between:

User Input

and:

API

---

## 31. Common Mistakes

Let's discuss some common mistakes students make.

### Mistake 1 — Forgetting `value`

They write:

```
<input
 onChange={handleChange}
>
```

but don't connect the input to state.

For a controlled component, we normally use:

```
value={name}
```

and:

```
onChange={handleChange}
```

---

### Mistake 2 — Forgetting `onChange`

They write:

```
<input
 value={name}
>
```

without providing a way to update the value.

A controlled input needs a mechanism to update the state.

---

### Mistake 3 — Forgetting `preventDefault()`

They write:

```
const handleSubmit = () => {
 console.log(formData);
};
```

but don't prevent the browser's default submission behavior.

Usually we want:

```
const handleSubmit = (e) => {
 e.preventDefault();
};
```

---

### Mistake 4 — Using `value` for checkbox state

For checkboxes, use:

```
e.target.checked
```

rather than relying on:

```
e.target.value
```

---

### Mistake 5 — Forgetting `name`

When using a generic handler like:

```
[name]: value
```

every input must have the correct:

`name`

attribute.

For example:

```
<input
 name="email"
>
```

---

## 32. Important Concept — Controlled vs Uncontrolled

There are two broad approaches to form inputs in React.

### Controlled

React state controls the input.

```
<input
 value={name}
 onChange={handleChange}
>
```

The data flow is:

```
State
↓
Input

Input
↓
Event
↓
State
```

This is the approach we're focusing on today.

---

### Uncontrolled

The DOM itself keeps track of the value, often accessed using a ref.

For example:

```
<input
 ref={inputRef}
>
```

We aren't going to focus on uncontrolled forms in this lecture.

For beginners, controlled forms are an important foundation because they make the data flow explicit.

---

## 33. Best Practices

When creating React forms:

### 1. Keep form state organized

For larger forms:

```
const [formData, setFormData] =
 useState({});
```

can be convenient.

---

### 2. Use meaningful names

For example:

```
firstName
lastName
email
phone
```

rather than:

```
a
b
c
d
```

---

### 3. Validate user input

Don't blindly send invalid data to your API.

---

### 4. Keep submit logic separate

Use:

```
handleSubmit
```

for submission logic.



---

## 5. Use reusable input components for large applications

When your application becomes bigger, you can create reusable components such as:

TextInput  
SelectInput  
Checkbox  
FormField

We'll keep today's implementation simple.

---

## 34. Practice Exercise

Now give your students this exercise.

### Exercise — Employee Registration Form

Create a React form containing:

Employee Name  
Email  
Phone  
Salary  
Department  
Gender  
Skills  
Accept Terms  
Submit

Requirements:

16. Use `useState`.
17. Use controlled components.
18. Use `onChange`.
19. Use a single `handleChange` where appropriate.
20. Handle form submission using `onSubmit`.
21. Call `preventDefault()`.
22. Validate Employee Name.
23. Validate Email.
24. Validate Department.
25. Display the submitted object in the console.

The expected flow is:

User enters data



React state



Submit



Validation



Console

---

## 35. Interview Questions

Let's finish with some common interview questions.

### Question 1

What is a controlled component?

**Answer:**

A controlled component is a form element whose value is controlled by React state.

---

### Question 2

How do you get the value from a text input?

`e.target.value`

---

### Question 3

How do you get the checked state of a checkbox?

`e.target.checked`

---

### Question 4

Why do we use `preventDefault()`?

It prevents the browser's default form submission behavior so that React can handle the submission.

---

### Question 5

What is onChange used for?

It is used to respond to changes in form elements and commonly update React state.

---

### Question 6

Why do we use the name attribute?

When using a generic form change handler, name identifies which field is being updated.

For example:

```
<input
 name="email"
>
```

allows us to do:

```
setFormData({
 ...formData,
 [name]: value
});
```

---

## 36. Lecture Summary

Today we learned how to work with forms in React.

We covered:

```
HTML Forms
 ↓
React Events
 ↓
onChange
 ↓
Controlled Components
 ↓
useState
```

↓  
Text Inputs  
↓  
Number Inputs  
↓  
Checkbox  
↓  
Radio Buttons  
↓  
Select  
↓  
Multiple Fields  
↓  
Form Submission  
↓  
preventDefault()  
↓  
Validation

The most important pattern to remember is:

```
const [name, setName] =
 useState("");

<input
 value={name}
 onChange={(e) =>
 setName(e.target.value)
 }
>
```

For multiple fields:

```
const [formData, setFormData] =
 useState({
 name: "",
 email: ""
 });
```

And:

```
const handleChange = (e) => {

 const {
 name,
 value
 } = e.target;

 setFormData({
```

```
 ...formData,
 [name]: value
 });
};
```

For submission:

```
const handleSubmit = (e) => {
 e.preventDefault();
 console.log(formData);
};
```

---

## 37. Final Takeaway

If you remember only one concept from today's lecture, remember this:

**In a controlled React form, the input value comes from React state, and user changes update that state.**

The basic flow is:

```
User types
 ↓
onChange
 ↓
setState
 ↓
React state changes
 ↓
Component renders
 ↓
Input displays new value
```

And when the user submits:

```
Submit
 ↓
onSubmit
 ↓
preventDefault()
```

↓  
Validate  
↓  
Use form data

This is the foundation we'll use for the API operations coming next.

# Lecture 20 — Creating a Mock API

## 1. Opening

Hello everyone, and welcome back.

So far in our React series, we have worked mostly on the frontend.

We learned:

- JSX
- Components
- Props
- Events
- State
- Hooks
- Routing
- Forms

In the previous lecture, we learned how to create and manage forms in React.

But now we have a question.

Suppose we create a product management application.

We have a form like:

Product Name  
Price  
Quantity  
Category

[Save Product]

When the user clicks **Save Product**, where should the data go?

We don't want the data to disappear when the page refreshes.

We want to store it somewhere.

This is where APIs come into the picture.

In today's lecture, we'll understand:

- What is an API?

- Why does React need an API?
- What is a REST API?
- What is JSON?
- What are HTTP methods?
- What is a mock API?
- Why are we using a mock API?
- How to create a mock API
- How to create sample data
- How to test API endpoints
- How this API will be used in our next lectures

Let's begin.

---

## 2. Frontend vs Backend

Before understanding APIs, let's understand a typical web application.

Imagine we are building an employee management application.

The user opens our React application.

The React application runs in the browser.

This is the:

Frontend

The frontend is responsible for things like:

User Interface

Buttons

Forms

Tables

Menus

Navigation

Validation

But employee data needs to be stored somewhere.

For example:

Employee ID

Employee Name

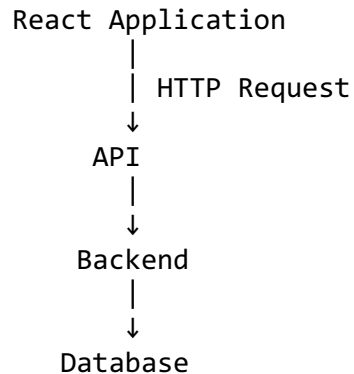
Email



Salary  
Department

Usually, this information is stored on a backend/server and database.

So we can think about our application like this:



The React application doesn't normally communicate directly with the database.

Instead, it communicates with the backend through APIs.

---

### 3. What Is an API?

API stands for:

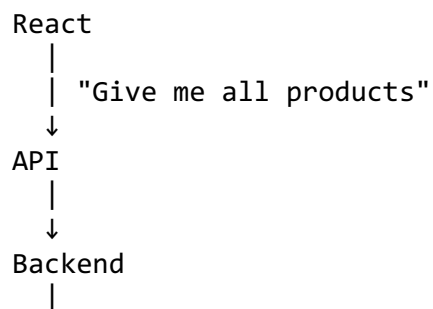
Application Programming Interface

Don't worry about the complicated name.

For our React course, think of an API as:

A way for our frontend application to communicate with a backend application.

For example:





## Database

The backend gets the data and sends it back.

For example:

```
[
 {
 "id": 1,
 "name": "Laptop",
 "price": 50000
 },
 {
 "id": 2,
 "name": "Mouse",
 "price": 1000
 }
]
```

React can then display this data.

---

## 4. Real-World Example

Let's take an e-commerce website.

Suppose the user opens:

Products

The React application needs product data.

It might make a request such as:

GET /products

The API responds:

```
[
 {
 "id": 1,
 "name": "Laptop",
 "price": 50000
 },
 {
 "id": 2,
 "name": "Keyboard",
 "price": 1000
 }
]
```

```
 "price": 2000
 }
]
```

React receives this data and renders:

Laptop       ₹50,000

Keyboard     ₹2,000

So the API acts as the communication layer between frontend and backend.

---

## 5. What Is JSON?

When frontend and backend communicate, they need a common format for exchanging data.

One of the most common formats is:

JSON

JSON stands for:

JavaScript Object Notation

For example:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 50000
}
```

It looks very similar to a JavaScript object.

But JSON has some rules.

Property names are generally written using double quotes:

```
{
 "name": "Laptop"
}
```

And strings use double quotes:

```
" Laptop "
```

rather than JavaScript-style single quotes.

## 6. JSON Array

An API can also return an array.

For example:

```
[
 {
 "id": 1,
 "name": "Laptop",
 "price": 50000
 },
 {
 "id": 2,
 "name": "Mouse",
 "price": 1000
 },
 {
 "id": 3,
 "name": "Keyboard",
 "price": 2000
 }
]
```

This represents multiple products.

Our React application can use this array to render a product list.

That's exactly what we'll do in Lecture 21.

## 7. HTTP Methods

When working with APIs, we need to tell the server what operation we want to perform.

The most common HTTP methods we'll use are:

GET  
POST  
PUT  
PATCH  
DELETE

Let's understand them.

---

## GET

GET is generally used to retrieve data.

For example:

GET /products

means:

Give me the products.

---

## POST

POST is generally used to create new data.

For example:

POST /products

with:

```
{
 "name": "Monitor",
 "price": 15000
}
```

means:

Create a new product.

---

## PUT

PUT is generally used to update an existing resource.

For example:

PUT /products/2

means:

Update product with ID 2.

---

## PATCH

PATCH is generally used for a partial update.

For example:

PATCH /products/2

might update only the price.

---

## DELETE

DELETE removes data.

For example:

DELETE /products/2

means:

Delete product with ID 2.

---

## 8. HTTP Methods and CRUD

We can connect these methods with CRUD operations.

CRUD stands for:

Create  
Read  
Update  
Delete

The relationship is approximately:

CRUD	HTTP Method
Create	POST
Read	GET
Update	PUT/PATCH
Delete	DELETE

Our next lectures follow exactly this pattern.

Lecture 21 → GET ALL  
Lecture 22 → GET BY ID  
Lecture 23 → DELETE  
Lecture 24 → INSERT  
Lecture 25 → UPDATE

So today's lecture prepares us for the rest of the series.

---

## 9. What Is an Endpoint?

An API usually exposes URLs called endpoints.

For example:

`/products`

could be an endpoint for products.

We could have:

`GET /products`

to get all products.

And:

`GET /products/1`

to get product number 1.

Similarly:

`POST /products`

creates a product.

`DELETE /products/1`

deletes product number 1.

Think of an endpoint as a specific address where we can perform an API operation.

---

## 10. Why Are We Creating a Mock API?

Normally, a real application would have:

React  
↓  
Backend API  
↓  
Database

But creating a complete backend and database is outside the scope of our beginner React series.

We want to focus on React.

So we're going to create a:

## Mock API

A mock API behaves like a real API for learning and development.

Our architecture will look like:

React  
↓  
Mock API  
↓  
Mock Data

This allows us to practice:

GET  
POST  
PUT  
DELETE

without building a complete backend.

---

## 11. Tool We Will Use

For this course, we'll use:

JSON Server

JSON Server is useful for creating a simple REST API from a JSON file.

We will create a file such as:

db.json



and put our data inside it.

JSON Server will expose that data through API endpoints.

For example, if our data contains:

```
{
 "products": [
 {
 "id": 1,
 "name": "Laptop",
 "price": 50000
 }
]
}
```

JSON Server can expose:

/products

as an API endpoint.

This is perfect for our React learning project.

---

## 12. Setting Up the Mock API

Now let's move to the practical part.

We already have our React project.

For our mock API, we'll create a separate folder.

Our project structure can look like:

```
react-course
├── react-app
└── mock-api
```

Inside:

mock-api

we'll create:

db.json

## 13. Creating db.json

Let's create our database file.

```
{
 "products": [
 {
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
 },
 {
 "id": 2,
 "name": "Mouse",
 "price": 1000,
 "quantity": 25,
 "category": "Accessories"
 },
 {
 "id": 3,
 "name": "Keyboard",
 "price": 2000,
 "quantity": 15,
 "category": "Accessories"
 },
 {
 "id": 4,
 "name": "Monitor",
 "price": 15000,
 "quantity": 8,
 "category": "Electronics"
 }
]
}
```

Let's understand this.

We have one collection:

products

And inside products, we have multiple objects.

Each object represents one product.

For example:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
}
```

---

## 14. Why Do We Need an ID?

Notice that every product has:

```
"id": 1
```

or:

```
"id": 2
```

The ID uniquely identifies the record.

For example:

Product 1 → Laptop

Product 2 → Mouse

Product 3 → Keyboard

This becomes very important when we want to get one particular product.

For example:

GET /products/2

means:

Get the product whose ID is 2.

It also becomes important when deleting or updating.

For example:

DELETE /products/2

means:

Delete product 2.

## 15. Installing JSON Server

Now let's install JSON Server.

Open the terminal inside our mock API folder.

We can install it as a development dependency:

```
npm install json-server --save-dev
```

After installation, we should have:

```
node_modules
package.json
package-lock.json
db.json
```

---

## 16. Creating the Start Command

Open:

package.json

Inside the scripts section, we can add:

```
{
 "scripts": {
 "start": "json-server --watch db.json --port 3001"
 }
}
```

Now we can run:

```
npm start
```

The mock API will start on:

http://localhost:3001

We're deliberately using port 3001.

Our React development server may be running on another port, such as:

http://localhost:5173

So we'll have:

React  
`http://localhost:5173`

API  
`http://localhost:3001`

---

## 17. Understanding Our First Endpoint

Once JSON Server is running, open:

`http://localhost:3001/products`

We should get something similar to:

```
[
 {
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
 },
 {
 "id": 2,
 "name": "Mouse",
 "price": 1000,
 "quantity": 25,
 "category": "Accessories"
 }
]
```

Congratulations.

We now have an API.

---

## 18. Understanding the Request

When we open:

`/products`

we are effectively making:

GET `/products`

The server receives the request.

It looks at:

products

in:

db.json

and returns the data.

The flow is:

```
Browser
↓
GET /products
↓
JSON Server
↓
db.json
↓
products
↓
JSON response
↓
Browser
```

---

## 19. GET BY ID

Now let's try:

`http://localhost:3001/products/1`

We should get:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
}
```

This is different from:

`/products`

because:

/products

returns multiple records.

While:

/products/1

returns one record.

So remember:

GET /products

↓

All products

GET /products/1

↓

One product

We'll use this in Lecture 22.

---

## 20. Testing POST

Now let's test creating a new product.

We need to make a POST request.

We can use an API testing tool such as Postman or another REST client.

Create a request:

POST

http://localhost:3001/products

Set the request body to JSON:

```
{
 "name": "Webcam",
 "price": 3000,
 "quantity": 12,
 "category": "Accessories"
}
```

Then send the request.

The server should create a new product.

The response will contain the newly created object, including its ID.

For example:

```
{
 "id": 5,
 "name": "Webcam",
 "price": 3000,
 "quantity": 12,
 "category": "Accessories"
}
```

Now if we open:

GET /products

we should see the new product.

---

## 21. Testing DELETE

Now let's delete a product.

For example:

DELETE  
http://localhost:3001/products/5

Send the request.

The product with ID 5 should be removed.

Then call:

GET /products

again.

The product should no longer appear.

This demonstrates the CRUD behavior of our mock API.

---



## 22. Testing UPDATE

We can also update a product.

For example:

PUT

`http://localhost:3001/products/1`

Request body:

```
{
 "name": "Gaming Laptop",
 "price": 70000,
 "quantity": 5,
 "category": "Electronics"
}
```

After sending the request, product 1 will contain the updated information.

Again, this is something we'll perform from React in Lecture 25.

---

## 23. Our API Structure

At this point, our mock API supports the operations we need.

We can think of our API like this:

```
GET
/products
↓
Get all products

GET
/products/1
↓
Get product 1

POST
/products
↓
Create product

DELETE
/products/1
↓
Delete product 1
```

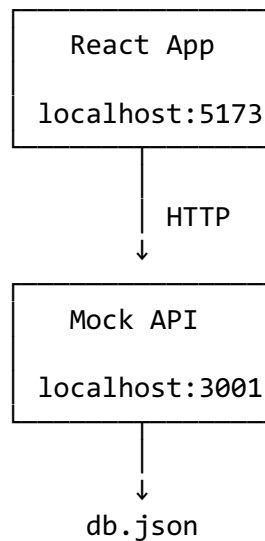
PUT  
/products/1  
↓  
Update product 1

This is essentially our complete CRUD API.

## 24. Connecting This With Our React Application

Now let's connect this lecture with the next few lectures.

Our architecture is now:



Our React application will make requests to:

`http://localhost:3001/products`

For example:

```
fetch(
 "http://localhost:3001/products"
)
```

The API will return JSON.

React will then store that data in state.

For example:

```
const [products, setProducts] =
 useState([]);
```

Then:

```
API response
 ↓
setProducts()
 ↓
React state
 ↓
Rendering
 ↓
Product list
```

This will be the topic of our next lecture.

---

## 25. Why Are We Using a Mock API?

Students may ask:

Why don't we just use a real backend?

That's a good question.

The goal of this series is to learn React.

If we build a complete backend, we would have to introduce:

- Node.js
- Express
- Database
- Authentication
- Database queries
- Server configuration

That would take our attention away from React.

Instead, the mock API gives us enough functionality to learn:

- API calls
- HTTP methods
- JSON
- Loading data
- Displaying data
- Creating data

Updating data  
Deleting data

without requiring a full backend.

---

## 26. Important Difference — Mock API vs Real API

A mock API is not necessarily a replacement for a production backend.

It is primarily useful for:

Learning  
Prototyping  
Frontend development  
Testing  
Demonstration

A real production application might use:

React  
↓  
REST/GraphQL API  
↓  
Backend  
↓  
Database

Our course uses:

React  
↓  
JSON Server  
↓  
db.json

The frontend concepts remain very similar.

---

## 27. Common Mistakes

Let's discuss some common mistakes.

### Mistake 1 — Invalid JSON

This is valid JSON:

```
{
 "name": "Laptop"
}
```

This is not valid JSON:

```
{
 name: "Laptop"
}
```

JSON requires property names to use double quotes.

---

## Mistake 2 — Duplicate IDs

Avoid manually creating multiple objects with the same ID.

For example, don't create:

```
{
 "id": 1,
 "name": "Laptop"
}
```

and:

```
{
 "id": 1,
 "name": "Mouse"
}
```

IDs should uniquely identify records.

---

## Mistake 3 — Wrong Port

If your API is running on:

3001

then your React code needs:

`http://localhost:3001`

not:

`http://localhost:3000`

or another port.

---

## Mistake 4 — API Server Not Running

If React tries to call:

`http://localhost:3001/products`

but JSON Server isn't running, the request will fail.

So when working with the project, we need both:

React server

+

API server

running.

---

## 28. Live Coding Demonstration Flow

For the actual video, I recommend doing the live coding in this exact order.

### Step 1

Open the existing React project.

Say:

“Our React application is already ready. Now I’m going to create a separate folder for our mock API.”

---

### Step 2

Create:

`mock-api`

---

### Step 3

Initialize npm:

`npm init -y`

---

## Step 4

Install JSON Server:

```
npm install json-server --save-dev
```

## Step 5

Create:

db.json

## Step 6

Add:

```
{
 "products": [
 {
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
 },
 {
 "id": 2,
 "name": "Mouse",
 "price": 1000,
 "quantity": 25,
 "category": "Accessories"
 },
 {
 "id": 3,
 "name": "Keyboard",
 "price": 2000,
 "quantity": 15,
 "category": "Accessories"
 }
]
}
```

## Step 7

Update package.json:

```
{
 "scripts": {
 "start": "json-server --watch db.json --port 3001"
 }
}
```

---

## Step 8

Run:

npm start

---

## Step 9

Open:

http://localhost:3001/products

Show the response.

---

## Step 10

Open:

http://localhost:3001/products/1

Explain the difference.

---

## Step 11

Use Postman or another REST client to demonstrate:

POST  
GET  
PUT  
DELETE

Don't spend too much time on the API testing tool.



The main purpose is to show students that our API actually works.

---

## 29. Suggested Screen Flow for the Video

For a 10–15 minute lecture, structure the screen recording like this:

0:00 – 1:30

Introduction

1:30 – 3:30

Frontend, backend and API concept

3:30 – 5:00

HTTP methods + CRUD

5:00 – 6:00

JSON and endpoints

6:00 – 9:00

Create JSON Server

9:00 – 11:00

Create db.json

11:00 – 13:00

Test GET, GET BY ID, POST

13:00 – 14:00

Explain DELETE and UPDATE

14:00 – 15:00

Connect with upcoming React lectures

---

## 30. Key Points to Remember

Let's summarize today's lecture.

An API allows:

Frontend ↔ Backend

JSON is a common format for exchanging data.

The main HTTP methods are:

GET  
POST  
PUT  
PATCH  
DELETE

CRUD means:

Create  
Read  
Update  
Delete

We created a mock API using JSON Server.

Our API is:

`http://localhost:3001`

Our product endpoint is:

`http://localhost:3001/products`

And a specific product can be accessed using:

`http://localhost:3001/products/1`

---

## 31. Final Takeaway

The most important thing to understand from today's lecture is the communication flow:

```
graph TD
 A[React Application] --> B[HTTP Request]
 B --> C[API]
 C --> D[Data]
 D --> E[HTTP Response]
 E --> F[React Application]
 F --> G[UI]
```

In the upcoming lectures, we're going to implement this flow from React.

In Lecture 21, we'll make our first API call from React:

GET /products

We'll retrieve all products from our mock API and display them in a React application.

# Lecture 21 — Consuming GET ALL API in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we created our mock API using JSON Server.

We created a `db.json` file containing product data, and JSON Server exposed that data through an API.

Our API is running at:

`http://localhost:3001`

And our products endpoint is:

`http://localhost:3001/products`

For example, if we open that URL in the browser, we get:

```
[
 {
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
 },
 {
 "id": 2,
 "name": "Mouse",
 "price": 1000,
 "quantity": 25,
 "category": "Accessories"
 }
]
```

But there is one problem.

Right now we're viewing the API directly in the browser.

What we actually want is:

API  
↓  
React  
↓  
React UI

We want React to call the API, receive the product data, store it in state, and display it on the screen.

That's exactly what we'll learn today.

---

## 2. What Does “Consuming an API” Mean?

When we say:

“We are consuming an API”

we simply mean:

Our application is making a request to an API and using the response data.

For example:

```
React Application
 |
 | GET request
 |
 ↓
http://localhost:3001/products
 |
 |
 ↓
Mock API
 |
 |
 ↓
JSON response
 |
 |
 ↓
React
 |
 |
 ↓
Display products
```

This is API consumption.

### 3. What Is GET?

In the previous lecture, we learned about HTTP methods.

For today's lecture, we're interested in:

GET

GET is generally used to retrieve data.

For example:

GET /products

means:

Give me all products.

And:

GET /products/1

means:

Give me product with ID 1.

Today we're focusing on:

GET /products

So our goal is:

Get ALL products

↓

Store them in React state

↓

Display them

---

### 4. How Does React Make an API Request?

There are several ways to make API requests in JavaScript.

One of the simplest is:

`fetch()`

For example:

```
fetch(
 "http://localhost:3001/products"
);
```

The `fetch()` function makes a network request.

But there's something important to understand.

`fetch()` is asynchronous.

That means the browser doesn't stop everything and wait for the API response.

Instead, the request happens asynchronously.

That's why we'll use:

```
fetch()
+
async/await
```

and:

```
useEffect()
```

---

## 5. Why Use `useEffect()`?

This is an important question.

Why don't we simply write:

```
function App() {

 fetch(
 "http://localhost:3001/products"
);

 return (...);

}
```

Because the component function runs during rendering.

If we make the API request directly inside the component body, we can accidentally create repeated API requests whenever the component renders.

We don't want that.

API calls are side effects.

And from Lecture 18, we already know that:

`useEffect`

is used for side effects.

So we'll put our API call inside:

`useEffect()`

The basic structure is:

```
useEffect(() => {
```

```
 // API call
```

```
}, []);
```

The empty dependency array:

```
[]
```

means the effect runs after the initial render.

For today's beginner example, this is exactly what we need.

---

## 6. First API Call

Let's start with a simple example.

```
import { useEffect } from "react";

function App() {

 useEffect(() => {

 fetch(
 "http://localhost:3001/products"
);

 }, []);

 return (
 <h1>
 Products
```



```
 </h1>
);
}
```

```
export default App;
```

At this point, we're making the API request.

But we're not doing anything with the response.

So let's understand the response.

---

## 7. Understanding the Fetch Promise

When we call:

```
fetch(
 "http://localhost:3001/products"
)
```

`fetch()` returns a Promise.

We can use:

```
.then()
```

to handle the response.

For example:

```
fetch(
 "http://localhost:3001/products"
)
 .then(response => {

 console.log(response);

 });
```

Now we'll see a Response object in the browser console.

It contains information about the HTTP response.

But we still don't have our JavaScript product array.

Why?

Because the response body needs to be converted from JSON.

We do that using:

```
response.json()
```

---

## 8. Converting Response to JSON

Let's modify the code:

```
fetch(
 "http://localhost:3001/products"
)
 .then(response => {

 return response.json();

 })
 .then(data => {

 console.log(data);

 });
```

Now the flow is:

```
fetch()
 ↓
HTTP Response
 ↓
response.json()
 ↓
JavaScript data
 ↓
data
```

The data variable should contain:

```
[
 {
 id: 1,
 name: "Laptop",
 price: 50000
 },
 {
 id: 2,
```

```
 name: "Mouse",
 price: 1000
 }
]
```

---

## 9. Using `async/await`

The `.then()` syntax works perfectly fine.

But many React developers prefer `async/await` because it can be easier to read.

Let's create a function:

```
const getProducts = async () => {

 const response =
 await fetch(
 "http://localhost:3001/products"
);

 const data =
 await response.json();

 console.log(data);

};
```

Then call it from `useEffect`:

```
useEffect(() => {

 getProducts();

}, []);
```

Now our code is easier to understand.

---

## 10. Important: Don't Make the Effect Callback Async

A common beginner mistake is:

```
useEffect(async () => {
```

```
// API call
```

```
}, []);
```

Don't do this.

Instead, create an async function inside the effect or outside it.

For example:

```
useEffect(() => {

 const getProducts = async () => {

 const response =
 await fetch(
 "http://localhost:3001/products"
);

 const data =
 await response.json();

 console.log(data);

 };

 getProducts();

}, []);
```

This is a good pattern for our course.

---

## 11. Storing API Data in State

Currently we're only doing:

```
console.log(data);
```

But our goal is to display the products.

So we need React state.

Let's create:

```
import {
 useEffect,
```

```
 useState
 } from "react";
```

Then:

```
const [products, setProducts] =
 useState([]);
```

Initially:

```
products = []
```

That's an empty array.

Once our API responds, we'll do:

```
setProducts(data);
```

Now the flow becomes:

```
API response
 ↓
data
 ↓
setProducts(data)
 ↓
products state
 ↓
React re-renders
```

---

## 12. Complete First Example

Let's put this together.

```
import {
 useEffect,
 useState
} from "react";

function App() {

 const [products, setProducts] =
 useState([]);

 useEffect(() => {

 const getProducts = async () => {
```

```
 const response =
 await fetch(
 "http://localhost:3001/products"
);

 const data =
 await response.json();

 setProducts(data);

 };

 getProducts();

}, []);

return (
 <div>

 <h1>
 Products
 </h1>

 <pre>
 {JSON.stringify(
 products,
 null,
 2
)}
 </pre>

 </div>
);
}

export default App;
```

Let's understand what's happening.

---

## 13. Step-by-Step Execution

When the component first renders:

`products = []`

React displays:

Products

`[]`

Then:

`useEffect()`

runs after the component renders.

Inside the effect:

`getProducts()`

is called.

Then:

`fetch(...)`

sends:

GET /products

to our API.

The API responds.

Then:

`response.json()`

converts the response into JavaScript data.

Then:

`setProducts(data)`

updates our state.

React renders again.

Now:

`products`

contains all the products.

This is the most important flow of today's lecture.

## 14. The Complete Data Flow

Draw this on the screen while teaching:

```
Component renders
 ↓
useEffect()
 ↓
getProducts()
 ↓
fetch()
 ↓
GET /products
 ↓
Mock API
 ↓
JSON response
 ↓
response.json()
 ↓
data
 ↓
setProducts(data)
 ↓
React state updates
 ↓
Component re-renders
 ↓
Products displayed
```

Tell students:

“If you understand this flow, you understand the basic pattern of consuming a GET API in React.”

## 15. Rendering Products

Now let's remove the `<pre>` and actually display the products.

We already learned array rendering in Lecture 12.

We can use:



`products.map()`

For example:

```
{
 products.map(product => (

 <div key={product.id}>

 <h2>
 {product.name}
 </h2>

 <p>
 Price: ₹{product.price}
 </p>

 </div>

))
}
```

This is where our previous knowledge of `.map()` becomes useful.

---

## 16. Complete Product List

Let's create the complete component.

```
import {
 useEffect,
 useState
} from "react";

function App() {

 const [products, setProducts] =
 useState([]);

 useEffect(() => {

 const getProducts = async () => {

 const response =
 await fetch(
 "http://localhost:3001/products"
)
 }
 })
}
```

```
);

 const data =
 await response.json();

 setProducts(data);

 };

 getProducts();

}, []);

return (
 <div>

 <h1>
 Product List
 </h1>

 {
 products.map(product => (

 <div
 key={product.id}
 >

 <h2>
 {product.name}
 </h2>

 <p>
 Price:
 ₹{product.price}
 </p>

 <p>
 Quantity:
 {product.quantity}
 </p>

 <p>
 Category:
 {product.category}
 </p>


```

```
 </div>
))
 }
 </div>
);
}
```

**export default** App;

Now the React application should display our products.

---

## 17. Why Do We Need `key`?

Notice this:

```
<div key={product.id}>
```

We learned about keys when rendering arrays.

When React renders a list, each item should have a unique key.

Our API already gives us:

```
product.id
```

So that's perfect.

We use:

```
key={product.id}
```

Don't use the array index when you already have a unique ID.

For example, prefer:

```
key={product.id}
```

over:

```
key={index}
```

when rendering data from an API.

---

## 18. Adding Loading State

Our application currently has a small problem.

When the page first loads, the API request takes some time.

During that time:

```
products = []
```

So the user might see an empty screen.

A better user experience is to display:

Loading products...

Let's create another state:

```
const [loading, setLoading] =
 useState(true);
```

Initially:

```
loading = true
```

After the API call finishes:

```
setLoading(false);
```

---

## 19. Complete Example With Loading

```
import {
 useEffect,
 useState
} from "react";

function App() {

 const [products, setProducts] =
 useState([]);

 const [loading, setLoading] =
 useState(true);

 useEffect(() => {

 const getProducts = async () => {
```

```
 const response =
 await fetch(
 "http://localhost:3001/products"
);

 const data =
 await response.json();

 setProducts(data);

 setLoading(false);

 };

 getProducts();

}, []);

return (
 <div>

 <h1>
 Product List
 </h1>

 {
 loading
 ? <p>
 Loading products...
 </p>
 : products.map(
 product => (

 <div
 key={
 product.id
 }
 >

 <h2>
 {
 product.name
 }
 </h2>

 <p>
```

```
 Price:
 ₹{
 product.price
 }
 </p>
</div>
)
}
</div>
);
}
```

**export default** App;

Now the user sees:

Loading products...

while the request is running.

Once the API responds, the products appear.

---

## 20. Adding Error Handling

What happens if the API isn't running?

For example:

JSON Server is stopped

Our application will fail to retrieve the products.

We should handle that situation.

Let's create:

```
const [error, setError] =
 useState("");
```

Then use try/catch.

```
try {
```

```
const response =
 await fetch(
 "http://localhost:3001/products"
);

const data =
 await response.json();

setProducts(data);
}
catch (error) {

 setError(
 "Unable to load products"
);

}
finally {

 setLoading(false);

}
```

This gives us a much better API consumption pattern.

---

## 21. Understanding `try/catch/finally`

Let's break this down.

`try`

The code that might fail:

```
try {

 // API request

}

catch
```

If an error occurs:

```
catch (error) {

 // Handle error

}

finally
```

This code runs after success or failure:

```
finally {

 setLoading(false);

}
```

This is useful because whether the API succeeds or fails, we don't want to keep showing:

Loading...

forever.

---

## 22. Checking `response.ok`

There's another important concept.

`fetch()` does not automatically throw an error for every HTTP error status.

So we should check:

```
if (!response.ok) {

 throw new Error(
 "Failed to fetch products"
);

}
```

Our code becomes:

```
const response =
 await fetch(
 "http://localhost:3001/products"
);

if (!response.ok) {
```



```
 throw new Error(
 "Failed to fetch products"
);
 }

 const data =
 await response.json();
```

This is a good habit when working with APIs.

---

## 23. Production-Style Beginner Example

Now let's create a clean version.

```
import {
 useEffect,
 useState
} from "react";

function App() {

 const [products, setProducts] =
 useState([]);

 const [loading, setLoading] =
 useState(true);

 const [error, setError] =
 useState("");

 useEffect(() => {

 const getProducts = async () => {

 try {

 const response =
 await fetch(
 "http://localhost:3001/products"
);

 if (!response.ok) {
```

```
 throw new Error(
 "Failed to fetch products"
);
 }

 const data =
 await response.json();

 setProducts(data);

 }
 catch (error) {

 setError(
 "Unable to load products"
);

 }
 finally {

 setLoading(false);

 }
};

getProducts();

}, []);

if (loading) {

 return (
 <h2>
 Loading products...
 </h2>
);
}

if (error) {

 return (
 <h2>
 {error}
 </h2>
);
}
```

```
);
 }

 return (
 <div>

 <h1>
 Product List
 </h1>

 {
 products.map(product => (

 <div
 key={product.id}
 >

 <h2>
 {product.name}
 </h2>

 <p>
 Price:
 ₹{product.price}
 </p>

 <p>
 Quantity:
 {product.quantity}
 </p>

 <p>
 Category:
 {product.category}
 </p>

 </div>

))
 }
 </div>
);
}

export default App;
```

This is a very useful pattern for beginners.

---

## 24. Let's Understand the Three States

At this point, our component has three important states.

### State 1 — Loading

`loading = true`

Display:

Loading products...

---

### State 2 — Error

`error = "Unable to load products"`

Display:

Unable to load products

---

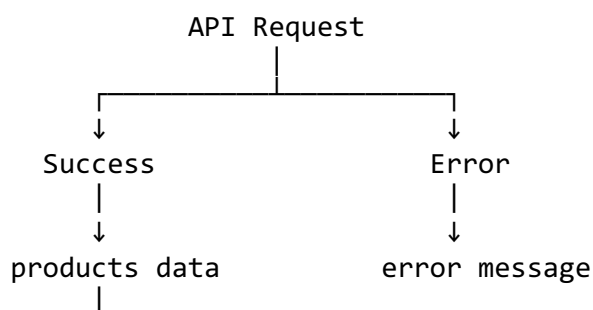
### State 3 — Success

`products = [...]`

Display:

Laptop  
Mouse  
Keyboard  
Monitor

So we can visualize:



↓  
Render UI

And while waiting:

API Request

↓  
Loading

This pattern is extremely common in React applications.

---

## 25. Refactoring the API URL

We currently have:

```
fetch(
 "http://localhost:3001/products"
);
```

As our application grows, we don't want to repeat the base URL everywhere.

Later, we could use environment variables or a centralized API configuration.

For now, let's create a constant:

```
const API_URL =
 "http://localhost:3001/products";
```

Then:

```
fetch(API_URL);
```

This makes our code easier to maintain.

---

## 26. Creating a Separate API Function

We can take this one step further.

Instead of putting everything inside `useEffect`, we can create a separate function.

For example:

```
const getProducts = async () => {

 const response =
```

```
 await fetch(
 "http://localhost:3001/products"
);

 if (!response.ok) {

 throw new Error(
 "Failed to fetch products"
);

 }

 return response.json();
 };
```

Then:

```
useEffect(() => {

 getProducts()
 .then(data => {
 setProducts(data);
 })
 .catch(() => {
 setError(
 "Unable to load products"
);
 })
 .finally(() => {
 setLoading(false);
 });

}, []);
```

For our beginner course, either approach is fine.

The important thing is understanding the flow.

---

## 27. Why `useEffect` Is Perfect Here

Let's connect today's lecture to Lecture 18.

We learned:

`useEffect` is used for side effects.

An API request is a side effect.

Therefore:

```
useEffect(() => {
 getProducts();
}, []);
```

is a natural place for our initial API request.

The empty dependency array:

```
[]
```

means:

Run this effect after the initial render and don't re-run it because of state changes inside this component.

For our current requirement—load products when the page opens—that is what we want.

---

## 28. Important Warning About Infinite Requests

Imagine we wrote:

```
useEffect(() => {
 getProducts();
});
```

without the dependency array.

The component renders.

The effect runs.

The API updates state.

The state update causes another render.

The effect runs again.

The API updates state again.

And this can create a repeated request loop.

Conceptually:

```
Render
↓
API
↓
setProducts
↓
Render
↓
API
↓
setProducts
↓
Render
↓
...
```

That's why understanding dependencies in `useEffect` is important.

For this example:

```
useEffect(() => {
 getProducts();
}, []);
```

---

## 29. Browser DevTools

Let's inspect what's actually happening.

Open browser DevTools.

Go to:

Network

Reload the React application.

You should see a request similar to:

products

Click it.



You can inspect:

Request URL  
Request Method  
Status Code  
Response

We should see something like:

Request URL:  
`http://localhost:3001/products`

Request Method:  
GET

Status:  
200

And the response should contain our products.

This is an excellent live demonstration.

Tell students:

“Whenever an API isn’t behaving as expected, the Network tab is one of the first places you should look.”

---

## 30. Understanding Status Code 200

When the request succeeds, we commonly get:

200 OK

This means the request was successfully processed.

Some other status codes students will encounter later include:

201 → Created  
400 → Bad Request  
401 → Unauthorized  
403 → Forbidden  
404 → Not Found  
500 → Server Error

We don’t need to memorize all of them today.

For today’s lecture, remember:

200 = successful request

---

## 31. Common Mistakes

### Mistake 1 — Forgetting `response.json()`

This:

```
const response =
 await fetch(url);
```

doesn't directly give you your product array.

You need:

```
const data =
 await response.json();
```

---

### Mistake 2 — Forgetting `await`

If we write:

```
const response =
 fetch(url);
```

then response is a Promise.

With `await`:

```
const response =
 await fetch(url);
```

we wait for the Promise to resolve.

---

### Mistake 3 — Calling API directly during render

Avoid:

```
function App() {

 getProducts();
```

```
 return (...);
}
```

For an initial data-fetching side effect, use:

```
useEffect(() => {
 getProducts();
}, []);
```

---

## Mistake 4 — Forgetting to update state

We successfully get:

```
const data =
 await response.json();
```

but if we only do:

```
console.log(data);
```

the UI won't know that it needs to display the data.

We need:

```
setProducts(data);
```

---

## Mistake 5 — API server isn't running

If JSON Server isn't running, the request can fail.

Make sure:

React Server  
+  
JSON Server

are both running.

---

## 32. Live Coding Flow for the Video

For your actual video recording, I recommend this sequence.

---

## Step 1 — Start both servers

Start React:

```
npm run dev
```

Start the mock API:

```
npm start
```

Show students that both applications are running.

---

## Step 2 — Verify API

Open:

```
http://localhost:3001/products
```

Show the JSON response.

Say:

“This is the data our React application is going to consume.”

---

## Step 3 — Create state

Write:

```
const [products, setProducts] =
 useState([]);
```

Explain why the initial value is an empty array.

---

## Step 4 — Add useEffect

Write:

```
useEffect(() => {

}, []);
```

Explain why API calls are placed inside the effect.

---

## Step 5 — Create API request

Write:

```
const response =
 await fetch(
 "http://localhost:3001/products"
);
```

---

## Step 6 — Convert JSON

Write:

```
const data =
 await response.json();
```

---

## Step 7 — Update state

Write:

```
setProducts(data);
```

---

## Step 8 — Display products

Use:

```
products.map(...)
```

Connect this back to Lecture 12.

---

## Step 9 — Add loading

Add:

```
const [loading, setLoading] =
 useState(true);
```

---

## Step 10 — Add error handling

Add:

```
try
catch
finally
```

---

## Step 11 — Show Network tab

Reload the page and demonstrate the GET request.

---

## 33. Recommended Final Code

For the end of the lecture, leave students with this version:

```
import {
 useEffect,
 useState
} from "react";

function App() {

 const [products, setProducts] =
 useState([]);

 const [loading, setLoading] =
 useState(true);

 const [error, setError] =
 useState("");

 useEffect(() => {

 const getProducts = async () => {

 try {

 const response =
 await fetch(
 "http://localhost:3001/products"
);

 if (!response.ok) {

 throw new Error(

```

```
 "Failed to fetch products"
);
 }

 const data =
 await response.json();

 setProducts(data);
 }
 catch (error) {
 setError(
 "Unable to load products"
);
 }
 finally {
 setLoading(false);
 }
};

getProducts();
}, []);

if (loading) {
 return (
 <h2>
 Loading products...
 </h2>
);
}

if (error) {
 return (
 <h2>
 {error}
 </h2>
);
}
```

```
);

 }

 return (
 <div>

 <h1>
 Product List
 </h1>

 {
 products.map(product => (

 <div
 key={product.id}
 >

 <h2>
 {product.name}
 </h2>

 <p>
 Price:
 ₹{product.price}
 </p>

 <p>
 Quantity:
 {product.quantity}
 </p>

 <p>
 Category:
 {product.category}
 </p>

 </div>

))
 }
 </div>
);
}

export default App;
```



## 34. Practice Exercise

Give students the following exercise.

### Exercise — Display Products From API

Use the mock API created in Lecture 20.

Endpoint:

GET `http://localhost:3001/products`

Create a React component that:

26. Calls the API when the component loads.
27. Stores the products in state.
28. Displays a loading message.
29. Displays an error message if the API fails.
30. Displays all products after the API succeeds.
31. Shows:
  - Product name
  - Price
  - Quantity
  - Category
32. Uses the product ID as the React key.

Expected UI:

Product List

Laptop

Price: ₹50000

Quantity: 10

Category: Electronics

Mouse

Price: ₹1000

Quantity: 25

Category: Accessories

Keyboard  
Price: ₹2000  
Quantity: 15  
Category: Accessories

---

## 35. Interview Questions

### Question 1

How do you make an API call in JavaScript?

Using functions such as:

`fetch()`

---

### Question 2

Why do we use `useEffect` for API calls?

Because fetching data is a side effect, and `useEffect` is designed to handle side effects in React components.

---

### Question 3

What does `response.json()` do?

It reads the response body and parses the JSON data into a JavaScript value.

---

### Question 4

Why do we use `setProducts(data)`?

To store the API response in React state, which causes the component to re-render with the new data.

---

### Question 5

What does an empty dependency array mean?

```
useEffect(() => {

}, []);
```

It tells React that the effect doesn't depend on changing values from the component and, for this pattern, should run after the initial render rather than after every render.

---

### Question 6

Why do we use `product.id` as the key?

Because each product has a unique identifier that React can use to identify list items.

---

### Question 7

What is the difference between `fetch()` and `response.json()`?

`fetch()` makes the HTTP request and gives us a Response object.

`response.json()` reads and parses the response body as JSON.

---

## 36. Lecture Summary

Today we learned how to consume a GET API from React.

The complete flow is:

```
React Component
 ↓
useEffect()
 ↓
fetch()
 ↓
GET /products
 ↓
Mock API
 ↓
Response
 ↓
response.json()
 ↓
setProducts()
 ↓
```

React State  
↓  
products.map()  
↓  
UI

We also learned how to handle:

Loading  
Error  
Success

This gives us a basic but realistic API integration pattern.

---

## 37. Final Takeaway

The three lines you should remember from today's lecture are:

```
const response =
 await fetch(url);

const data =
 await response.json();

setProducts(data);
```

Combined with:

```
useEffect(() => {
 getProducts();

}, []);
```

These lines form the foundation of API consumption in React.

# Lecture 22 — Consuming GET BY ID API in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned how to consume a **GET ALL API** in React.

We used:

`GET /products`

to retrieve all products.

Then we:

```
API
↓
fetch()
↓
response.json()
↓
setProducts()
↓
products.map()
↓
UI
```

Today we're going to take the next step.

Instead of displaying all products, suppose the user clicks on one product.

For example:

Product List

Laptop  
Mouse  
Keyboard  
Monitor

The user clicks on:

Laptop

We want to show:

Product Details

ID: 1

Name: Laptop

Price: ₹50000

Quantity: 10

Category: Electronics

We don't need to retrieve all products again.

We only need product number 1.

So today we'll consume:

GET /products/1

This is called a **GET BY ID API**.

---

## 2. GET ALL vs GET BY ID

Let's first understand the difference.

In Lecture 21, we used:

GET /products

The API returns:

```
[
 {
 "id": 1,
 "name": "Laptop"
 },
 {
 "id": 2,
 "name": "Mouse"
 },
 {
 "id": 3,
 "name": "Keyboard"
 }
]
```

We're asking:

Give me all products.

Today we'll use:

GET /products/1

The API returns:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
}
```

We're asking:

Give me the product whose ID is 1.

So remember:

GET /products

↓

Multiple products

GET /products/1

↓

One product

---

### 3. Why Do We Need the ID?

Imagine we have 100 products.

The user wants to see product number 57.

We don't want to retrieve all 100 products just to find product 57.

Instead, we can request:

GET /products/57

The backend can return only that product.

This is much more appropriate when displaying a detail page.

For example:

/products/1  
/products/2  
/products/3

Each URL represents a different product.

---

## 4. Connecting Routing With API

This is where the concepts from our earlier lectures become very useful.

In Lecture 15, we learned parameterized routes.

For example:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

Here:

:id

is a route parameter.

If the browser URL is:

/products/1

then:

id = 1

If the browser URL is:

/products/25

then:

id = 25

This is exactly what we need.

We can take that ID and use it to call our API.

---



## 5. The Complete Flow

Today's flow is:

User visits:

`/products/1`



React Router



`useParams()`



`id = 1`



`fetch("/products/1")`



API



Product data



`setProduct()`



Display product

This is the most important concept in today's lecture.

## 6. Our API

Let's make sure our mock API is running.

Our API is:

`http://localhost:3001`

All products:

`http://localhost:3001/products`

Product with ID 1:

`http://localhost:3001/products/1`

Open the second URL in the browser.

We should get something like:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
}
```

So our API is already ready.

Now we need to consume it from React.

---

## 7. Creating Product Details Component

Let's create a component:

`ProductDetails.jsx`

Our folder might look like:

```
src
├── App.jsx
├── ProductList.jsx
└── ProductDetails.jsx
```

We'll use this component to display one product.

Start with:

```
function ProductDetails() {

 return (
 <div>

 <h1>
 Product Details
 </h1>

 </div>
);
}
```

```
export default ProductDetails;
```

Now we need to create a route for this component.

---

## 8. Creating the Parameterized Route

In our router:

```
import {
 BrowserRouter,
 Routes,
 Route
} from "react-router-dom";

import ProductDetails
 from "./ProductDetails";

function App() {

 return (
 <BrowserRouter>

 <Routes>

 <Route
 path="/products/:id"
 element={
 <ProductDetails />
 }
 />
 </Routes>
 </BrowserRouter>
);
}
```

```
 }
 />

</Routes>

</BrowserRouter>
);
}

export default App;
```

Notice:

/products/:id

The :id means:

This part of the URL is dynamic.

So these URLs all match the same route:

```
/products/1
/products/2
/products/10
/products/50
```

---

## 9. Getting the ID Using `useParams`

Now we need to read the ID.

React Router provides a hook called:

`useParams`

Import it:

```
import {
 useParams
} from "react-router-dom";
```

Inside our component:

```
const params = useParams();
```

We can then access:

```
params.id
```

For example:

```
function ProductDetails() {
 const params = useParams();
 console.log(params);
 return (
 <h1>
 Product Details
 </h1>
);
}
```

If the URL is:

/products/1

the console will show something similar to:

```
{
 id: "1"
}
```

Notice something very important.

The ID is:

"1"

not:

1

It is a string because it came from the URL.

For today's API call, that's perfectly fine.

---

## 10. Destructuring `useParams`

Instead of:

```
const params = useParams();
console.log(params.id);
```

we can write:

```
const { id } = useParams();
```

Now we have:

id

directly available.

Our component becomes:

```
function ProductDetails() {

 const { id } = useParams();

 console.log(id);

 return (
 <h1>
 Product Details
 </h1>
);
}
```

If we visit:

/products/1

we get:

id = "1"

If we visit:

/products/3

we get:

id = "3"

This is exactly what we need for our API.

---

## 11. Building the API URL

Our API endpoint is:

<http://localhost:3001/products/1>

But we don't want to hard-code 1.

We want to use the ID from the URL.

So we can write:

```
const url =
 `http://localhost:3001/products/${id}`;
```

If:

id = 1

then:

```
url =
http://localhost:3001/products/1
```

If:

id = 5

then:

```
url =
http://localhost:3001/products/5
```

This makes our API request dynamic.

---

## 12. Creating Product State

Just like in Lecture 21, we need state.

But this time we're storing **one product**, not an array of products.

So instead of:

```
const [products, setProducts] =
 useState([]);
```

we'll use:

```
const [product, setProduct] =
 useState(null);
```

Why null?

Because initially we don't have any product data.

After the API call:

```
setProduct(data);
```

our state becomes:

```
{
 id: 1,
 name: "Laptop",
 price: 50000,
 quantity: 10,
 category: "Electronics"
}
```

So:

GET ALL

↓

products = []

GET BY ID

↓

product = null

This distinction is important.

---

## 13. Making the API Call

Now let's combine everything.

```
import {
 useEffect,
 useState
} from "react";

import {
 useParams
} from "react-router-dom";

function ProductDetails() {

 const { id } = useParams();

 const [product, setProduct] =
 useState(null);
```



```
useEffect(() => {

 const getProduct = async () => {

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`
);

 const data =
 await response.json();

 setProduct(data);

 };

 getProduct();

}, [id]);

return (
 <div>

 <h1>
 Product Details
 </h1>

 </div>
);
}
```

**export default** ProductDetails;

Let's understand the important part:

[id]

in the dependency array.

---

## 14. Why [id] Instead of []?

In Lecture 21, we used:

```
useEffect(() => {
 getProducts();
}, []);
```

because we wanted to load all products when the component mounted.

Here, our API request depends on:

id

Our URL is:

/products/\${id}

Therefore, the effect depends on id.

So we use:

```
useEffect(() => {
 getProduct();
}, [id]);
```

This means:

Run the effect when the component loads and whenever the id value changes.

For example:

/products/1

makes:

GET /products/1

If the route changes to:

/products/2

React can make:

GET /products/2

## 15. Displaying the Product

Now we need to display our product.

We can write:

```
return (
 <div>

 <h1>
 Product Details
 </h1>

 {
 product && (

 <div>

 <h2>
 {product.name}
 </h2>

 <p>
 Price:
 ₹{product.price}
 </p>

 <p>
 Quantity:
 {product.quantity}
 </p>

 <p>
 Category:
 {product.category}
 </p>

 </div>

)
 }

 </div>

```

Notice:

`product && (...)`

Why do we need this?

Because initially:

`product = null`

So we shouldn't try to access:

`product.name`

before the API has returned the data.

---

## 16. Adding Loading State

Just like Lecture 21, we should show a loading message.

Add:

```
const [loading, setLoading] =
 useState(true);
```

Then:

```
setLoading(false);
```

after the request finishes.

Our API code becomes:

```
useEffect(() => {

 const getProduct = async () => {

 try {

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`
);

 const data =
 await response.json();

 setProduct(data);
 }
 }
}
```

```
 }
 finally {
 setLoading(false);
 }
};

getProduct();
}, [id]));
```

Then:

```
if (loading) {
 return (
 <h2>
 Loading product...
 </h2>
);
}
```

Now the user gets feedback while the API request is running.

---

## 17. Adding Error Handling

Let's also handle errors.

Create:

```
const [error, setError] =
 useState("");
```

Then:

```
try {

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`
);

 if (!response.ok) {
```

```
 throw new Error(
 "Product not found"
);
 }

 const data =
 await response.json();

 setProduct(data);
}
catch (error) {
 setError(
 "Unable to load product"
);
}
finally {
 setLoading(false);
}
```

Now we can display the error:

```
if (error) {
 return (
 <h2>
 {error}
 </h2>
);
}
```

---

## 18. Complete `ProductDetails.jsx`

Let's put everything together.

```
import {
 useEffect,
 useState
} from "react";
```

```
import {
 useParams
} from "react-router-dom";

function ProductDetails() {

 const { id } = useParams();

 const [product, setProduct] =
 useState(null);

 const [loading, setLoading] =
 useState(true);

 const [error, setError] =
 useState("");

 useEffect(() => {

 const getProduct = async () => {

 try {

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`
);

 if (!response.ok) {

 throw new Error(
 "Product not found"
);

 }

 const data =
 await response.json();

 setProduct(data);

 }
 catch (error) {

 setError(
```

```
 "Unable to load product"
);

 }
 finally {
 setLoading(false);
 }
 };

 getProduct();
}, [id]));

if (loading) {
 return (
 <h2>
 Loading product...
 </h2>
);
}

if (error) {
 return (
 <h2>
 {error}
 </h2>
);
}

return (
 <div>

 <h1>
 Product Details
 </h1>

 <h2>
 {product.name}
 </h2>

 <p>
```



```
 ID: {product.id}
 </p>

 <p>
 Price: ₹{product.price}
 </p>

 <p>
 Quantity: {product.quantity}
 </p>

 <p>
 Category: {product.category}
 </p>
 </div>
);
}
```

**export default** ProductDetails;

This is our complete GET BY ID implementation.

---

## 19. Testing the Application

Now let's test.

Start the React application and JSON Server.

Then open:

/products/1

We should see:

Product Details

Laptop

ID: 1

Price: ₹50000

Quantity: 10

Category: Electronics

Now change the URL:

/products/2

We should see:

Product Details

Mouse

ID: 2

Price: ₹1000

Quantity: 25

Category: Accessories

Then:

/products/3

should show:

Keyboard

Our component is now completely dynamic.

---

## 20. Demonstrating the Network Tab

This is a good point to open browser DevTools.

Go to:

Network

Then open:

/products/1

You should see:

GET products/1

Click the request.

Show:

Request Method: GET

Status Code: 200

Request URL:

http://localhost:3001/products/1

Then change the URL to:

/products/2

You'll see:

GET products/2

This makes the concept very clear to students.

---

## 21. Connecting the Product List to Product Details

Now let's make the application more realistic.

Currently, we have:

Product List

and:

Product Details

We want the user to click a product from the list.

For example:

Product List

Laptop  
Mouse  
Keyboard  
Monitor

When the user clicks Laptop:

/products/1

When the user clicks Mouse:

/products/2

So we need to use React Router's:

Link

component.

---

## 22. Using `Link`

In our Product List component:

```
import {
 Link
} from "react-router-dom";
```

Then:

```
{
 products.map(product => (

 <div
 key={product.id}
 >

 <h2>
 {product.name}
 </h2>

 <Link
 to={` /products/${product.id}`}
 >
 View Details
 </Link>

 </div>

))
}
```

Now every product has a dynamic link.

For product 1:

View Details  
↓  
/products/1

For product 2:

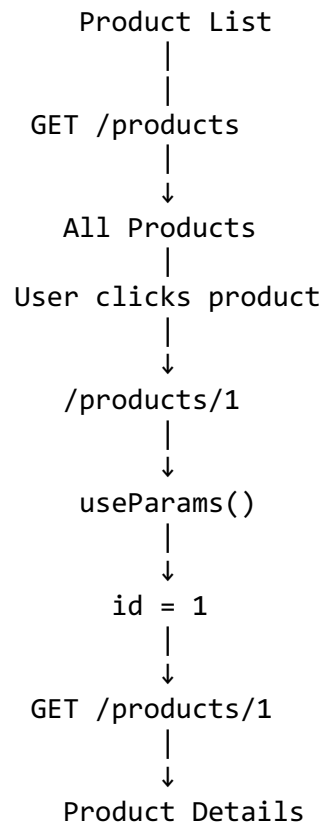
View Details  
↓  
/products/2

And so on.

## 23. Full Application Flow

Now we have a much more realistic application.

The flow is:



This is an important milestone in our React series.

We are combining:

React Router  
+  
useParams  
+  
useEffect  
+  
useState  
+  
fetch  
+  
REST API

## 24. Important Difference Between `useParams` and API Data

Students sometimes confuse these two.

When we write:

```
const { id } = useParams();
```

the `id` comes from the **URL**.

For example:

`/products/5`

gives:

```
id = "5"
```

Then we use that ID to make the API request:

GET `/products/5`

The API gives us the actual product:

```
{
 "id": 5,
 "name": "Monitor",
 "price": 15000
}
```

So:

URL parameter



`useParams()`



API request



API response



Product state

Keep these concepts separate.

---

## 25. What Happens If the Product Doesn't Exist?

Suppose we visit:

/products/999

but product 999 doesn't exist.

Our API should return an appropriate response.

We check:

```
if (!response.ok) {

 throw new Error(
 "Product not found"
);

}
```

Then our catch block handles it.

We can show:

Unable to load product

Or, even better:

Product not found

This is an important real-world scenario.

Users can manually enter URLs.

We cannot assume that every ID is valid.

---

## 26. Better Error Message

We can improve our error state.

Instead of:

```
setError(
 "Unable to load product"
);
```

we can use:

```
setError(
 "Product not found"
);
```

Then the UI can show:

Product not found

This is more useful to the user.

---

## 27. Handling API URL Carefully

Notice our URL:

```
`http://localhost:3001/products/${id}`
```

The backticks are important.

We're using a JavaScript template literal.

For example:

```
const id = 5;

const url =
 `http://localhost:3001/products/${id}`;
```

produces:

```
http://localhost:3001/products/5
```

This is a common technique when building dynamic API URLs.

---

## 28. Common Mistakes

### Mistake 1 — Forgetting `useParams`

If your route is:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

you need:

```
const { id } = useParams();
```

to get the value.



---

## Mistake 2 — Hard-Coding the ID

Don't write:

```
fetch(
 "http://localhost:3001/products/1"
);
```

inside a component that should display any product.

Instead:

```
fetch(
 `http://localhost:3001/products/${id}`
);
```

---

## Mistake 3 — Wrong Dependency Array

Don't blindly copy:

```
[]
```

from the GET ALL example.

Here the API depends on:

```
id
```

So:

```
[id]
```

is appropriate.

---

## Mistake 4 — Accessing `product.name` Too Early

Initially:

```
product = null
```

So this can cause an error:

```
product.name
```

before the data is loaded.

Our loading/error flow ensures we only render the product after the API response is available.

---

## Mistake 5 — Forgetting the Route

If you create:

`ProductDetails.jsx`

but don't create:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

the component won't be displayed for that URL.

---

## 29. Live Coding Flow for the Video

For the actual recording, I recommend the following sequence.

### Step 1 — Review Lecture 21

Start with:

`GET /products`

and briefly remind students how we fetched all products.

Say:

“Today we're going to modify this idea so that instead of retrieving all products, we'll retrieve one product based on the ID in the URL.”

---

### Step 2 — Test the API

Open:

`http://localhost:3001/products/1`

Show the response.

---

### Step 3 — Create ProductDetails.jsx

Create the component.

---

### Step 4 — Create the route

Add:

```
<Route
 path="/products/:id"
 element={<ProductDetails />}
/>
```

---

### Step 5 — Get ID

Add:

```
const { id } = useParams();
```

Then demonstrate:

```
console.log(id);
```

---

### Step 6 — Create product state

```
const [product, setProduct] =
 useState(null);
```

Explain why we use null.

---

### Step 7 — API request

```
fetch(
 `http://localhost:3001/products/${id}`
);
```

---

### Step 8 — Store the response

```
setProduct(data);
```

---

## Step 9 — Display product

Show:

Name  
Price  
Quantity  
Category

---

## Step 10 — Add loading and error states

Add:

Loading...

and:

Product not found

---

## Step 11 — Connect list to details

Add:

```
<Link
 to={` /products/${product.id}`}
>
 View Details
</Link>
```

---

## Step 12 — Demonstrate

Click:

Laptop

Show:

/products/1

Then click:

Mouse

Show:

/products/2

Finally manually enter:

/products/999

and demonstrate the error state.

---

## 30. Suggested Video Timeline

For a 10–15 minute video:

0:00 – 1:30

Introduction + GET ALL vs GET BY ID

1:30 – 3:00

Parameterized route review

3:00 – 4:30

useParams()

4:30 – 7:00

Build ProductDetails API call

7:00 – 9:00

Display product + loading/error

9:00 – 11:00

Connect Product List with Link

11:00 – 12:30

Network tab demonstration

12:30 – 14:00

Common mistakes

14:00 – 15:00

Summary + next lecture

---

## 31. Interview Questions

### Question 1

What is a GET BY ID API?

An API endpoint that retrieves one resource using its unique identifier.

Example:

GET /products/10

---

### Question 2

How do you get route parameters in React Router?

Using:

`useParams()`

For example:

```
const { id } = useParams();
```

---

### Question 3

Why is the dependency array `[id]`?

Because the API request depends on the route ID.

If the ID changes, the API request should run again.

---

### Question 4

What is the difference between:

`/products`

and:

`/products/:id`

`/products` represents the collection.

`/products/:id` represents a specific product.

---

### Question 5

Why initialize product with null?

Because initially we haven't received the product from the API.

After the API call succeeds, we replace null with the product object.

---

### Question 6

What happens when /products/2 is opened?

React Router extracts:

`id = 2`

Then React makes:

`GET /products/2`

The API returns product 2, and React displays it.

---

## 32. Practice Exercise

Ask students to build a product details page.

### Requirements

Create:

`/products/:id`

When the user opens the URL:

`/products/1`

the application should:

33. Read the ID using `useParams()`.

34. Call:

`GET http://localhost:3001/products/1`

3. Store the response in state.

#### 4. Display:

- ID
- Name
- Price
- Quantity
- Category

#### 5. Display a loading message.

#### 6. Display an error message if the product doesn't exist.

#### 7. Add a "Back to Products" link.

Expected UI:

Product Details

ID: 1

Name: Laptop

Price: ₹50000

Quantity: 10

Category: Electronics

[Back to Products]

---

## 33. Final Summary

Let's summarize today's lecture.

We learned how to consume a **GET BY ID API** in React.

The main steps are:

1. Create parameterized route  
↓
2. Get ID using useParams()  
↓
3. Build dynamic API URL  
↓
4. Call fetch()  
↓
5. Convert response using response.json()  
↓
6. Store result using setProduct()





## 7. Display product

The most important code is:

```
const { id } = useParams();

useEffect(() => {

 const getProduct = async () => {

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`
);

 const data =
 await response.json();

 setProduct(data);

 };

 getProduct();

}, [id]);
```

And the overall architecture is:

```
URL
↓
/products/1
↓
useParams()
↓
id = 1
↓
GET /products/1
↓
API
↓
Product JSON
↓
setProduct()
↓
Product Details UI
```

# Lecture 23 — Consuming DELETE API in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned how to consume a **GET BY ID API**.

We used a URL such as:

/products/1

Then we:

```
URL
↓
useParams()
↓
Get ID
↓
GET /products/1
↓
API response
↓
Display product
```

So far, our application has only been **reading data**.

Today we're going to learn how to **delete data**.

For example, suppose our product list contains:

Laptop  
Mouse  
Keyboard  
Monitor

And the user clicks:

Delete

for the Mouse.

We want to send a request to the server:

DELETE /products/2

After the server deletes the product, our React application should update the screen.

So today's complete flow is:

```
User clicks Delete
 ↓
Get Product ID
 ↓
DELETE /products/:id
 ↓
API
 ↓
Product deleted
 ↓
Update React state
 ↓
Product disappears
```

That is what we're going to build today.

---

## 2. What Is the DELETE HTTP Method?

Let's first understand the HTTP method.

We already know:

GET

is used to retrieve data.

For example:

GET /products

means:

Give me all products.

And:

GET /products/1

means:

Give me product 1.

Today we'll use:

## DELETE

DELETE is used to remove a resource.

For example:

```
DELETE /products/2
```

means:

Delete product with ID 2.

So our basic HTTP methods are becoming:

GET

→ Read data

POST

→ Create data

PUT/PATCH

→ Update data

DELETE

→ Delete data

These operations are commonly referred to as **CRUD**.

We'll discuss CRUD more as we continue the remaining API lectures.

---

## 3. GET vs DELETE

Let's compare the requests.

For GET:

```
GET /products/2
```

The server responds with:

```
{
 "id": 2,
 "name": "Mouse",
 "price": 1000
}
```

We're asking the server:

Give me this resource.

For DELETE:

DELETE /products/2

We're asking:

Remove this resource.

The server doesn't need to send the complete product back.

The important thing is that the operation was successful.

---

## 4. Our Mock API

Let's verify that JSON Server is running.

Our API:

`http://localhost:3001`

All products:

`GET http://localhost:3001/products`

One product:

`GET http://localhost:3001/products/1`

And to delete product 1:

`DELETE http://localhost:3001/products/1`

We don't normally type a DELETE URL into the browser.

The browser address bar makes a GET request.

Instead, we'll send the DELETE request from JavaScript using:

`fetch()`

---

## 5. DELETE Using `fetch()`

The syntax is very similar to GET.

For GET:

```
fetch(
 "http://localhost:3001/products/1"
);
```

By default, `fetch()` makes a GET request.

For DELETE, we need to specify the HTTP method:

```
fetch(
 "http://localhost:3001/products/1",
 {
 method: "DELETE"
 }
);
```

The important part is:

```
method: "DELETE"
```

So:

```
fetch()
+
method: "DELETE"
```

sends a DELETE request.

---

## 6. Understanding the DELETE Request

Let's break this down.

```
fetch(
 "http://localhost:3001/products/1",
 {
 method: "DELETE"
 }
);
```

The first argument is:

URL

The second argument is an options object.

Inside the options object:

```
{
 method: "DELETE"
}
```

tells fetch():

Don't make the default GET request. Make a DELETE request instead.

---

## 7. Creating a Delete Function

Let's create a simple function.

```
const deleteProduct = async (id) => {

 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "DELETE"
 }
);

};
```

Notice that we're using:

`${id}`

So this function can delete any product.

If:

`id = 1`

the request becomes:

`DELETE /products/1`

If:

`id = 5`

the request becomes:

`DELETE /products/5`

This makes our function reusable.

---

## 8. Connecting Delete to a Button

Now let's imagine our product list.

```
{
 products.map(product => (
 <div key={product.id}>
 <h2>
 {product.name}
 </h2>
 <p>
 ₹{product.price}
 </p>
 <button>
 Delete
 </button>
 </div>
))
}
```

We need to call our delete function when the user clicks the button.

We can write:

```
<button
 onClick={() =>
 deleteProduct(product.id)
 }
>
 Delete
</button>
```

Now the flow is:

```
User clicks Delete
 ↓
onClick
 ↓
deleteProduct(product.id)
 ↓
DELETE /products/:id
```



## 9. Important: Don't Call the Function Immediately

This is a very common beginner mistake.

Don't write:

```
<button
 onClick={
 deleteProduct(product.id)
 }
>
 Delete
</button>
```

Why?

Because:

```
deleteProduct(product.id)
```

is executed while React is rendering.

We don't want that.

We want the function to run **when the user clicks**.

So use:

```
<button
 onClick={() =>
 deleteProduct(product.id)
 }
>
 Delete
</button>
```

The arrow function delays the execution until the click happens.

## 10. Basic Delete Implementation

Let's create a basic example.

```
import {
 useEffect,
```

```
 useState
 } from "react";

function ProductList() {

 const [products, setProducts] =
 useState([]);

 useEffect(() => {

 const getProducts = async () => {

 const response =
 await fetch(
 "http://localhost:3001/products"
);

 const data =
 await response.json();

 setProducts(data);

 };

 getProducts();

 }, []);

 const deleteProduct = async (id) => {

 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "DELETE"
 }
);

 };

 return (
 <div>

 <h1>
 Product List
 </h1>

 </div>
);
}
```

```
{
 products.map(product => (

 <div
 key={product.id}
 >

 <h2>
 {product.name}
 </h2>

 <p>
 ₹{product.price}
 </p>

 <button
 onClick={() =>
 deleteProduct(
 product.id
)
 }
 >
 Delete
 </button>

 </div>

))
}

</div>
);
}
```

```
export default ProductList;
```

We have a problem, though.

The product will be deleted from the API, but will it disappear from the React UI?

Not necessarily.

And this is an important concept.

## 11. Server State vs React State

Suppose our React state contains:

```
[
 {
 id: 1,
 name: "Laptop"
 },
 {
 id: 2,
 name: "Mouse"
 },
 {
 id: 3,
 name: "Keyboard"
 }
]
```

We click Delete for:

Mouse

The API deletes product 2.

The server now contains:

```
[
 {
 id: 1,
 name: "Laptop"
 },
 {
 id: 3,
 name: "Keyboard"
 }
]
```

But React may still have:

```
[
 {
 id: 1,
 name: "Laptop"
 },
 {
 id: 2,
 name: "Mouse"
 },
]
```

```
{
 id: 3,
 name: "Keyboard"
}
```

So we have two different states:

Server  
↓  
Laptop  
Keyboard

React  
↓  
Laptop  
Mouse  
Keyboard

We need to update the React state.

---

## 12. Updating React State After DELETE

After the DELETE request succeeds, we can remove the product from our local state.

We already know how to use:

`setProducts()`

We can use:

```
setProducts(
 products.filter(
 product =>
 product.id !== id
)
);
```

Let's understand this carefully.

Suppose:

```
products = [
 { id: 1, name: "Laptop" },
 { id: 2, name: "Mouse" },
 { id: 3, name: "Keyboard" }
];
```

And:

`id = 2`

Then:

```
products.filter(
 product =>
 product.id !== 2
)
```

returns:

```
[
 { id: 1, name: "Laptop" },
 { id: 3, name: "Keyboard" }
]
```

Then:

`setProducts(...)`

updates React state.

React re-renders.

The Mouse disappears from the UI.

---

## 13. Complete Delete Function

So our function becomes:

```
const deleteProduct = async (id) => {

 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "DELETE"
 }
);

 setProducts(
 products.filter(
 product =>
 product.id !== id
)
);
};
```

```
};
```

Now the complete flow is:

Click Delete



DELETE API



Server deletes product



Filter product from React state



setProducts()



React re-renders



Product disappears

---

## 14. Why Use `filter()`?

We learned array methods earlier.

`filter()` creates a new array containing only the elements that satisfy a condition.

For example:

```
const numbers = [1, 2, 3, 4, 5];
```

```
const result =
 numbers.filter(
 number => number !== 3
);
```

Result:

```
[1, 2, 4, 5]
```

For our products:

```
products.filter(
 product =>
 product.id !== id
)
```

means:

Keep every product except the one we just deleted.

This is a very useful React pattern.

---

## 15. Don't Directly Modify State

A common mistake would be trying to directly modify the array.

For example:

```
products.splice(...)
```

Avoid directly modifying React state.

Instead, create a new array:

```
products.filter(...)
```

and then:

```
setProducts(newProducts);
```

React works very well with immutable state updates.

For deletion, `filter()` is an excellent choice.

---

## 16. Adding Confirmation

Deleting data is a destructive operation.

We don't want the user to accidentally delete a product.

So before making the DELETE request, we can ask for confirmation.

The simplest browser approach is:

```
const confirmed =
 window.confirm(
 "Are you sure you want to delete this product?"
);
```

If the user clicks Cancel:



```
if (!confirmed) {
 return;
}
```

So:

```
const deleteProduct = async (id) => {

 const confirmed =
 window.confirm(
 "Are you sure you want to delete this product?"
);

 if (!confirmed) {
 return;
 }

 // DELETE request

};
```

This is a simple but useful user-experience improvement.

---

## 17. Adding Loading State for Delete

Suppose the API takes two seconds to respond.

The user clicks:

Delete

and nothing appears to happen.

They might click the button again.

We can prevent that by tracking which product is being deleted.

Create:

```
const [deletingId, setDeletingId] =
 useState(null);
```

Initially:

```
deletingId = null
```

When deleting product 2:

```
setDeletingId(2);
```

After the operation:

```
setDeletingId(null);
```

Now we know which product is currently being deleted.

---

## 18. Delete Button With Loading State

We can write:

```
<button
 disabled={
 deletingId === product.id
 }
 onClick={() =>
 deleteProduct(product.id)
 }
>
 {
 deletingId === product.id
 ? "Deleting..."
 : "Delete"
 }
</button>
```

If we're deleting product 2:

Laptop	Delete
Mouse	Deleting...
Keyboard	Delete

This is much better than allowing repeated clicks.

---

## 19. Error Handling

What if the DELETE request fails?

We should handle it.

Let's create:

```
const [error, setError] =
 useState("");
```

Then:

```
try {

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "DELETE"
 }
);

 if (!response.ok) {

 throw new Error(
 "Failed to delete product"
);

 }

 setProducts(
 products.filter(
 product =>
 product.id !== id
)
);

}
catch (error) {

 setError(
 "Unable to delete product"
);

}
finally {

 setDeletingId(null);

}
```

This gives us a proper success/failure flow.

## 20. Important: Update State Only After Success

This is a very important concept.

Don't immediately do:

```
setProducts(
 products.filter(
 product =>
 product.id !== id
)
);
```

before knowing whether the API request succeeded.

Otherwise, this could happen:

React UI  
↓  
Product disappears

API  
↓  
Delete failed

Now the UI says the product is gone, but the server still has it.

That's a synchronization problem.

Instead:

```
DELETE API
 ↓
Success?
 / \
Yes No
 ↓ ↓
Update Show error
state
```

So we update React state **after successful deletion**.

---

## 21. Complete Product List Example

Let's put everything together.

```
import {
 useEffect,
 useState
} from "react";

function ProductList() {

 const [products, setProducts] =
 useState([]);

 const [loading, setLoading] =
 useState(true);

 const [error, setError] =
 useState("");

 const [deletingId, setDeletingId] =
 useState(null);

 useEffect(() => {

 const getProducts = async () => {

 try {

 const response =
 await fetch(
 "http://localhost:3001/products"
);

 if (!response.ok) {

 throw new Error(
 "Failed to fetch products"
);

 }

 const data =
 await response.json();

 setProducts(data);

 }
 catch (error) {
```

```
 setError(
 "Unable to load products"
);
 }
 finally {
 setLoading(false);
 }
};

getProducts();
}, []);

const deleteProduct = async (id) => {
 const confirmed =
 window.confirm(
 "Are you sure you want to delete this product?"
);

 if (!confirmed) {
 return;
 }

 try {
 setDeletingId(id);

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "DELETE"
 }
);

 if (!response.ok) {
 throw new Error(
 "Failed to delete product"
);
 }
 }
};
```

```
 }

 setProducts(
 products.filter(
 product =>
 product.id !== id
)
);
 }
 catch (error) {

 setError(
 "Unable to delete product"
);

 }
 finally {

 setDeletingId(null);

 }
};

if (loading) {

 return (
 <h2>
 Loading products...
 </h2>
);
}

if (error && products.length === 0) {

 return (
 <h2>
 {error}
 </h2>
);
}

return (
 <div>

 <h1>
```

```
Product List
</h1>

{error && (
 <p>
 {error}
 </p>
)}

{
 products.map(product => (
 <div
 key={product.id}
 >

 <h2>
 {product.name}
 </h2>

 <p>
 Price:
 ₹{product.price}
 </p>

 <p>
 Quantity:
 {product.quantity}
 </p>

 <p>
 Category:
 {product.category}
 </p>

 <button
 disabled={
 deletingId ===
 product.id
 }
 onClick={() =>
 deleteProduct(
 product.id
)
 }
 >
 {
```



```
 deletingId ===
 product.id
 ? "Deleting..."
 : "Delete"
 }
 </button>
 </div>
))
}
</div>
);
}
```

**export default** ProductList;

This is our complete beginner-friendly DELETE implementation.

---

## 22. Let's Understand the Code Flow

Suppose the user clicks Delete for:

Mouse  
ID = 2

The following function runs:

```
deleteProduct(2);
```

First:

```
window.confirm(...)
```

asks:

Are you sure you want to delete this product?

If the user selects Cancel:

return

Nothing happens.

If the user selects OK:

```
setDeletingId(2);
```

The button changes to:

Deleting...

Then:

```
fetch(
 "http://localhost:3001/products/2",
 {
 method: "DELETE"
 }
);
```

sends:

DELETE /products/2

The API deletes product 2.

Then:

```
setProducts(
 products.filter(
 product =>
 product.id !== 2
)
);
```

removes product 2 from the React state.

React re-renders.

The Mouse disappears.

Finally:

```
setDeletingId(null);
```

resets the delete state.

---

## 23. Demonstrating the Network Tab

This is an important live demonstration.

Open:

DevTools  
→ Network

Then click Delete.

You should see something like:

Request URL:  
`http://localhost:3001/products/2`

Request Method:  
DELETE

This is different from our earlier GET request.

Earlier:

GET /products

Today:

DELETE /products/2

This is a great opportunity to reinforce HTTP methods.

---

## 24. Checking the API After Deletion

After deleting product 2, open:

`http://localhost:3001/products`

Product 2 should no longer be present.

For example, before:

```
[
 {
 "id": 1,
 "name": "Laptop"
 },
 {
 "id": 2,
 "name": "Mouse"
 },
 {
 "id": 3,
 "name": "Keyboard"
 }
]
```

```
}
]
```

After deleting product 2:

```
[
 {
 "id": 1,
 "name": "Laptop"
 },
 {
 "id": 3,
 "name": "Keyboard"
 }
]
```

This demonstrates that the data was actually deleted from the API, not merely hidden from the UI.

---

## 25. Refresh the Page

Now refresh the React application.

The deleted product should still be gone.

Why?

Because we didn't only change React state.

We also deleted the record from the API.

When the page reloads:

```
React
↓
GET /products
↓
API
↓
Deleted product isn't returned
```

This is an important distinction.

## 26. Two Things Are Happening

When we delete a product, there are actually two different operations.

### Operation 1 — Server

```
DELETE /products/2
```

The database/API data changes.

### Operation 2 — React

```
setProducts(...)
```

The UI state changes.

So:

Server state

+

Client state

need to stay synchronized.

This concept becomes very important as applications become larger.

---

## 27. Alternative Approach: Fetch Again

There is another way to update the UI after deleting.

Instead of:

```
setProducts(
 products.filter(
 product =>
 product.id !== id
)
);
```

we could call the GET ALL API again:

```
DELETE /products/2
 ↓
GET /products
 ↓
setProducts(data)
```

The flow would be:

```
await deleteProduct();

const response =
 await fetch(
 "http://localhost:3001/products"
);

const data =
 await response.json();

setProducts(data);
```

This works.

But it makes another network request.

For this simple application, using:

```
filter()
```

is more efficient because we already know which product was deleted.

---

## 28. Which Approach Should Beginners Use?

For this lecture, I recommend:

```
setProducts(
 products.filter(
 product =>
 product.id !== id
)
);
```

Why?

Because students can clearly see:

```
Delete product with ID 2
 ↓
Remove ID 2 from local array
 ↓
Render remaining products
```

Later, when applications become more complex, refetching or using a data-fetching library can also be appropriate.

## 29. A Note About Functional State Updates

There's one improvement we can make.

Instead of:

```
setProducts(
 products.filter(
 product =>
 product.id !== id
)
);
```

we can use a functional state update:

```
setProducts(
 currentProducts =>
 currentProducts.filter(
 product =>
 product.id !== id
)
);
```

This is useful because React state updates can be asynchronous, and using the current state value directly can sometimes lead to stale-state problems.

For our course, students should understand both forms.

The functional form is:

```
setProducts(
 currentProducts =>
 currentProducts.filter(
 product =>
 product.id !== id
)
);
```

I recommend using this version in the final code.

---

## 30. Improved Delete Function

Our final delete function can therefore be:

```
const deleteProduct = async (id) => {
```

```
const confirmed =
 window.confirm(
 "Are you sure you want to delete this product?"
);

if (!confirmed) {
 return;
}

try {

 setDeletingId(id);

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "DELETE"
 }
);

 if (!response.ok) {

 throw new Error(
 "Failed to delete product"
);

 }

 setProducts(
 currentProducts =>
 currentProducts.filter(
 product =>
 product.id !== id
)
);

}
catch (error) {

 setError(
 "Unable to delete product"
);

}
finally {
```



```
 setDeletingId(null);
 }
};
```

This is the version I recommend students keep.

---

## 31. Adding a Delete Button to Our Product List

Our UI can now look like:

Product List

Laptop  
Price: ₹50000  
[View Details] [Delete]

Mouse  
Price: ₹1000  
[View Details] [Delete]

Keyboard  
Price: ₹2000  
[View Details] [Delete]

For example:

```
<div key={product.id}>
 <h2>
 {product.name}
 </h2>

 <p>
 Price: ₹{product.price}
 </p>

 <button
 onClick={() =>
 deleteProduct(product.id)
 }
 >
 Delete
 </button>
```

</div>

Later, we'll combine this with the Link from Lecture 22.

---

## 32. Combining View Details and Delete

Now our product item can have two actions:

View Details  
Delete

For example:

```
<div key={product.id}>

 <h2>
 {product.name}
 </h2>

 <p>
 Price: ₹{product.price}
 </p>

 <Link
 to={` /products/${product.id}`}
 >
 View Details
 </Link>

 <button
 onClick={() =>
 deleteProduct(product.id)
 }
 >
 Delete
 </button>

</div>
```

So the application now supports:

GET ALL  
↓  
Product List

GET BY ID



Product Details

DELETE



Remove Product

We're beginning to build a real CRUD application.

---

## 33. CRUD Connection

CRUD stands for:

C → Create

R → Read

U → Update

D → Delete

So far we've covered:

### Read

GET /products

GET /products/:id

### Delete

DELETE /products/:id

In the next two lectures, we'll work on:

POST

and:

PUT

So eventually our application will support all four CRUD operations.

---

## 34. Common Mistakes

### Mistake 1 — Forgetting the HTTP method

This:

```
fetch(url)
```

makes a GET request.

For DELETE:

```
fetch(url, {
 method: "DELETE"
});
```

---

## Mistake 2 — Deleting without an ID

We need:

/products/2

not simply:

/products

because we need to identify which product should be deleted.

---

## Mistake 3 — Not Updating React State

The API can successfully delete the product while the React UI continues displaying it.

Remember:

API state

+

React state

both need to be synchronized.

---

## Mistake 4 — Updating UI Before API Success

Don't remove the product from the UI before confirming that the server successfully deleted it.

Prefer:

DELETE

↓

Success

↓  
setProducts()

---

## Mistake 5 — Forgetting Confirmation

For destructive actions, confirmation is usually a good idea.

For our demo:

```
window.confirm(...)
```

is enough.

---

## Mistake 6 — Calling the Function Immediately

Don't:

```
onClick={
 deleteProduct(product.id)
}
```

Use:

```
onClick={() =>
 deleteProduct(product.id)
}
```

---

## 35. Live Coding Flow

For your video, I recommend this sequence.

### Step 1 — Review previous lectures

Briefly show:

```
GET /products
GET /products/:id
```

Then introduce:

```
DELETE /products/:id
```

---

---

## Step 2 — Demonstrate the API

Show your `db.json`.

Then show:

`GET /products`

and explain which product we're going to delete.

---

## Step 3 — Create Delete Function

Start simple:

```
const deleteProduct = async (id) => {

 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "DELETE"
 }
);

};
```

---

## Step 4 — Add Delete Button

```
<button
 onClick={() =>
 deleteProduct(product.id)
 }
>
 Delete
</button>
```

---

## Step 5 — Test

Click Delete.

Show the Network tab.

Point out:

Request Method: DELETE

---

## Step 6 — Update React State

Add:

```
setProducts(
 currentProducts =>
 currentProducts.filter(
 product =>
 product.id !== id
)
);
```

Demonstrate that the product disappears.

---

## Step 7 — Add Confirmation

Add:

```
window.confirm(...)
```

---

## Step 8 — Add Error Handling

Add:

```
try
catch
finally
```

---

## Step 9 — Add Deleting State

Show:

Delete

changing to:

Deleting...

---

## Step 10 — Refresh

Refresh the browser and show that the deleted product is still gone.

This confirms that the server data was actually modified.

---

## 36. Suggested Video Timeline

0:00 – 1:30

Introduction + DELETE concept

1:30 – 3:00

GET vs DELETE + HTTP methods

3:00 – 5:00

Basic fetch DELETE request

5:00 – 7:00

Delete button + product ID

7:00 – 9:00

Updating React state with `filter()`

9:00 – 11:00

Confirmation + error handling

11:00 – 12:30

Deleting loading state

12:30 – 14:00

Network tab + testing

14:00 – 15:00

Summary + exercise

---

## 37. Interview Questions

### Question 1

What HTTP method is used to delete a resource?

DELETE

---



## Question 2

How do you send a DELETE request using fetch?

```
fetch(url, {
 method: "DELETE"
});
```

---

## Question 3

Why do we pass the product ID?

Because the API needs to know which resource should be deleted.

Example:

DELETE /products/5

---

## Question 4

Why do we update React state after deletion?

Because deleting data on the server doesn't automatically remove the item from React's local state.

---

## Question 5

Why is `filter()` useful after deleting an item?

Because it creates a new array containing every product except the deleted product.

---

## Question 6

Why should we update state only after a successful DELETE?

Otherwise, the UI could show that the product was deleted even though the server operation failed.

---

## Question 7

What is CRUD?

Create  
Read  
Update  
Delete

---

## 38. Practice Exercise

Ask students to implement the following.

### Product Delete Exercise

Create a product list using:

GET /products

For every product, display:

Name  
Price  
Quantity  
Category

Add a:

Delete

button.

When the user clicks Delete:

35. Ask for confirmation.

36. Send:

DELETE /products/:id

8. Handle API errors.

9. Show Deleting... while the request is running.

10. Remove the deleted product from React state.

11. Verify that the product is also removed from the API.

---

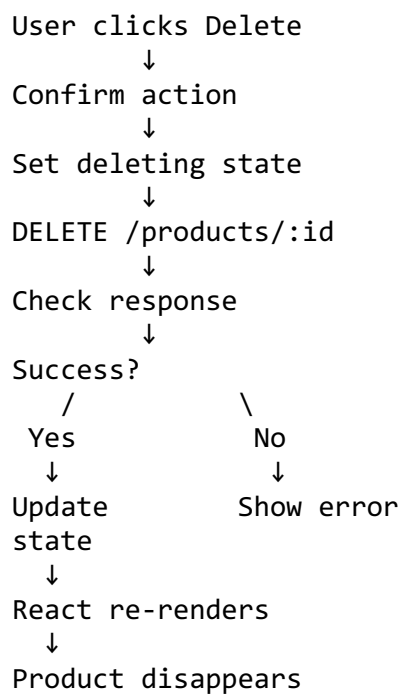
## 39. Final Summary

Today we learned how to consume a DELETE API in React.

```
await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "DELETE"
 }
);
```

But building a good application requires more than just sending the request.

The complete pattern is:



The key React code is:

```
setProducts(
 currentProducts =>
 currentProducts.filter(
 product =>
 product.id !== id
)
);
```

**Remember: DELETE changes the server, and setProducts() changes what React displays**

# Lecture 24 — Consuming INSERT API in React

## 1. Opening

Hello everyone, and welcome back.

In the previous lecture, we learned how to **delete data using a DELETE API**.

We learned how to send:

```
DELETE /products/:id
```

and then update our React state after the product was successfully deleted.

Until now, our application can:

GET



Read products

GET BY ID



Read one product

DELETE



Delete a product

But there is one important operation missing.

What if we want to **add a new product**?

For example, currently our database contains:

Laptop

Mouse

Keyboard

Monitor

Now suppose the user wants to add:

Name: Webcam

Price: 3000

Quantity: 15

Category: Electronics

We need to take this information from a React form and send it to our API.

For that, we'll use the:

POST

HTTP method.

Today's complete flow is:

```
React Form
 ↓
User enters data
 ↓
React State
 ↓
Submit Form
 ↓
POST API
 ↓
JSON Server
 ↓
New Product Created
 ↓
Update UI
```

---

## 2. What Is POST?

Let's first understand the HTTP method.

We already know:

GET

is used to retrieve data.

For example:

GET /products

means:

Give me all products.

And:

DELETE /products/5

means:

Delete product 5.

Now:

POST /products

means:

Create a new product.

So our HTTP methods are becoming:

GET

→ Read

POST

→ Create

PUT/PATCH

→ Update

DELETE

→ Delete

These four operations are commonly called:

CRUD

which means:

Create

Read

Update

Delete

Today we're focusing on the **Create** operation.

---

### 3. POST vs GET

This is an important distinction.

When we use GET:

GET /products

we're primarily asking the server for information.

With POST:

POST /products

we are sending information to the server.

For example:

```
{
 "name": "Webcam",
 "price": 3000,
 "quantity": 15,
 "category": "Electronics"
}
```

The server receives this data and creates a new product.

So:

GET

Client ← Server

POST

Client → Server

A POST request can also receive a response from the server after the resource is created.

---

## 4. Our Product Object

Let's look at the product structure we've been using.

For example:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
}
```

When creating a new product, we generally don't need to provide the id when using JSON Server.

We can send:

```
{
 "name": "Webcam",
 "price": 3000,
 "quantity": 15,
 "category": "Electronics"
}
```

JSON Server will create the new resource and assign an ID.

So our React application needs to collect:

Name  
Price  
Quantity  
Category

---

## 5. Creating the Insert Page

Let's create a component.

For example:

```
src
└── components
 └── AddProduct.jsx
```

Inside:

```
function AddProduct() {

 return (
 <div>

 <h1>
 Add Product
 </h1>

 </div>
);
}
```

```
export default AddProduct;
```

Our first step is simply to create the page.



## 6. Creating the Form

Now let's add our form.

```
function AddProduct() {
 return (
 <div>

 <h1>
 Add Product
 </h1>

 <form>

 <div>
 <label>
 Product Name
 </label>

 <input
 type="text"
 />
 </div>

 <div>
 <label>
 Price
 </label>

 <input
 type="number"
 />
 </div>

 <div>
 <label>
 Quantity
 </label>

 <input
 type="number"
 />
 </div>
 </div>
)
}
```

```
 <div>
 <label>
 Category
 </label>

 <input
 type="text"
 />
 </div>

 <button>
 Add Product
 </button>

 </form>
 </div>
);
}
```

At this point we have a normal HTML form inside React.

But we haven't connected the inputs to React yet.

---

## 7. Why Do We Need React State?

Suppose the user types:

Webcam

into the name field.

We need React to know:

name = "Webcam"

Then the user enters:

3000

We need:

price = 3000

And:

quantity = 15

So our React component needs to maintain form data.

We can use:

`useState()`

which we already learned.

---

## 8. Creating Form State

Let's import `useState`.

```
import { useState } from "react";
```

Then create state:

```
const [product, setProduct] = useState({
 name: "",
 price: "",
 quantity: "",
 category: ""
});
```

Our initial state is:

```
{
 name: "",
 price: "",
 quantity: "",
 category: ""
}
```

As the user types, we'll update these values.

---

## 9. What Is a Controlled Input?

This is an important React concept.

Consider:

```
<input
 type="text"
 value={product.name}
/>
```

The value of the input comes from React state.

We call this a:

Controlled component

because React controls the value of the input.

But there's one problem.

If the user types something, we need to update the state.

So we also need:

onChange

For example:

```
<input
 type="text"
 value={product.name}
 onChange={...}
/>
```

---

## 10. Handling Input Changes

Let's create a function:

```
const handleChange = (event) => {

 setProduct({
 ...product,
 [event.target.name]:
 event.target.value
 });

};
```

This code is extremely important.

Let's understand it carefully.

Suppose we have:

```
<input
 name="name"
 value={product.name}
 onChange={handleChange}
/>
```

When the user types:

Webcam

we get:

```
event.target.name
```

which is:

name

And:

```
event.target.value
```

is:

Webcam

So React updates:

```
{
 name: "Webcam"
}
```

---

## 11. Understanding the Computed Property

This part:

```
[event.target.name]:
 event.target.value
```

might look confusing.

Let's take a simple example.

If:

```
event.target.name = "name"
```

then:

```
[event.target.name]
```

becomes:

```
["name"]
```

So this:

```
{
 [event.target.name]:
 event.target.value
}
```

becomes conceptually:

```
{
 name: "Webcam"
}
```

If the input name is:

price

then it becomes:

```
{
 price: "3000"
}
```

This allows one `handleChange()` function to manage multiple inputs.

---

## 12. Why the Spread Operator?

Our code is:

```
setProduct({
 ...product,
 [event.target.name]:
 event.target.value
});
```

The:

```
...product
```

copies the existing properties.

Suppose state is:

```
{
 name: "Webcam",
 price: "",
 quantity: "",
 category: ""
}
```

Then the user enters:

3000

into the price field.

We want:

```
{
 name: "Webcam",
 price: "3000",
 quantity: "",
 category: ""
}
```

We don't want to lose the existing name.

That's why we use:

...product

---

## 13. Adding `name` to Every Input

Now let's update our form.

```
<input
 type="text"
 name="name"
 value={product.name}
 onChange={handleChange}
>
```

Price:

```
<input
 type="number"
 name="price"
 value={product.price}
>
```

```
 onChange={handleChange}
 />
```

Quantity:

```
<input
 type="number"
 name="quantity"
 value={product.quantity}
 onChange={handleChange}
/>
```

Category:

```
<input
 type="text"
 name="category"
 value={product.category}
 onChange={handleChange}
/>
```

Now one function can handle all four fields.

---

## 14. Testing Form State

Before we connect the API, let's test the state.

Inside `handleChange()`:

```
console.log(
 event.target.name,
 event.target.value
);
```

Now type:

Webcam

You should see:

name Webcam

Then type:

3000

You should see:



price 3000

This is a good live-coding demonstration.

You can also:

```
console.log(product);
```

to see the entire object.

---

## 15. Handling Form Submit

Now we need to submit the form.

Currently:

```
<form>
```

We want:

```
<form onSubmit={handleSubmit}>
```

Then create:

```
const handleSubmit = (event) => {
 event.preventDefault();
};
```

Why do we need:

```
event.preventDefault();
```

Because normal HTML form submission causes the browser to reload/navigate.

We don't want that in our React application.

We want React to handle the submission.

So:

Normal HTML



Browser reload

React



Prevent default



Run our JavaScript

---

## 16. Creating the POST Request

Now we're ready for the important part.

Inside `handleSubmit()`:

```
const response = await fetch(
 "http://localhost:3001/products",
 {
 method: "POST"
 }
);
```

But this isn't enough.

We need to send our product data.

---

## 17. Sending JSON Data

The request becomes:

```
const response = await fetch(
 "http://localhost:3001/products",
 {
 method: "POST",

 headers: {
 "Content-Type":
 "application/json"
 },

 body: JSON.stringify(product)
 }
);
```

There are three important parts:

method  
headers  
body

Let's understand each one.

---

## 18. Method

We specify:

method: `"POST"`

This tells the server:

I want to create a new resource.

---

## 19. Headers

We write:

```
headers: {
 "Content-Type":
 "application/json"
}
```

This tells the server:

The data I'm sending is JSON.

This is important because our request body contains JSON.

---

## 20. Body

We write:

body: `JSON.stringify(product)`

Suppose our React state is:

```
{
 name: "Webcam",
 price: "3000",
}
```

```
 quantity: "15",
 category: "Electronics"
}
```

JavaScript has this as an object.

But the HTTP request body needs to contain JSON text.

So we use:

```
JSON.stringify(product)
```

which converts the JavaScript object into a JSON string.

Conceptually:

```
JavaScript object
 ↓
JSON.stringify()
 ↓
JSON text
 ↓
HTTP request
```

---

## 21. JSON.stringify() VS JSON.parse()

This is worth revising.

When receiving JSON from an API, we use:

```
response.json()
```

When sending a JavaScript object as JSON, we use:

```
JSON.stringify()
```

So remember:

Receiving:  
`response.json()`

Sending:  
`JSON.stringify()`

This distinction is very important when working with APIs.

---

## 22. Complete `handleSubmit()`

Now our function becomes:

```
const handleSubmit = async (event) => {

 event.preventDefault();

 const response = await fetch(
 "http://localhost:3001/products",
 {
 method: "POST",

 headers: {
 "Content-Type":
 "application/json"
 },

 body: JSON.stringify(product)
 }
);

 const data =
 await response.json();

 console.log(data);

};
```

Now we have a complete POST request.

---

## 23. What Will the API Return?

Suppose we send:

```
{
 "name": "Webcam",
 "price": "3000",
 "quantity": "15",
 "category": "Electronics"
}
```

The API may return something like:

```
{
 "name": "Webcam",
 "price": "3000",
 "quantity": "15",
 "category": "Electronics",
 "id": "5"
}
```

The important point is that the newly created product now has an ID assigned by the server.

We can inspect it:

```
console.log(data);
```

---

## 24. Checking the Network Tab

This is a very good point to demonstrate browser DevTools.

Open:

DevTools  
→ Network

Submit the form.

You should see something similar to:

Request URL:  
`http://localhost:3001/products`

Request Method:  
POST

Then inspect the request payload.

You should see:

```
{
 "name": "Webcam",
 "price": "3000",
 "quantity": "15",
 "category": "Electronics"
}
```

This is an excellent way to teach students what actually happens when the form is submitted.

---

## 25. Handling API Errors

We shouldn't assume every request succeeds.

Let's use:

```
if (!response.ok) {

 throw new Error(
 "Failed to create product"
);

}
```

So:

```
const response = await fetch(
 "http://localhost:3001/products",
 {
 method: "POST",
 headers: {
 "Content-Type":
 "application/json"
 },
 body: JSON.stringify(product)
 }
);

if (!response.ok) {

 throw new Error(
 "Failed to create product"
);

}
```

---

## 26. Using `try...catch`

Now let's make the function more robust.

```
const handleSubmit = async (event) => {
```

```
event.preventDefault();

try {

 const response =
 await fetch(
 "http://localhost:3001/products",
 {
 method: "POST",

 headers: {
 "Content-Type":
 "application/json"
 },

 body:
 JSON.stringify(product)
 }
);

 if (!response.ok) {

 throw new Error(
 "Failed to create product"
);

 }

 const data =
 await response.json();

 console.log(data);

}
catch (error) {

 console.error(error);

}

};
```

Now our application can handle a failed request.



## 27. Showing a Success Message

Let's add state:

```
const [message, setMessage] =
 useState("");
```

After successful creation:

```
setMessage(
 "Product added successfully"
);
```

Then in the JSX:

```
{
 message && (
 <p>
 {message}
 </p>
)
}
```

After submission:

Product added successfully

---

## 28. Clearing the Form

After successfully creating the product, we probably want to clear the form.

We can reset state:

```
setProduct({
 name: "",
 price: "",
 quantity: "",
 category: ""
});
```

So the flow becomes:

```
Submit
↓
POST API
↓
Success
```

↓  
Show message  
↓  
Clear form

---

## 29. Updating the Product List

There are two common approaches.

### Approach 1

After inserting the product, fetch the product list again.

```
POST
↓
GET /products
↓
setProducts()
```

### Approach 2

Use the response returned from POST.

For example:

```
const data =
 await response.json();

setProducts(
 currentProducts => [
 ...currentProducts,
 data
]
);
```

For this lecture, we're focusing on the insert API, so we'll keep the page simple.

In a real application, either approach can be appropriate depending on the architecture.

---

## 30. Adding Navigation After Insert

If our application has routing, we may want to navigate back to the product list after successfully adding a product.

For example:

/products/add

after successful insertion:

/products

We can use React Router's:

`useNavigate()`

Import:

```
import {
 useNavigate
} from "react-router-dom";
```

Then:

```
const navigate = useNavigate();
```

After successful creation:

```
navigate("/products");
```

So:

Add Product



POST API



Success



Navigate to Product List

This creates a much better user experience.

---

## 31. Complete Add Product Component

Now let's combine everything.

```
import {
 useState
} from "react";
```

```
import {
 useNavigate
```

```
} from "react-router-dom";

function AddProduct() {

 const navigate = useNavigate();

 const [product, setProduct] =
 useState({
 name: "",
 price: "",
 quantity: "",
 category: ""
 });

 const [message, setMessage] =
 useState("");

 const [error, setError] =
 useState("");

 const [saving, setSaving] =
 useState(false);

 const handleChange = (event) => {

 setProduct({
 ...product,
 [event.target.name]:
 event.target.value
 });

 };

 const handleSubmit = async (event) => {

 event.preventDefault();

 setMessage("");
 setError("");

 try {

 setSaving(true);

 const response =
 await fetch(
```

```
 "http://localhost:3001/products",
 {
 method: "POST",

 headers: {
 "Content-Type":
 "application/json"
 },

 body:
 JSON.stringify(product)
 }
);

 if (!response.ok) {

 throw new Error(
 "Failed to create product"
);

 }

 const data =
 await response.json();

 console.log(
 "Created product:",
 data
);

 setMessage(
 "Product added successfully"
);

 setProduct({
 name: "",
 price: "",
 quantity: "",
 category: ""
 });

 navigate("/products");
}
catch (error) {

 setError(
```

```
 "Unable to add product"
);

 }
 finally {
 setSaving(false);
 }
};

return (
 <div>

 <h1>
 Add Product
 </h1>

 {
 message && (
 <p>
 {message}
 </p>
)
 }

 {
 error && (
 <p>
 {error}
 </p>
)
 }

 <form
 onSubmit={handleSubmit}
 >

 <div>

 <label>
 Product Name
 </label>

 <input
```

```
 type="text"
 name="name"
 value={product.name}
 onChange={handleChange}
 />
 </div>

 <div>

 <label>
 Price
 </label>

 <input
 type="number"
 name="price"
 value={product.price}
 onChange={handleChange}
 />

 </div>

 <div>

 <label>
 Quantity
 </label>

 <input
 type="number"
 name="quantity"
 value={product.quantity}
 onChange={handleChange}
 />

 </div>

 <div>

 <label>
 Category
 </label>

 <input
 type="text"
 name="category"
```

```
 value={product.category}
 onChange={handleChange}
 />

 </div>

 <button
 type="submit"
 disabled={saving}
 >

 {
 saving
 ? "Saving..."
 : "Add Product"
 }

 </button>

 </form>

</div>

);
}

export default AddProduct;
```

## 32. Explaining the Complete Flow

Let's walk through this code as a teacher.

The user opens:

/products/add

React displays:

Add Product

Product Name: [            ]

Price:            [            ]



Quantity: [            ]

Category: [            ]

[Add Product]

The user enters:

Name        → Webcam  
Price        → 3000  
Quantity    → 15  
Category    → Electronics

Every input triggers:

`handleChange()`

which updates:

product

state.

When the user clicks:

Add Product

React executes:

`handleSubmit()`

First:

`event.preventDefault();`

prevents the browser's default form submission.

Then:

POST /products

is sent.

The request contains:

```
{
 "name": "Webcam",
 "price": "3000",
 "quantity": "15",
}
```

```
 "category": "Electronics"
 }
```

The server creates the product.

Then React receives the response.

Finally:

```
navigate("/products");
```

takes the user back to the product list.

---

### 33. Important Concept: `event.preventDefault()`

Let's emphasize this because students frequently forget it.

Without:

```
event.preventDefault();
```

the browser may perform a normal HTML form submission.

That can cause:

Page reload

But React applications are generally designed to handle the form submission in JavaScript.

So:

```
const handleSubmit = (event) => {
 event.preventDefault();

 // React code
};
```

is a common pattern.

---

### 34. Important Concept: `name` Attribute

Students should also understand why we use:

```
name="name"
```

```
name="price"
```

```
name="quantity"
```

```
name="category"
```

Because our generic handler reads:

```
event.target.name
```

For example:

Input

```
name = "price"
```

```
value = "3000"
```

The handler knows:

Update price to 3000

This lets us avoid creating four different functions.

We only need:

```
handleChange()
```

---

## 35. Why Use One State Object?

We could create:

```
const [name, setName] =
 useState("");
```

```
const [price, setPrice] =
 useState("");
```

```
const [quantity, setQuantity] =
 useState("");
```

```
const [category, setCategory] =
 useState("");
```

This works.

But then we need four different state variables.

Instead, we can use:

```
const [product, setProduct] =
 useState({
 name: "",
 price: "",
 quantity: "",
 category: ""
 });
```

This keeps related form data together.

For this simple form, that's a clean approach.

---

## 36. Number Inputs and Data Types

There's one subtle point students should know.

Even if we use:

```
<input
 type="number"
>
```

the value received from:

```
event.target.value
```

is generally a string.

So:

```
3000
```

may be stored as:

```
"3000"
```

rather than:

```
3000
```

For a beginner application, this may be acceptable.

But if our API or application requires actual numbers, we can convert them.

For example:

```
Number(event.target.value)
```

or:

```
parseInt(event.target.value)
```

depending on the requirement.

For today's basic POST lecture, we can keep the values as strings and discuss data conversion as a practical consideration.

---

## 37. Validation

Before sending data to the API, we should make sure the user has entered the required information.

For example:

```
if (
 !product.name ||
 !product.price ||
 !product.quantity ||
 !product.category
) {

 setError(
 "Please fill all fields"
);

 return;
}
```

Then:

```
Submit
 ↓
Validate
 ↓
Valid?
 / \
No Yes
 ↓ ↓
Error POST
```

This prevents empty products from being submitted.

---

## 38. Adding Basic Validation

Our submit function can begin with:

```
const handleSubmit = async (event) => {

 event.preventDefault();

 setMessage("");
 setError("");

 if (
 !product.name ||
 !product.price ||
 !product.quantity ||
 !product.category
) {

 setError(
 "Please fill all fields"
);

 return;
 }

 // POST request
};
```

Now students understand that validation happens **before** the API call.

---

## 39. Improved Final `handleSubmit()`

A good version for this lecture is:

```
const handleSubmit = async (event) => {

 event.preventDefault();

 setMessage("");
 setError("");

 if (
 !product.name ||
 !product.price ||
 !product.quantity ||
 !product.category
) {

 setError(
 "Please fill all fields"
);

 return;
 }

 // POST request
};
```

```
 !product.quantity ||
 !product.category
) {

 setError(
 "Please fill all fields"
);

 return;
 }

 try {

 setSaving(true);

 const response =
 await fetch(
 "http://localhost:3001/products",
 {
 method: "POST",

 headers: {
 "Content-Type":
 "application/json"
 },

 body:
 JSON.stringify(product)
 }
);

 if (!response.ok) {

 throw new Error(
 "Failed to create product"
);

 }

 const data =
 await response.json();

 console.log(
 "Created product:",
 data
);
 }
}
```

```
 setProduct({
 name: "",
 price: "",
 quantity: "",
 category: ""
 });

 navigate("/products");

 }
 catch (error) {

 setError(
 "Unable to add product"
);

 }
 finally {

 setSaving(false);

 }

};
```

---

## 40. Live Coding Demonstration

For the actual lecture, I recommend building this incrementally rather than pasting the complete code immediately.

### Step 1 — Create the component

AddProduct.jsx

---

### Step 2 — Create the form

Add:

Name  
Price  
Quantity  
Category  
Submit



---

### Step 3 — Add state

```
const [product, setProduct] =
 useState({
 name: "",
 price: "",
 quantity: "",
 category: ""
 });
```

---

### Step 4 — Connect inputs

Add:

value  
name  
onChange

---

### Step 5 — Create `handleChange()`

```
const handleChange = (event) => {

 setProduct({
 ...product,
 [event.target.name]:
 event.target.value
 });

};
```

---

### Step 6 — Create `handleSubmit()`

First show:

```
event.preventDefault();
```

Then add the POST request.

---

## Step 7 — Explain POST options

Highlight:

method: "POST"

then:

headers

then:

body

---

## Step 8 — Explain `JSON.stringify()`

Demonstrate the difference between:

product

and:

`JSON.stringify(product)`

---

## Step 9 — Test the API

Open:

Network → POST /products

Show the request payload.

---

## Step 10 — Check `db.json`

Show that the new product was created.

---

## Step 11 — Add validation

Prevent an empty form from being submitted.

## Step 12 — Add loading state

Change:

Add Product

to:

Saving...

while the request is running.

---

## Step 13 — Navigate back

After successful creation:

```
navigate("/products");
```

---

## 41. Suggested Video Timeline

0:00 - 1:30

Introduction + CRUD

1:30 - 3:00

What is POST?

3:00 - 4:30

Create AddProduct form

4:30 - 6:00

useState + controlled inputs

6:00 - 7:30

handleChange()

7:30 - 9:30

POST API + JSON.stringify()

9:30 - 11:00

API response + Network tab

11:00 - 12:30

Validation + error handling

12:30 – 14:00

Loading state + navigation

14:00 – 15:00

Summary + exercise

---

## 42. Common Mistakes

### Mistake 1 — Forgetting `method: "POST"`

Without:

`method: "POST"`

`fetch()` defaults to GET.

---

### Mistake 2 — Forgetting `Content-Type`

Use:

```
headers: {
 "Content-Type":
 "application/json"
}
```

when sending JSON.

---

### Mistake 3 — Forgetting `JSON.stringify()`

Don't send:

`body: product`

when the API expects JSON.

Use:

`body: JSON.stringify(product)`

## Mistake 4 — Forgetting `event.preventDefault()`

Without it, the browser may perform a normal form submission.

---

## Mistake 5 — Forgetting the `name` attribute

Our generic handler depends on:

```
event.target.name
```

so every input needs an appropriate name.

---

## Mistake 6 — Not checking the response

Don't assume the API always succeeds.

Use:

```
if (!response.ok) {
 throw new Error(
 "Failed to create product"
);
}
```

---

## Mistake 7 — Not clearing or navigating after success

After successful insertion, give the user clear feedback.

For example:

Product added successfully

and/or navigate back to the product list.

---

# 43. Interview Questions

## Question 1

Which HTTP method is generally used to create a resource?

POST

## Question 2

How do you send JSON data using `fetch()`?

```
fetch(url, {
 method: "POST",
 headers: {
 "Content-Type":
 "application/json"
 },
 body: JSON.stringify(data)
});
```

## Question 3

Why do we use `JSON.stringify()`?

To convert a JavaScript object into a JSON string suitable for sending in the request body.

## Question 4

Why do we use `event.preventDefault()`?

To prevent the browser's default form submission and allow React to handle the submission.

## Question 5

What is a controlled input?

An input whose value is controlled by React state.

## Question 6

Why do we use the `name` attribute?

So a generic `handleChange()` function can determine which property of the form state should be updated.

## Question 7

What does CRUD stand for?

Create  
Read  
Update  
Delete

## Question 8

Which CRUD operation are we implementing today?

Create

using:

POST

---

## 44. Practice Exercise

Ask students to create an **Add Product** page.

The form should contain:

Product Name

Price

Quantity

Category

Requirements:

37. Use `useState()` for the form.
38. Use controlled inputs.
39. Create one `handleChange()` function.
40. Handle form submission using `handleSubmit()`.
41. Prevent the default form submission.
42. Validate that all fields are filled.
43. Send:

POST /products

8. Send JSON using:

`JSON.stringify()`

9. Show `Saving...` while the API request is running.
  10. Show an error if the request fails.
  11. After success, navigate back to the product list.
  12. Verify the new product exists in JSON Server.
-

## 45. Final Summary

Today we learned how to create a new resource using a POST API.

The basic structure is:

```
await fetch(
 "http://localhost:3001/products",
 {
 method: "POST",

 headers: {
 "Content-Type":
 "application/json"
 },

 body:
 JSON.stringify(product)
 }
);
```

The complete flow is:

```
graph TD
 A[User enters data] --> B[Controlled inputs]
 B --> C[React state]
 C --> D[Submit form]
 D --> E[Validate]
 E --> F[POST request]
 F --> G[JSON.stringify()]
 G --> H[API]
 H --> I[Product created]
 I --> J[Navigate / Update UI]
```

The three things I especially want students to remember are:

1. POST is used to create data.
2. JSON.stringify() converts our



JavaScript object into JSON.

3. A React form can be controlled using `useState()`.

# Lecture 25 — Consuming UPDATE API in React

## 1. Opening

Hello everyone, and welcome to the final lecture of our React API series.

In the previous lecture, we learned how to **insert new data** into our API using the POST method.

We created an Add Product form and followed this flow:

```
graph TD
 A[User enters product] --> B[React State]
 B --> C[POST API]
 C --> D[Product Created]
```

Today we're going to learn the final operation:

UPDATE

Suppose our product list contains:

Laptop  
Mouse  
Keyboard  
Monitor

And suppose the price of the laptop changes from:

50000

to:

55000

We don't want to create a new laptop.

We want to **modify the existing laptop**.

That's what an UPDATE API is used for.

By the end of today's lecture, we'll have completed CRUD:

C → Create → POST  
R → Read → GET  
U → Update → PUT  
D → Delete → DELETE

So today's lecture completes the basic CRUD cycle in React.

---

## 2. What Does UPDATE Mean?

Let's understand the basic idea first.

Suppose we have:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
}
```

Now the user wants to change the price:

50000 → 55000

We want the API to become:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 55000,
 "quantity": 10,
 "category": "Electronics"
}
```

The record is not being created again.

The existing record is being changed.

---

## 3. PUT API

For today's example, we'll use:

PUT

For example:

PUT /products/1

This means:

Update product whose ID is 1.

The general structure is:

PUT /products/:id

For example:

PUT /products/10

means:

Update product with ID 10.

---

## 4. CRUD Revision

Before we start coding, let's revise everything we've learned.

### CREATE

POST /products

Used to create a product.

---

### READ

GET /products

Used to retrieve all products.

---

### READ ONE

GET /products/10

Used to retrieve product 10.

---

## UPDATE

PUT /products/10

Used to update product 10.

---

## DELETE

DELETE /products/10

Used to delete product 10.

---

So our API table is:

Operation	HTTP Method	Example
Create	POST	/products
Read all	GET	/products
Read one	GET	/products/10
Update	PUT	/products/10
Delete	DELETE	/products/10

This table is worth showing on screen.

---

## 5. What Are We Going to Build?

Today we'll create an Edit Product page.

The user starts on:

/products

They see:

Laptop	₹50000	Edit
Mouse	₹500	Edit
Keyboard	₹1000	Edit

The user clicks:

Edit

for Laptop.

React navigates to:

`/products/edit/1`

The application gets the ID:

`1`

Then:

`GET /products/1`

retrieves the existing product.

The form is populated:

Product Name: Laptop  
Price: 50000  
Quantity: 10  
Category: Electronics

The user changes:

Price: 55000

Then clicks:

Update Product

React sends:

`PUT /products/1`

with the updated data.

That's our complete flow.

---

## 6. Creating the Edit Component

Let's create:

```
src
├── components
│ └── EditProduct.jsx
```

Start with:

```
function EditProduct() {
```

```
 return (
 <div>

 <h1>
 Edit Product
 </h1>

 </div>
);
}

export default EditProduct;
```

At the moment, this is just an empty component.

Now we'll make it functional.

---

## 7. We Need the Product ID

Our URL will look like:

/products/edit/1

The important information here is:

1

That's the product ID.

We need to get that ID from the URL.

Because we're using React Router, we can use:

`useParams()`

Import:

```
import {
 useParams
} from "react-router-dom";
```

Then:

```
const { id } = useParams();
```

Now:

URL :  
/products/edit/1

id:  
1

---

## 8. Why Do We Need the ID?

We need to tell the API exactly which product we're updating.

If the ID is:

1

our API URL becomes:

/products/1

If the ID is:

5

our URL becomes:

/products/5

So:

``http://localhost:3001/products/${id}``

creates the correct URL dynamically.

---

## 9. Creating Product State

We need to store the product in React state.

```
const [product, setProduct] =
 useState({
 name: "",
 price: "",
 quantity: "",
 category: ""
 });
```

Initially, the form is empty.



We'll later fetch the existing product and put its values into this state.

---

## 10. Loading the Existing Product

Now we need:

GET /products/:id

This is exactly what we learned in Lecture 22.

We'll use:

`useEffect()`

Import:

```
import {
 useEffect,
 useState
} from "react";
```

Then:

```
useEffect(() => {
 getProduct();
```

```
}, [id]);
```

We'll create:

```
const getProduct = async () => {
 // API call
};
```

---

## 11. Calling GET by ID

Our function becomes:

```
const getProduct = async () => {
 const response =
```

```
 await fetch(
 `http://localhost:3001/products/${id}`
);

 const data =
 await response.json();

 setProduct(data);
 };
```

Now let's understand what happens.

Suppose:

`id = 1`

The URL becomes:

`http://localhost:3001/products/1`

The API returns:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
}
```

Then:

```
setProduct(data);
```

puts that information into React state.

---

## 12. Why `useEffect()`?

We don't want to call the API every time the component renders.

We want to fetch the product when the component loads or when the ID changes.

That's why we use:

```
useEffect(() => {
```

```
getProduct();
```

```
}, [id]));
```

The dependency:

```
[id]
```

means the effect runs when id changes.

For example:

```
/products/edit/1
```

fetches product 1.

If the route changes to:

```
/products/edit/2
```

React can fetch product 2.

---

## 13. Creating the Edit Form

Now let's create our form.

```
<form>
```

```
 <div>
```

```
 <label>
```

```
 Product Name
```

```
 </label>
```

```
 <input
```

```
 type="text"
```

```
 name="name"
```

```
 value={product.name}
```

```
 onChange={handleChange}
```

```
 />
```

```
 </div>
```

```
 <div>
```

```
 <label>
```

```
 Price
 </label>

 <input
 type="number"
 name="price"
 value={product.price}
 onChange={handleChange}
 />

 </div>

 <div>

 <label>
 Quantity
 </label>

 <input
 type="number"
 name="quantity"
 value={product.quantity}
 onChange={handleChange}
 />

 </div>

 <div>

 <label>
 Category
 </label>

 <input
 type="text"
 name="category"
 value={product.category}
 onChange={handleChange}
 />

 </div>

 <button>
 Update Product
 </button>

</form>
```

This looks very similar to the form from Lecture 24.

That's intentional.

---

## 14. Reusing `handleChange()`

We can use the same pattern:

```
const handleChange = (event) => {

 setProduct({
 ...product,
 [event.target.name]:
 event.target.value
 });

};
```

Let's understand the flow.

Suppose the user changes:

Price

from:

50000

to:

55000

The input has:

`name="price"`

Therefore:

`event.target.name`

is:

price

and:

`event.target.value`

is:

55000

React updates:

```
{
 name: "Laptop",
 price: "55000",
 quantity: "10",
 category: "Electronics"
}
```

---

## 15. Handling the Update

Now we need to handle form submission.

```
<form
 onSubmit={handleSubmit}
>
```

Then:

```
const handleSubmit = async (event) => {

 event.preventDefault();

 // PUT API

};
```

Again, we use:

```
event.preventDefault();
```

because we don't want the browser to perform a normal form submission.

---

## 16. Creating the PUT Request

Now comes the most important part.

We'll use:

```
const response =
 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "PUT",

 headers: {
 "Content-Type":
 "application/json"
 },

 body:
 JSON.stringify(product)
 }
);
```

Notice how similar this is to POST.

The major difference is:

POST

versus:

PUT

and the URL contains the ID.

---

## 17. POST vs PUT

Let's compare them.

POST:

```
fetch(
 "http://localhost:3001/products",
 {
 method: "POST",
 ...
 }
);
```

PUT:

```
fetch(
 `http://localhost:3001/products/${id}`,
 {
 ...
 }
);
```

```
 method: "PUT",
 ...
 }
);
```

POST says:

Create a new product.

PUT says:

Update this specific product.

So the ID is very important for an update operation.

---

## 18. Sending the Updated Data

Suppose our state contains:

```
{
 name: "Laptop",
 price: "55000",
 quantity: "10",
 category: "Electronics"
}
```

We send:

body: `JSON.stringify(product)`

The server receives the updated product information.

For:

`id = 1`

the request is effectively:

PUT /products/1

with:

```
{
 "name": "Laptop",
 "price": "55000",
 "quantity": "10",
 "category": "Electronics"
}
```



## 19. Checking the Response

As with POST, we should check:

```
if (!response.ok) {

 throw new Error(
 "Failed to update product"
);

}
```

This allows us to handle API errors.

## 20. Complete `handleSubmit()`

Our function becomes:

```
const handleSubmit = async (event) => {

 event.preventDefault();

 try {

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "PUT",

 headers: {
 "Content-Type":
 "application/json"
 },

 body:
 JSON.stringify(product)
 }
);

 if (!response.ok) {
```

```
 throw new Error(
 "Failed to update product"
);
 }

 const data =
 await response.json();

 console.log(
 "Updated product:",
 data
);
}
catch (error) {
 console.error(error);
}
};
```

Now the API update is complete.

---

## 21. What Happens After Updating?

Once the update succeeds, we don't necessarily want to leave the user on the edit page.

We can take them back to:

/products

For that, we'll use:

`useNavigate()`

Import:

```
import {
 useNavigate
} from "react-router-dom";
```

Then:

```
const navigate = useNavigate();
```

After successful update:

```
navigate("/products");
```

Now our flow is:

Edit Product



PUT API



Success



Product List

---

## 22. Adding Loading State

While the PUT request is running, we can disable the button.

Create:

```
const [saving, setSaving] =
 useState(false);
```

Before the request:

```
setSaving(true);
```

After completion:

```
setSaving(false);
```

We can use finally:

```
finally {

 setSaving(false);

}
```

Then our button becomes:

```
<button
 type="submit"
 disabled={saving}
>
 {
 saving
 ? "Updating..."
```

```
 : "Update Product"
 }
 </button>
```

The user will see:

Updating...

while the API request is running.

## 23. Adding Validation

We should also validate the form before calling the API.

```
if (
 !product.name ||
 !product.price ||
 !product.quantity ||
 !product.category
) {
 setError(
 "Please fill all fields"
);
 return;
}
```

Create:

```
const [error, setError] =
 useState("");
```

Then display it:

```
{
 error && (
 <p>
 {error}
 </p>
)
}
```

The flow is now:

Submit  
↓

```
Validate
 ↓
Valid?
 / \
No Yes
 ↓ ↓
Error PUT
```

---

## 24. Complete Edit Product Component

Now let's put everything together.

```
import {
 useEffect,
 useState
} from "react";

import {
 useNavigate,
 useParams
} from "react-router-dom";

function EditProduct() {

 const { id } = useParams();

 const navigate = useNavigate();

 const [product, setProduct] =
 useState({
 name: "",
 price: "",
 quantity: "",
 category: ""
 });

 const [error, setError] =
 useState("");

 const [saving, setSaving] =
 useState(false);

 useEffect(() => {
```

```
 getProduct();

}, [id]));

const getProduct = async () => {

 try {

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`
);

 if (!response.ok) {

 throw new Error(
 "Product not found"
);

 }

 const data =
 await response.json();

 setProduct(data);

 }
 catch (error) {

 setError(
 "Unable to load product"
);

 }

};

const handleChange = (event) => {

 setProduct({
 ...product,
 [event.target.name]:
 event.target.value
 });

};
```

```
const handleSubmit = async (event) => {

 event.preventDefault();

 setError("");

 if (
 !product.name ||
 !product.price ||
 !product.quantity ||
 !product.category
) {

 setError(
 "Please fill all fields"
);

 return;
 }

 try {

 setSaving(true);

 const response =
 await fetch(
 `http://localhost:3001/products/${id}`,
 {
 method: "PUT",

 headers: {
 "Content-Type":
 "application/json"
 },

 body:
 JSON.stringify(product)
 }
);

 if (!response.ok) {

 throw new Error(
 "Failed to update product"
);
 }
 }
}
```

```
 }

 const data =
 await response.json();

 console.log(
 "Updated product:",
 data
);

 navigate("/products");
 }
 catch (error) {

 setError(
 "Unable to update product"
);

 }
 finally {

 setSaving(false);

 }

};

return (

 <div>

 <h1>
 Edit Product
 </h1>

 {
 error && (
 <p>
 {error}
 </p>
)
 }

 <form
```



```
 onSubmit={handleSubmit}
 >

 <div>

 <label>
 Product Name
 </label>

 <input
 type="text"
 name="name"
 value={product.name}
 onChange={handleChange}
 />

 </div>

 <div>

 <label>
 Price
 </label>

 <input
 type="number"
 name="price"
 value={product.price}
 onChange={handleChange}
 />

 </div>

 <div>

 <label>
 Quantity
 </label>

 <input
 type="number"
 name="quantity"
 value={product.quantity}
 onChange={handleChange}
 />

 </div>
```

```
 <div>

 <label>
 Category
 </label>

 <input
 type="text"
 name="category"
 value={product.category}
 onChange={handleChange}
 />

 </div>

 <button
 type="submit"
 disabled={saving}
 >

 {
 saving
 ? "Updating..."
 : "Update Product"
 }

 </button>

 </form>

 </div>

);

}

export default EditProduct;
```

## 25. Adding the Route

Now we need to configure React Router.

Suppose our existing routes look like:

```
<Route
 path="/products"
 element={<ProductList />}
/>
```

```
<Route
 path="/products/add"
 element={<AddProduct />}
/>
```

We add:

```
<Route
 path="/products/edit/:id"
 element={<EditProduct />}
/>
```

The important part is:

`:id`

This is a route parameter.

For example:

`/products/edit/1`

gives:

`id = 1`

And:

`/products/edit/25`

gives:

`id = 25`

---

## 26. Adding an Edit Button

Now let's go to our product list.

Suppose we're rendering products:

```
{
 products.map(product => (
```

```
 <div key={product.id}>
 <h3>
 {product.name}
 </h3>
 <p>
 {product.price}
 </p>
 </div>
))
}
```

We'll add an Edit button.

Using Link:

```
<Link
 to={` /products/edit/${product.id}`}
>
 Edit
</Link>
```

So a product might display:

Laptop  
₹50000

[Edit]

When the user clicks Edit:

/products/edit/1

opens.

---

## 27. Complete User Flow

Now let's understand the entire application flow.

The user opens:

/products

React gets the product list:

GET /products

The user sees:

Laptop	₹50000	Edit
Mouse	₹500	Edit
Keyboard	₹1000	Edit

They click:

Edit

for Laptop.

The application navigates to:

/products/edit/1

useParams() gives:

id = 1

Then:

GET /products/1

returns:

```
{
 "id": 1,
 "name": "Laptop",
 "price": 50000,
 "quantity": 10,
 "category": "Electronics"
}
```

React puts this into state.

The form displays the existing values.

The user changes:

50000 → 55000

Then clicks:

Update Product

React sends:

PUT /products/1

The API updates the product.

Then React navigates to:

/products

The user sees:

Laptop      ₹55000      Edit

The update is complete.

---

## 28. PUT Request in DevTools

This is another important live demonstration.

Open:

Browser DevTools  
→ Network

Edit a product.

Click:

Update Product

Find the request:

PUT /products/1

Show students:

### Request Method

PUT

### Request URL

http://localhost:3001/products/1

### Request Payload

```
{
 "name": "Laptop",
 "price": "55000",
 "quantity": "10",
 "category": "Electronics"
}
```

This makes the API process much easier to understand visually.

---

## 29. POST vs PUT — Very Important

Students often confuse these two.

Let's make the difference very clear.

### POST

POST /products

We don't specify an existing product ID.

We're saying:

Create a new product.

Example:

Webcam  
₹3000

becomes a new product.

---

### PUT

PUT /products/1

We specify an existing product ID.

We're saying:

Update product 1.

Example:

Laptop  
₹50000

becomes:

Laptop  
₹55000

---

## 30. Why Do We First GET the Product?

Students may ask:

Why don't we directly show an empty form and send PUT?

Technically, we could know some of the data already.

But the normal edit flow is:

```
GET existing record
 ↓
Display current values
 ↓
User modifies values
 ↓
PUT updated record
```

This is important because the user needs to see the existing information before modifying it.

For example, if the product has:

Name: Laptop  
Price: 50000  
Quantity: 10  
Category: Electronics

we don't want the user to type everything again.

We populate the form automatically.

---

## 31. Why Do We Need `useParams()`?

The route:

```
/products/edit/:id
```

contains dynamic information.

For example:

```
/products/edit/1
/products/edit/2
/products/edit/10
```

The component needs to know which product is being edited.



That's why:

```
const { id } = useParams();
```

is important.

It allows us to dynamically construct:

```
`/products/${id}`
```

---

## 32. Why Is `id` in the `useEffect` Dependency?

We use:

```
useEffect(() => {
 getProduct();
}, [id]);
```

This means:

Run this effect when the ID changes.

Suppose we edit product 1:

```
id = 1
```

Then we navigate to product 2:

```
id = 2
```

The effect can run again and retrieve product 2.

This is a good example of why dependencies matter in `useEffect`.

---

## 33. Handling Product Not Found

What if someone enters:

```
/products/edit/9999
```

but product 9999 doesn't exist?

Our API may return an error response.

We should handle that.

```
if (!response.ok) {
 throw new Error(
 "Product not found"
);
}
```

Then:

```
catch (error) {
 setError(
 "Unable to load product"
);
}
```

This is much better than leaving the page blank.

---

## 34. A Better Loading Experience

We can also create:

```
const [loading, setLoading] =
 useState(true);
```

When fetching:

```
setLoading(true);
```

After the request:

```
setLoading(false);
```

Then:

```
if (loading) {
 return (
 <p>
 Loading product...
 </p>
);
}
```

```
}
```

The user gets feedback while the API request is running.

---

## 35. Important Concept: Two API Calls

Notice something interesting.

When opening the Edit page, we make:

```
GET /products/:id
```

When submitting the form, we make:

```
PUT /products/:id
```

So the complete edit process involves two API calls:

```
Open Edit Page
```

↓

```
GET
```

↓

```
Display existing data
```

↓

```
User modifies data
```

↓

```
PUT
```

↓

```
Update server
```

This is a very common pattern in real-world applications.

---

## 36. CRUD Is Now Complete

At this point, our React application can perform all four basic operations.

### CREATE

```
POST /products
```

Add a product.

---

## READ

GET /products

Display all products.

---

## READ ONE

GET /products/:id

Display one product.

---

## UPDATE

PUT /products/:id

Modify a product.

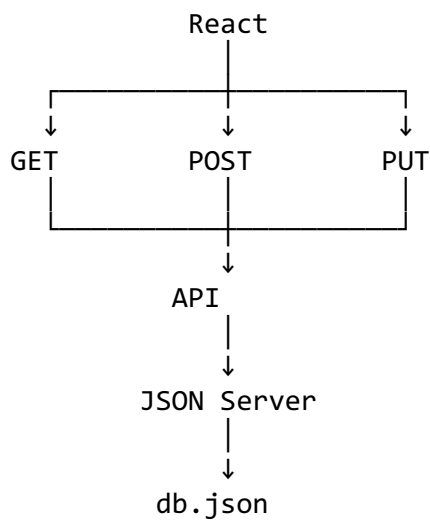
---

## DELETE

DELETE /products/:id

Remove a product.

So our complete architecture is:



And DELETE is also connected to the same API.

---

## 37. Complete CRUD Table

Show this on screen at the end of the lecture:

CRUD	HTTP Method	API
Create	POST	/products
Read	GET	/products
Read One	GET	/products/:id
Update	PUT	/products/:id
Delete	DELETE	/products/:id

Students should be able to explain this table without referring to their notes.

## 38. Common Mistakes

### Mistake 1 — Forgetting `:id`

Incorrect:

PUT /products

For our update operation, we need the specific product:

PUT /products/1

### Mistake 2 — Not using `useParams()`

If the ID is part of the URL:

/products/edit/:id

we need:

```
const { id } = useParams();
```

### Mistake 3 — Forgetting `JSON.stringify()`

Incorrect:

body: product

Use:

```
body: JSON.stringify(product)
```

---

## Mistake 4 — Forgetting Content-Type

Use:

```
headers: {
 "Content-Type":
 "application/json"
}
```

---

## Mistake 5 — Not preventing form submission

Use:

```
event.preventDefault();
```

---

## Mistake 6 — Updating the wrong product

Always verify the ID:

```
console.log(id);
```

and the URL:

```
console.log(
 `http://localhost:3001/products/${id}`
);
```

---

## Mistake 7 — Not loading the existing data

An edit form should normally retrieve the existing record first.

Use:

GET → Populate form → PUT

---

## 39. Interview Questions

### Question 1

Which HTTP method is commonly used for updating a resource?

PUT

---

### Question 2

How do you get a route parameter in React Router?

```
const { id } = useParams();
```

---

### Question 3

What does this route mean?

/products/edit/:id

It means id is a dynamic route parameter.

---

### Question 4

What is the difference between POST and PUT?

POST

→ Create a new resource

PUT

→ Update an existing resource

---

### Question 5

Why do we use GET before PUT?

To retrieve the existing resource and populate the edit form.

---

### Question 6

Why do we use `JSON.stringify()`?

To convert a JavaScript object into a JSON string for the request body.

---

### Question 7

Why do we use `response.ok`?

To determine whether the HTTP request was successful.

---

### Question 8

What does CRUD stand for?

Create  
Read  
Update  
Delete

---

## 40. Practical Exercise

Give students the following assignment.

Create a complete Product CRUD application.

The application should contain:

### Product List

`/products`

Requirements:

- Display all products.
- Show product name.
- Show price.
- Show quantity.
- Provide View button.
- Provide Edit button.



- Provide Delete button.
- 

## Add Product

/products/add

Requirements:

- Product name.
  - Price.
  - Quantity.
  - Category.
  - Form validation.
  - POST API.
- 

## Edit Product

/products/edit/:id

Requirements:

- Get ID using useParams().
  - Fetch existing product.
  - Populate form.
  - Allow editing.
  - Validate form.
  - Send PUT request.
  - Navigate back to product list.
- 

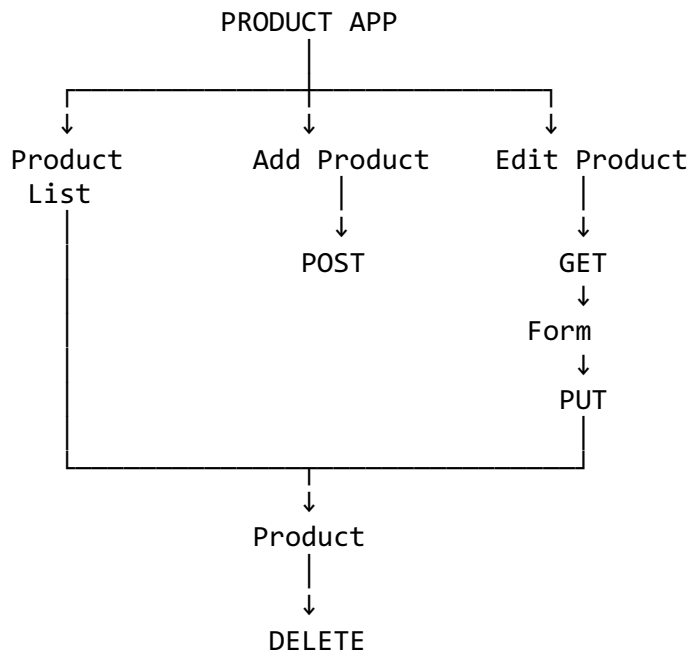
## Delete Product

Requirements:

- Ask for confirmation.
  - Send DELETE request.
  - Refresh/update the product list.
-

## 41. Final Project Flow

At the end of the series, students should be able to demonstrate:



This is the first point in the series where students can see how all the individual concepts they've learned work together.

## 42. Series Recap

Let's quickly review the entire React series.

### Lectures 1–5

We learned the React fundamentals:

- React
- JSX
- Expressions
- Styling
- Events

### Lectures 6–10

We learned components and props:

Components  
Function Components  
Class Components  
Component Files  
Props

## Lectures 11–15

We learned how to build more useful applications:

React DevTools  
Arrays  
Conditional Rendering  
Routing  
Parameterized Routes

## Lectures 16–19

We learned important React features:

Hooks  
useState  
useEffect  
Forms

## Lectures 20–25

We learned API integration and CRUD:

Mock API  
GET  
GET by ID  
DELETE  
POST  
PUT

So the progression is:

React Basics  
↓  
Components  
↓  
Props  
↓  
Routing  
↓  
Hooks  
↓  
Forms  
↓

APIs



CRUD



Complete React Application

---

## 43. Final Message to Students

At this point, you have learned the core concepts required to start building real React applications.

But remember:

Learning React is not about memorizing:

`useState()`  
`useEffect()`  
`fetch()`  
`useParams()`

The important thing is understanding **when and why** to use them.

For example:

Need to store changing data?



`useState`

Need to perform a side effect?



`useEffect`

Need to read URL parameters?



`useParams`

Need to communicate with an API?



`fetch`

Need to create data?



`POST`

Need to read data?



GET

Need to update data?



PUT

Need to delete data?



DELETE

Once you understand that decision-making process, React becomes much easier to work with.

---

## 44. Final Challenge

For the final challenge, don't simply copy the code from this series.

Build your own application.

For example:

Student Management System

or:

Employee Management System

or:

Product Management System

or:

Book Management System

Your application should support:

Create  
Read  
Update  
Delete

using React and an API.

Try adding:

- Search

- Filtering
- Sorting
- Form validation
- Loading indicators
- Error messages
- Confirmation dialogs
- Pagination

These features will help you move from a basic React application toward a more realistic project.

---

## 45. Closing

So this brings us to the end of our ReactJS lecture series.

We started with:

What is React?

and gradually moved through:

JSX  
Components  
Props  
Events  
Routing  
Hooks  
Forms  
APIs

and finally:

CRUD

Today we completed the final CRUD operation:

UPDATE

using:

PUT

The most important flow from today's lecture is:

Edit Button



Parameterized Route  
↓  
useParams()  
↓  
GET Existing Product  
↓  
Populate Form  
↓  
User Changes Data  
↓  
Submit  
↓  
PUT /products/:id  
↓  
API Updates Data  
↓  
Navigate to List

And now our complete CRUD flow is:

CREATE → POST  
READ → GET  
UPDATE → PUT  
DELETE → DELETE

That's the foundation of many real-world React applications.